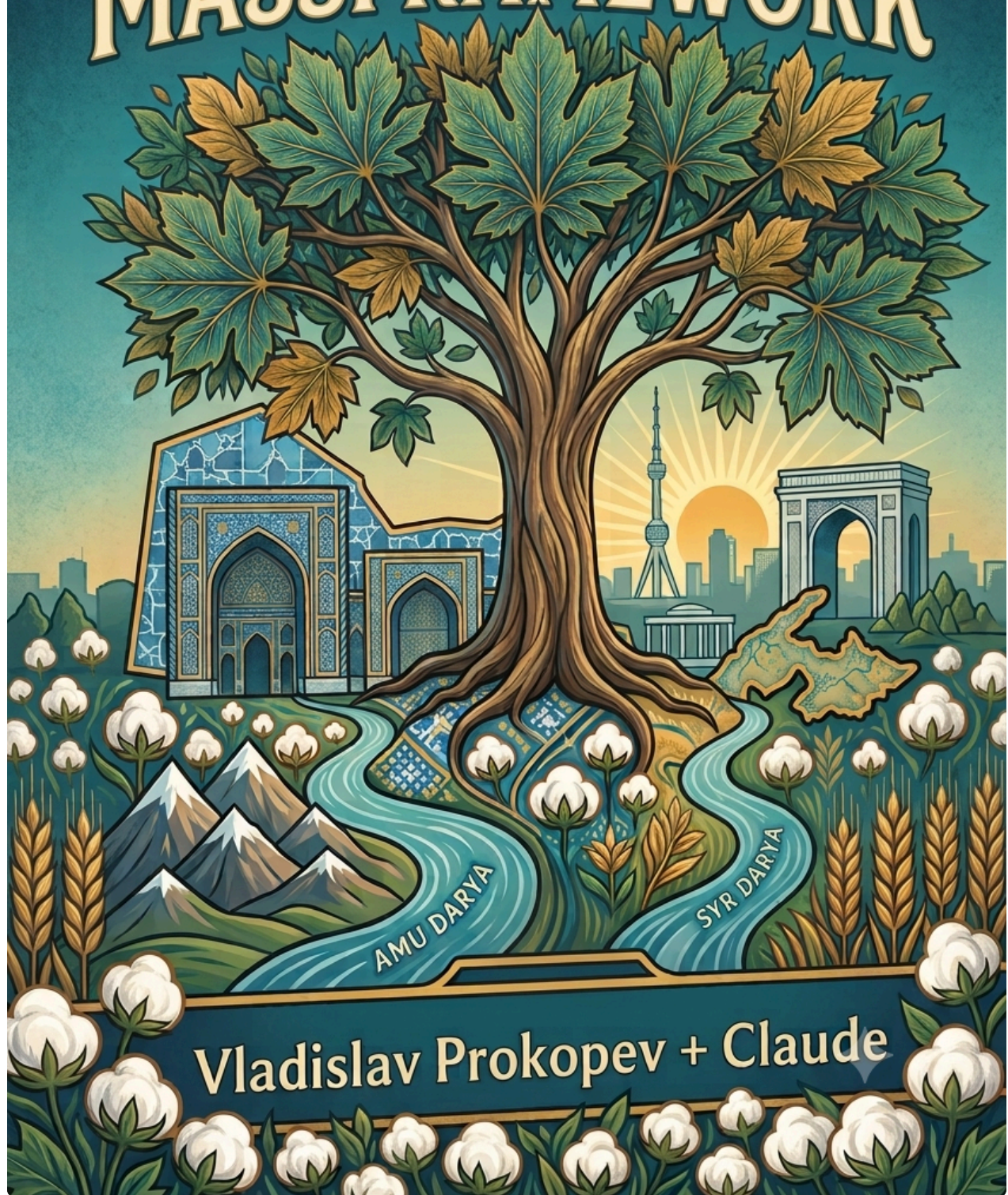


STATE TREE + MASSFRAMEWORK



Vladislav Prokopev + Claude

StateTree + Mass Framework в Unreal Engine

Полное руководство по модулю MassAIBehavior

Оглавление

Часть I. Ориентиры

1. Введение: задача, архитектура, карта модулей ← *текущая глава*
2. Mass за 30 минут: сущности, фрагменты, архетипы, процессоры, фазы
3. StateTree за 30 минут: состояния, узлы, instance data, схема, контекст
4. Точка стыковки: почему `UStateTreeComponent` не годится для Mass

Часть II. Базовые типы связки

5. `MassStateTreeTypes.h` — базовые классы узлов, сигналы, `FMassStateTreeInstanceHandle`
6. `MassStateTreeFragments.h` — фрагмент экземпляра и общий константный фрагмент
7. `StateTreeSchema.h` + `MassStateTreeSchema.h` — что схема разрешает и запрещает
8. Система зависимостей: `FStateTreeDependencyBuilder`, `FMassStateTreeDependency`, `GetDependencies()`

Часть III. Время жизни и исполнение

9. `UMassStateTreeSubsystem` — пул instance data, handle + generation, динамические процессоры
10. `StateTreeInstanceData.h` — `FStateTreeInstanceStorage`, `FStateTreeInstanceData`, временные данные
11. `StateTreeExecutionContext.h` — иерархия `ReadOnly` → `Minimal` → полный контекст, все методы
12. `FMassStateTreeExecutionContext` и `FMassExecutionExtension`
13. `MassStateTreeProcessors.h` — активация, разрушение, основной процессор; база `UMassProcessor`
14. `UMassSignalSubsystem` — событийная модель тика, отложенные и deferred-сигналы

Часть IV. Пишем узлы

15. Анатомия задачи: `FStateTreeTaskBase` — все флаги и все методы
16. Разбор `FMassLookAtTask` построчно
17. Разбор `FMassZoneGraphPathFollowTask` построчно

18. Эвалюаторы, условия и property functions в Mass-контексте

19. Своя задача с нуля: рецепт и типичные ошибки

Часть V. Сборка и эксплуатация

20. Трейты и конфигурация сущности: как ассет дерева попадает на агента

21. Сквозной пример: патрулирующий агент от пустого проекта до толпы

22. Отладка: Gameplay Debugger, visual log, симптомы и причины

23. Производительность: параллельный тик, стоимость сигналов, LOD, шардирование

24. Приложения: шпаргалка по типам, карта заголовков, чек-листы

Глава 1. Введение: задача и общая архитектура

1.1. Две технологии в одном предложении каждая

Mass Framework — это ECS (Entity Component System) внутри Unreal Engine. Вместо тысяч тяжёлых `AActor` с компонентами он хранит агентов как «сущности» — просто числовые идентификаторы, — а их данные раскладывает в плотные массивы структур (фрагментов), сгруппированных по архетипам. Логика живёт не в объектах, а в процессорах, которые пробегают по массивам пачками. Это даёт возможность держать в мире десятки тысяч агентов.

StateTree — это иерархическая машина состояний, объединённая с деревом поведения. Дерево состояний описывается в редакторе как ассет: состояния, вложенные состояния, задачи внутри состояний, условия входа, переходы по событиям и по завершению. StateTree рассчитан на дешёвое исполнение: скомпилированный ассет — это плоские массивы, а данные конкретного «экземпляра» дерева вынесены отдельно.

Связка нужна там, где надо задать **осмысленное поведение** (иди туда, посмотри на это, подожди, если увидел игрока — убегай) для **очень большого количества агентов**, которые физически не могут быть акторами.

1.2. Почему это не просто «повесить StateTree на актора»

В обычном игровом коде StateTree подключают через `UStateTreeComponent` — компонент актора. Он хранит `FStateTreeInstanceData` прямо в себе, тикает от движка через `TickComponent`, а узлы дерева обращаются к владельцу как к `AActor*`.

Для Mass это не работает по трём причинам:

Нет владельца-объекта. Сущность Mass — это `FMassEntityHandle`, пара из индекса и серийного номера. У неё нет `UObject`, к которому можно привязать компонент, нет `this`, нет виртуальных вызовов.

Нет места для данных экземпляра. `FStateTreeInstanceData` — довольно тяжёлая структура: в ней лежат экземпляры данных всех активных задач, стек активных состояний, очередь событий, отложенные переходы. Это не то, что кладут в фрагмент Mass: фрагменты должны быть маленькими POD-подобными структурами, чтобы плотно паковаться в чанки.

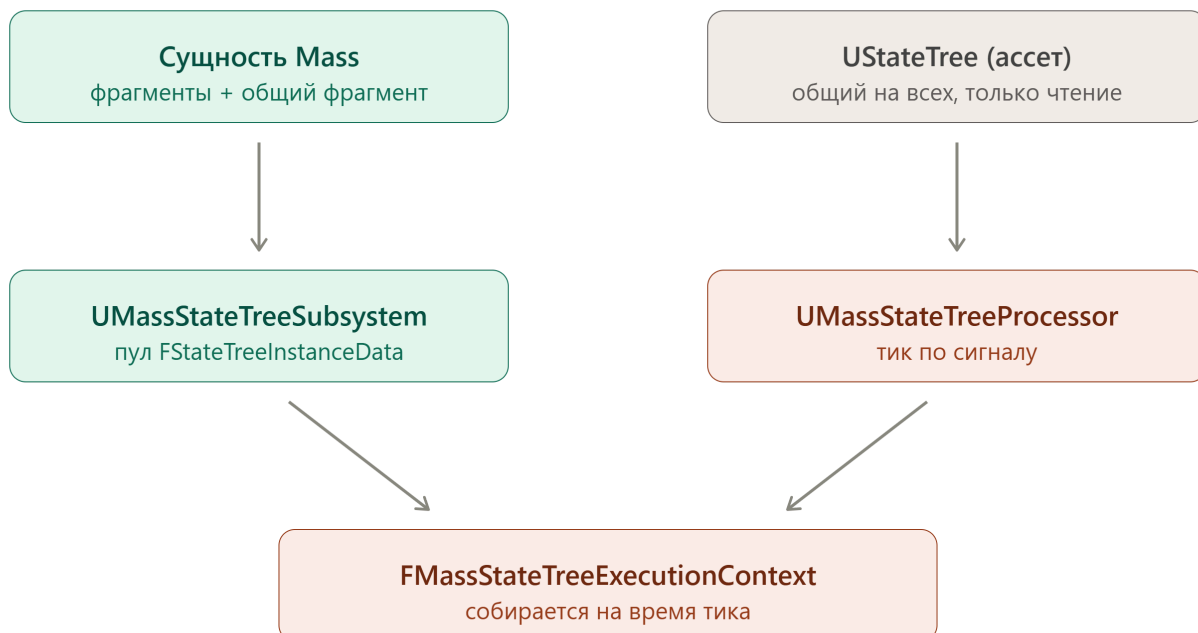
Нет обычного тика. В Mass никто не тикает по одному агенту. Процессор получает пачку чанков и обрабатывает их разом, а планировщик Mass должен заранее знать, к каким фрагментам процессор обращается, чтобы безопасно раскидать работу по потокам.

Модуль **MassAIBehavior** решает все три проблемы, и вся эта книга по сути — подробный разбор того, как именно.

1.3. Ключевая идея архитектуры

Идея в трёх словах: **данные экземпляра выносим в подсистему, на сущности храним только хендл, а исполнение запускаем из процессора через расширенный контекст.**

Вот как это выглядит на уровне блоков:



Разберём по стрелкам.

Сущность → подсистема. На агенте лежит `FMassStateTreeInstanceFragment`. Внутри него — всего два поля: `FMassStateTreeInstanceHandle` `InstanceHandle` и `double LastUpdateTimeInSeconds`. Хендл — это индекс в массиве подсистемы плюс «поколение» для защиты от переиспользованных индексов. Сам объёмный `FStateTreeInstanceData` живёт в `UMassStateTreeSubsystem::InstanceDataArray`. Фрагмент остаётся крошечным, чанки — плотными, а тяжёлые данные лежат в стороне.

Ассет → процессор. Какое именно дерево исполнять, задаётся через `FMassStateTreeSharedFragment` — это `FMassConstSharedFragment`, то есть один экземпляр на всю группу одинаково сконфигурированных сущностей. Тысяча агентов с одним поведением делят один указатель на `UStateTree`. Процессор при этом создаётся не один на всех: подсистема анализирует, к каким фрагментам обращается конкретный ассет, и порождает **динамический процессор** под каждый уникальный набор требований.

Подсистема + процессор → контекст. `FMassStateTreeExecutionContext` — это временный объект, который существует только внутри итерации по чанку. Он наследуется от `FStateTreeExecutionContext` и добавляет к нему `FMassEntityHandle` `Entity` и указатель на `FMassExecutionContext`. Благодаря этому задача внутри дерева может спросить «а какая сущность сейчас исполняется» и «дай мне её фрагмент движения».

1.4. Жизненный цикл поведения — обзорно

Полностью мы разберём это в главах 9–14, но общая канва такая:

Спавн. Сущность создаётся с трейтом, который добавляет `FMassStateTreeInstanceFragment` и `FMassStateTreeSharedFragment`. Наблюдатель на добавление фрагмента просит у подсистемы `AllocateInstanceData()` и записывает полученный хендл во фрагмент.

Активация. `UMassStateTreeActivationProcessor` находит сущности, у которых ещё нет тега `FMassStateTreeActivatedTag`, ставит тег и посылает сигнал `UE::Mass::Signals::StateTreeActivate`. Именно этот сигнал вызывает первый `Start()` дерева.

Исполнение. `UMassStateTreeProcessor` наследуется от `UMassSignalProcessorBase` — то есть он **не тикает каждый кадр по всем агентам**. Он просыпается только для тех сущностей, которым пришёл сигнал. Дерево, желающее тикать дальше, само переподписывается через `NewStateTreeTaskRequired` или через отложенный сигнал `DelayedTransitionWakeup`. Это ключевое решение по производительности: агент, стоящий в состоянии «жду», не стоит почти ничего.

Разрушение. `UMassStateTreeFragmentDestructor` — наблюдатель на удаление фрагмента. Он корректно останавливает дерево (чтобы у задач вызвался `ExitState`) и возвращает instance data в пул через `FreeInstanceData()`.

1.5. Карта модулей и файлов

Файлы, которые вы прислали, принадлежат четырём разным модулям. Полезно сразу понимать границы — они объясняют, почему одни вещи знают друг о друге, а другие нет.

Модуль	Файлы из пакета	Роль
<code>StateTreeModule</code>	<code>StateTreeTypes.h</code> , <code>StateTreeInstanceData.h</code> , <code>StateTreeExecutionContext.h</code> , <code>StateTreeTaskBase.h</code> , <code>StateTreeEvaluatorBase.h</code> , <code>StateTreeSchema.h</code>	Ядро <code>StateTree</code> . О <code>Mass</code> не знает ничего.
<code>MassEntity</code>	<code>MassEntityTypes.h</code> , <code>MassProcessor.h</code>	Ядро ECS: фрагменты, теги, архетипы, процессоры. О <code>StateTree</code> не знает ничего.
<code>MassSignals</code>	<code>MassSignalSubsystem.h</code>	Событийная шина <code>Mass</code> . Тоже независима.
<code>MassAIBehavior</code>	все <code>MassStateTree*.h</code> , <code>MassLookAtTask.h</code> , <code>MassZoneGraphPathFollowTask.h</code>	Клей. Единственный модуль, который знает про обе стороны.

Обратите внимание на важное свойство дизайна: зависимость односторонняя.

`StateTreeModule` не подозревает о существовании `Mass`, `MassEntity` — о существовании `StateTree`. Вся адаптация сосредоточена в `MassAIBehavior`. Именно поэтому там появляются собственные базовые структуры `FMassStateTreeTaskBase`, `FMassStateTreeConditionBase` и т. д. — они добавляют к обычным узлам `StateTree` единственный виртуальный метод `GetDependencies()`, через который узел сообщает планировщику `Mass`, к каким фрагментам он собирается обращаться.

Внутри `MassAIBehavior` файлы делятся по ролям так:

- **Типы и контракты:** `MassStateTreeTypes.h`, `MassStateTreeSchema.h`,
`MassStateTreeFragments.h`

- **Хранение и жизненный цикл:** `MassStateTreeSubsystem.h`
- **Исполнение:** `MassStateTreeExecutionContext.h` , `MassStateTreeProcessors.h`
- **Готовые узлы как образцы:** `MassLookAtTask.h` , `MassZoneGraphPathFollowTask.h`

1.6. Словарь терминов

Дальше эти слова будут встречаться постоянно, поэтому зафиксируем значения сразу.

Сущность (entity) — `FMassEntityHandle` , пара индекс + серийный номер. Никаких данных сама по себе не несёт.

Фрагмент (fragment) — `FMassFragment` , структура данных на сущности. Аналог компонента, но без поведения. Хранится в массиве внутри чанка.

Тег (tag) — `FMassTag` , структура нулевого размера. Присутствие или отсутствие тега меняет архетип сущности и потому влияет на то, какие запросы её увидят.

Общий фрагмент (shared fragment) — данные, разделяемые группой сущностей.
`FMassConstSharedFragment` — неизменяемая версия; именно её использует
`FMassStateTreeSharedFragment` .

Архетип (archetype) — уникальная комбинация фрагментов и тегов. Все сущности одного архетипа лежат в одних и тех же чанках памяти.

Процессор (processor) — `UMassProcessor` , единица логики. Объявляет запрос (`FMassEntityQuery`) и в `Execute()` обходит подходящие чанки.

Сигнал (signal) — именованное событие `Mass`, адресованное конкретным сущностям. Разбудить процессор сигналом дешевле, чем тикать всех подряд.

Узел (node) — общее название для задачи, условия, эвалюатора и `property function` в `StateTree`. Все они наследуются от `FStateTreeNodeBase` .

Instance data — данные конкретного экземпляра узла или дерева. У задачи это её `FInstanceDataType` ; у дерева целиком — `FStateTreeInstanceData` .

Схема (schema) — `UStateTreeSchema` , объект, который решает, какие узлы и какие внешние данные допустимы в дереве. Для `Mass` это `UMassStateTreeSchema` .

Внешние данные (external data) — то, что узел запрашивает у контекста через `TStateTreeExternalDataHandle<T>` : подсистемы, фрагменты сущности. Именно этот механизм даёт задаче доступ к данным `Mass`.

Линковка (linking) — этап, на котором дерево разрешает все хендлы: узел в методе `Link()` регистрирует свои требования, а схема в `UMassStateTreeSchema::Link()`

собирает из них итоговый список Mass-зависимостей.

Что дальше

В следующей главе мы разберём Mass с нуля: как устроены сущности, фрагменты, архетипы и чанки, что такое `FMassEntityQuery` и `FMassExecutionContext`, как процессоры выстраиваются в граф зависимостей и почему объявление требований — не бюрократия, а условие потокобезопасности. Всё это понадобится, чтобы главы про процессоры `StateTree` читались осмысленно, а не как набор незнакомых имён.

Глава 2. Mass за 30 минут

Эта глава — сжатый, но честный ввод в Mass. Если вы уже работали с ECS, всё равно пробежите её: у Unreal своя терминология и свои особенности, а без них главы про процессоры `StateTree` читаются как шум.

2.1. Смена модели данных

В привычном Unreal логика и данные лежат вместе: `AActor` владеет компонентами, компоненты владеют состоянием, у всех есть `Tick`. Это удобно, пока агентов сотни. На десятках тысяч всё рушится: каждый актор — это `UObject` с накладными расходами на GC, репликацию и реестры, каждый `Tick` — виртуальный вызов с непредсказуемым доступом к памяти.

Mass переворачивает раскладку. Данные отделяются от логики:

- **Данные** — маленькие структуры без методов, лежащие в плотных массивах.
- **Логика** — процессоры, которые обходят эти массивы пачками.
- **Идентичность** — просто число.

Выигрыш не столько от «отсутствия виртуальных вызовов», сколько от локальности кэша: когда процессор движения читает подряд десять тысяч `FTransformFragment`, лежащих вплотную, процессор CPU предсказывает загрузку и работает на полной скорости.

2.2. Сущность

`FMassEntityHandle` — это пара `int32 Index` и `int32 SerialNumber`. Никаких данных в ней нет.

cpp

```
FMassEntityHandle Entity = EntityManager.CreateEntity(ArchetypeHandle);
```

Серийный номер решает классическую проблему висячих ссылок: индексы переиспользуются, и без него хендл на удалённую сущность внезапно указывал бы на новую. `EntityManager.IsValid(Entity)` проверяет обе половины.

Хендл — это то, что вы будете передавать между системами. В контексте нашей книги он появляется в `FMassStateTreeExecutionContext::Entity` и в `FMassExecutionExtension::Entity`.

2.3. Элементы: пять видов данных

Mass различает несколько категорий данных, которые можно повесить на сущность. В UE 5.8 базовые типы вынесены в `MassElement.h`, но суть та же.

`FMassFragment` — обычный фрагмент. Персональные данные сущности: позиция, скорость, здоровье. Один экземпляр на сущность, лежит в массиве внутри чанка.

cpp

```
USTRUCT()  
struct FMassVelocityFragment : public FMassFragment  
{  
    GENERATED_BODY()  
    FVector Value = FVector::ZeroVector;  
};
```

`FMassTag` — тег. Структура нулевого размера, не хранит ничего. Её ценность в том, что наличие тега меняет архетип, а значит меняет, какие запросы увидят сущность. Это дешёвая булева метка «на уровне памяти». В `MassStateTreeProcessors.h` вы уже видели пример:

cpp

```
USTRUCT()  
struct FMassStateTreeActivatedTag : public FMassTag  
{  
    GENERATED_BODY()  
};
```

`FMassChunkFragment` — данные на чанк, а не на сущность. Используются редко: например, кэш LOD-уровня для целого чанка.

`FMassSharedFragment` — данные, разделяемые группой сущностей, изменяемые. Один экземпляр на группу.

`FMassConstSharedFragment` — то же самое, но неизменяемое. Именно этот вид использует `FMassStateTreeSharedFragment`, который хранит `TObjectPtr<UStateTree>`. Логика простая: ассет дерева одинаков для всей толпы и никогда не меняется в рантайме, значит нет смысла держать по указателю на каждого агента и нет смысла разрешать запись.

Разница между обычным и общим фрагментом — это, пожалуй, первое место, где надо принять осознанное решение при проектировании своих данных. Правило: если значение одинаково у всех агентов данной конфигурации и не меняется — общий константный фрагмент; если у каждого своё — обычный.

2.4. Архетип и чанки

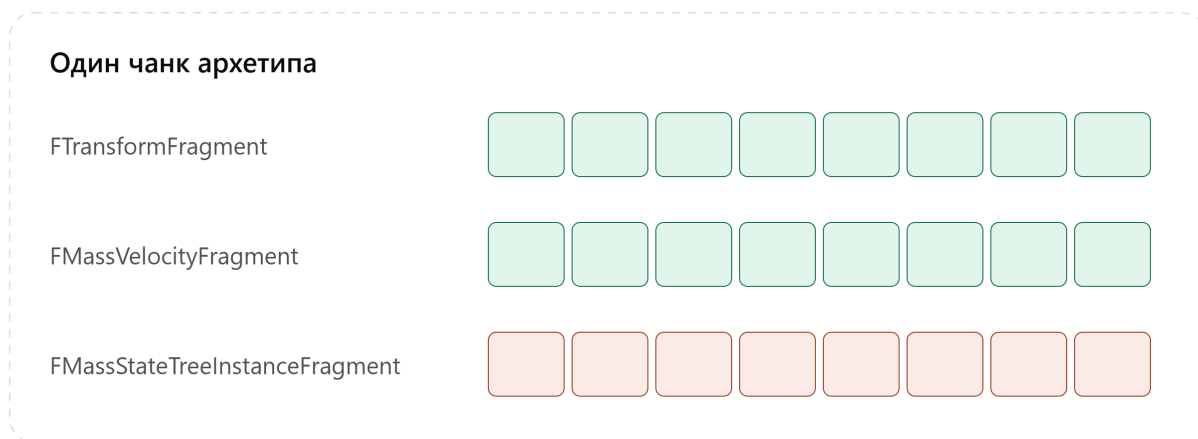
Архетип — уникальная комбинация фрагментов, тегов и общих фрагментов. Все сущности одного архетипа хранятся вместе.

Композиция описывается структурой `FMassArchetypeCompositionDescriptor`. Внутри она держит битовые множества по каждой категории — фрагменты, теги, чанк-фрагменты, изменяемые и константные общие фрагменты. В UE 5.8 они консолидированы в `FMassElementBitSet`, а старые отдельные геттеры (`GetChunkFragments()`, `GetSharedFragments()` и т. п.) помечены `UE_DEPRECATED(5.8, ...)` — их оставили ради совместимости, но новый код должен работать прямо с `FMassElementBitSet`.

Значения общих фрагментов хранит отдельная структура

`FMassArchetypeSharedFragmentValues` — она нужна потому, что битсет отвечает только на вопрос «есть ли такой фрагмент», а не «какое у него значение».

Память архетипа нарезана на **чанки**. Чанк — это блок фиксированного размера (по умолчанию `UE::Mass::ChunkSize`, при создании архетипа его можно переопределить через `FMassArchetypeCreationParams::ChunkMemorySize`). Внутри чанка данные хранятся не как массив структур, а как структура массивов:



Одна сущность — это одинаковый индекс во всех трёх массивах. Третий столбец сверху вниз — это трансформ, скорость и хендл instance data одного и того же агента.

Отсюда следует важное практическое правило: **добавление или удаление фрагмента/тега перемещает сущность в другой архетип**, то есть физически копирует её данные в другой чанк. Это не бесплатно. Поэтому теги вроде `FMassStateTreeActivatedTag` ставятся один раз за жизнь, а не переключаются каждый кадр.

2.5. FMassEntityManager

Менеджер сущностей — владелец всех архетипов и всех данных. Через него создают и уничтожают сущности, получают доступ к фрагментам, регистрируют наблюдателей. В процессорах он приходит как `FMassEntityManager&` в `Execute()`, а в `InitializeInternal()` — как `TSharedRef<FMassEntityManager>`.

Ключевой момент для нас: менеджер живёт в `TSharedPtr`. В `UMassStateTreeSubsystem` есть поле, которое его кэширует:

cpp

```
TSharedPtr<FMassEntityManager> EntityManager;
```

(да, там опечатка в исходниках Epic — первая буква `Ё`; в дальнейшем не удивляйтесь, это не ошибка распознавания).

2.6. Запрос: FMassEntityQuery

Процессор не обходит все сущности — он объявляет **запрос**, описывающий требуемый набор данных. Типичная конфигурация выглядит так:

cpp

```

void UMyProcessor::ConfigureQueries(const TSharedRef<FMassEntityManager>&
EntityManager)
{
    EntityQuery.AddRequirement<FTransformFragment>
(EMassFragmentAccess::ReadWrite);
    EntityQuery.AddRequirement<FMassVelocityFragment>
(EMassFragmentAccess::ReadOnly);
    EntityQuery.AddTagRequirement<FMassStateTreeActivatedTag>
(EMassFragmentPresence::All);
    EntityQuery.AddConstSharedRequirement<FMassStateTreeSharedFragment>();
    EntityQuery.AddSubsystemRequirement<UMassSignalSubsystem>
(EMassFragmentAccess::ReadWrite);
}

```

Здесь важны две оси:

Доступ (`EMassFragmentAccess`): `ReadOnly` или `ReadWrite`. Это не украшение — планировщик Mass строит по этим объявлениям граф зависимостей и решает, какие процессоры можно запускать параллельно. Два процессора, читающих один фрагмент, идут параллельно; читающий и пишущий — последовательно.

Присутствие (`EMassFragmentPresence`): `All` (обязателен), `Any`, `None` (не должно быть), `Optional` (может быть, доступ проверяется в рантайме).

Именно из-за этой механики появилась вся система `GetDependencies()` в `MassAIBehavior`: задачи `StateTree` обращаются к фрагментам, но процессор объявляет свои требования заранее, до того как узнает, какой ассет он будет исполнять. Без явного сбора зависимостей из дерева планировщик не смог бы гарантировать отсутствие гонок. Подробно — в главе 8.

2.7. FMassExecutionContext

Контекст исполнения — это то, что процессор получает на время обработки одного чанка. Он даёт:

- `GetNumEntities()` — сколько сущностей в текущем чанке;
- `GetEntity(int32 Index)` — хендл по индексу;
- `GetMutableFragmentView<T>()` / `GetFragmentView<T>()` — `TArrayView` на массив фрагментов чанка;
- `GetConstSharedFragment<T>()` — общий фрагмент;
- `GetMutableSubsystem<T>()` — подсистема, объявленная в запросе;
- `GetDeltaTimeSeconds()` ;
- `Defer()` — командный буфер (см. ниже).

Типичный обход:

cpp

```
EntityQuery.ForEachEntityChunk(Context,
    [](FMassExecutionContext& Context)
    {
        const TArrayView<FTransformFragment> Transforms =
Context.GetMutableFragmentView<FTransformFragment>();
        const TConstArrayView<FMassVelocityFragment> Velocities =
Context.GetFragmentView<FMassVelocityFragment>();

        for (int32 i = 0; i < Context.GetNumEntities(); ++i)
        {
            // ...
        }
    });
```

Есть и параллельный вариант — `ParallelForEachEntityChunk`. В `UMassStateTreeProcessor` выбор между ними управляется флагом `bProcessEntitiesInParallel`, который вы видели в заголовке.

Обратите внимание, что `FMassStateTreeExecutionContext` держит именно указатель на этот объект:

cpp

```
FMassExecutionContext* MassEntityExecutionContext = nullptr;
```

и предоставляет доступ через `GetMassEntityExecutionContext()` и `GetEntityManager()`. Это и есть мост, по которому задачи `StateTree` добираются до данных `Mass`.

2.8. `UMassProcessor` — подробно

Теперь разберём базовый класс всех процессоров, поскольку от него наследуются все три процессора `StateTree`.

Методы, которые переопределяют

cpp

```
virtual void InitializeInternal(UObject& Owner, const
TSharedRef<FMassEntityManager>& EntityManager);
virtual void ConfigureQueries(const TSharedRef<FMassEntityManager>&
```



```
EntityManager);  
virtual void Execute(FMassEntityManager& EntityManager, FMassExecutionContext&  
Context);
```

`InitializeInternal()` — место, где кэшируют подсистемы и другие внешние ссылки. Вызывается один раз, из `CallInitialize()`. Именно здесь `UMassStateTreeFragmentDestructor` получает `SignalSubsystem`.

`ConfigureQueries()` — объявление требований. Вызывается после инициализации; после этого запрос считается зафиксированным.

`Execute()` — собственно работа. Вызывается через `CallExecute()`, которая дополнительно проверяет флаги и состояние активации.

`ExportRequirements(FMassExecutionRequirements& OutRequirements)` — метод, который отдаёт агрегированные требования процессора наружу, для построения графа. Обычно базовая реализация сама собирает их из зарегистрированных запросов.

`UMassStateTreeProcessor` его переопределяет — потому что его требования не только из собственного запроса, но и из проанализированных ассетов `StateTree`.

Свойства, влияющие на планирование

`FMassProcessorExecutionOrder ExecutionOrder` — структура из трёх полей: `ExecuteInGroup` (имя группы; пусто = самая верхняя группа своей фазы), `ExecuteBefore` и `ExecuteAfter` — массивы имён процессоров или групп. Это декларативный способ сказать «я должен отработать после системы движения».

`EMassProcessingPhase ProcessingPhase` — фаза кадра, по умолчанию `PrePhysics`. Задаётся в конструкторе, потому что читается через CDO ещё до создания экземпляра.

`uint8 ExecutionFlags` — битовая маска `EProcessorExecutionFlags: Standalone / Server / Client / Editor`. Здесь стоит вернуться к `MassStateTreeTypes.h`:

cpp

```
namespace UE::MassStateTree  
{  
static constexpr EProcessorExecutionFlags ExecutionFlags(  
    EProcessorExecutionFlags::Standalone | EProcessorExecutionFlags::Server);  
}
```

То есть логика `StateTree` в Mass по умолчанию выполняется только на сервере и в одиночной игре. Клиент поведение не считает — он получает результаты через

репликацию или через LOD-представление. Это осознанное сетевое решение, и его стоит держать в голове.

`bAutoRegisterWithProcessingPhases` — нужно ли автоматически включать процессор в глобальный список. Для динамических процессоров ставится в `false`.

`bAllowMultipleInstances` — можно ли иметь несколько экземпляров класса в одном пайплайне. Для `UMassStateTreeProcessor` это критично: подсистема создаёт по процессору на каждый уникальный набор требований.

`bRequiresGameThreadExecution` — процессор обязан идти в игровом потоке.

`bAggregateRequirements` — объединять ли требования всех запросов процессора при разрешении конкуренции.

`int16 ExecutionPriority` — приоритет. Влияет не только на выбор следующего процессора, но и распространяется на зависимости: если высокоприоритетный процессор ждёт A и B, то A и B тоже поднимаются в приоритете.

`EActivationState ActivationState` — `Inactive`, `Active` или `OneShot`. Отключённый процессор остаётся в графе и корректно передаёт свои зависимости дальше, просто не исполняется. `OneShot` сам себя отключит после следующего `CallExecute()`.

`EMassQueryBasedPruning` — надо ли выкидывать процессор из графа, если под его требования не подходит ни один архетип. По умолчанию `Prune`.

Динамические процессоры

cpp

```
void MarkAsDynamic();
bool IsDynamic() const;
```

Динамический процессор — тот, который не регистрируется автоматически, а добавляется в фазу вручную в рантайме. Это позволяет иметь несколько экземпляров одного класса. Ровно этот механизм использует

`UMassStateTreeSubsystem::CreateProcessorForStateTree()`, и мы разберём его в главе 9.

2.9. UMassCompositeProcessor и граф

Процессоры не лежат плоским списком. `UMassCompositeProcessor` — контейнер, который держит дочерние процессоры и умеет строить из них граф исполнения (`BuildFlatProcessingGraph`, `UpdateProcessorsCollection`, `AddGroupedProcessor`).

Разрешением зависимостей занимается `FMassProcessorDependencySolver` — тот самый заголовок, который подключён в `MassStateTreeProcessors.h`.

Солвер берёт объявленные требования и порядок исполнения и выдаёт топологически отсортированный граф, где независимые ветви могут идти параллельно через `DispatchProcessorTasks`.

2.10. Наблюдатели

`UMassObserverProcessor` — специализация, которая срабатывает не каждый кадр, а на изменение состава сущности. Тип операции задаётся перечислением:

cpp

```
enum class EMassObservedOperation : uint8
{
    AddElement,        // элемент добавлен существующей сущности
    RemoveElement,     // элемент удалён
    DestroyEntity,     // сущность уничтожена (частный случай Remove)
    CreateEntity,      // сущность создана (частный случай Add)
    MAX,
    // Add и Remove – устаревшие значения
};
```

Есть и флаговая версия `EMassObservedOperationFlags`, где `Add = AddElement | CreateEntity`, `Remove = RemoveElement | DestroyEntity`, `All = Add | Remove`.

`UMassStateTreeFragmentDestructor` — как раз наблюдатель: он подписан на удаление `FMassStateTreeInstanceFragment`, чтобы успеть корректно остановить дерево до того, как данные исчезнут.

2.11. Трейты и шаблоны сущностей

Вручную перечислять фрагменты при спавне неудобно. Mass предлагает **трейты** — `UMassEntityTraitBase`. Трейт — это объект конфигурации, который в методе `BuildTemplate()` добавляет в шаблон сущности нужные фрагменты, теги и общие фрагменты.

Набор трейтов собирается в ассет `UMassEntityConfigAsset`, из которого строится `FMassEntityTemplate` — готовый рецепт архетипа. Именно так на агента попадают `FMassStateTreeInstanceFragment` и `FMassStateTreeSharedFragment`: через трейт, в котором дизайнер выбирает ассет дерева. Подробно — в главе 20.

2.12. Командный буфер

Изменять состав сущности прямо во время обхода чанков нельзя — это сдвинуло бы данные под ногами у итератора. Поэтому такие операции откладываются:

cpp

```
Context.Defer().AddTag<FMassStateTreeActivatedTag>(Entity);  
Context.Defer().DestroyEntity(Entity);
```

`FMassCommandBuffer` копит команды и исполняет их в безопасной точке кадра. Отсюда же растут «deferred»-варианты сигналов в `UMassSignalSubsystem`:

cpp

```
void SignalEntityDeferred(FMassExecutionContext& Context, FName SignalName,  
const FMassEntityHandle Entity);
```

Разница простая: обычный `SignalEntity()` можно звать вне обхода, deferred-версия кладёт команду в буфер и потому безопасна изнутри `Execute()`.

2.13. Что из этого критично для StateTree

Соберём в короткий список то, что реально понадобится дальше:

1. **Фрагмент должен быть маленьким.** Отсюда — вынос `FStateTreeInstanceData` в подсистему.
2. **Ассет — общий константный фрагмент.** Одно дерево на всю толпу, ноль дублирования.
3. **Требования объявляются заранее и определяют параллелизм.** Отсюда — вся система `GetDependencies()` и динамические процессоры под каждый уникальный набор требований.
4. **Тег меняет архетип, поэтому его ставят один раз.** Отсюда — `FMassStateTreeActivatedTag` как одноразовая метка активации.
5. **Изменения состава откладываются в командный буфер.** Отсюда — deferred-сигналы.
6. **Наблюдатели — единственный корректный способ отреагировать на появление и исчезновение данных.** Отсюда — `UMassStateTreeFragmentDestructor`.

Что дальше

В главе 3 разберём вторую половину связки — StateTree сам по себе: как устроено дерево состояний, чем задача отличается от эвалюатора, что такое instance data узла, как работают переходы и события, и почему у StateTree такая необычная схема доступа к внешним данным через хендлы. После этого мы будем готовы к разбору собственно `MassAIBehavior`.

Глава 3. StateTree за 30 минут

Теперь вторая половина связки. StateTree — технология относительно молодая (появилась в UE 5.0 как экспериментальная), и она сознательно сделана не так, как Behavior Tree. Понять эти отличия важно, иначе весь дизайн `MassAIBehavior` будет выглядеть произвольным.

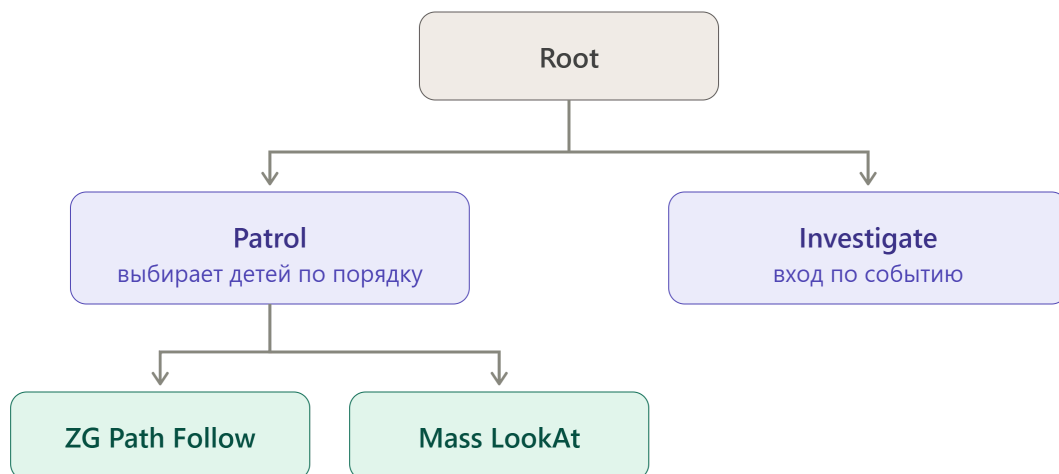
3.1. Что это за зверь

StateTree — гибрид **иерархической машины состояний** и **дерева поведения**.

От машины состояний он берёт главное: в каждый момент активен не один узел, а целый **стек состояний** — от корня до листа. Если активен лист «Идти к точке», то активны и все его родители, и задачи родителей тоже работают.

От дерева поведения он берёт **выбор состояния**: переход указывает не на конкретный лист, а на состояние, внутрь которого исполнение затем «проваливается» по правилам, проверяя условия входа на каждом уровне. Если условия не пропустили — выбор откатывается и пробуются следующий кандидат.

Вот минимальный пример поведения:



Нижний ряд — это **задачи внутри** состояния Patrol, а не дочерние состояния. Разница принципиальная: дочерние состояния взаимоисключающи (активно одно), а задачи одного состояния работают **одновременно**, пока состояние активно. Агент одновременно идёт по маршруту и смотрит на цель.

3.2. Типы состояний

Перечисление `EStateTreeStateType` из `StateTreeTypes.h` :

Значение	Смысл
State	Обычное состояние: содержит задачи и, возможно, дочерние состояния
Group	Только дочерние состояния, без задач — чистая группировка
Linked	Ссылка на другое состояние в этом же дереве; исполнение продолжается там
LinkedAsset	Ссылка на другой ассет StateTree; исполнение продолжается с его корня
Subtree	Поддерево, на которое можно сослаться из Linked

`LinkedAsset` — важный механизм для Mass: он позволяет собирать поведение из переиспользуемых кусков. Именно с ним связан метод `OnLinkedStateTreeOverridesSet()` в `FMassExecutionExtension`, который мы разберём в главе 12.

3.3. Как выбирается состояние

`EStateTreeStateSelectionBehavior` определяет, что происходит, когда исполнение «рассматривает» состояние как кандидата:

Значение	Поведение
None	Состояние нельзя выбрать напрямую
TryEnterState	Войти в него, даже если есть дети
TrySelectChildrenInOrder	Пробовать детей по порядку объявления
TrySelectChildrenAtRandom	Перемешать детей и пробовать
TrySelectChildrenWithHighestUtility	Выбрать ребёнка с наибольшей полезностью; при равенстве — по порядку
TrySelectChildrenAtRandomWeightedByUtility	Случайно, с вероятностью пропорциональной нормированной полезности

Значение	Поведение
TryFollowTransitions	Вместо входа попытаться сработать переходами

Последние три — это то, что делает StateTree похожим на utility AI. «Полезность» считают специальные узлы-considerations, а комбинируются результаты по правилам

EStateTreeExpressionOperand :

cpp

```
enum class EStateTreeExpressionOperand : uint8
{
    Copy UMETA(Hidden),
    And,           // для условий – AND, для полезности – Min(a, b)
    Or,            // для условий – OR, для полезности – Max(a, b)
    Multiply,      // (a * b), только для полезности
};
```

3.4. Четыре вида узлов

Все узлы наследуются от FStateTreeNodeBase и живут в дереве как FInstancedStruct .

Задача (FStateTreeTaskBase) — то, что делает работу, пока состояние активно. Полный разбор — в главе 15, пока три главных метода:

cpp

```
virtual EStateTreeRunStatus EnterState(FStateTreeExecutionContext& Context,
const FStateTreeTransitionResult& Transition) const;
virtual void ExitState(FStateTreeExecutionContext& Context, const
FStateTreeTransitionResult& Transition) const;
virtual EStateTreeRunStatus Tick(FStateTreeExecutionContext& Context, const
float DeltaTime) const;
```

Заметьте const в конце каждого. **Задача не имеет изменяемого состояния.** Экземпляр задачи в ассете один на всех; всё, что меняется, лежит в instance data. Это фундаментально для Mass, где одну задачу одновременно исполняют тысячи агентов.

Есть ещё два метода:

cpp


```
virtual void StateCompleted(FStateTreeExecutionContext& Context, const
EStateTreeRunStatus CompletionStatus, const FStateTreeActiveStates&
CompletedActiveStates) const;
virtual void TriggerTransitions(FStateTreeExecutionContext& Context) const;
```

StateCompleted() вызывается после завершения состояния, но **до** выбора нового, причём в обратном порядке — чтобы задача могла передать информацию задачам, стоящим выше по дереву. Важная оговорка из комментария в исходниках: он не вызывается, если состояние сменил условный переход.

TriggerTransitions() вызывается только если у задачи выставлен флаг bShouldAffectTransitions .

Условие (FStateTreeConditionBase) — проверка, возвращающая bool. Используется в условиях входа в состояние и в условиях переходов. Условия **не имеют персональной instance data в обычном смысле**: они работают на общих (SharedInstanceData) или временных (EvaluationScopeInstanceData) данных, потому что вычисляются в моменты, когда состояние ещё не активно.

Эвалюатор (FStateTreeEvaluatorBase) — вычисляет и публикует данные для принятия решений. Три метода:

cpp

```
virtual void TreeStart(FStateTreeExecutionContext& Context) const {}
virtual void TreeStop(FStateTreeExecutionContext& Context) const {}
virtual void Tick(FStateTreeExecutionContext& Context, const float DeltaTime)
const {}
```

Разница с задачей: эвалюатор глобален для дерева, а не привязан к состоянию, и работает всё время его исполнения. Типичное применение — «замечать» вещи: посчитать расстояние до игрока, чтобы условия могли на него смотреть.

Property function (FStateTreePropertyFunctionBase) — небольшая функция, вычисляющая значение прямо в момент привязки свойства. Позволяет писать Duration = Random(1.0, 3.0) без отдельного узла.

3.5. Статус исполнения

EStateTreeRunStatus — то, что возвращают EnterState() и Tick() :

- Running — работа продолжается;
- Succeeded — завершено успешно;

- `Failed` — завершено с ошибкой;
- `Stopped` — дерево остановлено извне;
- `Unset` — значение не задано.

Возврат `Succeeded` или `Failed` из `EnterState()` немедленно завершает состояние, не доводя дело до тика. Это удобный способ реализовать «мгновенную» задачу.

Когда состояние завершается, срабатывают переходы с триггером `OnStateCompleted` / `OnStateSucceeded` / `OnStateFailed`.

3.6. Переходы

Триггеры — битовая маска:

cpp

```
enum class EStateTreeTransitionTrigger : uint8
{
    None = 0,
    OnStateCompleted = 0x1 | 0x2, // = Succeeded | Failed
    OnStateSucceeded = 0x1,
    OnStateFailed = 0x2,
    OnTick = 0x4,
    OnEvent = 0x8,
    OnDelegate = 0x10,
};
```

Типы целей задаёт `EStateTreeTransitionType`: `None`, `Succeeded`, `Failed` (остановить дерево или поддереву с этим результатом), `GotoState`, `Parent`, `NextState`, `NextSelectableState`, `NextParent`, `NextSelectableParent`. Варианты с «Selectable» пробуют следующего соседа, пропуская тех, чьи условия входа не прошли.

Приоритеты — `EStateTreeTransitionPriority`: `Low`, `Normal`, `Medium`, `High`, `Critical`. При одновременном срабатывании нескольких переходов побеждает первый из самых приоритетных. Именно это поле есть у задачи: `EStateTreeTransitionPriority`
`TransitionHandlingPriority = Normal`.

Отдельно стоит `EStateTreeStateSelectionRules` — набор флагов, регулирующих тонкости пересоздания состояний при переходе. По умолчанию включены два:

- `CompletedTransitionStatesCreateNewStates` — завершённые состояния при переходе пересоздаются заново, иначе они могли бы остаться активными и никогда больше не завершиться;

- `CompletedStateBeforeTransitionSourceFailsTransition` — переход не срабатывает, если раньше по цепочке есть завершённое состояние; без этого правила переход по завершению мог бы занимать лишний тик.

Есть ещё `ReselectedStateCreatesNewStates` (по умолчанию выключено) и `None` — режим совместимости с правилами UE 5.6.

3.7. Отложенные переходы

Переход можно задержать. За это отвечает `FStateTreeRandomTimeDuration` в описании перехода и `FStateTreeTransitionDelayedState` в рантайме.

Для нас это ключевой момент, потому что в базовом `StateTree` отложенный переход реализуется просто продолжением тика каждый кадр — а в `Mass`, где мы тикаем по сигналу, это не годится. Отсюда переопределение:

cpp

```
virtual void BeginDelayedTransition(const FStateTreeTransitionDelayedState&
DelayedState) override;
```

в `FMassStateTreeExecutionContext` и сигнал

`UE::Mass::Signals::DelayedTransitionWakeup`. Механика: вместо ожидания тика `Mass` ставит отложенный сигнал ровно на нужное время и засыпает.

3.8. События

События — способ прервать поведение извне. Событие — это `FStateTreeEvent` с тегом (`FGameplayTag`), опциональной полезной нагрузкой (`FInstancedStruct`) и источником. Отправляется через контекст:

cpp

```
Context.SendEvent(EventTag, Payload, Origin);
```

Событие попадает в очередь в `instance data` и обрабатывается на ближайшем тике: переходы с триггером `OnEvent` сверяют свой тег с очередью.

Описание ожидаемых событий в скомпилированном ассете хранит `FCompactEventDesc` — это позволяет дереву заранее знать, какие события ему интересны, и не будить исполнение зря.

3.9. Разделение ассета и данных экземпляра

Это самая важная архитектурная идея StateTree, и именно она делает возможной работу в Mass.

Ассет (UStateTree) после компиляции — это набор плоских массивов:

FCompactStateTreeState , FCompactStateTransition , FCompactStateTreeFrame , FCompactStateTreeParameters , плюс массив узлов. Всё это read-only и разделяется всеми исполнителями.

Instance data (FStateTreeInstanceData) — всё изменяемое: стек активных состояний, экземпляры данных активных задач, очередь событий, отложенные переходы, запросы переходов.

Разные категории данных различает EStateTreeDataSourceType . Перечисление длинное, но логика в нём чёткая:

Группа	Значения	Что это
Глобальные	GlobalInstanceData , GlobalInstanceDataObject	Данные глобальных задач и эвалюаторов
Активного состояния	ActiveInstanceData , ActiveInstanceDataObject	Данные задач активных состояний
Общие	SharedInstanceData , SharedInstanceDataObject	Для условий, considerations и привязок функций
Временные	EvaluationScopeInstanceData , ...Object	Создаются и уничтожаются немедленно при вычислении
Рантайм-узла	ExecutionRuntimeData , ...Object , ...Any	Данные узла на время исполнения
Контекст	ContextData	Контекстные объекты и параметры дерева
Внешние	ExternalData	То, что узлы запрашивают у контекста
Параметры	GlobalParameterData , ExternalGlobalParameterData , SubtreeParameterData , StateParameterData	Параметры разных уровней
События	TransitionEvent , StateEvent	Событие, участвующее в переходе или выборе

Адресация внутри всего этого — `FStateTreeDataHandle` : компактная структура, хранящая тип источника плюс индекс. Именно хендлы, а не указатели, потому что данные могут переезжать в памяти.

Суффикс `Object` во многих значениях означает, что `instance data` узла — это не структура, а `UObject` (так работают Blueprint-узлы).

3.10. Внешние данные

Узел не может просто взять и обратиться к миру. Он должен объявить, что ему нужно, — на этапе линковки:

cpp

```
virtual bool Link(FStateTreeLinker& Linker) override
{
    Linker.LinkExternalData(MassSignalSubsystemHandle);
    Linker.LinkExternalData(LookAtHandle);
    return true;
}
```

Хендл имеет тип `TStateTreeExternalDataHandle<T, Requirement>`, где требование — это

cpp

```
enum class EStateTreeExternalDataRequirement : uint8
{
    Required,    // дерево нельзя исполнить без этих данных
    Optional,    // данные необязательны
};
```

В `FMassLookAtTask` видно оба варианта:

cpp

```
TStateTreeExternalDataHandle<UMassSignalSubsystem> MassSignalSubsystemHandle;
TStateTreeExternalDataHandle<FMassLookAtFragment,
    EStateTreeExternalDataRequirement::Optional> LookAtHandle;
```

Подсистема сигналов обязательна — без неё задача не сможет сообщить о завершении. Фрагмент `FMassLookAtFragment` опционален: агент может физически не иметь головы, и задача должна это пережить.

В рантайме доступ идёт через контекст:

cpp

```
UMassSignalSubsystem& SignalSubsystem =  
Context.GetExternalData(MassSignalSubsystemHandle);
```

Разрешает ли схема данный тип внешних данных вообще — решает `IsExternalItemAllowed()`. Для Mass это переопределено в `UMassStateTreeSchema` так, чтобы пропускать фрагменты и подсистемы Mass.

3.11. Привязки свойств

`EStateTreePropertyUsage` размечает свойства instance data узла:

cpp

```
enum class EStateTreePropertyUsage : uint8  
{  
    Invalid,  
    Context,      // контекстный объект  
    Input,        // значение приходит извне через привязку  
    Parameter,    // настраивается в редакторе  
    Output,       // значение отдаётся наружу  
};
```

В редакторе это выражается категорией `UPROPERTY`. Вернитесь к `FMassLookAtTaskInstanceData`:

cpp

```
UPROPERTY(EditAnywhere, Category = Input, meta = (Optional))  
FMassEntityHandle TargetEntity;  
  
UPROPERTY(EditAnywhere, Category = Parameter)  
float Duration = 0.f;  
  
UPROPERTY()  
float Time = 0.f;
```

`TargetEntity` — вход: дизайнер привяжет его к выходу эвалюатора «ближайшая цель». `Duration` — параметр, настраивается вручную. `Time` — без категории и без `EditAnywhere`: это чисто рабочее состояние, накопитель времени, который редактору показывать незачем, но который обязан жить в instance data, потому что у каждого агента он свой.

Отдельный механизм — `FStateTreePropertyRef` : ссылка на чужое свойство, а не копия. Он используется в `FMassZoneGraphPathFollowTaskInstanceData` :

cpp

```
UPROPERTY(EditAnywhere, Category = Input, meta=(RefType =  
"/Script/MassAIBehavior.MassZoneGraphTargetLocation"))  
FStateTreePropertyRef TargetLocation;
```

Задача не копирует к себе целевую точку — она берёт ссылку на структуру, которую заполнил другой узел. Для больших структур это заметная экономия.

3.12. Схема

`UStateTreeSchema` — фильтр, определяющий, что вообще может быть в дереве. Все его методы виртуальные и по умолчанию либо запрещают, либо разрешают:

cpp

```
virtual bool IsStructAllowed(const UScriptStruct* InScriptStruct) const {  
    return false; }  
virtual bool IsClassAllowed(const UClass* InScriptStruct) const { return  
    false; }  
virtual bool IsExternalItemAllowed(const UStruct& InStruct) const { return  
    false; }  
virtual bool IsScheduledTickAllowed() const { return false; }  
virtual bool IsStateSelectionAllowed(EStateTreeStateSelectionBehavior  
    InBehavior) const { return true; }  
virtual bool IsStateTypeAllowed(EStateTreeStateType InStateType) const {  
    return true; }  
virtual TConstArrayView<FStateTreeExternalDataDesc> GetContextDataDescs()  
    const { return {}; }  
virtual EStateTreeParameterDataType GetGlobalParameterDataType() const;  
virtual EStateTreeStateSelectionRules GetStateSelectionRules() const;  
virtual bool Link(FStateTreeLinker& Linker) { return true; }
```

Обратите внимание на дефолты: три первых метода возвращают `false` . Схема по умолчанию **не разрешает ничего** — это правильный подход к безопасности, каждая схема обязана явно перечислить, что ей подходит.

Плюс блок `#if WITH_EDITOR` с разрешениями для редактора: `AllowEnterConditions()` , `AllowUtilityConsiderations()` , `AllowEvaluators()` , `AllowMultipleTasks()` , `AllowGlobalParameters()` , `AllowTasksCompletion()` , `AllowQueuedCompilation()` . Последний как раз переопределён в `UMassStateTreeSchema` :

cpp

```
virtual bool AllowQueuedCompilation() const override
{
    // Если поставить в очередь, компиляция может произойти в рабочем потоке
    Mass
    // через CompileIfNeededSynchronously.
    return false;
}
```

Это чистой воды защита от гонки: Mass может дёрнуть компиляцию из своего потока, и отложенная компиляция там небезопасна.

Есть ещё удобный статический хелпер:

cpp

```
static bool IsChildOfBlueprintBase(const UClass* InClass);
```

— чтобы одной строкой пропустить все Blueprint-наследуемые узлы в `IsClassAllowed()`.

3.13. Контекст исполнения — предварительно

`FStateTreeExecutionContext` — объект, который связывает воедино владельца, ассет, instance data и предоставляет всё это узлам. Он существует только на время операции и создаётся заново при каждом обращении к дереву.

Иерархия из `StateTreeExecutionContext.h` трёхступенчатая:

```
FStateTreeReadOnlyExecutionContext
└─ FStateTreeMinimalExecutionContext
   └─ FStateTreeExecutionContext
```

Только-читающий вариант нужен отладчику и внешним наблюдателям; минимальный умеет посылать события, но не исполнять; полный умеет всё. Разбирать все методы будем в главе 11 — там их несколько десятков.

Плюс есть `FStateTreeWeakExecutionContext` и асинхронный вариант — для случаев, когда результат приходит из другого потока или через колбэк.

3.14. Что критично для связки с Mass

Соберём то, что понадобится дальше:

1. **Узлы `const` и без состояния.** Один экземпляр задачи обслуживает всю толпу — идеально ложится на ECS.
 2. **Instance data отделена от ассета.** Значит, её можно хранить где угодно, в том числе в пуле подсистемы.
 3. **Внешние данные объявляются на этапе линковки.** Значит, из ассета можно заранее вычислить, к каким фрагментам Mass он обратится, — на этом строится вся система зависимостей.
 4. **Схема запрещает всё по умолчанию.** Значит, `UMassStateTreeSchema` явно определяет границу того, что допустимо в Mass-поведении.
 5. **Отложенные переходы — переопределяемая точка.** Значит, их можно перевести с покадрового ожидания на сигналы.
 6. **События — очередь в instance data.** Значит, внешний мир может влиять на поведение, не имея ссылки на объект-агента.
-

Что дальше

Глава 4 — короткая, но важная: мы сопоставим два подхода к запуску `StateTree`, разберём, что именно делает `UStateTreeComponent` и почему каждый его шаг ломается в Mass, и получим точный перечень задач, которые решает модуль `MassAIBehavior`. Это будет мостик к части II, где начнётся строчный разбор исходников.

Глава 4. Точка стыковки

Эта глава — мостик. Мы сопоставим два способа исполнять `StateTree`, увидим, где именно ломается привычный подход, и получим точный список задач, которые решает `MassAIBehavior`. Дальше начнётся строчный разбор исходников, и без этой карты он превратится в перечисление незнакомых имён.

4.1. Как это работает на акторе

Классический путь — `UStateTreeComponent` на `AActor`. Его жизненный цикл:

`BeginPlay()` — компонент берёт ассет из своего `UPROPERTY`, создаёт `FStateTreeExecutionContext`, передаёт ему себя как владельца и собственное поле `InstanceData`, вызывает `Start()`.

`TickComponent()` — каждый кадр движок дёргает компонент, тот собирает контекст заново и вызывает `Tick(DeltaTime)`.

Доступ к данным — схема `UStateTreeComponentSchema` объявляет контекстные данные: `AActor` (владелец) и `UStateTreeComponent`. Узлы получают их через `GetExternalData()` и дальше работают привычными средствами: `GetOwner()`, `FindComponentByClass()`, вызовы методов.

`EndPlay()` — `Stop()`, задачам приходит `ExitState()`, `InstanceData` уничтожается вместе с компонентом.

Всё просто ровно потому, что есть `UObject`, к которому можно всё привязать.

4.2. Шесть мест, где это ломается

Нет владельца. Конструктор `FStateTreeExecutionContext` требует `UObject& InOwner`. У сущности `Mass` владельца нет. Что подставить?

Некуда положить instance data. `FStateTreeInstanceData` — это несколько `TArray`, `FInstancedStructContainer`, очередь событий. Класть такое во фрагмент нельзя: фрагменты копируются при смене архетипа, пакуются в чанки фиксированного размера и должны быть дешёвыми. Куда её деть?

Нет покадрового тика. Никто не тикает сущности по одной. И даже если бы тикал — тикать десять тысяч деревьев каждый кадр слишком дорого. Кто и когда вызывает `Tick()`?

Требования неизвестны заранее. Процессор обязан объявить, к каким фрагментам он обращается, в `ConfigureQueries()`. Но обращается-то не процессор, а задачи внутри ассета, который процессор увидит только в рантайме. Откуда взять список требований?

Отложенные переходы предполагают тик. Механизм «подожди 2 секунды и перейди» в базовом `StateTree` реализован ожиданием в тике. Если мы не тикаем — переход не сработает никогда.

Нет очевидной точки уничтожения. У компонента есть `EndPlay()`. У сущности `Mass` — только момент, когда её фрагменты исчезают. Кто вызовет `Stop()`, чтобы задачи корректно отработали `ExitState()` и освободили ресурсы?

4.3. Ответы `MassAIBehavior`

Проблема	Решение	Где смотреть
Нет владельца	Владельцем становится <code>UMassStateTreeSubsystem</code> ; идентичность агента передаётся отдельно, полем <code>FMassEntityHandle Entity</code> в контексте,	<code>MassStateTreeExecutionCor</code> гл. 12

Проблема	Решение	Где смотреть
	плюс <code>FMassExecutionExtension</code> для описания инстанса	
Некуда положить instance data	Пул в подсистеме: <code>TArray<FMassStateTreeInstanceDataItem></code> ; на сущности — только <code>FMassStateTreeInstanceHandle</code> (индекс + поколение)	<code>MassStateTreeSubsystem.h</code> <code>MassStateTreeFragments.h</code>
Нет покадрового тика	Исполнение по сигналам: <code>UMassStateTreeProcessor</code> наследует <code>UMassSignalProcessorBase</code> и просыпается только для сигнализированных сущностей	<code>MassStateTreeProcessors.h</code> <code>MassSignalSubsystem.h</code> , гл.
Требования неизвестны заранее	Каждый Mass-узел реализует <code>GetDependencies()</code> ; схема собирает их при линковке; подсистема создаёт динамический процессор под каждый уникальный набор	<code>MassStateTreeTypes.h</code> , <code>MassStateTreeSchema.h</code> , гл.
Отложенные переходы	Переопределён <code>BeginDelayedTransition()</code> : вместо ожидания ставится отложенный сигнал <code>DelayedTransitionWakeup</code>	<code>MassStateTreeExecutionCor</code> гл. 12
Нет точки уничтожения	Наблюдатель <code>UMassStateTreeFragmentDestructor</code> на удаление фрагмента: останавливает дерево и возвращает данные в пул	<code>MassStateTreeProcessors.h</code>

Обратите внимание на симметрию: почти каждое решение — это либо **вынос данных наружу**, либо **замена неявного механизма явным**. Там, где на акторе всё держалось на «у меня есть указатель на владельца» и «меня тикает движок», в Mass приходится проговаривать каждую связь.

4.4. Полная цепочка от спавна до поведения

Соберём всё в один сквозной сценарий. Пока — на уровне «кто кого зовёт», детали в частях III и IV.

Шаг 1. Конфигурация. Дизайнер создаёт `UMassEntityConfigAsset` , добавляет туда трейт поведения и выбирает в нём ассет `UStateTree` со схемой `Mass Behavior`.

Шаг 2. Построение шаблона. Трейт в `BuildTemplate()` добавляет в шаблон `FMassStateTreeInstanceFragment` и `FMassStateTreeSharedFragment` со ссылкой на ассет. Формируется архетип.

Шаг 3. Регистрация процессора. При первом обращении к ассету

`UMassStateTreeSubsystem::CreateProcessorForStateTree()` вытаскивает из схемы список `FMassStateTreeDependency`, считает по нему хеш, ищет в `RequirementsHashToProcessor` подходящий процессор и либо переиспользует существующий, либо создаёт новый динамический экземпляр `UMassStateTreeProcessor`.

Шаг 4. Спавн. Сущность создаётся. Наблюдатель на добавление

`FMassStateTreeInstanceFragment` вызывает `AllocateInstanceData()` и записывает хендл во фрагмент.

Шаг 5. Активация. `UMassStateTreeActivationProcessor` находит сущности без тега

`FMassStateTreeActivatedTag`, ставит тег через командный буфер и посылает сигнал `StateTreeActivate`.

Шаг 6. Первый тик. Процессор просыпается по сигналу. Для каждой сущности он

собирает `FMassStateTreeExecutionContext`, подставляет instance data из пула, вызывает `SetEntity()` и `Start()`. Дерево выбирает первое состояние, задачам приходит `EnterState()`.

Шаг 7. Работа. Задача вроде `FMassZoneGraphPathFollowTask` в `EnterState()` заполняет

`FMassMoveTargetFragment` — и дальше её работу делают **обычные процессоры движения**. Дерево при этом может спать: его разбудят, только когда что-то произойдёт.

Шаг 8. Пробуждение. Есть три источника инициативы: сигнал извне (кто-то заметил игрока), отложенный сигнал `DelayedTransitionWakeup`, или сигнал завершения от процессора-исполнителя (`LookAtFinished`, `StandTaskFinished`, `AnimateTaskFinished`). В любом случае процессор снова тикает дерево для конкретной сущности.

Шаг 9. Смерть. Сущность уничтожается. `UMassStateTreeFragmentDestructor` перехватывает удаление фрагмента, вызывает `Stop()` (чтобы прошли все `ExitState()`), затем `FreeInstanceData()` — индекс возвращается во фрилист, поколение инкрементируется.

4.5. Ключевая мысль: дерево решает, но не делает

Это, пожалуй, главное, что надо усвоить перед частью II.

На акторе задача `StateTree` обычно **сама выполняет работу**: вызывает `MoveTo`, играет анимацию, крутит меш. В `Mass` задача почти никогда не выполняет работу. Она **записывает намерение в фрагмент** и выходит из `EnterState()` со статусом `Running`.

Посмотрите на набор внешних данных `FMassZoneGraphPathFollowTask`:

cpp

```
TStateTreeExternalDataHandle<FMassZoneGraphLaneLocationFragment>  
LocationHandle;  
TStateTreeExternalDataHandle<FMassMoveTargetFragment> MoveTargetHandle;  
TStateTreeExternalDataHandle<FMassZoneGraphPathRequestFragment>  
PathRequestHandle;  
TStateTreeExternalDataHandle<FMassZoneGraphShortPathFragment> ShortPathHandle;  
TStateTreeExternalDataHandle<FMassZoneGraphCachedLaneFragment>  
CachedLaneHandle;  
TStateTreeExternalDataHandle<FAgentRadiusFragment> AgentRadiusHandle;  
TStateTreeExternalDataHandle<FMassMovementParameters> MovementParamsHandle;  
TStateTreeExternalDataHandle<UZoneGraphSubsystem> ZoneGraphSubsystemHandle;
```

Это всё — данные, которые задача заполняет или читает, чтобы **другие процессоры** потом двигали агента. Сама задача агента не двигает ни на сантиметр.

Из этого следует практическое правило проектирования: если вы пишете Mass-задачу и в ней появляется цикл, тяжёлые вычисления или обращение к сцене — почти наверняка вы пишете её неправильно. Задача должна быть тонким слоем «поставить намерение / проверить, выполнено ли». Тяжёлая работа — в процессорах, где её можно векторизовать и распараллелить.

4.6. Границы: чего связка не делает

Полезно сразу знать, что **не** входит в `MassAIBehavior`, чтобы не искать несуществующее:

Не исполняется на клиенте. `UE::MassStateTree::ExecutionFlags` — только `Standalone` | `Server`. Поведение считается авторитетно на сервере.

Нет привязки к актору. Если агенту нужен настоящий актор (`UMassActorSubsystem`, LOD-представление), задача может его получить, но это всегда обходной путь через подсистему, а не естественная часть модели.

Нет автоматического пробуждения. Дерево, которое не подписалось на сигнал и не поставило отложенный, никогда не тикнет снова. Это самая частая ошибка новичков, и мы будем к ней возвращаться.

Нет универсальных Blueprint-узлов. Схема Mass фильтрует то, что можно вставить в дерево: узел обязан быть наследником Mass-базы, чтобы уметь объявить свои зависимости. Обычные BP-задачи с доступом к актору сюда не подходят.

Нет пошагового отладчика по одному агенту в привычном виде. Отладка идёт через `Gameplay Debugger` и `FMassExecutionExtension::GetInstanceDescription()` — метод, который существует ровно для того, чтобы в логах и отладчике вместо «инстанс №4718» было понятное описание сущности.

4.7. Инварианты, которые стоит запомнить

Шесть утверждений, которые будут верны на протяжении всей книги:

1. Узел `StateTree` не имеет изменяемого состояния — только `const`-методы и `instance data`.
 2. `Instance data` агента живёт в подсистеме, а не на сущности; на сущности только хендл.
 3. Ассет дерева разделяется всеми агентами через константный общий фрагмент.
 4. Всё, к чему узел обращается извне, объявлено дважды: через `Link()` для `StateTree` и через `GetDependencies()` для `Mass`.
 5. Тик происходит только по сигналу; отсутствие сигнала — это не «пауза», а «навсегда».
 6. Задача выражает намерение, работу делают процессоры.
-

Что дальше

Часть I закончена — общая картина собрана. С главы 5 начинается построчный разбор.

Первым пойдёт `MassStateTreeTypes.h`: четыре базовые структуры узлов и зачем каждой из них `GetDependencies()`, полный список сигналов `UE::Mass::Signals` с разбором, кто их шлёт и кто на них реагирует, флаги исполнения, и подробный разбор

`FMassStateTreeInstanceHandle` — включая то, зачем нужно поколение и как эта схема защищает от классических багов с переиспользованием индексов.

Глава 5. `MassStateTreeTypes.h`

Файл небольшой — 133 строки, — но это фундамент всего модуля. В нём четыре смысловых блока: флаги исполнения, каталог сигналов, базовые структуры узлов и хендл `instance data`. Разберём каждый.

5.1. Заголовки и что из них следует

cpp

```
#include "MassProcessingTypes.h"
#include "StateTreeConditionBase.h"
#include "StateTreeEvaluatorBase.h"
#include "StateTreePropertyFunctionBase.h"
#include "StateTreeTaskBase.h"
#if UE_ENABLE_INCLUDE_ORDER_DEPRECATED_IN_5_6
```

```
#include "CoreMinimal.h"
#endif
```

Обратите внимание на то, чего здесь **нет**: `MassEntityTypes.h`, `MassEntityManager.h`, ничего про сущности вообще. Файл сознательно лёгкий — он подключается практически в каждый Mass-узел, и таскать за собой ядро ECS было бы дорого по времени компиляции.

Блок `UE_ENABLE_INCLUDE_ORDER_DEPRECATED_IN_5_6` вокруг `CoreMinimal.h` — это часть большой кампании Epic по чистке инклюдов (IWYU, Include What You Use). Раньше `CoreMinimal.h` тянули везде подряд; теперь его оставили только под макросом совместимости, чтобы старый код продолжал собираться. В новом коде на него полагаться нельзя.

Дальше — две форвард-декларации:

cpp

```
enum class EProcessorExecutionFlags : uint8;
namespace UE::MassBehavior
{
    struct FStateTreeDependencyBuilder;
}
```

Первая — трюк, доступный только для `enum class` с явно указанным базовым типом: компилятор знает размер, значит полное определение не нужно. Вторая — билдер зависимостей, определённый в `MassStateTreeDependency.h`.

Здесь стоит сделать пометку на будущее: пространств имён у модуля **два**.

`UE::MassStateTree` — для флагов, `UE::MassBehavior` — для системы зависимостей. Исторически модуль называется `MassAIBehavior`, отсюда второе имя. Логика в этом разделении немного, но помнить полезно: когда будете писать свои узлы, `FStateTreeDependencyBuilder` живёт именно в `UE::MassBehavior`.

5.2. Флаги исполнения

cpp

```
namespace UE::MassStateTree
{
    /** Flags to indicates to the MassStateTree processors and traits in which
        contexts
        they can execute (e.g., Standalone, Server, etc.) */
    static constexpr EProcessorExecutionFlags ExecutionFlags(
```

```
EProcessorExecutionFlags::Standalone | EProcessorExecutionFlags::Server));
```

(Двойная точка с запятой в конце — опечатка в исходниках Epic. Компилятор её игнорирует.)

Смысл: **вся логика StateTree в Mass выполняется только в одиночной игре и на сервере**. Клиент дерево не тикает.

Это не оптимизация, а сетевая модель. Поведение — авторитетная симуляция; клиент получает результат через репликацию (для важных агентов) или вообще не получает, а просто отображает толпу по LOD-данным. Если бы клиент тикал деревья самостоятельно, поведение немедленно разошлось бы с сервером: у StateTree есть случайный выбор состояний, есть RandomSeed, есть зависимость от порядка событий.

Практическое следствие: **не пытайтесь отлаживать Mass-поведение в PIE-клиенте**. Ставьте Standalone или подключайтесь к серверной стороне. Это одна из самых частых причин «у меня ничего не работает, дерево не тикает».

Константа применяется в конструкторах процессоров модуля:

cpp

```
UMassStateTreeProcessor::UMassStateTreeProcessor(...)
{
    ExecutionFlags = static_cast<int32>(UE::MassStateTree::ExecutionFlags);
    // ...
}
```

Технически static constexpr на уровне пространства имён в заголовке даёт внутреннюю компоновку — своя копия в каждой единице трансляции. Для константы это безобидно.

5.3. Каталог сигналов

cpp

```
namespace UE::Mass::Signals
{
    const FName StateTreeActivate = FName(TEXT("StateTreeActivate"));
    const FName LookAtFinished = FName(TEXT("LookAtFinished"));
    const FName NewStateTreeTaskRequired =
FName(TEXT("NewStateTreeTaskRequired"));
    const FName StandTaskFinished = FName(TEXT("StandTaskFinished"));
}
```



```

    const FName AnimateTaskFinished = FName(TEXT("AnimateTaskFinished"));
    const FName DelayedTransitionWakeup =
FName(TEXT("DelayedTransitionWakeup"));
    // @todo MassStateTree: move this to its game plugin when possible
    const FName ContextualAnimTaskFinished =
FName(TEXT("ContextualAnimTaskFinished"));
}

```

Это **единственный способ разбудить дерево**. Разберём каждый сигнал по схеме «кто шлёт → кто ловит → что происходит».

StateTreeActivate

Отправитель — `UMassStateTreeActivationProcessor`, один раз на агента. Получатель — `UMassStateTreeProcessor`. Действие — первый запуск: вызывается `FMassStateTreeExecutionContext::Start()`, дерево выбирает начальное состояние, задачам приходит `EnterState()`.

Почему это отдельный сигнал, а не работа в наблюдателе на создание фрагмента? Потому что на момент создания сущности её данные могут быть ещё не полностью проинициализированы другими трейтами. Отложенная активация через отдельный процессор гарантирует, что все фрагменты уже на месте и заполнены.

NewStateTreeTaskRequired

Отправитель — сама логика `StateTree`, когда задача завершилась и нужно выбрать новое состояние. Получатель — тот же процессор. Действие — очередной `Tick()`.

Этот сигнал — механизм «самопродолжения». Когда `Tick()` задачи возвращает `Succeeded`, дереву нужно ещё раз проснуться, чтобы обработать переход. Без сигнала оно бы просто зависло в завершённом состоянии.

DelayedTransitionWakeup

Отправитель — `FMassStateTreeExecutionContext::BeginDelayedTransition()`, через `DelaySignalEntity()`. Получатель — процессор. Действие — тик ровно в момент, когда истекла задержка перехода.

Это тот самый перевод отложенных переходов с покадрового ожидания на событийную модель. Агент, ждущий три секунды, эти три секунды не стоит **ничего**: он не в запросе, не в чанке, его никто не трогает. В отличие от ВТ-агента, который все три секунды тикает таймер.

`LookAtFinished`, `StandTaskFinished`, `AnimateTaskFinished`, `ContextualAnimTaskFinished`

Это сигналы **завершения работы**, и они отлично иллюстрируют мысль из главы 4 про «дерево решает, но не делает».

Схема одна и та же для всех четырёх:

1. Задача в `EnterState()` записывает намерение во фрагмент (например, `FMassLookAtFragment`).
2. Задача возвращает `Running` и... всё. Дерево засыпает.
3. Специализированный процессор (`UMassLookAtProcessor` и его аналоги) выполняет работу — крутит голову, проигрывает анимацию.
4. Закончив, процессор шлёт сигнал сущности.
5. Дерево просыпается, задача понимает, что работа сделана, и возвращает `Succeeded`.

Комментарий `@todo MassStateTree: move this to its game plugin when possible` у `ContextualAnimTaskFinished` — честное признание Epic: этот сигнал относится к системе контекстных анимаций, которая логически не часть базового Mass AI, но выковырять её пока не дошли руки. Такие пометки в движке стоит читать: они показывают, какие границы модулей ещё будут двигаться.

Почему `FName`, а не `enum` или `FGameplayTag`

`FName` в Unreal — это индекс в глобальной таблице строк. Сравнение двух `FName` — сравнение двух чисел, то есть практически бесплатно. При этом, в отличие от `enum`, `FName` **расширяем**: ваш игровой модуль может завести собственные сигналы, не трогая код движка.

От `FGameplayTag` отличается тем, что не требует регистрации в реестре тегов и не поддерживает иерархию — здесь она и не нужна.

Технический нюанс: `const FName` в области видимости пространства имён в заголовке имеет внутреннюю компоновку, то есть в каждой единице трансляции создаётся свой объект. Но поскольку `FName` — это по сути индекс, все копии равны между собой, и сравнение работает корректно.

Как объявить свой сигнал

cpp

```
// MyGameSignals.h
namespace MyGame::Signals
{
    const FName CombatTargetLost = FName(TEXT("CombatTargetLost"));
}
```

Никакой регистрации не нужно. Достаточно, чтобы отправитель и получатель использовали одно и то же имя. Единственное требование — процессор, который должен реагировать, обязан подписаться на сигнал в `InitializeInternal()` через `SubscribeToSignal()`. Подробно — в главе 14.

5.4. Четыре базовые структуры узлов

Дальше идут четыре почти идентичные структуры. Приведём одну целиком:

cpp

```
/**
 * Base struct for all Mass StateTree Tasks.
 */
USTRUCT(meta = (Hidden, DisplayName = "Mass Task Base"))
struct FMassStateTreeTaskBase : public FStateTreeTaskBase
{
    GENERATED_BODY()

    /**
     * Appends this task's Mass dependencies to the given Builder.
     * This is done once for every task instance, when the state tree asset is
     loaded or compiled.
     */
    virtual void
    GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
    {
    }
};
```

Остальные три — `FMassStateTreeEvaluatorBase`, `FMassStateTreeConditionBase`, `FMassStateTreePropertyFunctionBase` — устроены абсолютно так же, наследуясь от соответствующих баз `StateTree`.

Разбор метаданных

`meta = (Hidden, ...)` — структура не показывается в списке узлов, которые можно добавить в дерево. Она абстрактная по смыслу, хотя технически не абстрактная по C++.

`DisplayName = "Mass Task Base"` — имя в редакторе для случаев, когда структура всё же где-то отображается (в подсказках типов, в фильтрах).

Зачем эти структуры вообще нужны

Здесь два независимых основания, и оба важны.

Первое: место для `GetDependencies()`. Модуль `StateTreeModule` ничего не знает про `Mass` и не может завести такой метод в `FStateTreeTaskBase`. Значит, нужен промежуточный слой в `MassAIBehavior`.

Второе: маркер типа для схемы. Вспомните дефолт

`UStateTreeSchema::IsStructAllowed()` — он возвращает `false`. Схема `Mass` реализует его так, чтобы пропускать наследников этих четырёх баз (плюс общие узлы вроде `FStateTreeTaskCommonBase`). Наследование от `FMassStateTreeTaskBase` — это буквально пропуск в дерево `Mass`-поведения. Попробуете вставить обычную задачу — редактор её не покажет.

Почему пустая реализация, а не `= 0`

`GetDependencies()` не чисто виртуальный, а с пустым телом. Причин две.

Практическая: узел может не иметь `Mass`-зависимостей вообще. Задача «подожди N секунд» не трогает ни одного фрагмента — заставлять её писать пустой оверрайд было бы шумом.

Техническая: `USTRUCT` не может быть абстрактным в смысле `UObject`-системы. Рефлексия `Unreal` создаёт экземпляры структур по умолчанию (для CDO-подобных операций, для сериализации), а чисто виртуальный метод это запретил бы.

Почему `const` и почему передача по неконстантной ссылке

Метод `const`, потому что все узлы `StateTree` неизменяемы (см. главу 3). Билдер передаётся по неконстантной ссылке, потому что узел в него **пишет** — это классический паттерн «выходной параметр-аккумулятор»:

c++

```
void
FMassLookAtTask::GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder
& Builder) const
{
    Builder.AddReadWrite<FMassLookAtFragment>();
    Builder.AddReadWrite<UMassSignalSubsystem>();
}
```

Когда вызывается

Комментарий в исходниках отвечает точно: «*This is done once for every task instance, when the state tree asset is loaded or compiled*» — один раз на экземпляр узла, при загрузке или компиляции ассета.

Это не рантайм. К моменту, когда сущности начнут спавниться, список зависимостей уже посчитан, хеш вычислен, процессор создан. Стоимость нулевая в горячем пути.

Осторожно: два независимых объявления

Здесь кроется самая коварная ошибка при написании своих узлов. Внешние данные объявляются **дважды**, в двух разных местах, и компилятор не проверяет согласованность:

cpp

```
// 1. Для StateTree – чтобы получить доступ через Context.GetExternalData()
virtual bool Link(FStateTreeLinker& Linker) override
{
    Linker.LinkExternalData(LookAtHandle);
    return true;
}

// 2. Для Mass – чтобы планировщик знал про доступ к фрагменту
virtual void GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder&
Builder) const override
{
    Builder.AddReadWrite<FMassLookAtFragment>();
}
```

Забыли первое — узел упадёт при линковке или получит невалидный хендл. Забыли второе — узел будет **работать**, но процессор не объявит доступ к фрагменту, и планировщик может запустить его параллельно с другим процессором, пишущим в тот же фрагмент. Получите гонку данных, которая проявляется раз в тысячу кадров и только под нагрузкой.

Держите эти два метода рядом в коде и правьте всегда вместе. К этому правилу мы вернёмся в главе 19.

5.5. FMassStateTreeInstanceHandle

Последний блок файла:

cpp

```
/**
 * A handle pointing to a StateTree instance data in UMassStateTreeSubsystem.
 */
struct FMassStateTreeInstanceHandle
{
```

```

    FMassStateTreeInstanceHandle() = default;

    /** Initializes new handle based on an index */
    static FMassStateTreeInstanceHandle Make(const int32 InIndex, const int32
InGeneration)
    {
        return FMassStateTreeInstanceHandle(InIndex, InGeneration);
    }

    /** @returns index the handle points to */
    int32 GetIndex() const { return Index; }

    /** @returns generation of the handle, used to identify recycled indices.
*/
    int32 GetGeneration() const { return Generation; }

    /** @returns true if the handle is valid. */
    bool IsValid() const { return Index != INDEX_NONE; }

protected:
    FMassStateTreeInstanceHandle(const int32 InIndex, const int32
InGeneration)
        : Index(InIndex), Generation(InGeneration) {}

    int32 Index = INDEX_NONE;
    int32 Generation = 0;
};

```

Восемь байт. Именно они лежат на каждой сущности вместо тяжёлой
FStateTreeInstanceData .

Зачем поколение

Классическая проблема пулов: индекс освобождается и переиспользуется. Сценарий без поколения:

1. Агенту А выдан индекс 42.
2. Агент А умирает, индекс 42 возвращается во фрилист.
3. Спавнится агент В, ему выдаётся тот же индекс 42.
4. Где-то остался старый хендл на А с индексом 42 — и он молча начинает читать данные В.

Такие баги отвратительны: данные валидные, крешей нет, поведение просто «странное». Воспроизводимость околонулевая.

Поклоение решает это. `UMassStateTreeSubsystem` хранит

cpp

```
USTRUCT()
struct FMassStateTreeInstanceDataItem
{
    GENERATED_BODY()

    UPROPERTY()
    FStateTreeInstanceData InstanceData;

    UPROPERTY()
    int32 Generation = 0;
};
```

и проверяет обе половины:

cpp

```
bool IsValidHandle(const FMassStateTreeInstanceHandle Handle) const
{
    UE_MT_SCOPED_READ_ACCESS(InstanceDataMTDetector);
    return InstanceDataArray.IsValidIndex(Handle.GetIndex())
        && InstanceDataArray[Handle.GetIndex()].Generation ==
        Handle.GetGeneration();
}
```

При освобождении слота поколение инкрементируется. Старый хендл теперь имеет поколение 3, а слот — 4, проверка не проходит, `GetInstanceData()` вернёт `nullptr`.

Это ровно та же идея, что в `FMassEntityHandle` с его `SerialNumber`, и вообще стандартный приём для пулов в игровых движках.

Разбор дизайна класса

Конструктор по умолчанию публичный, параметризованный — `protected`. Публично создать хендл «из головы» нельзя. Единственный путь — статический `Make()`, и вызывать его должна подсистема, которая знает актуальное поколение. Это не защита от злого умысла, а защита от опечатки: собрать хендл руками — почти всегда ошибка.

Почему `protected`, а не `private`? Чтобы наследники могли переиспользовать конструктор. В движке такой практики пока нет, но задел оставлен.

`IsValid()` **проверяет только индекс**. Метод отвечает на вопрос «хендл вообще был кому-то выдан?», а не «указывает ли он на живые данные». Полную проверку делает `IsValidHandle()` подсистемы, потому что только она знает актуальные поколения.

Отсюда практика: `IsValid()` — дешёвая предварительная проверка, `IsValidHandle()` — настоящая. В горячем коде можно отсечь заведомо пустые хендлы первой, не трогая массив подсистемы.

Это не `USTRUCT`. Ни макроса, ни `GENERATED_BODY()`. Следствия:

- хендл не участвует в рефлексии и не может быть `UPROPERTY`;
- он не сериализуется автоматически;
- он не виден в редакторе и в Blueprint.

Именно поэтому в `FMassStateTreeInstanceFragment` поле объявлено голым:

cpp

```
USTRUCT()
struct FMassStateTreeInstanceFragment : public FMassFragment
{
    GENERATED_BODY()

    /** Handle to a StateTree instance data in MassStateTreeSubsystem. */
    FMassStateTreeInstanceHandle InstanceHandle;    // без UPROPERTY

    double LastUpdateTimeInSeconds = 0.;
};
```

Это осознанно. Хендл — рантайм-значение, привязанное к состоянию конкретной подсистемы в конкретном мире. Сохранять его в ассет или реплицировать бессмысленно: на другой стороне он будет указывать в никуда.

Нет `operator==`, **нет** `GetTypeHash()`. Хендл нельзя положить в `TMap` как ключ и нельзя сравнить напрямую. По коду модуля этого и не требуется — сравнение всегда идёт через `IsValidHandle()`. Если понадобится в вашем коде, придётся сравнивать поля вручную:

cpp

```
const bool bSame = A.GetIndex() == B.GetIndex()
                  && A.GetGeneration() == B.GetGeneration();
```

Размер и упаковка. Два `int32` — восемь байт, без выравнивающих дыр. Вместе с `double LastUpdateTimeInSeconds` фрагмент занимает 16 байт. Это очень дёшево: в чанке

помещается много таких записей подряд, обход по ним идеально ложится на кэш.

5.6. Что даёт этот файл в сумме

Подведём итог по четырём блокам:

Блок	Роль
<code>UE::MassStateTree::ExecutionFlags</code>	Сетевая политика: поведение считается на сервере и в Standalone
<code>UE::Mass::Signals</code>	Единственный способ разбудить дерево; расширяемый каталог имён
Четыре базы узлов	Пропуск в дерево Mass + точка объявления зависимостей
<code>FMassStateTreeInstanceHandle</code>	8-байтовая замена тяжёлой instance data на сущности

И три вещи, которые стоит унести из главы в практику:

1. **Не отлаживайте Mass-поведение в PIE-клиенте** — оно там не исполняется по определению.
2. `Link()` и `GetDependencies()` **правятся только вместе** — их рассогласование даёт гонку, а не ошибку компиляции.
3. **Дерево не проснётся само** — если ни один сигнал не запланирован, агент замер навсегда.

Что дальше

Глава 6 — `MassStateTreeFragments.h`. Файл ещё короче, всего 33 строки, но за ними стоит важное проектное решение: почему instance data вынесена в подсистему, почему ассет — константный общий фрагмент, зачем нужно поле `LastUpdateTimeInSeconds` и как из него считается дельта времени при событийном тике, когда «прошлый кадр» может быть десять секунд назад.

Глава 6. `MassStateTreeFragments.h`

Тридцать три строки, две структуры. Но за этой краткостью — два ключевых проектных решения, и оба стоит разобрать подробно, потому что по их образцу вы будете

проектировать собственные данные.

6.1. Файл целиком

cpp

```
#pragma once

#include "MassEntityTypes.h"
#include "MassStateTreeSubsystem.h"
#include "MassStateTreeFragments.generated.h"

USTRUCT()
struct FMassStateTreeInstanceFragment : public FMassFragment
{
    GENERATED_BODY()

    FMassStateTreeInstanceFragment() = default;

    /** Handle to a StateTree instance data in MassStateTreeSubsystem. */
    FMassStateTreeInstanceHandle InstanceHandle;

    /** The last update time use to calculate ticking delta time. */
    double LastUpdateTimeInSeconds = 0.;
};

USTRUCT()
struct FMassStateTreeSharedFragment : public FMassConstSharedFragment
{
    GENERATED_BODY()

    FMassStateTreeSharedFragment() = default;

    UPROPERTY()
    TObjectPtr<UStateTree> StateTree = nullptr;
};
```

Сразу отметим асимметрию, которая объясняет весь дизайн: **персональные данные** агента — это два безымянных для рефлексии поля, а **общие** данные — это `UPROPERTY` с `TObjectPtr`. Дальше станет ясно, почему именно так.

6.2. Инклюд, который многое говорит

cpp

```
#include "MassStateTreeSubsystem.h"
```

Файл фрагментов тянет за собой заголовок подсистемы. Это выглядит избыточно — но `FMassStateTreeInstanceHandle` определён в `MassStateTreeTypes.h`, который подключается транзитивно через подсистему.

Смысловая нагрузка тут важнее технической: **фрагмент бессмыслен без подсистемы**. Он не самодостаточен, он — ссылка. Инклюд это честно отражает.

6.3. `FMassStateTreeInstanceFragment` : СКОЛЬКО МЫ СЭКОНОМИЛИ

Посчитаем. `FStateTreeInstanceData` содержит (упрощённо):

- `FStateTreeInstanceStorage` — а внутри него `FInstancedStructContainer` с данными всех активных задач;
- массив активных состояний `FStateTreeActiveStates`;
- очередь событий (`TArray<FStateTreeSharedEvent>`);
- отложенные переходы (`TArray<FStateTreeTransitionDelayedState>`);
- запросы переходов;
- массив `TObjectPtr` на instance-объекты Blueprint-узлов;
- служебное состояние исполнения (`FStateTreeExecutionState`).

Даже пустая, эта структура — заметно больше сотни байт, а с данными десятка активных задач легко переваливает за килобайт. И размер **непостоянен**: он зависит от того, какие состояния активны прямо сейчас.

Теперь представим, что мы положили её во фрагмент. Что сломается:

Чанки станут огромными. Чанк — это блок фиксированного размера. Если один элемент занимает килобайт, в чанк поместится десяток агентов вместо сотен. Все преимущества плотной упаковки испаряются.

Смена архетипа станет дорогой. Добавление любого тега агенту (`FMassStateTreeActivatedTag`, например) копирует все его фрагменты в другой чанк. Копировать килобайт вместо шестнадцати байт — на десяти тысячах агентов это заметно.

Обход разрушит кэш. Процессор, который просто хочет проверить `LastUpdateTimeInSeconds`, будет тащить в кэш килобайт данных на каждого агента. Полезной информации — восемь байт из тысячи.

Размер перестанет быть фиксированным. Это фатально: ECS предполагает, что элемент массива имеет постоянный размер, известный на этапе создания архетипа. Динамические контейнеры внутри фрагмента формально возможны (TArray — это указатель плюс два int), но данные окажутся в куче, размазанные по памяти. Ровно то, от чего ECS уходит.

Наш фрагмент вместо этого занимает **16 байт**: восемь на хендл, восемь на время. Никаких аллокаций, никаких указателей на кучу, идеальное выравнивание.

Цена — одна косвенная адресация при обращении к настоящим данным:

cpp

```
FStateTreeInstanceData* InstanceData =  
StateTreeSubsystem.GetInstanceData(Fragment.InstanceHandle);
```

Но эта косвенность оплачивается только в момент, когда дерево реально тикает — а тикает оно, напомним, редко и только для сигнализированных агентов.

Это общий паттерн, который стоит запомнить: **если данные большие, переменного размера или нужны редко — во фрагменте держите хендл, а данные храните в подсистеме.**

6.4. LastUpdateTimeInSeconds : дельта при событийном тике

cpp

```
/** The last update time use to calculate ticking delta time. */  
double LastUpdateTimeInSeconds = 0.;
```

Поле маленькое, но без него событийная модель не работает.

Проблема

В обычном тике дельта времени приходит сама: Tick(DeltaTime) — это время с прошлого кадра. В Mass с сигналами такого нет. Агент мог не тикать десять секунд, пока ждал DelayedTransitionWakeup. Если передать ему Context.GetDeltaTimeSeconds() (дельту **кадра**, скажем 16 мс), задача, накапливающая время, посчитает неверно.

Вернитесь к FMassLookAtTaskInstanceData :

cpp

```
UPROPERTY(EditAnywhere, Category = Parameter)
float Duration = 0.f;

UPROPERTY()
float Time = 0.f;
```

Задача в `Tick()` делает примерно `InstanceData.Time += DeltaTime` и сравнивает с `Duration`. Если `DeltaTime` будет кадровым, а тики — редкими, таймер поедет в разы.

Решение

Процессор перед тиком считает реальную дельту сам:

cpp

```
const double CurrentTime = World->GetTimeSeconds();
const float DeltaTime = static_cast<float>(CurrentTime -
StateTreeInstanceFragment.LastUpdateTimeInSeconds);
StateTreeInstanceFragment.LastUpdateTimeInSeconds = CurrentTime;

// ... дальше DeltaTime уходит в StateTreeContext.Tick(DeltaTime)
```

Дельта — это время с прошлого **тика дерева**, а не с прошлого кадра. Задача получает честные десять секунд и корректно завершается.

Почему double, а не float

Это не педантизм. `float` даёт около семи значащих десятичных цифр. Игровое время в секундах: через час игры это 3600, через десять часов — 36000. При таких значениях шаг `float` становится порядка 0.004 секунды, а дальше — хуже. Разность двух близких больших чисел в `float` теряет точность катастрофически: вы можете получить 0 там, где реально прошло 3 мс.

`double` даёт около пятнадцати значащих цифр — точности хватит на годы непрерывной работы сервера. Для выделенных серверов, живущих неделями, это не теоретическая проблема.

Обратите внимание: хранится `double`, а в задачу передаётся `float`. Правильный компромисс — точность там, где накапливается ошибка (абсолютное время), экономия там, где значение малó (дельта).

Инициализация

Значение по умолчанию — `0.`, что означает «дерево ещё не тикало». Первый тик при наивном расчёте дал бы дельту, равную всему времени с начала игры. Поэтому активация — отдельный этап: при `Start()` поле проставляется текущим временем, и первый настоящий `Tick()` получает корректную дельту.

6.5. `FMassStateTreeSharedFragment` : одно дерево на всех

cpp

```
USTRUCT()
struct FMassStateTreeSharedFragment : public FMassConstSharedFragment
{
    GENERATED_BODY()

    FMassStateTreeSharedFragment() = default;

    UPROPERTY()
    TObjectPtr<UStateTree> StateTree = nullptr;
};
```

Почему общий, а не обычный

Указатель на ассет одинаков у всех агентов данной конфигурации. Хранить его персонально — это восемь байт на агента впустую: десять тысяч агентов, 80 килобайт, при том что полезной информации — один указатель.

Но экономия памяти тут вторична. Главное — **семантика группировки**. Общий фрагмент участвует в определении архетипа: агенты с разными деревьями оказываются в разных архетипах автоматически. Процессору не нужно проверять в цикле «а какое у этого агента дерево» — он получает чанк, в котором дерево гарантированно одно, и достаёт его один раз на весь чанк:

cpp

```
const FMassStateTreeSharedFragment& SharedFragment =
Context.GetConstSharedFragment<FMassStateTreeSharedFragment>();
const UStateTree* StateTree = SharedFragment.StateTree;
// дальше цикл по 128 агентам – StateTree уже в регистре
```

Почему константный

`FMassConstSharedFragment`, а не `FMassSharedFragment`. Разница существенная.

Изменяемый общий фрагмент — это разделяемая изменяемая память. Планировщик Mass обязан учитывать её при построении графа: два процессора, пишущие в один общий фрагмент, нельзя запускать параллельно. Это ограничивает параллелизм.

Константный общий фрагмент таких ограничений не создаёт: читать одновременно можно откуда угодно. А поскольку менять указатель на дерево в рантайме никто и не собирается — выбор очевиден.

Дополнительный бонус: Mass **дедуплицирует** константные общие фрагменты по хешу содержимого. Если сто разных конфигураций агентов используют один и тот же ассет дерева, в памяти будет один экземпляр фрагмента, а не сто.

Создаётся он через менеджер сущностей:

cpp

```
const FConstSharedStruct SharedFragment =  
    EntityManager.GetOrCreateConstSharedFragment<FMassStateTreeSharedFragment>  
(StateTreeAsset);
```

Менеджер считает хеш, ищет существующий экземпляр, при совпадении возвращает его.

Почему UPROPERTY и TSharedPtr

Здесь ровно противоположная ситуация по сравнению с InstanceHandle , и контраст поучителен.

UStateTree — это UObject . Значит, на него распространяется сборка мусора. Если хранить сырой UStateTree* без UPROPERTY , GC не увидит ссылку и вправе уничтожить ассет, оставив висячий указатель. UPROPERTY регистрирует ссылку в графе достижимости.

TSharedPtr<T> вместо T* — современный стандарт UE5. Внешне ведёт себя как указатель, но добавляет поддержку отложенной загрузки и проверки доступа в редакторных сборках. В шипящей сборке компилируется в обычный указатель без накладных расходов.

A FMassStateTreeInstanceHandle — не UObject и не USTRUCT , это чистое значение, ничего удерживать не нужно, поэтому UPROPERTY там не только не требуется, но и невозможен.

6.6. Как фрагменты попадают на сущность

Полностью — в главе 20, но общая схема нужна уже сейчас.

Трейт (`UMassEntityTraitBase`) в методе построения шаблона делает примерно следующее:

cpp

```
void UMassStateTreeTrait::BuildTemplate(FMassEntityTemplateBuildContext&
BuildContext,
                                     const UWorld& World) const
{
    BuildContext.AddFragment<FMassStateTreeInstanceFragment>();

    const FConstSharedStruct StateTreeFragment =

    EntityManager.GetOrCreateConstSharedFragment<FMassStateTreeSharedFragment>
(StateTree);
    BuildContext.AddConstSharedFragment(StateTreeFragment);
}
```

Ассет `UStateTree` — это `UPROPERTY` самого трейта, который дизайнер выбирает в редакторе. Обычно там стоит фильтр по схеме:

cpp

```
UPROPERTY(EditAnywhere, Category = "", meta = (RequiredAssetDataTags =
"Schema=/Script/MassAIBehavior.MassStateTreeSchema"))
TObjectPtr<UStateTree> StateTree;
```

Это тот самый механизм из комментария в `StateTreeSchema.h` : каждый ассет `StateTree` записывает имя своей схемы в теги данных ассета, и по ним можно фильтровать выпадающий список. Дизайнер физически не сможет выбрать дерево не той схемы.

Обратите внимание: тег `FMassStateTreeActivatedTag` трейт **не** добавляет. Его ставит процессор активации в рантайме — потому что это состояние, а не конфигурация.

6.7. Что происходит, если хендл невалиден

Ситуация реальная: фрагмент есть, а `InstanceHandle.IsValid()` возвращает `false` . Такое бывает между созданием сущности и срабатыванием наблюдателя, который выделит данные.

Правильная реакция процессора — **пропустить агента без всякого шума**:

cpp


```

if (!StateTreeInstanceFragment.InstanceHandle.IsValid())
{
    continue;
}

FStateTreeInstanceData* InstanceData =
Subsystem.GetInstanceData(StateTreeInstanceFragment.InstanceHandle);
if (InstanceData == nullptr)
{
    continue;
}

```

Две проверки, а не одна: первая дешёвая (просто сравнение с `INDEX_NONE`), вторая полная (сверка поколения внутри подсистемы). Смысл разделения был описан в главе 5.

Заметьте: **никаких** `check()` или `ensure()`. В Mass состав сущности меняется через командный буфер, то есть асинхронно относительно исполнения. Гонки в один кадр — нормальное явление, а не признак ошибки. Ассерт тут превратился бы в ложные срабатывания на ровном месте.

6.8. Расширение: как добавить свои данные

Допустим, вашему поведению нужен персональный «уровень тревоги» — число от 0 до 1, которое читают условия дерева и пишет отдельный процессор восприятия.

Правильно — отдельный фрагмент:

cpp

```

USTRUCT()
struct FMyAlertLevelFragment : public FMassFragment
{
    GENERATED_BODY()

    float Alert = 0.f;
};

```

Трейт добавляет его рядом с `FMassStateTreeInstanceFragment`. Условие в дереве объявляет его как внешние данные и в `GetDependencies()` пишет `Builder.AddReadOnly<FMyAlertLevelFragment>()`.

Неправильно — расширять `FMassStateTreeInstanceFragment`. Даже если бы вы могли (а вы не можете — это код движка), это было бы плохо: платили бы за поле все агенты, включая тех, кому тревога не нужна.

Философия ECS: **данные добавляются композицией, а не наследованием**. Много маленьких фрагментов лучше одного большого, потому что архетип формируется из тех, что реально нужны, а запросы точнее фильтруют.

Если данные общие для конфигурации (например, «скорость нарастания тревоги» — параметр типа агента, а не персональное значение), делайте константный общий фрагмент по образцу `FMassStateTreeSharedFragment`.

6.9. Типичные ошибки

Класть в фрагмент то, что должно быть в `instance data` задачи. Если значение нужно только одной задаче и только пока она активна (накопитель времени, кэш промежуточного результата), его место — в `FInstanceDataType` этой задачи. Фрагмент — для данных, которыми обмениваются с процессорами Mass.

Использовать `Context.GetDeltaTimeSeconds()` вместо расчёта по `LastUpdateTimeInSeconds`. Внутри задачи вы получаете дельту через параметр `Tick(Context, DeltaTime)` — она уже посчитана правильно. Лезть за кадровой дельтой в Mass-контекст напрямую — ошибка, которая проявится только на редко тикающих агентах.

Изменяемый общий фрагмент там, где хватило бы константного. Ограничивает параллелизм, отключает дедупликацию, ничего не даёт взамен. Изменяемый нужен только когда группа агентов действительно разделяет меняющееся состояние.

Забыть, что общий фрагмент влияет на архетип. Два агента с разными деревьями — это два архетипа. Если у вас пятьдесят разных ассетов поведения, у вас минимум пятьдесят архетипов, и чанки станут разреженными. Иногда лучше одно дерево с ветвлением, чем пятьдесят похожих.

Хранить `UObject*` без `UPROPERTY`. GC уничтожит объект, указатель повиснет. В фрагментах Mass это особенно неприятно, потому что креш случится глубоко внутри обхода чанков.

6.10. Итог главы

Элемент	Решение	Почему
Данные экземпляра дерева	Хендл во фрагменте, данные в подсистеме	Фрагмент 16 байт вместо килобайта, фиксированный размер
Ассет дерева	Константный общий фрагмент	Дедупликация, группировка по архетипам, ноль ограничений на параллелизм

Элемент	Решение	Почему
Время последнего тика	<code>double</code> во фрагменте	Событийная модель требует считать дельту вручную; <code>float</code> теряет точность на длинных сессиях
Тег активации	Не в трейте, а в рантайме	Это состояние, а не конфигурация

Что дальше

Глава 7 — схемы. Разберём `StateTreeSchema.h` целиком (все четырнадцать виртуальных методов, включая редакторный блок) и затем `MassStateTreeSchema.h`: как `IsStructAllowed()` пропускает Mass-узлы и общие узлы, как `IsExternalItemAllowed()` открывает доступ к фрагментам и подсистемам, что происходит в `Link()`, откуда берётся массив `Dependencies` и почему `AllowQueuedCompilation()` возвращает `false`. Это подготовит нас к главе 8, где мы разберём сбор зависимостей до последней детали.

Глава 7. Схемы: `StateTreeSchema.h` и `MassStateTreeSchema.h`

Схема — это привратник дерева. Она решает, какие узлы можно вставить, к каким данным разрешено обращаться и какие возможности редактора доступны. В обычном `StateTree` её роль скромная; в `Mass` она становится центральной, потому что именно схема собирает список Mass-зависимостей, от которого зависит вся дальнейшая машинерия процессоров.

Разберём сначала базу целиком, потом Mass-специализацию.

7.1. Зачем схема вообще нужна

Представьте, что схемы нет. Дизайнер открывает ассет `StateTree` и видит список всех задач, которые есть в проекте: задачи для NPC на акторах, задачи для Mass-агентов, задачи для UI, задачи из чужого плагина. Он собирает дерево, компилирует — всё зелёное. В рантайме дерево падает, потому что Mass-задача пытается получить `AActor*`, которого нет.

Схема превращает эту рантайм-ошибку в **невозможность выбора в редакторе**. Неподходящий узел просто не появится в списке.

Второе применение — фильтрация ассетов. Комментарий в заголовке объясняет механизм:

cpp

```
/**
 * Schema describing which inputs, evaluators, and tasks a StateTree can
 * contain.
 * Each StateTree asset saves the schema class name in asset data tags, which
 * can be
 * used to limit which StatTree assets can be selected per use case, i.e.:
 *
 * UPROPERTY(EditDefaultsOnly, Category = AI, meta=
 * (RequiredAssetDataTags="Schema=StateTreeSchema_SupaDupa"))
 * UStateTree* StateTree;
 */
```

Каждый ассет записывает имя своего класса схемы в теги данных ассета. Дальше любое `UPROPERTY` может потребовать конкретную схему, и в выпадающем списке останутся только подходящие деревья. Ровно это делает трейт `Mass`, о котором шла речь в главе 6.

Имена тегов заданы в `StateTreeTypes.h`:

cpp

```
namespace UE::StateTree
{
    inline const FName SchemaTag(TEXT("Schema"));
    inline const FName SchemaCanBeOverridenTag(TEXT("SchemaCanBeOverriden"));
}
```

Второй тег отмечает деревья, чью схему допустимо переопределить снаружи. Это нужно для `LinkedAsset`-состояний: поддереву, написанное под общую схему, может быть подключено в дерево с более узкой схемой.

7.2. Объявление класса

cpp

```
UCLASS(MinimalAPI, Abstract)
class UStateTreeSchema : public UObject
```

`Abstract` — экземпляры базового класса создавать нельзя, только наследников. Логично: базовая схема запрещает всё и потому бесполезна.

`MinimalAPI` — экспортируется минимум символов (в основном сгенерированные рефлексией). Остальное экспортируется точно через `UE_API`, который здесь раскрывается в `STATETREEMODULE_API`. Это стандартная практика UE5 для ускорения линковки: чем меньше экспортированных символов, тем быстрее собирается проект.

Схема — это `UObject`, а не структура. Значит, у неё есть CDO, её можно инстанцировать по классу, она участвует в рефлексии. Ассет `StateTree` хранит экземпляр схемы как вложенный объект (`EditInlineNew` у наследников это подчёркивает).

7.3. Методы рантайма: полный разбор

`IsStructAllowed`

cpp

```
/** @return True if specified struct is supported */
virtual bool IsStructAllowed(const UScriptStruct* InScriptStruct) const
{
    return false;
}
```

Главный фильтр. Вызывается для каждой структуры-кандидата: задач, условий, эвалюаторов, property functions. Возврат `false` по умолчанию — принцип «запрещено всё, что не разрешено явно».

Типичная реализация — цепочка проверок наследования:

cpp

```
bool UMyMSchema::IsStructAllowed(const UScriptStruct* InScriptStruct) const
{
    return InScriptStruct->IsChildOf(FMyTaskBase::StaticStruct())
        || InScriptStruct-
>IsChildOf(FStateTreeConditionCommonBase::StaticStruct());
}
```

Обратите внимание на параметр: `const UScriptStruct*`, то есть работа идёт с типом, а не с экземпляром. Метод отвечает на вопрос «допустим ли такой вид узла в принципе», а не «подходит ли этот конкретный настроенный узел».

`IsClassAllowed`

cpp

```
/** @return True if specified class is supported */
virtual bool IsClassAllowed(const UClass* InScriptStruct) const
{
    return false;
}
```

(Имя параметра `InScriptStruct` — опечатка Epic, тип-то `UClass*`.)

Аналог предыдущего, но для узлов, реализованных как `UObject` — то есть для Blueprint-узлов. Именно здесь пригождается статический хелпер:

cpp

```
/**
 * Helper function to check if a class is any of the Blueprint extendable item
 * classes (Eval, Task, Condition).
 * Can be used to quickly accept all of those classes in IsClassAllowed().
 */
UE_API static bool IsChildOfBlueprintBase(const UClass* InClass);
```

Одна строка `return IsChildOfBlueprintBase(InClass);` — и схема пропускает все Blueprint-узлы.

Для Mass это, как правило, **не** делается. Blueprint-узел не может объявить `GetDependencies()`, значит планировщик не узнает о его доступах к данным. Плюс исполнение Blueprint-графа для десяти тысяч агентов — это ровно та цена, от которой Mass уходит.

IsExternalItemAllowed

cpp

```
/** @return True if specified struct/class is supported as external data */
virtual bool IsExternalItemAllowed(const UStruct& InStruct) const
{
    return false;
}
```

Проверяет не сам узел, а то, что узел **запрашивает**. Каждый `TStateTreeExternalDataHandle<T>` из главы 3 в конечном счёте проходит через этот метод.

Параметр — `const UStruct&` (не указатель, не `UScriptStruct`). `UStruct` — общий предок и `UScriptStruct`, и `UClass`, поэтому одна сигнатура покрывает и структуры-фрагменты, и классы-подсистемы.

Для Mass это критическая точка: именно здесь открывается доступ к фрагментам и подсистемам.

`IsScheduledTickAllowed`

cpp

```
/** @return True if the execution context can sleep or the next tick delayed.
 */
virtual bool IsScheduledTickAllowed() const
{
    return false;
}
```

Относительно новая возможность `StateTree`: дерево может само сказать «мне ничего не нужно ближайшие N секунд, не тикайте меня». Владелец (компонент) вправе это использовать, чтобы снизить частоту тика.

Для Mass это выключено, и по интересной причине: **у Mass своя, более сильная система сна**. Дерево тут не «тикает реже» — оно не тикает вообще, пока не придёт сигнал. Включать поверх этого механику запланированного тика значило бы иметь две конкурирующие системы управления пробуждением.

Косвенно этот флаг влияет и на задачи: у `FStateTreeTaskBase` есть флаг

cpp

```
uint8 bConsideredForScheduling : 1;
```

с комментарием, что задача учитывается при расчёте запланированного тика и что на сам процесс тика это не влияет.

`IsStateSelectionAllowed`

cpp

```
/** @return True if the state selection behavior is supported. */
virtual bool IsStateSelectionAllowed(EStateTreeStateSelectionBehavior
InBehavior) const
{
```

```
    return true;
}
```

Дефолт `true` — разрешены все поведения выбора из главы 3. Схема может запретить, например, `utility`-варианты, если считает их слишком дорогими.

IsStateTypeAllowed

cpp

```
/** @return True if the state type is supported. */
virtual bool IsStateTypeAllowed(EStateTreeStateType InStateType) const
{
    return true;
}
```

То же для типов состояний (`State`, `Group`, `Linked`, `LinkedAsset`, `Subtree`). Схема может запретить `LinkedAsset`, если не готова обрабатывать деревья-ссылки.

GetContextDataDescs

cpp

```
/** @return List of context objects (UObjects or UScriptStructs) enforced by
the schema.
    They must be provided at runtime through the execution context. */
virtual TConstArrayView<FStateTreeExternalDataDesc> GetContextDataDescs()
const
{
    return {};
}
```

Контекстные данные — это то, что схема **гарантирует** всем узлам. Для схемы компонента это `AActor` и `UStateTreeComponent`: любая задача может на них рассчитывать без объявления.

Ключевое слово в комментарии — *enforced*. Это не «доступно, если есть», а «обязано быть предоставлено при запуске». Если владелец не подставит эти данные, дерево не запустится.

GetGlobalParameterDataType

cpp


```
/** @return the global parameter type used by the schema. */
UE_API virtual EStateTreeParameterDataType GetGlobalParameterDataType() const;
```

Возвращает одно из двух значений (из главы 3): `GlobalParameterData` — параметры хранятся в instance data дерева, или `ExternalGlobalParameterData` — приходят снаружи. Второй вариант позволяет владельцу подменять параметры, не трогая instance data.

Реализация в `.cpp`, дефолт — обычные глобальные параметры.

GetStateSelectionRules

cpp

```
/** @return the selection rules used by the schema. */
UE_API virtual EStateTreeStateSelectionRules GetStateSelectionRules() const;
```

Те самые флаги из главы 3, регулирующие пересоздание состояний при переходах. Схема может вернуть `None` для совместимости с поведением UE 5.6 — это спасение для проектов, где обновление движка сломало отлаженные деревья.

Link

cpp

```
/** Resolves schema references to other StateTree data. */
virtual bool Link(FStateTreeLinker& Linker)
{
    return true;
}
```

Вызывается при линковке дерева — там же, где `Link()` каждого узла. Схема получает возможность разрешить собственные ссылки. Возврат `false` означает провал линковки, дерево окажется нерабочим.

Для Mass это **самый важный метод во всём файле**, и мы вернёмся к нему через несколько разделов.

7.4. Редакторный блок

cpp

```
#if WITH_EDITOR
    virtual bool AllowEnterConditions() const { return true; }
```

```

virtual bool AllowUtilityConsiderations() const { return true; }
virtual bool AllowEvaluators() const { return true; }
virtual bool AllowMultipleTasks() const { return true; }
virtual bool AllowGlobalParameters() const { return true; }
virtual bool AllowTasksCompletion() const { return true; }
virtual bool AllowQueuedCompilation() const { return true; }
#endif // WITH_EDITOR

```

Все семь возвращают `true` по умолчанию — здесь принцип обратный: разрешено всё, схема сужает по необходимости.

Это чисто редакторные ограничения. В шипящей сборке методов нет вообще, потому что решения уже приняты на этапе компиляции ассета.

Метод	Что запрещает при <code>false</code>
<code>AllowEnterConditions</code>	Условия входа в состояния
<code>AllowUtilityConsiderations</code>	Узлы полезности (и, соответственно, utility-выбор)
<code>AllowEvaluators</code>	Эвалюаторы — только задачи и условия
<code>AllowMultipleTasks</code>	Больше одной задачи в состоянии
<code>AllowGlobalParameters</code>	Глобальные параметры дерева
<code>AllowTasksCompletion</code>	Настройку того, какие задачи влияют на завершение состояния; вместо этого — «любая»
<code>AllowQueuedCompilation</code>	Отложенную компиляцию ассета

`AllowTasksCompletion` связан с двумя полями `FStateTreeTaskBase`:

cpp

```

#if WITH_EDITORONLY_DATA
    /** True if the task is considered for completion.
        False if the task runs in the background without affecting the state
        completion. */
    UPROPERTY()
    uint8 bConsideredForCompletion : 1;

    /** True if the user can edit bConsideredForCompletion in the editor. */
    UPROPERTY()
    uint8 bCanEditConsideredForCompletion : 1;
#endif

```

То есть «фоновая» задача, которая работает, но не завершает состояние своим `Succeeded`. Если схема запрещает настройку, используется правило «любая завершившаяся задача завершает состояние».

7.5. `UMassStateTreeSchema` : объявление

Переходим к Mass-специализации.

c++

```
#define UE_API MASSAIBEHAVIOR_API

/**
 * StateTree for Mass behaviors.
 */
UCLASS(MinimalAPI, BlueprintType, EditInlineNew, CollapseCategories,
        meta = (DisplayName = "Mass Behavior", CommonSchema))
class UMassStateTreeSchema : public UStateTreeSchema
{
    GENERATED_BODY()
}
```

Разберём спецификаторы:

`BlueprintType` — тип доступен в Blueprint. Не для написания узлов, а чтобы можно было, например, проверить схему ассета из BP-кода.

`EditInlineNew` — экземпляр создаётся встроенно, внутри владеющего ассета, а не как отдельный объект. Схема — часть ассета дерева.

`CollapseCategories` — косметика: свойства показываются плоским списком, без группировки по категориям. Здесь это не важно, свойство одно и оно `Transient`.

`DisplayName = "Mass Behavior"` — то, что дизайнер видит при создании нового ассета `StateTree`. Именно эту строку он выбирает в диалоге.

`CommonSchema` — метка «общая схема». Она относится к механизму `SchemaCanBeOverridden` из раздела 7.1: дерево с общей схемой может быть подключено как `LinkedAsset` в дерево с другой схемой. Практический смысл — библиотека переиспользуемых поддеревьев.

7.6. Публичный интерфейс

c++

```

public:
    /** Fetches a read-only view of the Mass-relevant requirements of the
    associated StateTee */
    const TArray<FMassStateTreeDependency>& GetDependencies() const;

```

и реализация в конце файла:

cpp

```

//-----
-
// INLINE
//-----
-
inline const TArray<FMassStateTreeDependency>&
UMassStateTreeSchema::GetDependencies() const
{
    return Dependencies;
}

```

Единственный публичный метод, добавленный к базе. Это **точка выхода** всей системы зависимостей: подсистема спрашивает у схемы «что нужно этому дереву» и по ответу строит процессор.

Возврат по константной ссылке — ноль копирований. `inline` в заголовке — вызов раскрывается на месте, без обращения к другой единице трансляции.

Обратите внимание на асимметрию доступа: **публичный геттер**, **protected поле**. Заполнять массив может только сам класс (в `Link()`), читать — кто угодно. Правильная инкапсуляция для данных, вычисляемых один раз.

7.7. AllowQueuedCompilation

cpp

```

#if WITH_EDITOR
    virtual bool AllowQueuedCompilation() const override
    {
        // If queued, they might be compiled in a mass worker thread via
        CompileIfNeededSynchronously.
        return false;
    }
#endif

```

Единственный редакторный оверрайд, и комментарий объясняет причину точно.

Отложенная компиляция — оптимизация редактора: вместо того чтобы пересобирать дерево немедленно после каждой правки, изменения копятся и компилируются пачкой. Хорошо для интерактивности.

Проблема в том, что в Mass дерево может понадобиться **из рабочего потока**. Процессор, обрабатывающий чанк, обнаруживает, что ассет не скомпилирован, и вызывает `CompileIfNeededSynchronously()`. Компиляция `StateTree` — тяжёлая операция, трогающая `UObject`-граф, работающая с рефлексией, потенциально создающая объекты. Делать это не в игровом потоке нельзя.

Возврат `false` гарантирует, что к моменту, когда Mass дотянется до ассета, тот уже скомпилирован в игровом потоке.

Это хороший пример того, как ограничения одной системы протекают в настройки другой. И хороший повод запомнить: **если ваша схема используется из Mass, отключайте отложенную компиляцию**.

7.8. Защищённые методы

cpp

```
protected:
    UE_API virtual bool IsStructAllowed(const UScriptStruct* InScriptStruct)
const override;
    UE_API virtual bool IsExternalItemAllowed(const UStruct& InStruct) const
override;
    UE_API virtual bool Link(FStateTreeLinker& Linker) override;
```

Три оверрайда. Их реализации — в `MassStateTreeSchema.cpp`, которого в нашем пакете нет, поэтому дальше я разбираю **контракт и логику**, восстанавливая структуру реализации по остальному коду модуля. Конкретные строки могут отличаться, смысл — нет.

`IsStructAllowed`

Задача метода — пропустить два класса узлов.

Mass-специфичные — наследники четырёх баз из главы 5: `FMassStateTreeTaskBase`, `FMassStateTreeConditionBase`, `FMassStateTreeEvaluatorBase`, `FMassStateTreePropertyFunctionBase`. Они умеют объявлять зависимости.

Общеприменимые — наследники `...CommonBase : FStateTreeTaskCommonBase , FStateTreeConditionCommonBase` и аналоги. Помните комментарий из `StateTreeTaskBase.h` ?

cpp

```
/**
 * Base class (namespace) for all common Tasks that are generally applicable.
 * This allows schemas to safely include all conditions child of this struct.
 */
USTRUCT(meta = (Hidden))
struct FStateTreeTaskCommonBase : public FStateTreeTaskBase
{
    GENERATED_BODY()
};
```

Такой же есть у эвалюаторов:

cpp

```
/**
 * Base class (namespace) for all common Evaluators that are generally
 applicable.
 */
USTRUCT(Meta=(Hidden))
struct FStateTreeEvaluatorCommonBase : public FStateTreeEvaluatorBase
```

Смысл: узлы, не трогающие внешний мир, безопасны в любой схеме. Сравнение двух чисел, задержка, случайный выбор — им не нужны ни актор, ни фрагмент. Разрешать их можно смело.

Логика метода примерно такая:

cpp

```
// реконструкция
bool UMassStateTreeSchema::IsStructAllowed(const UScriptStruct*
InScriptStruct) const
{
    return InScriptStruct->IsChildOf(FMassStateTreeTaskBase::StaticStruct())
        || InScriptStruct-
>IsChildOf(FMassStateTreeConditionBase::StaticStruct())
        || InScriptStruct-
>IsChildOf(FMassStateTreeEvaluatorBase::StaticStruct())
```

```

    || InScriptStruct-
>IsChildOf(FMassStateTreePropertyFunctionBase::StaticStruct())
    || InScriptStruct->IsChildOf(FStateTreeTaskCommonBase::StaticStruct())
    || InScriptStruct-
>IsChildOf(FStateTreeConditionCommonBase::StaticStruct())
    || InScriptStruct-
>IsChildOf(FStateTreeEvaluatorCommonBase::StaticStruct())
    || InScriptStruct-
>IsChildOf(FStateTreePropertyFunctionCommonBase::StaticStruct());
}

```

Обратите внимание, чего в списке **нет**: `IsClassAllowed` не переопределён вовсе, то есть остаётся базовый `return false`. Blueprint-узлы в Mass-деревья не допускаются — по причинам, названным в разделе 7.3.

IsExternalItemAllowed

Здесь открывается доступ к данным Mass. Пропускать нужно три категории:

Фрагменты — наследники `FMassFragment`. Это то, ради чего всё затевалось:

`FMassLookAtFragment`, `FMassMoveTargetFragment` и десятки других.

Общие фрагменты — наследники `FMassSharedFragment` и `FMassConstSharedFragment`.

Подсистемы — наследники `USubsystem` (на практике `UWorldSubsystem` и `UMassSubsystemBase`). Сюда попадают `UMassSignalSubsystem`, `UZoneGraphSubsystem`, `UMassStateTreeSubsystem`.

cpp

```

// реконструкция
bool UMassStateTreeSchema::IsExternalItemAllowed(const UStruct& InStruct)
const
{
    return InStruct.IsChildOf(FMassFragment::StaticStruct())
        || InStruct.IsChildOf(FMassSharedFragment::StaticStruct())
        || InStruct.IsChildOf(FMassConstSharedFragment::StaticStruct())
        || InStruct.IsChildOf(USubsystem::StaticClass());
}

```

Единая сигнатура с `UStruct&` здесь окупается: `IsChildOf` работает и для структур, и для классов.

Именно этот метод делает легальной строку из `FMassLookAtTask`:

cpp

```
TStateTreeExternalDataHandle<FMassLookAtFragment,  
EStateTreeExternalDataRequirement::Optional> LookAtHandle;
```

Если бы `FMassLookAtFragment` не проходил проверку, линковка провалилась бы.

Link — сердце главы

cpp

```
UE_API virtual bool Link(FStateTreeLinker& Linker) override;
```

Комментарий у поля объясняет назначение:

cpp

```
/**  
 * The Mass-relevant requirements of the associated StateTree, collected during  
 StateTree's linking  
 * @see UStateTree::Link  
 * @see UMassStateTreeSchema::Link  
 */  
UPROPERTY(Transient)  
TArray<FMassStateTreeDependency> Dependencies;
```

Что происходит по шагам:

1. `UStateTree::Link()` начинает линковку ассета.
2. Он проходит по всем узлам дерева и вызывает `Link()` каждого — узлы регистрируют свои внешние данные.
3. Он вызывает `Link()` схемы.
4. Схема **ещё раз** обходит все узлы, приводит каждый к соответствующей Mass-базе и вызывает `GetDependencies(Builder)`.
5. Билдер накапливает требования, схлопывая дубликаты и объединяя доступы (если один узел просит фрагмент на чтение, а другой на запись — итог `ReadWrite`).
6. Результат сохраняется в `Dependencies`.

Логика примерно такая:

cpp


```

// реконструкция
bool UMassStateTreeSchema::Link(FStateTreeLinker& Linker)
{
    Dependencies.Reset();

    UE::MassBehavior::FStateTreeDependencyBuilder Builder(Dependencies);

    const UStateTree* StateTree = Cast<UStateTree>(GetOuter());
    for (const FStateTreeNode& Node : StateTree->GetNodes())
    {
        if (const FMassStateTreeTaskBase* Task = Node.GetPtr<const
FMassStateTreeTaskBase>())
        {
            Task->GetDependencies(Builder);
        }
        else if (const FMassStateTreeEvaluatorBase* Eval = Node.GetPtr<const
FMassStateTreeEvaluatorBase>())
        {
            Eval->GetDependencies(Builder);
        }
        // ... условия и property functions аналогично
    }

    return true;
}

```

Обратите внимание: приведение через `GetPtr<T>()` возвращает `nullptr`, если узел другого типа. Общие узлы (`FStateTreeTaskCommonBase`) не пройдут ни одну проверку — и правильно, у них нет зависимостей от Mass.

7.9. Почему `Dependencies` объявлен `Transient`

cpp

```

UPROPERTY(Transient)
TArray<FMassStateTreeDependency> Dependencies;

```

`Transient` — не сериализуется, не сохраняется в ассет.

Причина в том, что зависимости **выводимы**. Они полностью определяются содержимым дерева, а дерево линкуется при каждой загрузке. Сохранять производные данные в ассет — значит создать возможность рассогласования: кто-то поменял код узла, зависимости изменились, а в ассете лежит старая версия. Классический источник тяжёлых багов.

При этом `UPROPERTY` (пусть и `Transient`) всё равно нужен: `FMassStateTreeDependency` может содержать `TObjectPtr` на `UClass` подсистемы, и GC должен видеть эту ссылку.

Разница между «нет `UPROPERTY`» и «`UPROPERTY(Transient)`» здесь принципиальна: первое означает «рефлексия не знает об этом поле вообще», второе — «знает, отслеживает ссылки, но не пишет на диск».

7.10. Полная цепочка от узла до процессора

Соберём всё, что мы уже знаем, в одну последовательность:

Этап 1, компиляция ассета (редактор). Узлы проверяются через `IsStructAllowed()`, их внешние данные — через `IsExternalItemAllowed()`. Неподходящее не попадает в ассет.

Этап 2, линковка (загрузка ассета). Узлы регистрируют внешние данные через `Link()`. Схема в своём `Link()` собирает `Dependencies` через `GetDependencies()` каждого узла.

Этап 3, регистрация процессора (первое обращение).

`UMassStateTreeSubsystem::CreateProcessorForStateTree()` вызывает `Schema->GetDependencies()`, конвертирует список в `FMassFragmentRequirements` и `FMassSubsystemRequirements`, считает хеш.

Этап 4, поиск или создание процессора. По хешу ищется существующий процессор в `RequirementsHashToProcessor`. Если есть — ассет добавляется к нему через `AddHandledStateTree()`. Если нет — создаётся новый динамический экземпляр, ему передаются требования через `SetExecutionRequirements()`.

Этап 5, построение графа. Солвер зависимостей `Mass` получает требования процессора через `ExportRequirements()` и размещает его в графе так, чтобы не было гонок.

Этап 6, рантайм. Процессор тикает деревья по сигналам, и планировщик уже знает, что можно распараллелить, а что нельзя.

Шаги 3–5 разберём детально в главах 8 и 9.

7.11. Своя схема на основе Mass

Зачем это может понадобиться:

Сузить набор узлов. Например, схема для «мирных жителей», где боевые задачи недоступны — дизайнер физически не сможет ошибиться.

Добавить контекстные данные. Через `GetContextDataDescs()` можно гарантировать всем узлам доступ к какому-то игровому объекту.

Изменить правила выбора состояний через `GetStateSelectionRules()`.

Скелет:

cpp

```
UCLASS(BlueprintType, EditInlineNew, CollapseCategories,
        meta = (DisplayName = "My Peaceful Mass Behavior"))
class UMyPeacefulMassSchema : public UMassStateTreeSchema
{
    GENERATED_BODY()

protected:
    virtual bool IsStructAllowed(const UScriptStruct* InScriptStruct) const
    override
    {
        // сначала общие правила Mass
        if (!Super::IsStructAllowed(InScriptStruct))
        {
            return false;
        }
        // затем собственный запрет
        return !InScriptStruct->IsChildOf(FMyCombatTaskBase::StaticStruct());
    }
};
```

Три правила при написании своей схемы:

1. **Всегда вызывайте** `Super::` в `IsStructAllowed` и `IsExternalItemAllowed`, иначе потеряете базовую функциональность Mass.
2. **Не переопределяйте** `Link()`, **не вызвав** `Super::Link()` — иначе `Dependencies` останется пустым, процессор не объявит требований, и вы получите гонки данных вместо ошибки.
3. **Не включайте обратно** `AllowQueuedCompilation()` — причина в разделе 7.7.

7.12. Диагностика: узел не появляется в редакторе

Самая частая проблема при написании своих узлов. Порядок проверки:

1. **Наследуется ли узел от Mass-базы?** Не от `FStateTreeTaskBase`, а именно от `FMassStateTreeTaskBase`.
2. **Есть ли** `USTRUCT()` **и** `GENERATED_BODY()` ? Без рефлексии узла для редактора не существует.
3. **Не помечен ли узел** `meta = (Hidden)` ? Скопировали объявление базы вместе с метаданными — частая опечатка.

4. **Правильная ли схема у ассета?** Дерево создано как «Mass Behavior», а не как «StateTree Component»?
5. **Собран ли модуль?** Новый USTRUCT требует регенерации заголовков; горячая перезагрузка иногда не подхватывает.

Если узел появляется, но линковка падает — проблема в `IsExternalItemAllowed`: вы запрашиваете тип, который схема не пропускает. Проверьте, что запрашиваемая структура действительно наследник `FMassFragment`, а класс — наследник `USubsystem`.

7.13. Итог главы

Механизм	Роль
Теги данных ассета	Фильтрация деревьев по схеме в выпадающих списках редактора
<code>IsStructAllowed</code>	Какие узлы допустимы: Mass-специфичные плюс общеприменимые
<code>IsClassAllowed</code>	Не переопределён — Blueprint-узлов в Mass нет
<code>IsExternalItemAllowed</code>	Какие данные допустимы: фрагменты, общие фрагменты, подсистемы
<code>Link</code>	Сбор <code>Dependencies</code> со всех узлов дерева
<code>GetDependencies</code>	Точка выхода: подсистема читает результат и строит процессор
<code>AllowQueuedCompilation = false</code>	Защита от компиляции ассета в рабочем потоке Mass
<code>Transient y Dependencies</code>	Данные выводимы из дерева, сохранять нельзя

Что дальше

Глава 8 — система зависимостей целиком. Разберём `FMassStateTreeDependency` и `FStateTreeDependencyBuilder`: как узел объявляет доступ, как объединяются требования от разных узлов, как список превращается в `FMassFragmentRequirements` и `FMassSubsystemRequirements`, зачем нужен хеш и почему процессоров получается несколько. Разберём и обратную сторону — что происходит, если объявить лишнее (потеря параллелизма) или недостаточно (гонка данных), и как это диагностировать.

Глава 8. Система зависимостей

Это, пожалуй, самая необычная часть связки. В обычном StateTree ничего подобного нет: задача просто берёт актора и делает с ним что хочет. В Mass так нельзя — планировщик обязан заранее знать все доступы к данным, иначе параллельное исполнение превращается в лотерею.

Глава разбирает механизм от объявления в узле до размещения процессора в графе.

8.1. Проблема, которую решаем

Напомню суть из главы 2: процессор объявляет требования в `ConfigureQueries()`, а солвер по ним строит граф исполнения. Два процессора, читающих один фрагмент, идут параллельно; читающий и пишущий — последовательно.

Теперь посмотрим на `UMassStateTreeProcessor`. Что он должен объявить?

Он не знает. В момент `ConfigureQueries()` ему известно только, что он будет тикать какие-то деревья. К каким фрагментам обратятся задачи внутри этих деревьев — зависит от содержимого ассетов, которые дизайнер собрал в редакторе.

Три негодных решения, которые напрашиваются:

Объявить всё на запись. Процессор требует `ReadWrite` на все фрагменты движка. Работает — и полностью убивает параллелизм: с ним теперь конфликтует буквально каждый процессор. Симуляция становится однопоточной.

Не объявлять ничего. Задачи всё равно доберутся до данных через контекст. Работает ровно до первой гонки: процессор движения пишет `FMassMoveTargetFragment`, а задача StateTree в другом потоке его читает. Результат — испорченные данные, воспроизводимость околонулевая.

Проверять в рантайме. Перед каждым обращением спрашивать «а можно?». Дорого, и всё равно не решает проблему: к моменту проверки другой поток уже пишет.

Правильное решение — **вывести требования из содержимого ассетов заранее**, на этапе загрузки. Это и делает система зависимостей.

8.2. `FMassStateTreeDependency`

Структура объявлена в `MassStateTreeDependency.h`, который в наш пакет не попал (он подключён из `MassStateTreeSchema.h`). По использованию восстанавливается однозначно:

cpp

```
// реконструкция
struct FMassStateTreeDependency
{
    /** Тип: наследник FMassFragment / FMassSharedFragment / USubsystem */
    TObjectPtr<const UStruct> Type;

    /** Требуемый уровень доступа */
    EMassFragmentAccess Access;
};
```

Пара «тип + доступ». Ничего больше и не нужно.

`EMassFragmentAccess` — знакомое по главе 2 перечисление:

- `ReadOnly` — только чтение;
- `ReadWrite` — чтение и запись.

Массив таких пар и есть `Dependencies` в схеме.

Тип хранится как `const UStruct*`, а не как отдельные поля для структур и классов, — по той же причине, что и в `IsExternalItemAllowed() : UStruct` покрывает и `UScriptStruct` (фрагменты), и `UClass` (подсистемы).

8.3. FStateTreeDependencyBuilder

Билдер живёт в `UE::MassBehavior` и передаётся в `GetDependencies()`:

cpp

```
namespace UE::MassBehavior
{
    struct FStateTreeDependencyBuilder;
}
```

Его интерфейс виден по использованию в узлах модуля:

cpp

```
// реконструкция
struct FStateTreeDependencyBuilder
{
    template<typename T>
    void AddReadOnly();
```

```

template<typename T>
void AddReadWrite();

void Add(const UStruct& Type, EMassFragmentAccess Access);

// ...
};

```

Типичное применение:

cpp

```

void
FMassLookAtTask::GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder
& Builder) const
{
    Builder.AddReadWrite<FMassLookAtFragment>();
    Builder.AddReadWrite<UMassSignalSubsystem>();
}

```

Шаблонные методы — это сахар: `AddReadOnly<T>()` разворачивается в `Add(*T::StaticStruct(), EMassFragmentAccess::ReadOnly)` для структур и в `Add(*T::StaticClass(), ...)` для классов. Различение делается через трейты типов на этапе компиляции.

Что билдер делает с дубликатами

Главная его работа — не накопление, а **слияние**. Разные узлы одного дерева часто просят одно и то же:

```

Задача A: FMassMoveTargetFragment, ReadWrite
Задача B: FMassMoveTargetFragment, ReadOnly
Условие C: FMassMoveTargetFragment, ReadOnly

```

В итоговом списке должна остаться одна запись, и уровень доступа — **максимальный** из запрошенных:

```

FMassMoveTargetFragment, ReadWrite

```

Логика простая: если хоть кто-то пишет, процессор в целом пишет. Понизить нельзя — потеряем корректность. Повышать без нужды не хочется — потеряем параллелизм.

cpp

```

// реконструкция
void FStateTreeDependencyBuilder::Add(const UStruct& Type, EMassFragmentAccess
Access)
{
    for (FMassStateTreeDependency& Existing : Dependencies)
    {
        if (Existing.Type == &Type)
        {
            // ReadWrite «сильнее» ReadOnly
            if (Access == EMassFragmentAccess::ReadWrite)
            {
                Existing.Access = EMassFragmentAccess::ReadWrite;
            }
            return;
        }
    }
    Dependencies.Emplace(&Type, Access);
}

```

Почему билдер, а не прямая запись в массив

Три причины, и все практические.

Инкапсуляция слияния. Логика «максимум доступа» написана один раз, а не в каждом узле.

Устойчивость к изменениям. Если завтра появится третий уровень доступа или потребуется учитывать присутствие (Optional / None), меняется только билдер. Сотни узлов в движке и в проектах трогать не придётся.

Единообразный интерфейс. Автор узла пишет одну строку и не думает о том, как устроено хранилище.

Это классический паттерн «строитель-аккумулятор», и вы встретите его в Unreal повсеместно.

8.4. Полный путь: от узла до требований процессора

Соберём цепочку целиком.

Шаг 1. Узел объявляет

cpp


```

void
FMassZoneGraphPathFollowTask::GetDependencies(UE::MassBehavior::FStateTreeDepen-
dencyBuilder& Builder) const
{
    Builder.AddReadOnly<FMassZoneGraphLaneLocationFragment>();
    Builder.AddReadWrite<FMassMoveTargetFragment>();
    Builder.AddReadWrite<FMassZoneGraphPathRequestFragment>();
    Builder.AddReadWrite<FMassZoneGraphShortPathFragment>();
    Builder.AddReadWrite<FMassZoneGraphCachedLaneFragment>();
    Builder.AddReadOnly<FAgentRadiusFragment>();
    Builder.AddReadOnly<FMassMovementParameters>();
    Builder.AddReadOnly<UZoneGraphSubsystem>();
}

```

(Точное распределение `ReadOnly` / `ReadWrite` — реконструкция; принцип виден по смыслу полей: то, куда задача пишет намерение, идёт на запись, справочные данные — на чтение.)

Сопоставьте это с объявленными хендлами задачи из главы 4 — списки должны совпадать один в один. Именно здесь и живёт та «двойная бухгалтерия», о которой я предупреждал в главе 5.

Шаг 2. Схема собирает

`UMassStateTreeSchema::Link()` обходит все узлы дерева и вызывает `GetDependencies()` каждого, передавая один общий билдер. Результат — `Dependencies`, дедуплицированный список для **всего ассета**.

Важный нюанс: собираются требования **всех** узлов, включая те, что лежат в состояниях, которые агент, может быть, никогда не посетит. Это неизбежно: планировщик строит статический граф, он не может знать, в какое состояние агент придёт.

Отсюда практическое следствие, к которому мы вернёмся в разделе 8.9: **чем разнообразнее дерево, тем шире его требования и тем меньше параллелизма**.

Шаг 3. Подсистема конвертирует

`UMassStateTreeSubsystem::CreateProcessorForStateTree()` берёт список и раскладывает его на две структуры `Mass`:

c++

```

// реконструкция
FMassFragmentRequirements FragmentRequirements(EntityManager.ToSharedRef());

```

```

FMassSubsystemRequirements SubsystemRequirements;

for (const FMassStateTreeDependency& Dependency : Schema->GetDependencies())
{
    if (Dependency.Type->IsChildOf(FMassFragment::StaticStruct()))
    {
        FragmentRequirements.AddRequirement(
            CastChecked<UScriptStruct>(Dependency.Type),
            Dependency.Access,
            EMassFragmentPresence::Optional);
    }
    else if (Dependency.Type->IsChildOf(USubsystem::StaticClass()))
    {
        SubsystemRequirements.AddSubsystemRequirement(
            CastChecked<UClass>(Dependency.Type),
            Dependency.Access);
    }
    // общие фрагменты — аналогично
}

```

Обратите внимание на `EMassFragmentPresence::Optional`. Это принципиально важно.

Если бы требования были `All` (обязательны), процессор обрабатывал бы только тех агентов, у которых есть **все** фрагменты, упомянутые в дереве. Но дерево может содержать ветку для летающих агентов и ветку для пеших — и ни один агент не имеет фрагментов обеих веток сразу. С `All` процессор не увидел бы никого.

`Optional` означает «если фрагмент есть — дай доступ, если нет — всё равно обработай агента». Планировщик учитывает доступ для построения графа, но не отфильтровывает сущности. А сама задача проверяет наличие данных в рантайме — вспомните `EStateTreeExternalDataRequirement::Optional` у `LookAtHandle` из главы 3. Два механизма, работающих согласованно.

Шаг 4. Хеш и выбор процессора

cpp

```

/**
 * The key represents a hash of mass requirements calculated from a StateTree
 assets.
 * Using the hash rather than a StateTree asset pointer since multiple assets
 can have identical requirements,
 * and we want just one dynamic processor to handle all of them.
 */

```

```
UPROPERTY()
```

```
TMap<uint32, TObjectPtr<UMassStateTreeProcessor>> RequirementsHashToProcessor;
```

Комментарий Epic объясняет всё: ключ — хеш требований, а не указатель на ассет, потому что разные ассеты часто имеют идентичные требования, и плодить под них отдельные процессоры бессмысленно.

Пример: у вас двадцать вариантов поведения горожан. Все используют один набор узлов — ходьба по ZoneGraph, взгляд, ожидание. Требования у всех двадцати одинаковые. Хеш совпадает — один процессор на все двадцать ассетов.

cpp

```
/**
 * Adds StateTree to the collection of the assets this specific processor
 instance will handle.
 */
MASSAIBEHAVIOR_API void AddHandledStateTree(TNotNull<const UStateTree*>
StateTree);
```

Ассеты копятя в процессоре:

cpp

```
/** The assets handled by this processor - entities utilizing any of these
assets will be processed by this processor */
UPROPERTY()
TArray<TObjectPtr<const UStateTree>> HandledStateTrees;
```

A StateTreeToProcessor даёт обратное отображение:

cpp

```
/**
 * Mapping StateTree assets to the dynamic processors handling them. Note that
 it's not 1:1,
 * a single processor can handle multiple assets.
 */
TMap<TObjectKey<UStateTree>, TObjectPtr<UMassStateTreeProcessor>>
StateTreeToProcessor;
```

TObjectKey вместо TObjectPtr — деталь, которую стоит отметить: TObjectKey не удерживает объект от сборки мусора. Если ассет выгрузят, ключ станет невалидным, но

висячего указателя не будет.

Шаг 5. Настройка процессора

cpp

```
/**
 * Called to configure dynamic processor's additional requirements that will
 * ensure its located
 * properly within Mass's processing graph. Calling this function is allowed
 * only until the
 * processor is Initialized. The function will ensure that's the case.
 */
MASSAIBEHAVIOR_API void SetExecutionRequirements(const
FMassFragmentRequirements& FragmentRequirements,
                                                const
FMassSubsystemRequirements& SubsystemRequirements);
```

Требования сохраняются в поле:

cpp

```
/**
 * Stores the additional requirements as configured by
 * SetExecutionRequirements.
 * These requirements ensure the processor will be placed at the right
 * location in the processing graph
 * to avoid data races.
 */
FMassExecutionRequirements ExecutionRequirements;
```

Ключевая фраза — *to avoid data races*. Требования нужны не для доступа к данным (доступ идёт через контекст задачи), а **исключительно для размещения в графе**.

Обратите внимание на ограничение из комментария: вызывать можно только до инициализации, и функция это проверяет через `ensure`. Требования — часть идентичности процессора, менять их на лету нельзя, иначе граф станет неверным.

Шаг 6. Экспорт в граф

cpp

```
MASSAIBEHAVIOR_API virtual void ExportRequirements(FMassExecutionRequirements&
OutRequirements) const override;
```

Базовая реализация `UMassProcessor::ExportRequirements()` собирает требования из зарегистрированных запросов. Здесь она переопределена, потому что к запросу процессора нужно **добавить** накопленные `ExecutionRequirements` :

cpp

```
// реконструкция
void UMassStateTreeProcessor::ExportRequirements(FMassExecutionRequirements&
OutRequirements) const
{
    Super::ExportRequirements(OutRequirements);
    OutRequirements.Append(ExecutionRequirements);
}
```

Дальше `FMassProcessorDependencySolver` получает объединённые требования и размещает процессор в графе.

8.5. Что реально происходит в графе

Разберём на конкретном примере. Допустим, в мире три процессора:

- **A** — движение: пишет `FTransformFragment` , читает `FMassMoveTargetFragment` ;
- **B** — наш StateTree-процессор: пишет `FMassMoveTargetFragment` , читает `FMassZoneGraphLaneLocationFragment` ;
- **C** — восприятие: читает `FTransformFragment` , пишет `FMassAlertFragment` .

Солвер строит связи:

- **B → A**: B пишет `FMassMoveTargetFragment` , A его читает. Последовательно, B раньше.
- **A → C**: A пишет `FTransformFragment` , C его читает. Последовательно, A раньше.
- **B и C**: общих данных нет. Могли бы идти параллельно, но C зависит от A, а A от B — так что фактический порядок $B \rightarrow A \rightarrow C$.

Теперь представим, что автор дерева добавил задачу, читающую `FMassAlertFragment` . Появляется связь **C → B**, и вместе с **B → A → C** получается **цикл**. Солвер такое не разрешит: либо выдаст ошибку, либо разорвёт цикл по правилам `ExecuteBefore` / `ExecuteAfter` , и вы получите данные с задержкой в кадр.

Отсюда практический вывод: **состав требований дерева влияет на глобальный порядок исполнения симуляции**. Добавление одного узла в одно дерево может перестроить весь граф.

8.6. Цена лишнего объявления

Допустим, вы на всякий случай написали `AddReadWrite` вместо `AddReadOnly`. Что произойдёт?

Ваш процессор теперь конфликтует со всеми, кто читает этот фрагмент. Вместо параллельного исполнения — последовательное. На четырёхъядерной машине разница может быть двукратной по времени кадра.

Хуже того: конфликты **транзитивны**. Ваш процессор блокирует процессор X, тот блокирует Y, и цепочка вытягивается в линию.

Правило: **объявляйте минимально достаточный доступ**. `ReadOnly` там, где не пишете. Если задача пишет в фрагмент только в `EnterState()`, а в `Tick()` читает — всё равно `ReadWrite`, но стоит подумать, нельзя ли вынести запись в отдельный узел.

8.7. Цена недостающего объявления

Обратная ошибка страшнее.

Задача обращается к фрагменту, но `GetDependencies()` о нём молчит. Что происходит:

1. Планировщик не знает о доступе.
2. Он размещает процессор `StateTree` параллельно с процессором, пишущим в тот же фрагмент.
3. Задача читает фрагмент в момент, когда другой поток его меняет.

Симптомы: значения «дёргаются», агенты изредка ведут себя странно, всё это не воспроизводится под отладчиком (там всё однопоточно) и проявляется только под нагрузкой на многоядерной машине.

Компилятор не поможет. Линковка пройдёт. Тесты, скорее всего, тоже.

Единственная защита — **дисциплина**. Держите `Link()` и `GetDependencies()` рядом, правьте вместе, при код-ревью проверяйте соответствие. Хороший приём — комментарий-якорь:

cpp

```
// ВНИМАНИЕ: список хендлов ниже должен точно соответствовать
GetDependencies()
TStateTreeExternalDataHandle<FMassMoveTargetFragment> MoveTargetHandle;
TStateTreeExternalDataHandle<FAgentRadiusFragment> AgentRadiusHandle;
```

Можно пойти дальше и написать автотест, который для каждого Mass-узла сравнивает набор хендлов (через рефлекссию) с результатом `GetDependencies()`. В движке такого

нет, но в проекте сделать несложно.

8.8. Отладка: как посмотреть, что получилось

Несколько практических приёмов.

Дамп графа процессоров. Консольная команда

```
mass.debug.PrintProcessorGraph
```

(точное имя может отличаться между версиями — ищите в разделе `mass.debug`) выводит порядок исполнения и группы. Динамические процессоры `StateTree` там видны как отдельные экземпляры.

Отладочное описание процессора.

`UMassProcessor::DebugOutputDescription(FOutputDevice& Ar, int32 Indent)` печатает требования. Для нашего процессора это покажет объединённый список — свой запрос плюс `ExecutionRequirements`.

Логирование при загрузке. Простейший способ убедиться, что зависимости собрались, — временно вывести их в `Link()` схемы:

c++

```
for (const FMassStateTreeDependency& Dep : Dependencies)
{
    UE_LOG(LogMassBehavior, Log, TEXT(" %s : %s"),
        *Dep.Type->GetName(),
        Dep.Access == EMassFragmentAccess::ReadWrite ? TEXT("RW") :
        TEXT("RO"));
}
```

Счётчик процессоров. Если их получилось подозрительно много (по одному на каждый ассет), значит требования у деревьев различаются сильнее, чем вы думали. Это повод посмотреть, какие узлы вносят уникальные зависимости.

8.9. Проектные следствия

Система зависимостей задаёт неочевидные правила проектирования деревьев.

Узкие деревья лучше широких. Дерево, содержащее и боевые задачи, и социальные, и навигационные, объявит объединение всех их требований. Даже агент, который весь день стоит на месте, будет обработан процессором с максимально широкими требованиями.

Разделить на несколько ассетов — значит получить несколько процессоров с более узкими требованиями, которые лучше параллелятся.

Но не слишком узкие. Каждый уникальный набор требований — это отдельный процессор, отдельный узел в графе, отдельный обход архетипов. Сотня процессоров с почти одинаковыми требованиями хуже, чем пять с чуть более широкими.

Общие узлы бесплатны. Наследники `FStateTreeTaskCommonBase` не имеют `GetDependencies()` и не расширяют требования. Задержка, случайный выбор, сравнение — используйте смело.

Подсистемы дороже фрагментов. Требование на подсистему конфликтует со всеми процессорами, которые её используют, — а подсистема одна на весь мир. Вспомните трейты `UMassSignalSubsystem`:

cpp

```
template<>
struct TMassExternalSubsystemTraits<UMassSignalSubsystem> final
{
    enum
    {
        GameThreadOnly = false,
        // @todo this subsystem not being thread-safe when writing is an
        obstacle in
        // parallelizing multiple processors
        ThreadSafeWrite = false,
    };
};
```

Еріс прямо признаёт: небезопасность записи в подсистему сигналов мешает параллелить процессоры. Это одно из известных узких мест.

Optional-фрагменты — норма, а не исключение. Дерево, рассчитанное на разные типы агентов, неизбежно упоминает фрагменты, которых у конкретного агента нет. Проверять наличие в рантайме — часть нормального кода задачи, а не признак плохого дизайна.

8.10. Итог главы

Элемент	Что делает
<code>FMassStateTreeDependency</code>	Пара «тип + уровень доступа»
<code>FStateTreeDependencyBuilder</code>	Аккумулятор со слиянием дубликатов и повышением доступа до максимума

Элемент	Что делает
<code>GetDependencies()</code> узла	Объявление доступов конкретного узла
<code>UMassStateTreeSchema::Link()</code>	Сбор объявлений со всего дерева
Хеш требований	Ключ для переиспользования процессоров между ассетами
<code>SetExecutionRequirements()</code>	Передача требований процессору до инициализации
<code>ExportRequirements()</code>	Выдача требований солверу для построения графа
<code>EMassFragmentPresence::Optional</code>	Не фильтровать агентов, но учитывать доступ

И три правила на практику:

1. **Минимально достаточный доступ.** Лишний `ReadWrite` стоит параллелизма.
2. `Link()` и `GetDependencies()` — **единое целое.** Рассогласование даёт гонку без единой ошибки компиляции.
3. **Состав дерева влияет на весь граф.** Добавление узла может перестроить порядок исполнения симуляции.

Что дальше

Часть II закончена — контракты разобраны. С главы 9 начинается часть III, посвящённая времени жизни и исполнению.

Первым пойдёт `UMassStateTreeSubsystem`: разберём пул `instance data` и работу фрилиста, механику `AllocateInstanceData()` и `FreeInstanceData()`, многопоточный детектор доступа `UE_MT_DECLARE_TS_RW_ACCESS_DETECTOR` и почему создание `instance data` до сих пор не распараллелено, создание динамических процессоров через `CreateProcessorForStateTree()`, настройку класса процессора через `UMassBehaviorSettings` и консольную переменную `bDynamicSTProcessorsEnabled`, которая позволяет всю эту машинерию отключить.

Глава 9. `UMassStateTreeSubsystem`

Подсистема — центральный узел всей связки. Она держит данные экземпляров, раздаёт и переиспользует слоты, создаёт динамические процессоры и служит владельцем при построении контекста исполнения. Разберём её целиком.

9.1. Заголовки и переключатель

cpp

```
#include "MassStateTreeTypes.h"
#include "MassSubsystemBase.h"
#include "StateTreeExecutionContext.h"
#if UE_ENABLE_INCLUDE_ORDER_DEPRECATED_IN_5_7
#include "Mass/ExternalSubsystemTraits.h"
#endif
#include "Misc/MTTransactionallySafeAccessDetector.h"
#include "UObject/ObjectKey.h"
```

`StateTreeExecutionContext.h` тянется ради `FStateTreeInstanceData` — он нужен по значению, внутри `FMassStateTreeInstanceDataItem`, так что форвард-декларацией не обойтись.

`MTTransactionallySafeAccessDetector.h` — детекторы многопоточного доступа, о них подробно в разделе 9.7.

`ObjectKey.h` — ради `TObjectKey<UStateTree>` в карте процессоров.

Дальше — переключатель всей динамической машинерии:

cpp

```
namespace UE::Mass::StateTree
{
    extern bool bDynamicSTProcessorsEnabled;
}
```

Переменная объявлена `extern`, определена в `.cpp` и почти наверняка привязана к консольной переменной вида `mass.statetree.DynamicProcessorsEnabled`. Это аварийный выключатель: при `false` подсистема не создаёт процессоры под каждый ассет, а используется единый статически зарегистрированный процессор.

Зачем такой рубильник? Динамические процессоры — относительно новый механизм (появился, чтобы не объявлять требования «на всё подряд»). Если он ломается на конкретном проекте — например, солвер зависимостей начинает выдавать циклы, — нужен способ откатиться к старому поведению без перекомпиляции движка. Хорошая практика для рискованных оптимизаций.

Практический совет: если у вас после обновления движка развалился порядок процессоров, попробуйте выключить этот флаг и посмотреть, изменится ли картина. Это

быстро локализует проблему.

9.2. FMassStateTreeInstanceDataItem

cpp

```
USTRUCT()  
struct FMassStateTreeInstanceDataItem  
{  
    GENERATED_BODY()  
  
    UPROPERTY()  
    FStateTreeInstanceData InstanceData;  
  
    UPROPERTY()  
    int32 Generation = 0;  
};
```

Элемент пула: сами данные плюс счётчик поколения из главы 5.

Оба поля — `UPROPERTY`, и это не формальность. `FStateTreeInstanceData` содержит `TObjectPtr` на instance-объекты Blueprint-узлов и на другие `UObject`. Без `UPROPERTY` сборщик мусора их не увидит и уничтожит прямо под работающим деревом.

Отметьте: **элемент никогда не удаляется**. При освобождении данные сбрасываются, поколение растёт, индекс уходит во фрилист — но сам элемент остаётся в массиве. Массив только растёт. Последствия обсудим в разделе 9.9.

9.3. Объявление класса

cpp

```
#define UE_API MASSAIBEHAVIOR_API  
  
UCLASS(MinimalAPI)  
class UMassStateTreeSubsystem : public UMassSubsystemBase
```

`UMassSubsystemBase` — база для подсистем Mass. Она наследуется от `UWorldSubsystem`, то есть подсистема живёт при мире: создаётся вместе с ним, уничтожается вместе с ним, у каждого мира своя. Это важно для PIE: сервер и клиент в одном процессе имеют разные экземпляры.

Помимо жизненного цикла, `UMassSubsystemBase` даёт унифицированный доступ к менеджеру сущностей и корректную регистрацию в системе Mass.

Приём с `#define UE_API MASSAIBEHAVIOR_API` в начале файла и `#undef UE_API` в конце — свежая практика UE5. Она позволяет писать `UE_API` вместо длинного макроса модуля и, что важнее, безболезненно переносить код между модулями: достаточно поменять одну строку определения.

cpp

```
protected:
    // USubsystem BEGIN
    UE_API virtual void Initialize(FSubsystemCollectionBase& Collection)
override;
    // USubsystem END
```

`Initialize()` — стандартная точка входа подсистемы. Здесь происходит:

- получение и кэширование менеджера сущностей;
- получение `UMassSimulationSubsystem` для регистрации динамических процессоров;
- чтение `UMassBehaviorSettings` и установка `DynamicProcessorClass`.

Обратите внимание: метод `protected`. Вызывает его только движок.

9.4. Публичный API: четыре метода

`AllocateInstanceData`

cpp

```
/**
 * Allocates new instance data for specified StateTree.
 * @param StateTree StateTree to allocated the data for.
 * @return Handle to the data.
 */
UE_API FMassStateTreeInstanceHandle AllocateInstanceData(const UStateTree*
StateTree);
```

Вызывается наблюдателем при появлении `FMassStateTreeInstanceFragment` на сущности. Что делает:

1. Проверяет валидность ассета.
2. Убеждается, что для этого ассета уже создан процессор; если нет — вызывает `CreateProcessorForStateTree()`.
3. Берёт индекс: либо снимает с фрилиста, либо добавляет новый элемент в массив.
4. Инициализирует `FStateTreeInstanceData` под структуру данного дерева.

5. Собирает и возвращает хендл через `FMassStateTreeInstanceHandle::Make(Index, Generation)`.

Параметр `const UStateTree*` нужен по двум причинам сразу: и для инициализации данных (структура instance data зависит от состава узлов), и для регистрации процессора. Второе — неочевидный побочный эффект, но именно поэтому создание процессора привязано к первой аллокации, а не к загрузке ассета: подсистема узнаёт об ассете ровно тогда, когда появляется первый агент, который его использует.

FreeInstanceData

cpp

```
/**
 * Frees instance data.
 * @param Handle Instance data handle to free.
 */
UE_API void FreeInstanceData(const FMassStateTreeInstanceHandle Handle);
```

Обратная операция: проверить валидность, сбросить данные, инкрементировать поколение, вернуть индекс во фрилист.

Критически важно: **к моменту вызова дерево должно быть уже остановлено.**

`FreeInstanceData()` не вызывает `Stop()` — это ответственность вызывающего кода, то есть `UMassStateTreeFragmentDestructor`. Если освободить данные без остановки, задачи не получат `ExitState()` и не отпустят захваченные ресурсы (заявку на слот в очереди, зарезервированную точку интереса, что угодно).

Полный цикл слота:

V

visualize

V

visualize show_widget

Возврат по левой стрелке — это и есть момент, когда все ранее выданные хендлы на этот индекс становятся недействительными: поколение слота ушло вперёд, а в старых хендлах осталось прежнее.

GetInstanceData

cpp

```

/** @return Pointer to instance data held by the handle, or nullptr if handle
is not valid. */
FStateTreeInstanceData* GetInstanceData(const FMassStateTreeInstanceHandle
Handle)
{
    UE_MT_SCOPED_READ_ACCESS(InstanceDataMTDetector);
    return IsValidHandle(Handle) ?
    &InstanceDataArray[Handle.GetIndex()].InstanceData : nullptr;
}

```

Метод **inline в заголовке** — это горячий путь, вызывается для каждой сущности на каждом тике дерева. Раскрытие на месте экономит вызов.

Возврат `nullptr` вместо ассерта — сознательное решение. Как обсуждалось в главе 6, гонки на один кадр в Mass нормальны: фрагмент могли добавить, а данные ещё не выделили. Вызывающий код обязан проверять результат.

Обратите внимание, что метод **не** `const`, и указатель возвращается неконстантный — дерево будет писать в эти данные.

IsValidHandle

cpp

```

/** @return True if the handle points to active instance data. */
bool IsValidHandle(const FMassStateTreeInstanceHandle Handle) const
{
    UE_MT_SCOPED_READ_ACCESS(InstanceDataMTDetector);
    return InstanceDataArray.IsValidIndex(Handle.GetIndex())
        && InstanceDataArray[Handle.GetIndex()].Generation ==
    Handle.GetGeneration();
}

```

Две проверки: индекс в границах массива и совпадение поколений. Вторая — та самая защита от переиспользования из главы 5.

Порядок важен: `IsValidIndex()` идёт первым, иначе обращение к элементу вышло бы за границы. Оператор `&&` гарантирует короткое замыкание.

Метод `const` и публичный — его можно вызвать заранее, чтобы не получать указатель зря.

9.5. CreateProcessorForStateTree

cpp

```
protected:
    /**
     * Gathers Mass-relevant processing requirements from StateTree and spawns
     * a dynamic processor to handle entities using this given asset
     */
    UE_API void CreateProcessorForStateTree(TNotNull<const UStateTree*>
StateTree);
```

Метод, который связывает главу 8 с реальностью. Пошагово:

1. Проверить `UE::Mass::StateTree::bDynamicSTProcessorsEnabled` — если выключено, выйти.
2. Проверить `StateTreeToProcessor` — если ассет уже обработан, выйти.
3. Получить схему ассета, привести к `UMassStateTreeSchema`, вызвать `GetDependencies()`.
4. Преобразовать зависимости в `FMassFragmentRequirements` и `FMassSubsystemRequirements`.
5. Посчитать хеш требований.
6. Поискать процессор в `RequirementsHashToProcessor`.
7. Если нашёлся — вызвать `AddHandledStateTree(StateTree)`.
8. Если нет — создать экземпляр `DynamicProcessorClass`, вызвать `SetExecutionRequirements()`, `AddHandledStateTree()`, `MarkAsDynamic()`, зарегистрировать в `SimulationSubsystem`, положить в обе карты.

Тип параметра — `TNotNull<const UStateTree*>`. Это относительно новая обёртка UE5, документирующая контракт «указатель гарантированно не null» и проверяющая его в отладочных сборках. Полезно: сигнатура сама рассказывает, что проверка на null — забота вызывающего.

9.6. Поля хранения

cpp

```
TArray<int32> InstanceDataFreelist;

UPROPERTY(Transient)
TArray<FMassStateTreeInstanceDataItem> InstanceDataArray;
```

Фрилист — просто массив свободных индексов, используемый как стек (`Pop()` при выделении, `Push()` при освобождении). Не `UPROPERTY`, потому что это числа, ничего

удерживать не нужно.

Использование как стека, а не как очереди, — намеренно: только что освобождённый слот, скорее всего, ещё в кэше процессора, и переиспользовать его дешевле.

Массив данных — `UPROPERTY(Transient)`. Разберём оба атрибута:

`UPROPERTY` нужен ради GC: внутри `FStateTreeInstanceData` живут `TObjectPtr`.

`Transient` означает «не сохранять». И это интересный момент: **состояние поведения агентов не переживает сохранение мира**. При загрузке сейва дерева начнут с начала.

Для типичного применения Mass (толпа горожан, трафик, фоновые NPC) это приемлемо: агенты обычно спавнятся динамически по мере приближения игрока. Но если вы делаете что-то, где поведение агента должно сохраняться, — придётся реализовывать сериализацию самостоятельно, и это нетривиально: `FStateTreeInstanceData` содержит хендлы, привязанные к конкретной скомпилированной версии ассета.

9.7. Детектор многопоточного доступа

cpp

```
/**
 * Multithread access detector to prevent attempts to allocate new instance
 data
 * from multiple threads, which is not supported by the current
 implementation.
 * @see AllocateInstanceData, FreeInstanceData, CreateProcessorForStateTree
 * @todo Instance data creation needs to be refactored in order to allow
 parallelization of StateTree activations
 */
UE_MT_DECLARE_TS_RW_ACCESS_DETECTOR(InstanceDataMTDetector);
```

Комментарий Epic предельно честен: аллокация из нескольких потоков **не поддерживается**, и это признанное ограничение с открытым todo.

Как работает детектор

Макросы `UE_MT_*` — это инструмент отладки, а не синхронизация. Они **не блокируют** ничего. Они регистрируют, кто и как обращается к данным, и срабатывают ассертом при обнаружении конфликта:

- несколько писателей одновременно;
- писатель и читатель одновременно.

c++

```
UE_MT_SCOPED_READ_ACCESS(InstanceDataMTDetector);    // в GetData,
IsValidHandle
UE_MT_SCOPED_WRITE_ACCESS(InstanceDataMTDetector);    // в AllocateInstanceData,
FreeInstanceData
```

В шипящей сборке макросы компилируются в ничто — нулевые накладные расходы.

Суффикс TS в UE_MT_DECLARE_TS_RW_ACCESS_DETECTOR означает «transactionally safe» — корректная работа в условиях транзакционной памяти (AutoRTFM), новой технологии Unreal для откатываемых операций.

Что это значит на практике

Чтение параллельно — можно. Много процессоров могут одновременно вызывать `GetInstanceData()`. Именно поэтому `UMassStateTreeProcessor` вообще может иметь флаг `bProcessEntitiesInParallel`.

Аллокация параллельно — нельзя. Наблюдатель, выделяющий данные, должен работать в одном потоке. Массив может перевыделиться при росте, и все указатели, полученные другими потоками, повиснут.

Аллокация одновременно с чтением — тоже нельзя. По той же причине.

Отсюда практическое ограничение: **массовый спавн агентов не параллелится**. Если вы разом создаёте десять тысяч агентов с деревьями, все аллокации пойдут последовательно. Это заметный, но обычно разовый расход. Если он мешает — спавните пачками по кадрам.

9.8. Ссылки на другие системы

c++

```
/**
 * The relevant Entity Manager. Needed to build processing requirements for
 * dynamic processors.
 */
TSharedPtr<FMassEntityManager> EntityManager;
```

Да, первая буква — `Ё` (Е с огоньком, U+0118). Опечатка в исходниках Epic, живущая уже не одну версию. Компилятору всё равно — идентификаторы Unicode допустимы. Упоминаю, чтобы вы не решили, что у вас проблемы с кодировкой, и чтобы поиск по коду не давал пустой результат.

TSharedPtr, а не TSharedPtr, потому что на момент конструирования подсистемы менеджера ещё нет — он появляется в Initialize().

cpp

```
UPROPERTY()  
TMap<uint32, TObjectPtr<UMassStateTreeProcessor>> RequirementsHashToProcessor;  
  
TMap<TObjectKey<UStateTree>, TObjectPtr<UMassStateTreeProcessor>>  
StateTreeToProcessor;
```

Две карты из главы 8. Разница в объявлении показательна: первая UPROPERTY, вторая нет.

Дело в ключах. TObjectKey<UStateTree> — слабая ссылка, она не удерживает ассет от сборки мусора и не требует регистрации в рефлексии. А процессоры-значения всё равно удерживаются первой картой, так что вторая может быть обычным полем.

cpp

```
// @todo ask Patrick how would this behave when ST asset gets  
changed/recompiled or whatnot
```

Ещё один честный комментарий Epic, на этот раз с обращением к коллеге. Проблема реальная: если перекомпилировать ассет StateTree в редакторе на лету, его зависимости могут измениться, а созданный процессор останется со старыми требованиями. В PIE это может проявиться странным поведением после правки дерева.

Практический совет: **после существенных правок дерева перезапускайте PIE**, не полагайтесь на горячую перезагрузку.

cpp

```
/** Cached SimulationSubsystem for registering dynamic processors. */  
UPROPERTY()  
TObjectPtr<UMassSimulationSubsystem> SimulationSubsystem;
```

UMassSimulationSubsystem управляет фазами обработки и графом процессоров. Через него динамический процессор попадает в исполнение.

cpp

```

/**
 * Class to use when creating dynamic processors to handle given StateTree
 assets.
 * Set based on UMassBehaviorSettings.DynamicStateTreeProcessorClass
 */
UPROPERTY(Transient)
TSubclassOf<UMassStateTreeProcessor> DynamicProcessorClass;

```

Точка расширения для проекта. Вспомните комментарий из `MassStateTreeProcessors.h` :

cpp

```

/**
 * The processor that the UMassStateTreeSubsystem will instantiate for every
 unique StateTree Mass-requirements.
 * The user is not expected to instantiate these processors manually, but a
 project-specific extension can be implemented.
 * It needs to derive from UMassStateTreeProcessor and set as the value of
 UMassStateTreeSubsystem.DynamicProcessorClass.
 */

```

Чтобы подменить класс, нужно завести наследника и прописать его в настройках проекта:

ini

```

[/Script/MassAIBehavior.MassBehaviorSettings]
DynamicStateTreeProcessorClass=/Script/MyGame.MyStateTreeProcessor

```

Зачем это может понадобиться: добавить профилирование, изменить порядок в графе через `ExecutionOrder`, включить параллельный обход, добавить собственную логику перед тиком деревьев.

9.9. Практические характеристики

Массив только растёт

`InstanceDataArray` никогда не сжимается. Если у вас в пике было пятьдесят тысяч агентов, а потом осталась сотня, массив останется на пятьдесят тысяч элементов.

Насколько это плохо? Освобождённый `FStateTreeInstanceData` сбрасывает свои внутренние контейнеры, так что основная память возвращается. Остаётся оверхед пустых структур — заметный, но не катастрофический.

Полезная привычка: если ваш геймплей допускает резкие пики численности, замеряйте память подсистемы. Штатного способа ужать массив нет.

Стоимость аллокации

Обычный случай — снять индекс с фрилиста, инициализировать данные. Дёшево.

Плохой случай — фрилист пуст, массив нужно расширить. `TArray` при росте перевыделяет буфер и **копирует все элементы**. Копирование `FStateTreeInstanceData` — не тривиальная операция.

Если вы знаете ожидаемое количество агентов, имеет смысл прогреть пул: заспавнить их разом при старте уровня, а не наращивать постепенно во время игры.

Один экземпляр на мир

Подсистема привязана к миру. В PIE с сервером и клиентом их две, и данные не пересекаются. Учитывая, что деревья исполняются только на сервере (глава 5), клиентская подсистема будет практически пустой.

9.10. Диагностика

«Дерево не запускается». Проверьте по порядку: выделены ли данные (`InstanceHandle.IsValid()`), валиден ли хендл (`IsValidHandle()`), создан ли процессор для ассета (`StateTreeToProcessor` содержит ключ), пришёл ли сигнал активации.

«Данные чужого агента». Почти наверняка вы где-то сохранили хендл и используете его после уничтожения сущности. Проверка поколения должна это ловить — если не ловит, значит вы обходите `IsValidHandle()` и лезете в массив напрямую.

Ассерт детектора многопоточности. Вы вызываете `AllocateInstanceData()` или `FreeInstanceData()` не из игрового потока, либо параллельно с чтением. Смотрите, откуда пришёл вызов: скорее всего, наблюдатель попал в параллельный обход.

«Процессоров слишком много». Смотрите зависимости деревьев (раздел 8.8). Разные требования — разные процессоры.

«После правки дерева поведение странное». Перезапустите PIE. См. комментарий про Patrick в разделе 9.8.

9.11. Итог главы

Элемент	Роль
<code>bDynamicSTProcessorsEnabled</code>	Аварийный выключатель динамических процессоров
<code>FMassStateTreeInstanceDataItem</code>	Слот пула: данные + поколение
<code>AllocateInstanceData</code>	Выдача слота; попутно регистрирует процессор для ассета
<code>FreeInstanceData</code>	Возврат слота; дерево должно быть остановлено заранее
<code>GetInstanceData / IsValidHandle</code>	Горячий путь, inline, возврат <code>nullptr</code> вместо ассета
<code>CreateProcessorForStateTree</code>	Мост от зависимостей главы 8 к реальному процессору
<code>InstanceDataFreelist</code>	Стек свободных индексов
<code>InstanceDataArray</code>	Пул, <code>Transient</code> — состояние не сохраняется в сейв
MT-детектор	Отладочная проверка: параллельное чтение можно, аллокация нет
<code>DynamicProcessorClass</code>	Точка расширения через настройки проекта

Что дальше

Глава 10 — `StateTreeInstanceData.h`, 836 строк. Разберём, что именно лежит в тех данных, которые подсистема так бережно хранит: `FStateTreeInstanceStorage` и его внутреннее устройство, `FStateTreeInstanceData` и её API, работу с временными экземплярами, очередь событий, отложенные переходы и запросы переходов, а также механику копирования и сериализации. После этой главы станет понятно, почему структура тяжёлая и почему её нельзя положить во фрагмент.

Глава 10. `StateTreeInstanceData.h`

Мы уже знаем, что подсистема хранит `FStateTreeInstanceData` в пуле и что во фрагмент эту структуру класть нельзя. Теперь разберём, что именно там внутри — и заодно получим ответ на вопрос, почему `TArray<FMassStateTreeInstanceDataItem>` вообще работает.

Файл — 836 строк, самый большой из разобранных до сих пор. Идём по порядку.

10.1. Двухслойная конструкция

Главное архитектурное решение файла описано в комментарии:

cpp

```
/**
 * State Tree instance data is used to store the runtime state of a State
 * Tree. It is used together with FStateTreeExecution context to tick the state
 * tree.
 * You are supposed to use FStateTreeInstanceData as a property to store the
 * instance data. That ensures that any UObject references will get GC'd
 * correctly.
 *
 * The FStateTreeInstanceData wraps FStateTreeInstanceStorage, where the data
 * is actually stored. This indirection is done in order to allow the
 * FStateTreeInstanceData
 * to be bitwise relocatable (e.g. you can put it in an array), and we can
 * still allow delegates to bind to the instance data of individual tasks.
 *
 * Since the tasks in the instance data are stored in a array that may get
 * resized you will need to use TStateTreeInstanceDataStructRef
 * to reference a struct based task instance data.
 */
```

Расшифруем. Есть две структуры:

`FStateTreeInstanceStorage` — здесь лежат настоящие данные. Массивы, контейнеры, очереди.

`FStateTreeInstanceData` — тонкая обёртка, у которой ровно одно рабочее поле:

cpp

```
protected:
    /** Storage for the actual instance data, always stores
    FStateTreeInstanceStorage. */
    TSharedRef<FStateTreeInstanceStorage> InstanceStorage =
    MakeShared<FStateTreeInstanceStorage>();
```

Зачем такая косвенность? Комментарий называет причину прямо: **bitwise relocatable**, то есть «переносимая побитовым копированием». `TSharedRef` — это два указателя; перемещение обёртки в памяти не трогает сами данные.

И вот тут возникает прямая связь с главой 9. Помните

`TArray<FMassStateTreeInstanceDataItem>` в подсистеме? Массив растёт, перевыделяется, копирует элементы. Если бы данные лежали в `FStateTreeInstanceData` напрямую, любой указатель на instance data задачи повис бы при первом же росте массива. Благодаря косвенности `FStateTreeInstanceStorage` остаётся на месте в куче, а перемещается только пара указателей.

Комментарий даже приводит пример: «*e.g. you can put it in an array*». Подсистема Mass — ровно этот случай.

Вторая причина косвенности — делегаты. Задача может привязать колбэк к своим instance-данным; общий владелец через `TSharedRef` позволяет проверить, жив ли ещё объект.

10.2. Свободные функции доступа

cpp

```
namespace UE::StateTree::InstanceData
{
    /** @return data view of the specified handle relative to the given frame.
    */
    [[nodiscard]] UE_API FStateTreeDataView GetDataView(
        FStateTreeInstanceStorage& InstanceStorage,
        FStateTreeInstanceStorage* SharedInstanceStorage,
        const FStateTreeExecutionFrame& CurrentFrame,
        const FStateTreeDataHandle& Handle);

    /** @return data view of the specified handle relative to the given frame,
    or tries to find a matching temporary instance. */
    [[nodiscard]] UE_API FStateTreeDataView GetDataViewOrTemporary(...);

    UE_API bool DoesRequireExecutionContext(EStateTreeDataSourceType
    SourceType);
    UE_API bool DoesRequireInstanceStorage(EStateTreeDataSourceType
    SourceType);
}
```

Это низкоуровневый слой разрешения хендлов. Вспомните `FStateTreeDataHandle` из главы 3 — «тип источника плюс индекс». Функция `GetDataView()` превращает хендл в `FStateTreeDataView`, то есть в пару «указатель на память + описание типа».

Три параметра помимо хендла:

- `InstanceStorage` — данные конкретного экземпляра;
- `SharedInstanceStorage` — общие данные (для условий, которые вычисляются вне активного состояния); может быть `nullptr`;
- `CurrentFrame` — кадр исполнения. Это ключевое понятие: `FStateTreeExecutionFrame` описывает один уровень вложенности деревьев. Когда дерево через `LinkedAsset` подключает другое дерево, появляется новый кадр со своими базовыми индексами.

Пометка `[[nodiscard]]` — если вы вызвали функцию и проигнорировали результат, компилятор выдаст предупреждение. Разумно: у функции нет побочных эффектов, вызывать её ради ничего бессмысленно.

Обратите внимание на пару устаревших перегрузок:

cpp

```
UE_DEPRECATED(5.8, "Use the version without Parent Frame instead")
[[nodiscard]] inline FStateTreeView GetDataView(
    FStateTreeInstanceStorage& InstanceStorage,
    FStateTreeInstanceStorage* SharedInstanceStorage,
    const FStateTreeExecutionFrame* ParentFrame,
    const FStateTreeExecutionFrame& CurrentFrame,
    const FStateTreeDataHandle& Handle)
{
    return GetDataView(InstanceStorage, SharedInstanceStorage, CurrentFrame,
        Handle);
}
```

Раньше требовался родительский кадр; в 5.8 от него избавились. Классический приём совместимости: старая сигнатура остаётся `inline`-переходником, компилятор ругается, но код собирается.

Две проверочные функции отвечают на вопрос «что нужно, чтобы добраться до данных этого типа»:

- `DoesRequireExecutionContext()` — контекстные данные, внешние данные и данные области вычисления требуют полного `FStateTreeExecutionContext`;
- `DoesRequireInstanceStorage()` — этому типу достаточно хранилища, полный контекст не нужен.

Разделение объясняет, почему в `StateTree` три уровня контекста (глава 11): часть данных доступна и без полного контекста.

10.3. `FStateTreeTemporaryInstanceData`

cpp

```
/**
 * Holds temporary instance data created during state selection.
 * The data is identified by Frame and DataHandle.
 */
USTRUCT()
struct FStateTreeTemporaryInstanceData
{
    GENERATED_BODY()

    UE::StateTree::FActiveFrameID FrameID;

    UPROPERTY()
    FStateTreeDataHandle DataHandle = FStateTreeDataHandle::Invalid;

    UPROPERTY()
    FStateTreeIndex16 OwnerNodeIndex = FStateTreeIndex16::Invalid;

    UPROPERTY()
    FInstancedStruct Instance;
    // ... устаревшие поля под WITH_EDITORONLY_DATA
};
```

Зачем нужны временные данные? Вспомните механику выбора состояния из главы 3: дерево «примеряет» состояние, проверяя условия входа. Условия могут обращаться к данным задач этого состояния — но состояние ещё не активно, его instance data не выделена.

Решение: создать временный экземпляр, вычислить условие, а дальше либо превратить временные данные в постоянные (если состояние выбрано), либо выбросить (если не выбрано).

Идентификация — по паре «кадр + хендл данных», плюс индекс узла-владельца.

Поле `FrameID` — **без** `UPROPERTY`, в отличие от остальных. Это чисто рантайм-идентификатор, генерируемый через `GenerateUniqueId()`, сериализовать его бессмысленно.

Отдельно отметьте блок:

cpp

```
PRAGMA_DISABLE_DEPRECATION_WARNINGS
FStateTreeTemporaryInstanceData(const FStateTreeTemporaryInstanceData&
```

```
Other) = default;
    // ... остальные специальные функции
PRAGMA_ENABLE_DEPRECATED_WARNINGS
```

Структура содержит устаревшие поля (`StateTree` , `RootState` под `WITH_EDITORONLY_DATA`). Автоматически сгенерированные конструктор копирования и присваивание к ним обращаются, и компилятор выдал бы предупреждение на каждое использование. Явное объявление в обёртке из прагм подавляет шум. Приём, который вы встретите по всему движку.

10.4. Версионирование сериализации

cpp

```
struct FStateTreeInstanceStorageCustomVersion
{
    enum Type
    {
        // Before any version changes were made in the plugin
        BeforeCustomVersionWasAdded = 0,
        // Added custom serialization
        AddedCustomSerialization,

        // -----<new versions can be added above this line>-----
        VersionPlusOne,
        LatestVersion = VersionPlusOne - 1
    };

    /** The GUID for this custom version number */
    UE_API const static FGuid GUID;

private:
    FStateTreeInstanceStorageCustomVersion() = default;
};
```

Стандартный шаблон кастомной версии Unreal. Приватный конструктор — структура служит только пространством имён для перечисления, создавать её экземпляры незачем.

Идиома `VersionPlusOne / LatestVersion` избавляет от ручного обновления «последней версии» при добавлении новой: достаточно вписать значение перед разделительной строкой.

10.5. `FStateTreeInstanceStorage` : поля

Начнём с данных, а API разберём после — так понятнее.

cpp

```
private:
    /**
     * Struct for global and active instances.
     * The buffer format is:
     * for each frames
     *   Global parameters, if it's a global frame.
     *   Global node instances, if it's a global frame. (evaluator, global
tasks)
     *   Active state parameters
     *   Active node instances (tasks)
     * @note Not transient, as we use FStateTreeInstanceData to store default
values for instance data.
     */
    UPROPERTY()
    FInstancedStructContainer InstanceStructs;
```

Это главное хранилище. `FInstancedStructContainer` — контейнер структур **разных типов** в одном непрерывном буфере: не массив указателей, а плотно упакованные данные с таблицей смещений и типов.

Комментарий описывает раскладку буфера точно. Для каждого кадра исполнения подряд лежат: глобальные параметры (если кадр глобальный), экземпляры глобальных узлов (эвалюаторы и глобальные задачи), параметры активных состояний, экземпляры активных узлов.

Именно поэтому доступ идёт через **базовый индекс кадра плюс смещение из хендла**:

cpp

```
case EStateTreeDataSourceType::ActiveInstanceData:
    return Storage.GetMutableStruct(CurrentFrame.ActiveInstanceIndexBase.Get()
+ Handle.GetIndex());
```

И именно поэтому буфер **переупаковывается** при смене активных состояний — а значит, все указатели в него инвалидируются. К этому вернёмся в разделе 10.8.

Пометка `@note Not transient` объясняет отсутствие `Transient`: тот же тип используется в ассете для хранения **значений по умолчанию** instance data. Там сериализация нужна.

cpp

```

/** Execution state of the state tree instance. */
UPROPERTY(Transient)
FStateTreeExecutionState ExecutionState;

```

Состояние исполнения: стек активных кадров, активные состояния в каждом, текущий статус, служебные счётчики.

cpp

```

/**
 * Struct for the execution runtime data.
 * They stay alive until the owning execution context stops.
 */
UPROPERTY(Transient)
UE::StateTree::InstanceData::FInstanceContainer ExecutionRuntimeData;

/** Info to find the index where the execution runtime data starts for a
specific state tree. */
struct FExecutionRuntimeInfo
{
    FObjectKey StateTree;
    int32 StartIndex = 0;
};
TArray<FExecutionRuntimeInfo, TInlineAllocator<1>>
ExecutionRuntimeDataInfos;

```

Данные, живущие всё время исполнения дерева, а не только пока активно состояние. Соответствуют `EStateTreeDataSourceType::ExecutionRuntimeData` из главы 3.

Обратите внимание на `TInlineAllocator<1>` — типичный случай «почти всегда один элемент». Одно дерево без связанных ассетов даёт одну запись, и она помещается прямо в структуру без обращения к куче. Мелкая, но показательная оптимизация: в Unreal такие аллокаторы встречаются повсеместно.

cpp

```

/** Temporary instances */
UPROPERTY(Transient)
TArray<FStateTreeTemporaryInstanceData> TemporaryInstances;

/** Events (Transient) */
TSharedRef<FStateTreeEventQueue> EventQueue =
MakeShared<FStateTreeEventQueue>();

```

```

/** Array of broadcasted delegates. */
TArray<FStateTreeDelegateDispatcher> BroadcastedDelegates;

/** Requested transitions */
UPROPERTY(Transient)
TArray<FStateTreeTransitionRequest> TransitionRequests;

/** Global parameters */
UPROPERTY(Transient)
FInstancedStruct GlobalParameters;

/** Unique id. */
UPROPERTY(Transient)
uint32 UniqueIdGenerator = 0;

```

Очередь событий — `TSharedRef`, и это важная деталь, к которой вернёмся в разделе 10.6.

cpp

```

/**
 * Used to detect if we are using the instance data on multiple threads in
 a safe way.
 * The instance data supports multiple reader threads or a single writer
 thread.
 * The detector supports recursive access.
 */
UE_MT_DECLARE_MRSW_RECURSIVE_ACCESS_DETECTOR(AccessDetector);

/* True if the storage owns the event queue. */
bool bIsOwningEventQueue = true;

#if WITH_STATETREE_DEBUG
    TPimplPtr<UE::StateTree::Debug::FRuntimeValidationInstanceData>
    RuntimeValidationData;
#endif

    friend struct FStateTreeInstanceData;

```

`MRSW` — Multiple Readers, Single Writer. Та же модель, что у подсистемы `Mass` из главы 9, и это не совпадение: обе системы строят параллелизм на «читать можно многим, писать — одному».

`Recursive` означает, что вложенные захваты того же типа разрешены — иначе задача, вызывающая другую задачу, ловила бы ложное срабатывание.

`TPimplPtr` для отладочных данных — идиома «указатель на реализацию»: заголовок не знает устройства `FRuntimeValidationInstanceData`, что сокращает зависимости при компиляции.

Все поля `private`, а `FStateTreeInstanceData` объявлена другом. Обёртка имеет полный доступ, все остальные — только через методы.

10.6. API хранилища

Очередь событий

cpp

```
FStateTreeEventQueue& GetMutableEventQueue() { return *EventQueue; }
const FStateTreeEventQueue& GetEventQueue() const { return *EventQueue; }
bool IsOwningEventQueue() const { return bIsOwningEventQueue; }
const TSharedRef<FStateTreeEventQueue>& GetSharedMutableEventQueue() { return
EventQueue; }

/** Sets event queue from another storage. Marks the event queue not owned. */
UE_API void SetSharedEventQueue(const TSharedRef<FStateTreeEventQueue>&
InSharedEventQueue);
```

Очередь можно **разделить** между несколькими экземплярами данных. Зачем?

Сценарий — связанные деревья. Основное дерево через `LinkedAsset` подключает поддереву; у поддерева свои кадры исполнения, но события должны быть общими. Событие, отправленное в поддереве, обязано быть видно переходам основного дерева.

Флаг `bIsOwningEventQueue` фиксирует, кто хозяин: владелец очищает очередь и управляет её жизнью, «арендатор» только пользуется.

Для Mass это пока экзотика, но если вы строите поведение из связанных ассетов — механизм пригодится.

Запросы переходов

cpp

```
/**
 * Buffers a transition request to be sent to the State Tree.
 * @param Owner Optional pointer to an owner UObject that is used for logging
errors.
 * @param Request transition to request.
 */
```

```

UE_API void AddTransitionRequest(const UObject* Owner, const
FStateTreeTransitionRequest& Request);

TConstArrayView<FStateTreeTransitionRequest> GetTransitionRequests() const {
return TransitionRequests; }
UE_API void ResetTransitionRequests();

```

Запрос перехода — способ сказать «перейди туда» программно, а не через настроенный в редакторе переход. Запросы буферизуются и обрабатываются в безопасной точке тика: менять активные состояния прямо посреди обхода задач нельзя.

Параметр `Owner` нужен только для логирования — чтобы в сообщении об ошибке было видно, кто запросил переход. Для Mass это будет подсистема, а не агент, — отсюда важность `FMassExecutionExtension::GetInstanceDescription()` из главы 12.

Делегаты

cpp

```

/** Marks delegate as broadcasted. Use for transitions. */
UE_API void MarkDelegateAsBroadcasted(const FStateTreeDelegateDispatcher&
Dispatcher);
UE_API bool IsDelegateBroadcasted(const FStateTreeDelegateDispatcher&
Dispatcher) const;
UE_API void ResetBroadcastedDelegates();
UE_API TArray<FStateTreeDelegateDispatcher> StealBroadcastedDelegates();
UE_API bool HasBroadcastedDelegates() const;

```

Механизм триггера `EStateTreeTransitionTrigger::OnDelegate` из главы 3. Задача широковещательно вызывает делегат, факт вызова отмечается, переходы это видят.

`StealBroadcastedDelegates()` — «забрать и очистить» одним вызовом, без лишнего копирования. Типичный приём для обработки накопленного за кадр.

Связано с флагом задачи:

cpp

```

/** True if the node is bound to a task completion delegate listener. */
UPROPERTY()
uint8 bHasTaskCompletionDelegateDispatcher : 1;

```

Доступ к элементам

cpp

```
int32 Num() const { return InstanceStructs.Num(); }
bool IsValidIndex(const int32 Index) const { return
InstanceStructs.IsValidIndex(Index); }

bool IsObject(const int32 Index) const
{
    return InstanceStructs[Index].GetScriptStruct() ==
TBaseStructure<FStateTreeInstanceObjectWrapper>::Get();
}

FConstStructView GetStruct(const int32 Index) const { return
InstanceStructs[Index]; }
FStructView GetMutableStruct(const int32 Index) { return
InstanceStructs[Index]; }

const UObject* GetObject(const int32 Index) const
{
    const FStateTreeInstanceObjectWrapper& Wrapper =
InstanceStructs[Index].Get<const FStateTreeInstanceObjectWrapper>();
    return Wrapper.InstanceObject;
}
```

Интересный момент: **объекты хранятся в том же буфере, что и структуры**, завернутые в `FStateTreeInstanceObjectWrapper`. Это объясняет парность значений `EStateTreeDataSourceType` из главы 3 (`ActiveInstanceData / ActiveInstanceDataObject`): источник один, различается способ извлечения.

`IsObject()` проверяет тип обёртки, и по результату вызывающий выбирает `GetStruct()` или `GetObject()`.

Заметьте странность: `GetMutableObject()` объявлен `const`. Формально это корректно — мы не меняем сам контейнер, а возвращаем неконстантный указатель на объект, который контейнер только адресует. Но читается неочевидно.

cpp

```
/** @return true if all instances are valid. */
UE_API bool AreAllInstancesValid() const;
```

Диагностический метод. Полезен при подозрении на порчу данных — можно вставить в отладочную сборку проверку перед тиком.

Состояние исполнения и рантайм-данные

cpp

```
const FStateTreeExecutionState& GetExecutionState() const { return
ExecutionState; }
FStateTreeExecutionState& GetMutableExecutionState() { return ExecutionState;
}

[[nodiscard]] const UE::StateTree::InstanceData::FInstanceContainer&
GetExecutionRuntimeData() const;
[[nodiscard]] UE::StateTree::InstanceData::FInstanceContainer&
GetExecutionRuntimeData();

/**
 * Add or reuse the execution runtime data for the state tree.
 * Return the base index for the execution runtime data.
 */
[[nodiscard]] UE_API int32 AddExecutionRuntimeData(TNotNull<UObject*> Owner,
UE::StateTree::FExecutionFrameHandle FrameHandle);
```

Формулировка *add or reuse* существенна: при повторном входе в то же дерево рантайм-данные переиспользуются, а не создаются заново. Возвращается базовый индекс — та самая точка отсчёта, к которой прибавляются смещения хендлов.

Временные экземпляры

cpp

```
UE_API FStructView AddTemporaryInstance(UObject& InOwner, const
FStateTreeExecutionFrame& Frame,
const FStateTreeIndex16 OwnerNodeIndex, const FStateTreeDataHandle
DataHandle, FConstStructView NewInstanceData);

UE_API void RemoveTemporaryInstance(const UE::StateTree::FActiveFrameID
FrameID,
const FStateTreeIndex16 OwnerNodeIndex, const FStateTreeDataHandle
DataHandle);

UE_API FStructView GetMutableTemporaryStruct(UE::StateTree::FActiveFrameID
FrameID, const FStateTreeDataHandle DataHandle);
UE_API UObject* GetMutableTemporaryObject(UE::StateTree::FActiveFrameID
FrameID, const FStateTreeDataHandle DataHandle);
UE_API void ResetTemporaryInstances();
```

```
TArrayView<FStateTreeTemporaryInstanceData> GetMutableTemporaryInstances() {  
    return TemporaryInstances; }  

```

Обратите внимание на устаревшие перегрузки:

cpp

```
UE_DEPRECATED(5.7, "Use the version with FrameID instead.")  
FStructView GetMutableTemporaryStruct(const FStateTreeExecutionFrame& Frame,  
    const FStateTreeDataHandle DataHandle)  
{  
    return GetMutableTemporaryStruct(Frame.FrameID, DataHandle);  
}
```

Раньше передавали весь кадр, теперь — только его идентификатор. Логичное сужение: искать нужно по ID, весь кадр для этого не требуется, а передача ссылки на кадр создавала лишнюю связанность.

Глобальные параметры и прочее

cpp

```
UE_API void SetGlobalParameters(FConstStructView Parameters);  
  
UE_DEPRECATED(5.8, "Use the version with a FConstStructView")  
UE_API void SetGlobalParameters(const FInstancedPropertyBag& Parameters);  
  
FConstStructView GetGlobalParameters() const { return GlobalParameters; }  
FStructView GetMutableGlobalParameters() { return GlobalParameters; }  
  
/** @return a unique number used to make active frame id and active state id.  
 */  
UE_API uint32 GenerateUniqueId(); //@TODO rename to GenerateUniqueID  
  
UE_API void AddStructReferencedObjects(FReferenceCollector& Collector);  
UE_API void Reset();
```

Переход с `FInstancedPropertyBag` на `FConstStructView` в 5.8 — обобщение: теперь параметрами может быть любая структура, а не только property bag.

`GenerateUniqueId()` — источник идентификаторов кадров и состояний, обеспечивающий их различимость даже при переиспользовании индексов. Ровно та же идея, что с поколениями в главе 5.

Комментарий `//@TODO rename to GenerateUniqueID` — Epic заметила несоответствие своему же стандарту именования. Мелочь, но показывает, что живой код всегда содержит незакрытые мелкие долги.

cpp

```
/** Start the invalid multithreading read-only access detection. */
UE_API void AcquireReadAccess();
UE_API void ReleaseReadAccess();
UE_API void AcquireWriteAccess();
UE_API void ReleaseWriteAccess();
```

Ручное управление детектором — для случаев, когда область доступа не совпадает с областью видимости и `UE_MT_SCOPED_*` не подходит. Например, контекст исполнения захватывает доступ в конструкторе и освобождает в деструкторе.

10.7. FStateTreeInstanceData : API обёртки

cpp

```
/**
 * Note: If FStateTreeInstanceData is placed on an struct, you must call
 AddStructReferencedObjects() manually,
 *      as it is not automatically called recursively.
 * Note: Serialization is supported only for
 FArchive::IsModifyingWeakAndStrongReferences(), that is replacing object
 references.
 */
```

Оба примечания критичны для Mass.

Первое: если вложить `FStateTreeInstanceData` в другую структуру, надо вручную пробросить сбор ссылок. Именно поэтому `FMassStateTreeInstanceDataItem` объявляет поле как `UPROPERTY` — рефлексия делает это за нас.

Второе уточняет вывод главы 9. Сериализация поддерживается **только** для замены ссылок на объекты, а не для сохранения на диск. То есть даже без `Transient` состояние деревьев в сейв не попало бы. Наблюдение из главы 9 верно, но причина глубже: структура принципиально не рассчитана на персистентность.

Создание и наполнение

cpp

```

struct FAddArgs
{
    STATETREEMODULE_API static FAddArgs Default;
    /** Duplicate the object contained by object wrapper. */
    bool bDuplicateWrappedObject = true;
};

UE_API void Init(UObject& InOwner, TConstArrayView<FInstancedStruct>
InStructs, FAddArgs Args = FAddArgs::Default);
UE_API void Init(UObject& InOwner, TConstArrayView<FConstStructView>
InStructs, FAddArgs Args = FAddArgs::Default);

UE_API void Append(UObject& InOwner, TConstArrayView<FInstancedStruct>
InStructs, FAddArgs Args = FAddArgs::Default);
UE_API void Append(UObject& InOwner, TConstArrayView<FConstStructView>
InStructs, FAddArgs Args = FAddArgs::Default);
UE_API void Append(UObject& InOwner, TConstArrayView<FConstStructView>
InStructs, TConstArrayView<FInstancedStruct*> InInstancesToMove, FAddArgs Args
= FAddArgs::Default);
UE_API void Append(UObject& InOwner, TConstArrayView<FConstStructView>
InStructs, TConstArrayView<TOptional<FInstancedStruct*>> InInstancesToMove,
FAddArgs Args = FAddArgs::Default);

```

Везде первым параметром — `UObject& InOwner`. Владелец нужен для двух вещей: как `Outer` при создании instance-объектов и как контекст для сообщений об ошибках. В `Mass` владельцем выступает подсистема.

Варианты `Append` с `InInstancesToMove` — это оптимизация перехода между состояниями. Когда состояние пересоздаётся, данные некоторых задач можно **переместить**, а не создавать заново. Версия с `TOptional<FInstancedStruct*>` позволяет перемещать выборочно: часть данных переносится, часть создаётся с нуля.

cpp

```

/** Shrinks the array sizes to specified lengths. Sizes must be small or equal
than current size. */
UE_API void ShrinkTo(const int32 Num);

/** Shares the layout from another instance data, and copies the data over. */
UE_API void CopyFrom(UObject& InOwner, const FStateTreeInstanceData& InOther);

/** Resets the data to empty. */
UE_API void Reset();

```

`ShrinkTo()` — при выходе из состояний хвост буфера обрезается. `CopyFrom()` копирует раскладку и данные целиком; в Mass пригодились бы для клонирования агента вместе с состоянием поведения.

Доступ и служебное

cpp

```
UE_API FStateTreeInstanceStorage& GetMutableStorage();
UE_API const FStateTreeInstanceStorage& GetStorage() const;

UE_API TWeakPtr<FStateTreeInstanceStorage> GetWeakMutableStorage();
UE_API TWeakPtr<const FStateTreeInstanceStorage> GetWeakStorage() const;

UE_API int32 GetEstimatedMemoryUsage() const;
```

Слабые указатели на хранилище — основа безопасных отложенных обращений (раздел 10.8).

`GetEstimatedMemoryUsage()` — **очень полезный метод для Mass**. Он даёт оценку памяти одного экземпляра. Умножьте на количество агентов, и получите реальный вес поведения в вашем проекте. Если получается многовато — стоит подумать о сокращении числа одновременно активных задач.

cpp

```
/** Type traits */
UE_API bool Identical(const FStateTreeInstanceData* Other, uint32 PortFlags)
const;
UE_API void AddStructReferencedObjects(FReferenceCollector& Collector);
UE_API bool Serialize(FArchive& Ar);
UE_API void GetPreloadDependencies(TArray<UObject*>& OutDeps);
```

И регистрация трейтов:

cpp

```
template<>
struct TStructOpsTypeTraits<FStateTreeInstanceData> : public
TStructOpsTypeTraitsBase2<FStateTreeInstanceData>
{
    enum
    {
        WithIdentical = true,
```

```

        WithAddStructReferencedObjects = true,
        WithSerializer = true,
        WithGetPreloadDependencies = true,
    };
};

```

`TStructOpsTypeTraits` — способ сообщить системе рефлексии, что структура реализует специальные операции. Без этой регистрации методы просто не будут вызваны, даже если объявлены. Классическая ошибка: реализовать `Serialize()` и забыть трейт.

`WithAddStructReferencedObjects = true` — то, благодаря чему GC видит объекты внутри `instance data`. Для Mass это буквально условие того, что данные агентов не рассыплются при первой же сборке мусора.

10.8. TStateTreeInstanceDataStructRef

Последний и, пожалуй, самый практически важный элемент файла.

cpp

```

/**
 * Stores indexed reference to a instance data struct.
 * The instance data structs may be relocated when the instance data
 * composition changed. For that reason you cannot store pointers to the instance
 * data.
 * This is often needed for example when dealing with delegate lambda's.
 *
 * Note that the reference is valid only during the lifetime of a task
 * (between a call EnterState() and ExitState()).
 */

```

Проблема сформулирована прямо: **указатели на instance data хранить нельзя**. Буфер переупаковывается при смене активных состояний, и любой сохранённый указатель протухает.

Пример из комментария показывает типичный случай — таймер с лямбдой:

cpp

```

EStateTreeRunStatus FTestTask::EnterState(FStateTreeExecutionContext& Context,
const FStateTreeTransitionResult& Transition) const
{
    FInstanceDataType& InstanceData = Context.GetInstanceData(*this);

    Context.GetWorld()->GetTimerManager().SetTimer(

```

```

        InstanceData.TimerHandle,
        [InstanceDataRef = Context.GetInstanceDataStructRef(*this)]()
    {
        if (FInstanceDataType* InstanceData = InstanceDataRef.GetPtr())
        {
            ...
        }
    },
    Delay, true);

    return EStateTreeRunStatus::Running;
}

```

Лямбда захватывает не указатель, а **описание того, как найти данные**: слабую ссылку на хранилище, идентификатор кадра и хендл данных.

cpp

```

protected:
    TWeakPtr<FStateTreeInstanceStorage> WeakStorage = nullptr;
    UE::StateTree::FActiveFrameID FrameID;
    FStateTreeDataHandle DataHandle = FStateTreeDataHandle::Invalid;

```

Конструктор проверяет допустимость источника:

cpp

```

checkf(InDataHandle.GetSource() ==
EStateTreeDataSourceType::ActiveInstanceData
    || InDataHandle.GetSource() ==
EStateTreeDataSourceType::ActiveInstanceDataObject
    || InDataHandle.GetSource() ==
EStateTreeDataSourceType::GlobalInstanceData
    || InDataHandle.GetSource() ==
EStateTreeDataSourceType::GlobalInstanceDataObject,
    TEXT("TStateTreeInstanceDataStructRef supports only instance data.));

```

Только настоящие instance data — ни внешние, ни контекстные, ни параметры.

Как работает GetPtr()

Метод реализован в заголовке целиком, разберём его логику по шагам.

Шаг 1. Проверка жизни хранилища.

cpp

```
TSharedPtr<FStateTreeInstanceStorage> StoragePtr = WeakStorage.Pin();
if (StoragePtr == nullptr)
{
    return nullptr;
}
```

Если агент умер и данные освобождены, слабая ссылка не «пиннется». Это защита от обращения к мёртвому агенту из отложенного колбэка — в Mass ситуация абсолютно штатная.

Шаг 2. Поиск активного кадра.

cpp

```
const FStateTreeExecutionState& Exec = StoragePtr->GetExecutionState();
const FStateTreeExecutionFrame* CurrentFrame = Exec.FindActiveFrame(FrameID);
```

Кадр ищется по идентификатору. Если состояние уже покинуто, кадра в активных нет.

Шаг 3а. Кадр активен.

cpp

```
if (IsHandleSourceValid(Storage, *CurrentFrame, DataHandle))
{
    DataView = GetDataView(Storage, *CurrentFrame, DataHandle);
}
else
{
    DataView = Storage.GetMutableTemporaryStruct(CurrentFrame->FrameID,
DataHandle);
}
```

Сначала пробуем постоянные данные, при неудаче — временные.

Шаг 3б. Кадр не активен.

cpp

```
else
{
    // When selecting a state, the frame is not in the active list.
```



```
FStateTreeTemporaryInstanceData* ExistingInstance = StoragePtr->GetMutableTemporaryInstances().FindByPredicate(...);
```

Комментарий объясняет ситуацию: во время выбора состояния кадр ещё не в списке активных, данные существуют только как временные.

Шаг 4. Проверка типа.

cpp

```
if constexpr (!std::is_same_v<T, void>)
{
    const bool bIsValidType = DataView.GetStruct() == nullptr ||
    DataView.GetStruct()->IsChildOf<T>();
    if (!ensure(bIsValidType))
    {
        return nullptr;
    }
}
```

`if constexpr` — проверка на этапе компиляции: для `T = void` весь блок исчезает. Специализация `TStateTreeInstanceDataStructRef<void>` нужна для нетипизированных сценариев.

`ensure()` вместо `check()` — несоответствие типа сигнализируется, но не роняет игру.

Метод `IsHandleSourceValid`

cpp

```
case EStateTreeDataSourceType::ActiveInstanceData:
case EStateTreeDataSourceType::ActiveInstanceDataObject:
    return CurrentFrame.ActiveInstanceIndexBase.IsValid()
        && CurrentFrame.ActiveStates.Contains(Handle.GetState())
        && Storage.IsValidIndex(CurrentFrame.ActiveInstanceIndexBase.Get() +
        Handle.GetIndex());
```

Три проверки подряд, и средняя — самая содержательная: **состояние, которому принадлежат данные, всё ещё активно**. Если агент ушёл в другое состояние, данные задачи уже не его.

Что это значит для Mass

Правило простое и жёсткое: **никогда не сохраняйте указатель или ссылку на instance data за пределами одного вызова метода задачи.**

c++

```
// НЕЛЬЗЯ
FInstanceDataType* Saved = &Context.GetInstanceData(*this);
SomeSystem->RegisterCallback([Saved]() { Saved->Value = 1; }); // повиснет

// МОЖНО
SomeSystem->RegisterCallback([Ref = Context.GetInstanceDataStructRef(*this)]()
{
    if (FInstanceDataType* Data = Ref.GetPtr())
    {
        Data->Value = 1;
    }
});
```

В Mass это критичнее, чем на акторах: агентов много, умирают они постоянно, а асинхронные запросы (пути, восприятие) — норма.

10.9. Почему это нельзя во фрагмент

Теперь у нас есть точный ответ на вопрос из главы 6. Посчитаем, что означало бы размещение `FStateTreeInstanceData` во фрагменте:

Компонент	Что даёт
<code>TSharedRef<FStateTreeInstanceStorage></code>	Обязательная аллокация в куче при создании
<code>FInstancedStructContainer</code> <code>InstanceStructs</code>	Буфер переменного размера в куче
<code>TSharedRef<FStateTreeEventQueue></code>	Ещё одна аллокация
Пять <code>TArray</code>	Ещё до пяти аллокаций
<code>TPimplPtr</code> отладочных данных	Аллокация в отладочных сборках

Даже если бы размер самой структуры был приемлемым, каждое создание агента означало бы **горсть обращений к куче**. При спавне десяти тысяч агентов — десятки тысяч аллокаций. Это ровно то, чего ECS пытается избежать.

Вынос в подсистему не устраняет аллокации, но локализует их: они происходят при выделении слота, слоты переиспользуются через фрилист, а чанки Mass остаются

плотными и предсказуемыми.

10.10. Практические выводы для Mass

Измеряйте память. `GetEstimatedMemoryUsage()` — единственный штатный способ узнать реальный вес поведения. Замерьте на типичном агенте, умножьте на численность.

Сокращайте число одновременно активных задач. Каждая активная задача — это её `instance data` в буфере. Десять задач в стеке активных состояний вместо трёх — втрое больше данных на агента.

Не храните указатели. См. раздел 10.8. Используйте `GetInstanceDataStructRef()` для всего асинхронного.

Не рассчитывайте на сохранение. Состояние деревьев не сериализуется, и это свойство самой структуры, а не настройки подсистемы.

Помните про MRSW. Читать `instance data` параллельно можно, писать — только из одного потока. Если включаете `bProcessEntitiesInParallel` у процессора, убедитесь, что один агент обрабатывается ровно одним потоком (Mass это гарантирует, распределяя чанки, а не отдельные сущности).

10.11. Итог главы

Элемент	Роль
<code>FStateTreeInstanceData</code>	Тонкая обёртка; переносима побитово, потому и живёт в <code>TArray</code> подсистемы
<code>FStateTreeInstanceStorage</code>	Настоящее хранилище, всегда в куче через <code>TSharedRef</code>
<code>FInstancedStructContainer</code> <code>InstanceStructs</code>	Единый буфер данных всех узлов; раскладка по кадрам
<code>FStateTreeExecutionState</code>	Стек кадров и активных состояний
<code>TemporaryInstances</code>	Данные, создаваемые при примерке состояния
<code>EventQueue</code> в <code>TSharedRef</code>	Может разделяться между связанными деревьями
MRSW-детектор	Много читателей или один писатель, рекурсия разрешена
<code>TStateTreeInstanceDataStructRef</code>	Единственный корректный способ сослаться на данные асинхронно
<code>GetEstimatedMemoryUsage()</code>	Инструмент оценки стоимости поведения

Что дальше

Глава 11 — `StateTreeExecutionContext.h`, 1817 строк, самый крупный файл пакета. Разберём трёхуровневую иерархию контекстов и то, почему уровней именно три; полный API запуска, тика и остановки; работу с событиями и переходами; доступ к instance data и внешним данным; механику `FStateTreeExecutionExtension`, которую переопределяет `Mass`; асинхронные и слабые варианты контекста. Это фундамент для главы 12, где мы наконец разберём `FMassStateTreeExecutionContext` во всех деталях.

Глава 11. `StateTreeExecutionContext.h`

Самый большой файл пакета — 1817 строк. Контекст исполнения связывает воедино владельца, ассет, instance data, внешние данные и предоставляет всё это узлам. Разбираться будем послойно, потому что слоёв в нём буквально три.

11.1. Контекст временный

Первое, что нужно усвоить, сказано в комментарии прямо:

cpp

```
/**
 * StateTree Execution Context is a helper that is used to update StateTree
 * instance data.
 *
 * The context is meant to be temporary, you should not store a context across
 * multiple frames.
 *
 * The owner is used as the owner of the instantiated UObject's in the instance
 * data and logging,
 * it should have same or greater lifetime as the InstanceData.
 */
```

Контекст — не объект, который живёт вместе с агентом. Он создаётся на стеке, используется и уничтожается. В `Mass` это выражается буквально:

`FMassStateTreeExecutionContext` конструируется внутри обхода чанка и умирает вместе с итерацией.

Отсюда — все конструкторы копирования и присваивания удалены:

cpp

```
private:
    FStateTreeExecutionContext(const FStateTreeExecutionContext&) = delete;
    FStateTreeExecutionContext& operator=(const FStateTreeExecutionContext&) =
delete;
```

Контекст нельзя скопировать, положить в контейнер или вернуть из функции. Только создать на месте и использовать.

11.2. Три уровня и зачем они

```
FStateTreeReadOnlyExecutionContext    - только чтение
├── FStateTreeMinimalExecutionContext - + события и запланированный тик
│   └── FStateTreeExecutionContext   - + полное исполнение
```

Разделение не косметическое. Комментарий у среднего уровня объясняет:

cpp

```
/**
 * Minimal execution context to interact with the state tree instance data.
 * A regular execution context requires the context data and external data to
 * be valid to execute all possible operations.
 * The minimal execution context doesn't requires those data but supports only
 * a subset of operations.
 */
```

Полный контекст требует, чтобы **все** контекстные и внешние данные были собраны и валидны. Это дорогая операция: нужно пройти по описаниям, найти каждую подсистему, каждый фрагмент, заполнить массив представлений.

Но не всякая операция этого требует. Чтобы узнать имя активного состояния — достаточно instance data. Чтобы отправить событие — тоже. Собирать ради этого внешние данные бессмысленно.

Помните пару функций из главы 10?

cpp

```
UE_API bool DoesRequireExecutionContext(EStateTreeDataSourceType SourceType);
UE_API bool DoesRequireInstanceStorage(EStateTreeDataSourceType SourceType);
```

Это ровно та же граница, выраженная на уровне типов данных.

Для Mass практический смысл такой: если процессору нужно просто узнать, в каком состоянии агент (для отладки, для статистики, для UI), незачем строить полный FMassStateTreeExecutionContext — хватит read-only варианта.

11.3. Вспомогательные функции

Перед классами файл объявляет набор свободных функций:

cpp

```
namespace UE::StateTree::ExecutionContext
{
    UE_API bool MarkDelegateAsBroadcasted(FStateTreeDelegateDispatcher
    Dispatcher, const FStateTreeExecutionContext& CurrentFrame,
    FStateTreeInstanceStorage& Storage);
    UE_API EStateTreeRunStatus GetPriorityRunStatus(EStateTreeRunStatus A,
    EStateTreeRunStatus B);
    UE_API UE::StateTree::ETaskCompletionStatus
    CastToTaskStatus(EStateTreeFinishTaskType FinishTask);
    UE_API EStateTreeRunStatus CastToRunStatus(EStateTreeFinishTaskType
    FinishTask);
    UE_API UE::StateTree::ETaskCompletionStatus
    CastToTaskStatus(EStateTreeRunStatus InStatus);
    UE_API EStateTreeRunStatus
    CastToRunStatus(UE::StateTree::ETaskCompletionStatus InStatus);

    using FTaskCompletionDelegateDispatcherContainer =
    TArray<FStateTreeDelegateDispatcher, TInlineAllocator<3>>;
    /** Find the dispatcher for a task completion if one matches the task node
    and condition. */
    UE_API FTaskCompletionDelegateDispatcherContainer
    GetTaskCompletionDispatcher(TNotNull<const UStateTree*> StateTree, int32
    NodeIndex, UE::StateTree::ETaskCompletionStatus TaskStatus);
}
```

GetPriorityRunStatus(A, B) заслуживает отдельного внимания. Когда в состоянии несколько задач и они вернули разные статусы, нужно свести их к одному. Приоритет примерно такой: Failed сильнее Succeeded, Succeeded сильнее Running. То есть одна провалившаяся задача роняет всё состояние.

Пара функций CastToTaskStatus / CastToRunStatus — конвертация между «статусом завершения задачи» и «статусом исполнения дерева». Два похожих, но не тождественных перечисления.

Снова `TInlineAllocator<3>` — «обычно диспетчеров не больше трёх». Мелкие оптимизации такого рода разбросаны по всему файлу.

11.4. Делегат сбора внешних данных

cpp

```
/**
 * Delegate used by the execution context to collect external data views for a
 * given StateTree asset.
 * The caller is expected to iterate over the ExternalDataDescs array, find
 * the matching external data,
 * and store it in the OutDataViews at the same index
 */
DECLARE_DELEGATE_RetVal_FourParams(bool, FOnCollectStateTreeExternalData,
    const FStateTreeExecutionContext& /*Context*/,
    const UStateTree* /*StateTree*/,
    TArrayView<const FStateTreeExternalDataDesc> /*ExternalDataDescs*/,
    TArrayView<FStateTreeDataView> /*OutDataViews*/);
```

Это ключевая точка расширения для Mass.

Дерево говорит: «мне нужны вот эти внешние данные» (список `ExternalDataDescs`). Владелец обязан их найти и разложить по индексам в `OutDataViews`. Как именно — его дело.

Для актора это выглядит как поиск компонентов и подсистем. Для Mass — как обращение к `FMassExecutionContext` за фрагментами текущей сущности:

cpp

```
// реконструкция логики Mass
bool CollectExternalData(const FStateTreeExecutionContext& Context, const
UStateTree* StateTree,
    TArrayView<const FStateTreeExternalDataDesc> Descs,
    TArrayView<FStateTreeDataView> OutViews)
{
    for (int32 Index = 0; Index < Descs.Num(); ++Index)
    {
        const FStateTreeExternalDataDesc& Desc = Descs[Index];

        if (Desc.Struct->IsChildOf(FMassFragment::StaticStruct()))
        {
            // фрагмент текущей сущности из чанка
            OutViews[Index] = MakeViewFromFragment(MassContext, EntityIndex,
```

```

Desc.Struct);
    }
    else if (Desc.Struct->IsChildOf(USubsystem::StaticClass()))
    {
        OutViews[Index] =
FStateTreeDataView(MassContext.GetMutableSubsystem(...));
    }
}
return true;
}

```

Именно здесь `TStateTreeExternalDataHandle<FMassLookAtFragment>` превращается в реальный указатель на данные конкретного агента.

Обратите внимание на комментарий у конструктора полного контекста:

cpp

```

* In case the State Tree links to other state tree assets, the collect
external data might get called
* multiple times, once for each asset.

```

Связанные деревья вызывают сбор отдельно для каждого ассета — у каждого свой набор внешних данных.

11.5. FStateTreeReadOnlyExecutionContext

cpp

```

/**
 * Read-only execution context to interact with the state tree instance data.
Only const and read accesses are available.
 * Multiple FStateTreeReadOnlyExecutionContext can coexist on different
threads as long no other (minimal, weak, regular) execution context exists.
 * The user is responsible for preventing invalid multi-threaded access.
 */
struct FStateTreeReadOnlyExecutionContext
{
    UE_API explicit FStateTreeReadOnlyExecutionContext(TNotNull<UObject*>
Owner, TNotNull<const UStateTree*> StateTree, FStateTreeInstanceData&
InInstanceData);
    UE_API explicit FStateTreeReadOnlyExecutionContext(TNotNull<UObject*>
Owner, TNotNull<const UStateTree*> StateTree, FStateTreeInstanceStorage&

```



```
Storage);  
UE_API virtual ~FStateTreeReadOnlyExecutionContext();
```

Комментарий фиксирует ту же модель MRSW из главы 10: несколько read-only контекстов параллельно — можно, но только если нет ни одного пишущего. И ответственность за это лежит на пользователе.

Два конструктора: от `FStateTreeInstanceData` (обёртка) и от `FStateTreeInstanceStorage` (хранилище напрямую). Второй нужен, когда у вас уже есть хранилище и не хочется лишней косвенности.

Поля

cpp

```
protected:  
    /** Owner of the instance data. */  
    UObject& Owner;  
  
    /** The StateTree asset the context is initialized for */  
    const UStateTree& RootStateTree;  
  
    /** Data storage of the instance data. */  
    FStateTreeInstanceStorage& Storage;
```

Три ссылки. Не указатели — контекст не может существовать без них.

`RootStateTree` — именно **корневой** ассет. При связанных деревьях в кадрах исполнения будут другие ассеты, но корень один.

Методы

cpp

```
bool IsValid() const { return RootStateTree.IsReadyToRun(); }
```

Проверяет, что ассет не пустой и успешно слинкован. Комментарий уточняет: «*would be able to run the instance of the associated StateTree asset with a regular execution context*». То есть это проверка ассета, а не данных.

cpp

```
TNotNull<UObject*> GetOwner() const { return &Owner; }  
UWorld* GetWorld() const { return Owner.GetWorld(); }
```

```
TNotNull<const UStateTree*> GetStateTree() const { return &RootStateTree; }
```

`GetWorld()` через владельца — важная деталь для Mass. Владелец там подсистема, привязанная к миру, так что мир получится корректный. Но `GetOwner()` вернёт **подсистему, а не агента** — это то, к чему надо привыкнуть.

cpp

```
/** @return true if there is a pending event with specified tag. */
bool HasEventToProcess(const FGameplayTag Tag) const
{
    return Storage.GetEventQueue().GetEventsView().ContainsByPredicate([Tag]
(const FStateTreeSharedEvent& Event)
    {
        check(Event.IsValid());
        return Event->Tag.MatchesTag(Tag);
    });
}
```

`MatchesTag` — иерархическое сравнение: событие `AI.Alert.Combat` подойдёт под запрос `AI.Alert`.

cpp

```
UE_API FStateTreeScheduledTick GetNextScheduledTick() const;
UE_API EStateTreeRunStatus GetStateTreeRunStatus() const;
UE_API EStateTreeRunStatus GetLastTickStatus() const;
UE_API TConstArrayView<FStateTreeExecutionFrame> GetActiveFrames() const;
UE_API FString GetActiveStateName() const;
UE_API TArray<FName> GetActiveStateNames() const;
```

`GetActiveStateNames()` возвращает **весь стек** от корня до листа — то самое свойство иерархической машины состояний из главы 3. Для отладки Mass это первое, что стоит вывести.

Разница между `GetStateTreeRunStatus()` и `GetLastTickStatus()`: первый — общий статус дерева, второй — результат последнего тика. Дерево может быть `Running`, а последний тик вернуть `Succeeded` для завершившегося состояния.

Отладочные блоки

cpp

```

#if WITH_GAMEPLAY_DEBUGGER
    UE_API FString GetDebugInfoString() const;
#endif

#if WITH_STATETREE_DEBUG
    UE_API int32 GetStateChangeCount() const;
    UE_API void DebugPrintInternalLayout();
#endif

#if WITH_STATETREE_TRACE
    UE_API FStateTreeInstanceDebugId GetInstanceDebugId() const;
    FString GetInstanceDebugDescription() const { return
GetInstanceDescriptionInternal(); }
    void SetOuterTraceId(const uint64 Id) const;
    uint64 GetOuterTraceId() const;
    void
SetNodeCustomDebugTraceData(UE::StateTreeTrace::FNodeCustomDebugData&&
DebugData) const;
    UE::StateTreeTrace::FNodeCustomDebugData StealNodeCustomDebugTraceData()
const;
#else
    void SetOuterTraceId(const uint64 Id) const {}
    uint64 GetOuterTraceId() const { return 0; }
#endif

```

Три разных макроса — три уровня отладки: Gameplay Debugger (экранный оверлей), внутренние проверки StateTree, трассировка для StateTree Debugger.

Обратите внимание на `#else`-ветку: методы остаются, но пустые. Это позволяет вызывающему коду не обкладываться `#if` — стандартный приём, экономящий много шума.

`DebugPrintInternalLayout()` печатает раскладку буфера instance data из главы 10. Полезно, когда подозреваете, что хендлы указывают не туда.

`mutable` у полей трассировки:

cpp

```

#if WITH_STATETREE_TRACE
    mutable UE::StateTreeTrace::FNodeCustomDebugData NodeCustomDebugTraceData;
    mutable uint64 OuterTraceId = 0;
#endif

```

— чтобы отладочные сеттеры можно было звать из `const`-методов. Логически они не меняют состояние исполнения.

cpp

```
protected:
    /** @return Description used as prefix by STATETREE_LOG and
    STATETREE_CLOG, Owner name by default. */
    UE_API FString GetInstanceDescriptionInternal() const;
```

Вот отсюда растёт `FMassExecutionExtension::GetInstanceDescription()`. По умолчанию в лог пишется имя владельца — для Mass это была бы подсистема, одна и та же для всех агентов. Бесполезно. Расширение позволяет подставить описание конкретной сущности.

11.6. FStateTreeMinimalExecutionContext

Добавляет ровно две вещи: запланированный тик и отправку событий.

cpp

```
/**
 * Adds a scheduled tick request.
 * The result of GetNextScheduledTick is affected by the request.
 * This allows a specific task to control when the tree ticks.
 * @note A request with a higher priority will supersede all other requests.
 * ex: Task A request a custom time of 1FPS and Task B request a custom time
 * of 2FPS. Both tasks will tick at 1FPS.
 */
UE_API UE::StateTree::FScheduledTickHandle
AddScheduledTickRequest(FStateTreeScheduledTick ScheduledTick);
UE_API void UpdateScheduledTickRequest(UE::StateTree::FScheduledTickHandle
Handle, FStateTreeScheduledTick ScheduledTick);
UE_API void RemoveScheduledTickRequest(UE::StateTree::FScheduledTickHandle
Handle);
```

Механизм, который `UMassStateTreeSchema` отключает через `IsScheduledTickAllowed() = false` (глава 7). Пример в комментарии показывает логику разрешения конфликтов: побеждает **более частый** запрос. Если одна задача просит 1 FPS, а другая 2 FPS, дерево тикает на 1 FPS — то есть реже. Формулировка «higher priority» здесь означает «более сильное требование», а более сильным считается более редкий тик.

Для Mass это не используется: там сна добиваются полным отсутствием сигналов.

cpp

```

/** Sends event for the StateTree. */
UE_API void SendEvent(const FGameplayTag Tag, const FConstStructView Payload =
FConstStructView(), const FName Origin = FName());

```

Отправка события. Доступна уже на минимальном уровне — значит, разбудить дерево извне можно, не собирая внешние данные. Для Mass это ценно: процессор восприятия может послать событие агенту, не строя полный контекст.

cpp

```

protected:
    /**
     * Get ExecutionExtension from InstanceStorage for less indirections
     * The user is responsible for validity of the result, re-call the getter
    when needed
     * @return ExecutionExtension from InstanceStorage. could be null if
    Extension hasn't been set or Instance Storage has been reset.
     */
    UE_API FStateTreeExecutionExtension* GetMutableExecutionExtension() const;

    /** Informs the owner when the instance of the tree must woke up from a
    scheduled tick sleep. */
    UE_API void ScheduleNextTick(UE::StateTree::ETickReason Reason =
    UE::StateTree::ETickReason::None);

protected:
    /** The context is processing the tree. We do not need to inform the owner
    that something changed. */
    bool bAllowedToScheduleNextTick = true;

```

Расширение (FStateTreeExecutionExtension) хранится в **instance data**, а не в контексте. Логично: контекст временный, расширение должно пережить его. Именно поэтому FMassExecutionExtension передаётся при старте и остаётся жить вместе с данными агента.

Флаг bAllowedToScheduleNextTick подавляет уведомления во время самого тика: незачем дёргать владельца, пока мы и так его обрабатываем.

11.7. FStateTreeExecutionContext : конструирование

cpp

```

UE_API FStateTreeExecutionContext(UObject& InOwner, const UStateTree&
InStateTree, FStateTreeInstanceData& InInstanceData,

```

```

    const FOnCollectStateTreeExternalData& CollectExternalDataCallback = {},
    const EStateTreeRecordTransitions RecordTransitions =
EStateTreeRecordTransitions::No);

/** Construct an execution context from a parent context and another tree.
Useful to run a subtree from the parent context with the same schema. */
UE_API FStateTreeExecutionContext(const FStateTreeExecutionContext&
InContextToCopy, const UStateTree& InStateTree, FStateTreeInstanceData&
InInstanceData);

```

Плюс версии с `TNotNull<>` — старые с голыми ссылками остаются для совместимости.

Второй конструктор — «дочерний контекст»: тот же владелец, те же настройки, но другое дерево. Так исполняются поддеревья.

`EStateTreeRecordTransitions` включает запись истории переходов:

cpp

```

/** Captured snapshots for transition results that can be used to recreate
transitions. This array is only populated if bRecordTransitions is true. */
TArray<FRecordedStateTreeTransitionResult> RecordedTransitions;

```

Это отладочный механизм: можно потом воспроизвести цепочку решений. В продакшене для тысяч агентов включать не стоит.

Паттерн настройки

Комментарий у структуры содержит канонический пример:

cpp

```

FStateTreeExecutionContext Context(*GetOwner(), *StateTreeRef.GetStateTree(),
InstanceData);
if (SetContextRequirements(Context))
{
    Context.Tick(DeltaTime);
}

```

и метод настройки:

cpp

```
bool UMyComponent::SetContextRequirements(FStateTreeExecutionContext& Context)
{
    if (!Context.IsValid())
    {
        return false;
    }
    Context.SetContextDataByName(...);

    Context.SetCollectExternalDataCallback(FOnCollectStateTreeExternalData::Create
    UObject(this, &UMyComponent::CollectExternalData));
    return Context.AreContextDataViewsValid();
}
```

Три шага: проверить ассет, задать контекстные данные, подключить сбор внешних данных, проверить результат.

В Mass этот паттерн скрыт внутри FMassStateTreeExecutionContext. Конструктор делает всё сам:

cpp

```
MASSAIBEHAVIOR_API FMassStateTreeExecutionContext(UObject& InOwner
, const UStateTree& InStateTree
, FStateTreeInstanceData& InInstanceData
, FMassExecutionContext& InContext);
```

Вы просто передаёте FMassExecutionContext — и сбор внешних данных настраивается автоматически. Это одна из главных удобств Mass-обёртки.

11.8. Запуск, тик, остановка

FStartParameters

cpp

```
struct FStartParameters
{
    /** Optional override of global parameters initial values. */
    FConstStructView InitialGlobalParameters;

    /** Optional extension for the execution context. */
    TInstancedStruct<FStateTreeExecutionExtension> ExecutionExtension;

    /** Optional event queue from another instance data. Marks the event queue
```

```

not owned. */
    const TSharedPtr<FStateTreeEventQueue> SharedEventQueue;

    /** Optional override of initial seed for RandomStream. By default
    FPlatformTime::Cycles() will be used. */
    TOptional<int32> RandomSeed;

    struct FStateToSelectOverrideArgs
    {
        FGameplayTag StateTag;
        UStateTree::EStateGameplayTagQueryMethod TagQueryMethod =
        UStateTree::EStateGameplayTagQueryMethod::MatchesExact;
    };

    /** Optional override of initial state to select. By default, root state
    will be selected. */
    TOptional<FStateToSelectOverrideArgs> SelectStateOverrideArgs;
};

```

Здесь три поля, важных для Mass.

`ExecutionExtension` — точка, через которую подставляется `FMassExecutionExtension`. Тип — `TInstancedStruct<>`, то есть полиморфная структура с владением.

`SharedEventQueue` — разделяемая очередь событий из главы 10.

`RandomSeed` — детерминизм. По умолчанию берётся `FPlatformTime::Cycles()`, то есть поведение недетерминировано. Для сетевой игры или для воспроизводимых тестов сид нужно задавать явно. В Mass это особенно актуально: тысяча агентов со случайным выбором состояний — источник рассинхрона, который потом невозможно отладить.

`FStateToSelectOverrideArgs` позволяет стартовать не с корня, а с состояния по тегу. Полезно для «оживления» агента в конкретной ситуации: заспавнить уже испуганным, уже работающим.

Методы

cpp

```

UE_API EStateTreeRunStatus Start();
UE_API EStateTreeRunStatus Start(FConstStructView InitialGlobalParameters);
UE_API EStateTreeRunStatus Start(FStartParameters Parameter);

UE_DEPRECATED(5.8, "Use the version with the FStartParameters")

```



```
UE_API EStateTreeRunStatus Start(const FInstancedPropertyBag*
InitialGlobalParameters, int32 RandomSeed = -1);
```

Обратите внимание на устаревшую сигнатуру — и сравните с
FMassStateTreeExecutionContext :

cpp

```
MASSAIBEHAVIOR_API EStateTreeRunStatus Start();
MASSAIBEHAVIOR_API EStateTreeRunStatus Start(const FInstancedPropertyBag*
InitialParameters, int32 RandomSeed = -1);
```

Mass-обёртка объявляет свои Start() , повторяя старую форму. Это скрывает (не переопределяет — методы не виртуальные) базовые версии. Когда будете писать код, помните: у Mass-контекста набор перегрузок свой.

cpp

```
/**
 * Stop executing if the tree is running.
 * @param CompletionStatus Status (and terminal state) reported in the
transition when the tree is stopped.
 */
UE_API EStateTreeRunStatus Stop(const EStateTreeRunStatus CompletionStatus =
EStateTreeRunStatus::Stopped);
```

Именно это вызывает UMassStateTreeFragmentDestructor перед освобождением данных. Без вызова задачи не получают ExitState() .

cpp

```
UE_API EStateTreeRunStatus Tick(const float DeltaTime);

/**
 * Tick the state tree logic partially, updates the tasks.
 * For full update TickTriggerTransitions() should be called after.
 */
UE_API EStateTreeRunStatus TickUpdateTasks(const float DeltaTime);

/**
 * Tick the state tree logic partially, triggers the transitions.
 * For full update TickUpdateTasks() should be called before.
```

```
*/
UE_API EStateTreeRunStatus TickTriggerTransitions();
```

Разделённый тик — интересная возможность для Mass. Можно обновить задачи всех агентов пачкой, а переходы обработать отдельным проходом. Теоретически это улучшает локальность кэша: один и тот же код работает подряд для многих агентов. В базовом `UMassStateTreeProcessor` это не используется, но в собственном наследнике (через `DynamicProcessorClass` из главы 9) — вполне реализуемо.

11.9. События

cpp

```
template<typename TFunc>
typename TEnableIf<TIsInvocable<TFunc, FStateTreeSharedEvent>::Value,
void>::Type ForEachEvent(TFunc&& Function) const;

template<typename TFunc>
typename TEnableIf<TIsInvocable<TFunc, FStateTreeEvent>::Value, void>::Type
ForEachEvent(TFunc&& Function) const;

TArrayView<FStateTreeSharedEvent> GetMutableEventsToProcessView();
TConstArrayView<FStateTreeSharedEvent> GetEventsToProcessView() const;

/** Consumes and removes the specified event from the event queue. */
UE_API void ConsumeEvent(const FStateTreeSharedEvent& Event);
```

Две перегрузки `ForEachEvent` различаются типом параметра лямбды, а выбор делается через `SFINAE` (`TEnableIf` + `TIsInvocable`). Комментарий советует версию с `FStateTreeSharedEvent`: она не разыменовывает разделяемый указатель.

`ConsumeEvent()` — «я обработал, остальным не показывать». Полезно, когда несколько узлов реагируют на один тег и нужно, чтобы сработал только первый.

11.10. Доступ к данным

Контекстные данные

cpp

```
TConstArrayView<FStateTreeExternalDataDesc> GetContextDataDescs() const {
return RootStateTree.GetContextDataDescs(); }

void SetContextData(const FStateTreeExternalDataHandle Handle,
```

```

FStateTreeView DataView);
UE_API bool SetContextDataByName(const FName Name, FStateTreeView
DataView);
UE_API FStateTreeView GetContextDataByName(const FName Name) const;
UE_API bool AreContextDataViewsValid() const;

```

Те самые данные, которые схема объявляет обязательными (глава 7).

UMassStateTreeSchema их не объявляет — в Mass нет объекта, который был бы «владельцем агента», — поэтому AreContextDataViewsValid() в Mass-сценарии проходит тривиально.

Внешние данные

cpp

```

template <typename T>
typename T::DataType& GetExternalData(const T Handle) const
{
    check(Handle.IsValid());
    check(Handle.DataHandle.GetSource() ==
EStateTreeDataSourceType::ExternalData);
    check(CurrentlyProcessedFrame);
    check(CurrentlyProcessedFrame->StateTree-
>ExternalDataDescs[Handle.DataHandle.GetIndex()].Requirement !=
EStateTreeExternalDataRequirement::Optional); // Optionals should query
pointer instead.
    return ContextAndExternalDataViews[CurrentlyProcessedFrame-
>ExternalDataBaseIndex.Get() + Handle.DataHandle.GetIndex()].template
GetMutable<typename T::DataType>();
}

template <typename T>
typename T::DataType* GetExternalDataPtr(const T Handle) const;

FStateTreeView GetExternalDataView(const FStateTreeExternalDataHandle
Handle);

```

Обратите особое внимание на четвёртый check. Он прямо говорит: для Optional - данных нужно использовать GetExternalDataPtr(), а не GetExternalData().

Это ровно случай FMassLookAtTask:

cpp

```
TStateTreeExternalDataHandle<FMassLookAtFragment,  
EStateTreeExternalDataRequirement::Optional> LookAtHandle;
```

Правильное обращение:

cpp

```
FMassLookAtFragment* LookAtFragment =  
Context.GetExternalDataPtr(LookAtHandle);  
if (LookAtFragment == nullptr)  
{  
    return EStateTreeRunStatus::Failed; // у агента нет головы  
}
```

Попытка вызвать `GetExternalData()` для опционального хендла упадёт в отладочной сборке. Это, пожалуй, самая частая ошибка при написании Mass-узлов.

Заметьте адресацию: `CurrentlyProcessedFrame->ExternalDataBaseIndex.Get() + Handle.DataHandle.GetIndex()`. Тот же принцип «база кадра плюс смещение», что и в главе 10 для instance data.

Instance data узла

cpp

```
template <typename T>  
T* GetInstanceDataPtr(const FStateTreeNodeBase& Node) const  
{  
    check(CurrentNodeDataHandle == Node.InstanceDataHandle);  
    return CurrentNodeInstanceData.template GetMutablePtr<T>();  
}  
  
template <typename T>  
T& GetInstanceData(const FStateTreeNodeBase& Node) const;  
  
template <typename T>  
typename T::FInstanceDataType& GetInstanceData(const T& Node) const  
{  
    static_assert(TIsDerivedFrom<T, FStateTreeNodeBase>::IsDerived, "Expecting  
Node to derive from FStateTreeNodeBase.");  
    check(CurrentNodeDataHandle == Node.InstanceDataHandle);  
    return CurrentNodeInstanceData.template GetMutable<typename  
T::FInstanceDataType>();  
}
```

Третья форма — самая удобная и самая используемая:

cpp

```
FInstanceDataType& InstanceData = Context.GetInstanceData(*this);
```

Тип выводится из `using FInstanceDataType = ...;`, объявленного в узле. Вспомните `FMassLookAtTask`:

cpp

```
using FInstanceDataType = FMassLookAtTaskInstanceData;
```

Без этого объявления шаблон не скомпилируется.

Ключевая проверка — `check(CurrentNodeDataHandle == Node.InstanceDataHandle)`. Контекст отдаёт данные **только текущего обрабатываемого узла**. Достать данные соседней задачи нельзя: обмен между узлами идёт через привязки свойств, а не через прямой доступ. Это дисциплинирующее ограничение, и оно правильное.

cpp

```
template <typename T>
TStateTreeInstanceDataStructRef<typename T::FInstanceDataType>
GetInstanceDataStructRef(const T& Node) const;
```

Тот самый безопасный референс из главы 10 — для лямбд и отложенных вызовов.

Данные времени исполнения

cpp

```
template <typename T>
T* GetExecutionRuntimeDataPtr(const FStateTreeNodeBase& Node) const;

template <typename T>
typename T::FExecutionRuntimeDataType& GetExecutionRuntimeData(const T& Node)
const;
```

Данные, живущие всё время исполнения дерева, а не только пока активно состояние (`EStateTreeDataSourceType::ExecutionRuntimeData`). Полезно для кэшей, которые незачем пересоздавать при каждом входе в состояние.

Текущий контекст обработки

cpp

```
FStateTreeIndex16 GetCurrentlyProcessedNodeIndex() const;
FStateTreeDataHandle GetCurrentlyProcessedNodeInstanceData() const;
FStateTreeStateHandle GetCurrentlyProcessedState() const;
const FStateTreeExecutionFrame* GetCurrentlyProcessedFrame() const;
const FStateTreeExecutionFrame* GetCurrentlyProcessedParentFrame() const;
TSharedPtr<UE::StateTree::ExecutionContext::ITemporaryStorage>
GetCurrentlyProcessedTemporaryStorage() const;
```

Набор «где я сейчас». Пригодается при написании отладочного вывода и при работе с временными данными.

11.11. Переходы и завершение

cpp

```
/**
 * Requests transition to a state.
 * If called during transition processing (e.g. from
 FStateTreeTaskBase::TriggerTransitions()) the transition
 * is attempted to be activate immediately (it can fail e.g. because of
 preconditions on a target state).
 * If called outside the transition handling, the request is buffered and
 handled at the beginning of next transition processing.
 */
UE_API void RequestTransition(const FStateTreeTransitionRequest& Request);
UE_API void RequestTransition(FStateTreeStateHandle TargetState,
    EStateTreeTransitionPriority Priority =
EStateTreeTransitionPriority::Normal,
    EStateTreeSelectionFallback Fallback = EStateTreeSelectionFallback::None);
```

Поведение зависит от момента вызова: внутри обработки переходов — немедленно, иначе — буферизуется. Отсюда `TransitionRequests` в хранилище из главы 10.

cpp

```
/**
 * Finishes a task. This fails if the Task is not currently the processed
 node.
 * ie. Must be called from inside a FStateTreeTaskBase EnterState, ExitState,
 StateCompleted, Tick, TriggerTransitions.
```

```

* If called during tick processing, then the state completes immediately.
* If called outside of the tick processing, then the request is buffered and
handled on the next tick.
*/
UE_API void FinishTask(const FStateTreeTaskBase& Task,
EStateTreeFinishTaskType FinishType);

```

Альтернатива возврату статуса из `Tick()`. Полезно, когда задача узнаёт о завершении не в момент тика — например, из колбэка.

Для `Mass` есть нюанс: «buffered and handled on the next tick» означает, что нужен **следующий тик**, а он придёт только по сигналу. Завершив задачу асинхронно, не забудьте послать `NewStateTreeTaskRequired`.

cpp

```

UE_API void BroadcastDelegate(const FStateTreeDelegateDispatcher& Dispatcher);
UE_API void BindDelegate(const FStateTreeDelegateListener& Listener,
FSimpleDelegate Delegate);
UE_API void UnbindDelegate(const FStateTreeDelegateListener& Listener);

```

Механизм делегатов, обеспечивающий триггер `OnDelegate`. Старые имена (`AddDelegateListener` / `RemoveDelegateListener`) помечены устаревшими в 5.6.

11.12. Точки расширения

Две — и обе используются в `Mass`.

cpp

```

UE_DEPRECATED(5.6, "Use FStateTreeExecutionExtension::GetInstanceDescription
instead")
/** @return Prefix that will be used by STATETREE_LOG and STATETREE_CLOG,
Owner name by default. */
UE_API virtual FString GetInstanceDescription() const final;

```

Метод одновременно `virtual` и `final` — переопределить нельзя, только вызвать. Это способ «запечатать» устаревшую точку расширения: раньше её переопределяли, теперь нужно использовать `FStateTreeExecutionExtension`. Тот же приём вы видели в `StateTreeEvaluatorBase.h`:

cpp

```

UE_DEPRECATED(5.8, "Use the version with the
FStateTreeReadOnlyExecutionContext.")
virtual void AppendDebugInfoString(FString& DebugString, const
FStateTreeExecutionContext& Context) const final
{
}

```

Отсюда в `MassStateTreeExecutionContext.h`:

cpp

```

USTRUCT()
struct FMassExecutionExtension : public FStateTreeExecutionExtension
{
    GENERATED_BODY()

public:
    virtual FString GetInstanceDescription(const FContextParameters& Context)
const override;
    // ...
    FMassEntityHandle Entity;
};

```

Вторая точка:

cpp

```

/** Callback when delayed transition is triggered. Contexts that are event
based can use this to trigger a future event. */
virtual void BeginDelayedTransition(const FStateTreeTransitionDelayedState&
DelayedState) {};

```

Комментарий буквально описывает Mass: «контексты, основанные на событиях, могут использовать это, чтобы вызвать будущее событие». Пустая реализация по умолчанию, переопределение в `FMassStateTreeExecutionContext`:

cpp

```

protected:
    MASSAIBEHAVIOR_API virtual void BeginDelayedTransition(const
FStateTreeTransitionDelayedState& DelayedState) override;

```


Реализация ставит отложенный сигнал `DelayedTransitionWakeup`. Детально — в главе 12.

11.13. Переопределения связанных деревьев

cpp

```
/**
 * Overrides for linked State Trees. This table is used to override State Tree
 * references on linked states.
 * If a linked state's tag is exact match of the tag specified on the table,
 * the reference from the table is used instead.
 */
UE_API void SetLinkedStateTreeOverrides(FStateTreeReferenceOverrides
InLinkedStateTreeOverrides);

/** @return the first state tree reference set by SetLinkedStateTreeOverrides
that matches the StateTag. Or null if not found. */
UE_API const FStateTreeReference* GetLinkedStateTreeOverrideForTag(const
FGameplayTag StateTag) const;
```

Механизм подмены поддеревьев по тегу. Дизайнер строит базовое поведение с состоянием `LinkedAsset`, помеченным тегом `Behavior.Idle`, а конкретный тип агента подставляет своё поддерево для этого тега.

Отсюда в Mass:

cpp

```
virtual void OnLinkedStateTreeOverridesSet(const FContextParameters& Context,
const FStateTreeReferenceOverrides& Overrides) override;

uint32 LinkedStateTreeOverridesHash = 0;
```

Хеш нужен, чтобы понимать, изменился ли набор переопределений: если да, требования могли поменяться, и это влияет на выбор процессора.

11.14. Внутреннее устройство

Поля полного контекста показывают, сколько всего он держит:

cpp

```
protected:
    /** Instance data used during current tick. */
    FStateTreeInstanceData& InstanceData;

    /** Events queue to use, cached for less indirections. */
    TSharedPtr<FStateTreeEventQueue> EventQueue;

    /** Current linked state tree overrides. */
    FStateTreeReferenceOverrides LinkedAssetStateTreeOverrides;

    /** Data view of the context data. */
    TArray<FStateTreeDataView> ContextAndExternalDataViews;

    FOnCollectStateTreeExternalData CollectExternalDataDelegate;

    struct FCollectedExternalDataCache
    {
        const UStateTree* StateTree = nullptr;
        FStateTreeIndex16 BaseIndex;
    };
    TArray<FCollectedExternalDataCache> CollectedExternalCache;
    bool bActiveExternalDataCollected = false;
```

ContextAndExternalDataViews — плоский массив представлений, куда сбор внешних данных складывает результаты. Один массив на все ассеты во всех кадрах; каждый кадр знает свой базовый индекс.

CollectedExternalCache — кэш «для этого ассета данные уже собраны, вот база». При связанных деревьях один и тот же ассет может встретиться дважды.

cpp

```
static constexpr int32 ExpectedEvaluationScopeCacheLength = 4;
TArray<FEvaluationScopeDataCache,
TInlineAllocator<ExpectedEvaluationScopeCacheLength>>
EvaluationScopeInstanceCaches;

UE_API void PushEvaluationScopeInstanceContainer(...);
UE_API void PopEvaluationScopeInstanceContainer(...);
```

Стек областей вычисления — для тех самых временных данных условий (EvaluationScopeInstanceData из главы 3).

cpp

```

    UE_API bool CollectActiveExternalData();
    UE_API bool CollectActiveExternalData(const
TArrayView<FStateTreeExecutionFrame> Frames);
    UE_API FStateTreeIndex16 CollectExternalData(const UStateTree* StateTree);

```

Внутренние методы сбора. Именно они дёргают `CollectExternalDataDelegate`.

11.15. Константное представление

cpp

```

/**
 * The const version of a StateTree Execution Context that prevents using the
 FStateTreeInstanceData with non-const member function.
 */
struct FConstStateTreeExecutionContextView
{
public:
    FConstStateTreeExecutionContextView(UObject& InOwner, const UStateTree&
InStateTree, const FStateTreeInstanceData& InInstanceData)
        : ExecutionContext(InOwner, InStateTree,
const_cast<FStateTreeInstanceData&>(InInstanceData))
    {}

    operator const FStateTreeExecutionContext& () { return ExecutionContext; }
    const FStateTreeExecutionContext& Get() const { return ExecutionContext; }

private:
    FStateTreeExecutionContext ExecutionContext;
};

```

Обёртка, позволяющая построить контекст поверх константных данных. Внутри — честный `const_cast`, снаружи — только константный доступ.

Приём спорный, но практичный: полный контекст физически требует неконстантную ссылку (он много чего кэширует), а вызывающему нужен только чтение. Обёртка фиксирует намерение на уровне типа.

Не путайте с `FStateTreeReadOnlyExecutionContext`: тот — отдельный лёгкий класс, этот — полный контекст с ограниченным интерфейсом.

11.16. Что из этого использует Mass

Соберём в таблицу, чтобы глава 12 читалась как продолжение:

Механизм контекста	Как использует Mass
<code>FOnCollectStateTreeExternalData</code>	Настраивается в конструкторе <code>FMassStateTreeExecutionContext</code> ; отдаёт фрагменты текущей сущности
<code>FStartParameters::ExecutionExtension</code>	Туда подставляется <code>FMassExecutionExtension</code> с хендлом сущности
<code>BeginDelayedTransition()</code>	Переопределён: вместо ожидания ставится отложенный сигнал
<code>GetInstanceDescriptionInternal()</code>	Заменяется на описание сущности через расширение
<code>OnLinkedStateTreeOverridesSet()</code>	Отслеживание изменений набора связанных деревьев по хешу
<code>Start()</code> / <code>Tick()</code> / <code>Stop()</code>	Вызываются процессорами по сигналам
<code>GetExternalDataPtr()</code>	Обязателен для опциональных фрагментов
<code>RandomSeed</code>	Стоит задавать явно ради детерминизма
Read-only контекст	Годится для отладки и статистики без сбора внешних данных

11.17. Итог главы

Уровень	Что умеет	Требует
<code>FStateTreeReadOnlyExecutionContext</code>	Читать состояние, активные кадры, имена состояний, проверять события	Только instance data
<code>FStateTreeMinimalExecutionContext</code>	То же плюс <code>SendEvent()</code> и запланированный тик	Только instance data
<code>FStateTreeExecutionContext</code>	Полное исполнение: <code>Start</code> , <code>Tick</code> , <code>Stop</code> , доступ к данным узлов и внешним данным, переходы	Контекстные и внешние данные должны быть собраны

Три вещи, которые стоит унести в практику:

1. **Контекст живёт один вызов.** Не сохраняйте его, не копируйте — это запрещено на уровне типа.

2. Для Optional -внешних данных используйте `GetExternalDataPtr()` .
`GetExternalData()` для них падает в отладочной сборке.
 3. **Задавайте** `RandomSeed` явно, если вам нужна воспроизводимость или сетевая синхронность.
-

Что дальше

Глава 12 — `MassStateTreeExecutionContext.h` . Файл всего 76 строк, но теперь мы готовы прочитать в нём всё: как `FMassExecutionExtension` подставляет описание сущности и отслеживает переопределения связанных деревьев, что делает конструктор помимо вызова базового, почему `Start()` объявлен заново, как устроен доступ к менеджеру сущностей через `FMassExecutionContext` , и главное — как `BeginDelayedTransition()` превращает ожидание в отложенный сигнал.

Глава 12. `FMassStateTreeExecutionContext` и `FMassExecutionExtension`

Семьдесят шесть строк — и в них сходится всё, что мы разбирали одиннадцать глав. Теперь у нас есть контекст, чтобы прочитать этот файл целиком, не оставив ни одной непонятной строки.

12.1. Заголовки

cpp

```
#if UE_ENABLE_INCLUDE_ORDER_DEPRECATED_IN_5_6
#include "MassEntityTypes.h"
#endif
#include "Mass/EntityHandle.h"
#include "MassExecutionContext.h"
#include "StateTreeExecutionContext.h"
#include "MassStateTreeExecutionContext.generated.h"

struct FMassExecutionContext;
struct FMassEntityManager;
class UMassSignalSubsystem;
struct FMassCommandBuffer;
```

Здесь видна та самая работа Eric по разгрузке инклюдов, о которой шла речь в главе 5. Раньше подключался тяжёлый `MassEntityTypes.h` (1192 строки); теперь — компактный `Mass/EntityHandle.h`, где живёт только `FMassEntityHandle`. Разница на масштабах движка ощутимая.

Четыре форвард-декларации. Обратите внимание на `UMassSignalSubsystem` и `FMassCommandBuffer`: в заголовке они больше нигде не используются. Это следы **устаревшего конструктора**, который раньше требовал подсистему сигналов явно, и намёк на реализацию в `.cpp`, где командный буфер точно понадобится для отложенных сигналов.

`MassExecutionContext.h` подключается полностью, а не форвардом, — потому что inline-методы обращаются к его членам.

12.2. `FMassExecutionExtension`

`cpp`

```
USTRUCT()
struct FMassExecutionExtension : public FStateTreeExecutionExtension
{
    GENERATED_BODY()

public:
    virtual FString GetInstanceDescription(const FContextParameters& Context)
    const override;
    virtual void OnLinkedStateTreeOverridesSet(const FContextParameters&
    Context, const FStateTreeReferenceOverrides& Overrides) override;

    FMassEntityHandle Entity;
    uint32 LinkedStateTreeOverridesHash = 0;
};
```

Компактно — но каждая строка нагружена смыслом.

Зачем расширение вообще

Вспомним проблему из главы 11.

`FStateTreeReadOnlyExecutionContext::GetInstanceDescriptionInternal()` по умолчанию возвращает имя владельца. В `Mass` владелец — `UMassStateTreeSubsystem`, **одна и та же на весь мир**. Логи от десяти тысяч агентов выглядели бы так:

```
[MassStateTreeSubsystem_0] Entering state 'Patrol'
[MassStateTreeSubsystem_0] Entering state 'Idle'
```

```
[MassStateTreeSubsystem_0] Transition failed
```

Понять, какой агент что сделал, невозможно.

Расширение решает это: оно хранит `FMassEntityHandle Entity` и подставляет его в описание. Логи становятся такими:

```
[Entity 4718:3] Entering state 'Patrol'
```

Реализация в `.cpp` наверняка сводится к чему-то вроде:

`cpp`

```
// реконструкция
FString FMassExecutionExtension::GetInstanceDescription(const
FContextParameters& Context) const
{
    return FString::Printf(TEXT("Entity [%s]"),
    *Entity.DebugGetDescription());
}
```

Где расширение живёт

Ключевой момент, который легко упустить: **расширение хранится не в контексте, а в instance data**. Вспомните метод из главы 11:

`cpp`

```
/**
 * Get ExecutionExtension from InstanceStorage for less indirections
 * The user is responsible for validity of the result, re-call the getter when
 * needed
 */
UE_API FStateTreeExecutionExtension* GetMutableExecutionExtension() const;
```

Из хранилища, не из контекста. И передаётся оно один раз, при старте:

`cpp`

```
struct FStartParameters
{
    /** Optional extension for the execution context. */
    TInstancedStruct<FStateTreeExecutionExtension> ExecutionExtension;
```

```
// ...  
};
```

Логика железная: контекст временный, живёт одну итерацию по чанку. Если бы расширение жило в нём, оно бы умирало вместе с ним, и на следующем тике пришлось бы создавать заново. Instance data живёт вместе с агентом — там ему и место.

Практическое следствие: Entity записывается в расширение **при** Start() и дальше не меняется. Хендл сущности постоянен на протяжении её жизни, так что это корректно.

LinkedListStateTreeOverridesHash

cpp

```
virtual void OnLinkedListStateTreeOverridesSet(const FContextParameters& Context,  
const FStateTreeReferenceOverrides& Overrides) override;  
  
uint32 LinkedListStateTreeOverridesHash = 0;
```

Механизм из раздела 11.13: подмена поддеревьев по тегу. Колбэк вызывается, когда набор переопределений установлен или изменён.

Зачем хеш? Потому что **изменение набора связанных деревьев меняет требования**.

Восстановим логику. Дерево A содержит состояние LinkedAsset с тегом Behavior.Combat. Требования дерева A посчитаны и по ним создан процессор (глава 8). Теперь кто-то подставляет вместо стандартного боевого поддерева своё, которое обращается к фрагментам, о которых процессор не знает.

Результат — ровно та ситуация из раздела 8.7: доступ к данным без объявления, гонка без единой ошибки компиляции.

Хеш позволяет это заметить. Реализация, вероятно, выглядит так:

cpp

```
// реконструкция  
void FMassExecutionExtension::OnLinkedListStateTreeOverridesSet(const  
FContextParameters& Context, const FStateTreeReferenceOverrides& Overrides)  
{  
    const uint32 NewHash = ComputeHash(Overrides);  
    if (NewHash != LinkedListStateTreeOverridesHash)  
    {  
        LinkedListStateTreeOverridesHash = NewHash;  
        // зарегистрировать зависимости подставленных деревьев  
    }  
}
```



```
}  
}
```

Практический совет: **если вы используете переопределения связанных деревьев в Mass, убедитесь, что подставляемые ассеты не расширяют набор требований.** Безопасный вариант — чтобы все взаимозаменяемые поддеревья работали с одним и тем же набором фрагментов.

Почему USTRUCT без UPROPERTY

Структура помечена `USTRUCT()`, но ни одно поле не `UPROPERTY`. Это не противоречие.

`USTRUCT()` нужен, потому что расширение хранится в `TInstancedStruct<FStateTreeExecutionExtension>` — полиморфном контейнере, которому требуется рефлексия типа, чтобы знать, что именно создавать.

A `UPROPERTY` не нужен, потому что оба поля — простые значения: хендл сущности (два `int32`) и хеш (`uint32`). Ни одной ссылки на `UObject`, удерживать нечего. Та же логика, что с `FMassStateTreeInstanceHandle` в главе 5.

12.3. Объявление контекста

cpp

```
/**  
 * Extends FStateTreeExecutionContext to provide additional data to Evaluators  
 and Tasks related to MassSimulation  
 */  
struct FMassStateTreeExecutionContext : public FStateTreeExecutionContext
```

Наследование от **полного** контекста, не от минимального. Логично: процессору нужно всё — запуск, тик, доступ к внешним данным.

Заметьте: это не `USTRUCT`. Контекст временный, в рефлексии не нуждается.

12.4. Конструкторы

cpp

```
MASSAIBEHAVIOR_API FMassStateTreeExecutionContext(UObject& InOwner  
    , const UStateTree& InStateTree  
    , FStateTreeInstanceData& InInstanceData  
    , FMassExecutionContext& InContext);
```

Четыре параметра:

- `InOwner` — `UMassStateTreeSubsystem` ;
- `InStateTree` — ассет из `FMassStateTreeSharedFragment` ;
- `InInstanceData` — данные из пула подсистемы по хендлу из фрагмента;
- `InContext` — контекст исполнения `Mass`, из которого берётся всё остальное.

Именно четвёртый параметр делает конструктор особенным. Реализация в `.cpp` наверняка настраивает делегат сбора внешних данных:

`cpp`

```
// реконструкция
FMassStateTreeExecutionContext::FMassStateTreeExecutionContext(UObject&
InOwner, const UStateTree& InStateTree,
    FStateTreeInstanceData& InInstanceData, FMassExecutionContext& InContext)
    : FStateTreeExecutionContext(InOwner, InStateTree, InInstanceData,
        FOnCollectStateTreeExternalData::CreateRaw(this,
&FMassStateTreeExecutionContext::CollectExternalData))
    , MassEntityExecutionContext(&InContext)
{
}
```

То есть весь паттерн `SetContextRequirements` из главы 11 сведён к передаче одного параметра. Пользователю `Mass` не нужно ничего настраивать вручную.

Устаревший конструктор

`cpp`

```
UE_DEPRECATED(5.6, "Use the other constructor that doesn't require
MassEntityManager and MassSignalSubSystem")
FMassStateTreeExecutionContext(UObject& InOwner
    , const UStateTree& InStateTree
    , FStateTreeInstanceData& InInstanceData
    , FMassEntityManager& InEntityManager
    , UMassSignalSubsystem& InSignalSubsystem
    , FMassExecutionContext& InContext)
    : FMassStateTreeExecutionContext(InOwner, InStateTree, InInstanceData,
InContext)
{
}
```

Раньше менеджер сущностей и подсистему сигналов приходилось передавать явно. В 5.6 выяснилось, что оба достижимы через `FMassExecutionContext`, и параметры убрали.

Реализация — делегирующий конструктор, просто игнорирующий лишние аргументы. Аккуратный способ сохранить совместимость: старый код собирается, компилятор предупреждает, поведение не меняется.

Именно этот конструктор объясняет форвард-декларации `FMassEntityManager` и `UMassSignalSubsystem` в начале файла.

12.5. `Start()`

cpp

```
/** Start executing. */
MASSAIBEHAVIOR_API EStateTreeRunStatus Start();
MASSAIBEHAVIOR_API EStateTreeRunStatus Start(const FInstancedPropertyBag*
InitialParameters, int32 RandomSeed = -1);
```

Два метода, повторяющие сигнатуры базового класса. И тут есть важная тонкость: **это не переопределение**. `FStateTreeExecutionContext::Start()` не виртуальный. Это **сокрытие** (name hiding): при вызове через `FMassStateTreeExecutionContext&` выберутся эти версии, при вызове через базовую ссылку — базовые.

Зачем понадобилось своё? Потому что Mass-версия должна сделать больше:

cpp

```
// реконструкция
EStateTreeRunStatus FMassStateTreeExecutionContext::Start()
{
    FStartParameters Parameters;

    FMassExecutionExtension Extension;
    Extension.Entity = Entity;
    Parameters.ExecutionExtension =
TInstancedStruct<FStateTreeExecutionExtension>::Make(Extension);

    return FStateTreeExecutionContext::Start(MoveTemp(Parameters));
}
```

Она подставляет расширение с хендлом сущности. Без этого шага логи и отладка потеряли бы привязку к агенту.

Обратите внимание на вторую сигнатуру: `const FInstancedPropertyBag*` и `int32 RandomSeed`. Это форма, которая в базовом классе помечена устаревшей в 5.8:

cpp

```
UE_DEPRECATED(5.8, "Use the version with the FStartParameters")
UE_API EStateTreeRunStatus Start(const FInstancedPropertyBag*
InitialGlobalParameters, int32 RandomSeed = -1);
```

Mass-обёртка её пока сохраняет. Скорее всего, в будущих версиях это выровняют. Практический вывод: при работе с Mass-контекстом ориентируйтесь на его собственный набор перегрузок, а не на базовый.

Отдельно отметьте `RandomSeed = -1` — «использовать значение по умолчанию», то есть `FPlatformTime::Cycles()`. Про детерминизм говорилось в главе 11: для сетевой игры сид нужно задавать явно и одинаково на всех машинах.

12.6. Доступ к Mass

cpp

```
FMassEntityManager& GetEntityManager() const
{
    return GetMassEntityExecutionContext().GetEntityManagerChecked();
}

FMassExecutionContext& GetMassEntityExecutionContext() const
{
    return *MassEntityExecutionContext;
}
```

Обе inline, обе `const`, обе возвращают **неконстантные** ссылки.

Это не небрежность. Константность метода относится к самому контексту исполнения: он не меняет своих полей. А то, на что он ссылается, менять можно и нужно — задачи пишут во фрагменты.

`GetEntityManagerChecked()` — вариант с проверкой: если менеджер не установлен, будет ассерт. В нормальном потоке исполнения (внутри `Execute()` процессора) он всегда есть.

`GetMassEntityExecutionContext()` разыменовывает указатель без проверки. Контракт: контекст создан из валидной ссылки, значит указатель не может быть null.

Что это даёт задаче

Через эти два метода Mass-задача получает всё:

cpp

```
EStateTreeRunStatus FMyTask::Tick(FStateTreeExecutionContext& Context, const
float DeltaTime) const
{
    FMassStateTreeExecutionContext& MassContext =
static_cast<FMassStateTreeExecutionContext*>(Context);

    const FMassEntityHandle Entity = MassContext.GetEntity();
    FMassExecutionContext& MassExecContext =
MassContext.GetMassEntityExecutionContext();

    // отложенная команда
    MassExecContext.Defer().AddTag<FMyTag>(Entity);

    return EStateTreeRunStatus::Running;
}
```

Приведение через `static_cast` — стандартная практика в Mass-узлах. Оно безопасно, потому что схема гарантирует: Mass-узел выполняется только Mass-контекстом. Но именно поэтому нельзя вставлять Mass-задачи в обычные деревья — приведение молча даст мусор.

Более аккуратный вариант — вспомогательная функция:

cpp

```
FMassStateTreeExecutionContext& MassContext =
FMassStateTreeExecutionContext::Get(Context);
```

В движке такой нет, но в проекте её несложно завести с `checkf` на тип.

12.7. Сущность

cpp

```
FMassEntityHandle GetEntity() const
{
    return Entity;
}
```

```
MASSAIBEHAVIOR_API void SetEntity(const FMassEntityHandle InEntity);

protected:
    FMassEntityHandle Entity;
```

Геттер inline и возвращает **по значению** — хендл всего восемь байт, копия дешевле косвенности.

Сеттер, наоборот, экспортируется из модуля и реализован в `.cpp`. Значит, он делает больше, чем присваивание. Вероятно, синхронизирует хендл с расширением в instance data:

cpp

```
// реконструкция
void FMassStateTreeExecutionContext::SetEntity(const FMassEntityHandle
InEntity)
{
    Entity = InEntity;

    if (FStateTreeExecutionExtension* Extension =
GetMutableExecutionExtension())
    {
        if (FMassExecutionExtension* MassExtension = ...)
        {
            MassExtension->Entity = InEntity;
        }
    }
}
```

Почему сущность — поле контекста, а не параметр

Это ключевое проектное решение, и стоит его осознать.

Контекст создаётся **один раз на чанк**, а не на каждого агента. Внутри цикла по сущностям вызывается `SetEntity()`, и контекст «перенацеливается»:

cpp

```
// типичный обход в процессоре
FMassStateTreeExecutionContext StateTreeContext(Subsystem, *StateTree,
DummyInstanceData, Context);

for (int32 EntityIndex = 0; EntityIndex < Context.GetNumEntities();
```

```

++EntityIndex)
{
    const FMassEntityHandle Entity = Context.GetEntity(EntityIndex);
    FMassStateTreeInstanceFragment& InstanceFragment =
    InstanceFragments[EntityIndex];

    FStateTreeInstanceData* InstanceData =
    Subsystem.GetInstanceData(InstanceFragment.InstanceHandle);
    if (InstanceData == nullptr)
    {
        continue;
    }

    StateTreeContext.SetEntity(Entity);
    // ... подставить InstanceData, посчитать DeltaTime, тикнуть
}

```

Альтернатива — создавать контекст на каждого агента — стоила бы дорого: контекст держит несколько TArray (представления данных, кэши сбора, стек областей вычисления). Аллокации на каждого из десяти тысяч агентов — недопустимо.

Побочный эффект: **контекст не потокобезопасен по сущности**. Один контекст — один агент в один момент. Если процессор работает в параллельном режиме (bProcessEntitiesInParallel), каждый поток обязан иметь свой контекст. Mass это обеспечивает, раздавая потокам разные чанки.

12.8. BeginDelayedTransition() — главное

cpp

```

protected:
    MASSAIBEHAVIOR_API virtual void BeginDelayedTransition(const
    FStateTreeTransitionDelayedState& DelayedState) override;

```

Одна строка, а за ней — весь смысл событийной модели.

Как это работает в обычном StateTree

Дизайнер настроил переход с задержкой в три секунды. Что происходит на акторе:

1. Условие перехода сработало.
2. StateTree создаёт FStateTreeTransitionDelayedState с оставшимся временем.
3. Компонент тикает каждый кадр.
4. На каждом тике время уменьшается.

5. Через три секунды (около 180 тиков при 60 FPS) переход срабатывает.

180 тиков ради одного события. На одном акторе — ерунда. На десяти тысячах агентов — 1.8 миллиона холостых тиков.

Как это работает в Mass

1. Условие перехода сработало.
2. `StateTree` вызывает `BeginDelayedTransition(DelayedState)`.
3. `Mass`-контекст берёт из состояния оставшееся время и ставит отложенный сигнал.
4. Дерево засыпает. **Ноль тиков.**
5. Через три секунды `UMassSignalSubsystem` шлёт `DelayedTransitionWakeup`.
6. Процессор просыпается ровно для этого агента, тикает дерево, переход срабатывает.

Реализация почти наверняка такая:

cpp

```
// реконструкция
void FMassStateTreeExecutionContext::BeginDelayedTransition(const
FStateTreeTransitionDelayedState& DelayedState)
{
    if (UMassSignalSubsystem* SignalSubsystem =
        GetMassEntityExecutionContext().GetMutableSubsystem<UMassSignalSubsystem>())
    {
        SignalSubsystem->DelaySignalEntity(
            UE::Mass::Signals::DelayedTransitionWakeup,
            Entity,
            DelayedState.TimeLeft);
    }
}
```

Сравните с комментарием базового класса из главы 11:

cpp

```
/** Callback when delayed transition is triggered. Contexts that are event
based can use this to trigger a future event. */
virtual void BeginDelayedTransition(const FStateTreeTransitionDelayedState&
DelayedState) {};
```

Еріс заложила эту точку расширения именно под такие случаи. `Mass` ей воспользовался буквально.

Экономия в цифрах

Прикинем на реалистичном сценарии: десять тысяч агентов-горожан, каждый в среднем половину времени ждёт (стоит у витрины, сидит на скамейке, идёт паузу между действиями).

Без событийной модели: $10\,000 \text{ агентов} \times 60 \text{ FPS} = 600\,000 \text{ тиков дерева в секунду}$.

С событийной моделью: тикают только те, кому пришёл сигнал. Если агент совершает действие раз в две секунды, это $10\,000 / 2 = 5000 \text{ тиков в секунду}$.

Разница более чем на два порядка. Именно это делает Mass пригодным для больших толп.

Оборотная сторона

За экономию платят предсказуемостью. Дерево, которое не подписалось на сигнал и не поставило отложенный, **не проснётся никогда**. Это упоминалось в главах 4 и 5, и теперь механизм ясен полностью.

Отсюда практическое правило: **любая задача, возвращающая Running и ожидающая внешнего события, обязана обеспечить себе пробуждение**. Либо через отложенный сигнал (как `FMassLookAtTask` с её `Duration`), либо через сигнал от процессора-исполнителя (как `LookAtFinished`).

12.9. Поля

cpp

```
protected:
    FMassExecutionContext* MassEntityExecutionContext = nullptr;
    FMassEntityHandle Entity;
```

Всего два поля, и оба минимальны.

`MassEntityExecutionContext` — указатель, а не ссылка. Формально контекст всегда конструируется с валидным аргументом, но указатель позволяет перемещать объект и упрощает реализацию.

Общий вес добавки: восемь байт указателя плюс восемь байт хендла. По сравнению с базовым классом (несколько `TArray`, делегат, кэши) — практически ничего.

12.10. Сквозной сценарий

Соберём всё в одну картину — как процессор тикает дерево одного агента.

1. Подготовка чанка. Процессор получает чанк, достаёт общий фрагмент и из него ассет:

cpp

```
const FMassStateTreeSharedFragment& SharedFragment =  
Context.GetConstSharedFragment<FMassStateTreeSharedFragment>();  
const UStateTree* StateTree = SharedFragment.StateTree;
```

2. Создание контекста. Один раз на чанк.

3. Цикл по сущностям. Для каждой:

- получить хендл сущности и фрагмент экземпляра;
- достать `FStateTreeInstanceData` из подсистемы по хендлу, пропустить при `nullptr`;
- посчитать дельту через `LastUpdateTimeInSeconds` (глава 6);
- вызвать `SetEntity()`;
- вызвать `Start()` или `Tick(DeltaTime)`.

4. Внутри тика. `StateTree` обращается к задачам. Задача просит внешние данные — срабатывает делегат сбора, который берёт фрагменты **текущей** сущности из `FMassExecutionContext` по индексу.

5. Задача пишет намерение во фрагмент и возвращает `Running`.

6. Если сработал отложенный переход — вызывается `BeginDelayedTransition()`, ставится сигнал.

7. Если задача завершилась — дерево выбирает новое состояние; при необходимости ставится `NewStateTreeTaskRequired`.

8. Логи и трассировка используют описание из `FMassExecutionExtension`.

Каждый шаг этой цепочки мы разобрали. Осталось посмотреть на сами процессоры — этим займёмся в главе 13.

12.11. Практические замечания

Не создавайте контекст на каждого агента. Один на чанк, дальше `SetEntity()`.

Не сохраняйте контекст. Он временный по контракту; копирование запрещено на уровне типа.

Приводите тип осознанно. `static_cast<FMassStateTreeExecutionContext&>` в `Mass-` узле безопасен, но только потому, что схема не пропустит такой узел в обычное дерево.

Помните про сокрытие `start()` . Через базовую ссылку вызовется базовая версия — и расширение не подставится.

Проверяйте пробуждение. Задача, вернувшая `Running` без запланированного сигнала, замораживает агента навсегда.

Осторожно с переопределениями связанных деревьев. Они могут расширить набор требований за спиной у процессора. Хеш это фиксирует, но не спасает автоматически.

12.12. Итог главы

Элемент	Роль
<code>FMassExecutionExtension::Entity</code>	Привязка логов и отладки к конкретной сущности
<code>FMassExecutionExtension::LinkStateTreeOverridHash</code>	Отслеживание изменений набора связанных деревьев
Конструктор с <code>FMassExecutionContext&</code>	Скрывает весь паттерн настройки внешних данных
Устаревший конструктор	Совместимость: менеджер и подсистема теперь достаются из контекста <code>Mass</code>
<code>Start()</code>	Соккрытие базового метода ради подстановки расширения
<code>GetEntityManager()</code> / <code>GetMassEntityExecutionContext()</code>	Мост к данным <code>Mass</code> для задач
<code>SetEntity()</code>	Перенацеливание одного контекста на разных агентов в цикле
<code>BeginDelayedTransition()</code>	Превращение ожидания в отложенный сигнал — основа экономии

Что дальше

Глава 13 — `MassStateTreeProcessors.h`. Разберём три процессора:

`UMassStateTreeActivationProcessor` (кто и когда запускает деревья),
`UMassStateTreeFragmentDestructor` (корректное завершение при уничтожении агента) и
`UMassStateTreeProcessor` (основной исполнитель, наследник
`UMassSignalProcessorBase`). Заодно посмотрим на `SignalEntities()` как на точку входа, разберём флаг `bProcessEntitiesInParallel` и то, при каких условиях его безопасно включать.

Глава 13. `MassStateTreeProcessors.h`

Три процессора, 111 строк. Это исполнительный слой: всё, что мы разбирали одиннадцать глав, приводится в движение именно здесь.

13.1. Заголовки

cpp

```
#include "MassSignalProcessorBase.h"
#if UE_ENABLE_INCLUDE_ORDER_DEPRECATED_IN_5_6
#include "MassStateTreeFragments.h"
#endif
#include "MassObserverProcessor.h"
#include "MassProcessorDependencySolver.h"
#if UE_ENABLE_INCLUDE_ORDER_DEPRECATED_IN_5_6
#include "MassLODTypes.h"
#endif

struct FMassStateTreeExecutionContext;
struct FMassSubsystemRequirements;
struct FMassFragmentRequirements;
class UStateTree;
```

Два инcludes под макросом совместимости — те же следы чистки, что и раньше.

Особенно интересен `MassLODTypes.h`. Раньше процессоры `StateTree` знали про LOD напрямую: логика тикалась чаще для близких агентов и реже для далёких. В новой версии это убрано — управление частотой полностью перешло к сигналам. Хорошая иллюстрация того, как одна архитектурная идея вытесняет другую.

`MassProcessorDependencySolver.h` подключён ради `FMassExecutionRequirements` — типа поля, а значит форвардом не обойтись.

13.2. UMassStateTreeFragmentDestructor

cpp

```
/**
 * Processor to stop and uninitialized StateTrees on entities.
 */
UCLASS(MinimalAPI)
class UMassStateTreeFragmentDestructor : public UMassObserverProcessor
{
    GENERATED_BODY()

public:
    MASSAIBEHAVIOR_API UMassStateTreeFragmentDestructor();

protected:
    MASSAIBEHAVIOR_API virtual void InitializeInternal(UObject& Owner, const
TSharedRef<FMassEntityManager>& EntityManager) override;
    MASSAIBEHAVIOR_API virtual void ConfigureQueries(const
TSharedRef<FMassEntityManager>& EntityManager) override;
    MASSAIBEHAVIOR_API virtual void Execute(FMassEntityManager& EntityManager,
FMassExecutionContext& Context) override;

    FMassEntityQuery EntityQuery;

    UPROPERTY(Transient)
    TObjectPtr<UMassSignalSubsystem> SignalSubsystem = nullptr;
};
```

Наблюдатель (глава 2), подписанный на удаление FMassStateTreeInstanceFragment .
Подписка задаётся в конструкторе:

cpp

```
// реконструкция
UMassStateTreeFragmentDestructor::UMassStateTreeFragmentDestructor()
    : EntityQuery(*this)
{
    ObservedType = FMassStateTreeInstanceFragment::StaticStruct();
    Operation = EMassObservedOperation::Remove;
    ExecutionFlags = static_cast<int32>(UE::MassStateTree::ExecutionFlags);
}
```

`EMassObservedOperation::Remove` — устаревшее имя, объединяющее `RemoveElement` и `DestroyEntity` (глава 2). То есть процессор срабатывает и когда фрагмент снимают с живой сущности, и когда сущность уничтожают целиком.

Что он делает

Порядок операций критичен:

1. Достать хендл из `FMassStateTreeInstanceFragment`.
2. Получить `FStateTreeInstanceData` из подсистемы.
3. Построить `FMassStateTreeExecutionContext`, вызвать `SetEntity()`.
4. Вызвать `Stop()` — задачам приходит `ExitState()`.
5. Вызвать `FreeInstanceData()` — слот возвращается в пул, поколение растёт.

Четвёртый шаг — та самая обязанность вызывающего кода, о которой шла речь в главе 9. `FreeInstanceData()` не останавливает дерево сам.

Почему это важнее, чем кажется

Без корректного `Stop()` ломается вот что:

Захваченные ресурсы не отпускаются. Задача «занять место на скамейке» в `EnterState()` пометила слот занятым, а в `ExitState()` должна освободить. Агент умер без `ExitState()` — скамейка занята призраком навсегда.

Намерения остаются во фрагментах. Задача записала цель движения; без `ExitState()` соседние системы могут продолжать её видеть (если фрагмент переиспользуется).

Внешние подписки повисают. Задача, зарегистрировавшая делегат или таймер, его не снимет.

Именно поэтому уничтожение агента с деревом — не бесплатная операция, а полноценный проход по стеку активных состояний.

Зачем ему подсистема сигналов

cpp

```
UPROPERTY(Transient)
TObjectPtr<UMassSignalSubsystem> SignalSubsystem = nullptr;
```

Кэшируется в `InitializeInternal()`. Зачем наблюдателю на удаление сигналы?

Причина в том, что `ExitState()` задачи может захотеть послать сигнал — например, сообщить связанному агенту «я ухожу». Плюс отложенные сигналы для умирающего агента желательно отменить, чтобы подсистема не будила несуществующую сущность (проверка поколения это отловит, но лишняя работа ни к чему).

`Transient`, потому что подсистема живёт при мире и восстанавливается при загрузке.

13.3. Тег активации

cpp

```
/**
 * Special tag to know if the state tree has been activated
 */
USTRUCT()
struct FMassStateTreeActivatedTag : public FMassTag
{
    GENERATED_BODY()
};
```

Пустая структура нулевого размера. Вспомните главу 2: тег меняет архетип, а значит меняет, какие запросы увидят сущность.

Здесь тег работает как **защёлка**: агент без него — кандидат на активацию, агент с ним — уже запущен. Проверять не нужно ни одного байта данных: сущности просто лежат в разных архетипах.

13.4. `UMassStateTreeActivationProcessor`

cpp

```
/**
 * Processor to send the activation signal to the state tree which will
 * execute the first tick */
UCLASS(MinimalAPI)
class UMassStateTreeActivationProcessor : public UMassProcessor
{
    GENERATED_BODY()
public:
    MASSAIBEHAVIOR_API UMassStateTreeActivationProcessor();
protected:
    MASSAIBEHAVIOR_API virtual void ConfigureQueries(const
TSharedRef<FMassEntityManager>& EntityManager) override;
    MASSAIBEHAVIOR_API virtual void Execute(FMassEntityManager& EntityManager,
FMassExecutionContext& Context) override;
```

```

    FMassEntityQuery EntityQuery;
};

```

Обычный процессор, не наблюдатель. Запрос выглядит примерно так:

cpp

```

// реконструкция
void UMassStateTreeActivationProcessor::ConfigureQueries(const
TSharedRef<FMassEntityManager>& EntityManager)
{
    EntityQuery.AddRequirement<FMassStateTreeInstanceFragment>
(EMassFragmentAccess::ReadOnly);
    EntityQuery.AddConstSharedRequirement<FMassStateTreeSharedFragment>();
    EntityQuery.AddTagRequirement<FMassStateTreeActivatedTag>
(EMassFragmentPresence::None);
}

```

Ключевое — `EMassFragmentPresence::None`: «сущности **без** этого тега». Как только тег поставлен, агент выпадает из запроса навсегда.

Исполнение:

cpp

```

// реконструкция
void UMassStateTreeActivationProcessor::Execute(FMassEntityManager&
EntityManager, FMassExecutionContext& Context)
{
    TArray<FMassEntityHandle> EntitiesToSignal;

    EntityQuery.ForEachEntityChunk(Context, [&EntitiesToSignal]
(FMassExecutionContext& Context)
    {
        for (int32 i = 0; i < Context.GetNumEntities(); ++i)
        {
            const FMassEntityHandle Entity = Context.GetEntity(i);
            Context.Defer().AddTag<FMassStateTreeActivatedTag>(Entity);
            EntitiesToSignal.Add(Entity);
        }
    });

    if (!EntitiesToSignal.IsEmpty())
    {

```



```
SignalSubsystem->SignalEntities(UE::Mass::Signals::StateTreeActivate,
EntitiesToSignal);
    }
}
```

Обратите внимание на две детали.

Тег ставится через `Defer()`. Менять состав сущности во время обхода чанков нельзя (глава 2). Команда исполнится в безопасной точке кадра.

Сигналы отправляются пачкой. `SignalEntities()` с массивом дешевле, чем `SignalEntity()` в цикле: один поиск делегата вместо тысячи.

Почему активация — отдельный процессор

Напрашивается вопрос: почему не сделать наблюдатель на **добавление** фрагмента, симметрично деструктору?

Причина в порядке инициализации. Наблюдатель на добавление срабатывает в момент, когда фрагмент появился, — но другие трейты того же агента могут ещё не отработать. Дерево запустилось бы на полупустых данных: задача в `EnterState()` полезла бы во фрагмент движения, которого ещё нет.

Отдельный процессор, работающий в обычной фазе кадра, гарантирует: к моменту активации сущность полностью собрана. Цена — задержка на один кадр между спавном и первым тиком поведения. Для толпы это несущественно.

Второе преимущество: тег остаётся видимым инструментом отладки. Можно посмотреть, у каких агентов дерево запущено, и это видно прямо в составе архетипа.

13.5. `UMassStateTreeProcessor` : объявление

cpp

```
/**
 * The processor that the UMassStateTreeSubsystem will instantiate for every
 * unique StateTree Mass-requirements.
 * The user is not expected to instantiate these processors manually, but a
 * project-specific extension can be implemented.
 * It needs to derive from UMassStateTreeProcessor and set as the value of
 * UMassStateTreeSubsystem.DynamicProcessorClass.
 */
UCLASS(MinimalAPI)
class UMassStateTreeProcessor : public UMassSignalProcessorBase
{
```

```
GENERATED_BODY()
```

```
public:  
    MASSAIBEHAVIOR_API UMassStateTreeProcessor(const FObjectInitializer&  
ObjectInitializer = FObjectInitializer::Get());
```

Комментарий повторяет то, что мы знаем из главы 9: экземпляров много, создаёт их подсистема, класс подменяется через настройки.

Конструктор с `FObjectInitializer` — нужен для динамического создания через `NewObject<>()` с указанием класса.

Ключевое отличие от предыдущих двух: база — `UMassSignalProcessorBase`, а не `UMassProcessor`.

Что даёт `UMassSignalProcessorBase`

Эта база превращает обычный процессор в событийный. Она:

- держит список сигналов, на которые подписан процессор;
- накапливает сущности, получившие эти сигналы, за кадр;
- вызывает не `Execute()`, а `SignalEntities()` — и только для накопленных сущностей;
- сама фильтрует по запросу процессора.

Подписка делается в `InitializeInternal()`:

cpp

```
// реконструкция  
void UMassStateTreeProcessor::InitializeInternal(UObject& Owner, const  
TSharedRef<FMassEntityManager>& EntityManager)  
{  
    Super::InitializeInternal(Owner, EntityManager);  
  
    SubscribeToSignal(UE::Mass::Signals::StateTreeActivate);  
    SubscribeToSignal(UE::Mass::Signals::NewStateTreeTaskRequired);  
    SubscribeToSignal(UE::Mass::Signals::DelayedTransitionWakeup);  
    SubscribeToSignal(UE::Mass::Signals::LookAtFinished);  
    SubscribeToSignal(UE::Mass::Signals::StandTaskFinished);  
    SubscribeToSignal(UE::Mass::Signals::AnimateTaskFinished);  
    SubscribeToSignal(UE::Mass::Signals::ContextualAnimTaskFinished);  
}
```

Вот где нужны все имена из `MassStateTreeTypes.h` (глава 5). Если ваш проект вводит собственные сигналы завершения, подписку придётся добавить — а для этого нужен свой наследник процессора.

Это, пожалуй, главная причина заводить `DynamicProcessorClass`. Не профилирование и не порядок в графе, а именно подписка на собственные сигналы.

13.6. Настройка требований

cpp

```
/**
 * Called to configure dynamic processor's additional requirements that will
 * ensure its located
 * properly within Mass's processing graph. Calling this function is allowed
 * only until the
 * processor is Initialized. The function will ensure that's the case.
 */
MASSAIBEHAVIOR_API void SetExecutionRequirements(const
FMassFragmentRequirements& FragmentRequirements, const
FMassSubsystemRequirements& SubsystemRequirements);

/**
 * Adds StateTree to the collection of the assets this specific processor
 * instance will handle.
 */
MASSAIBEHAVIOR_API void AddHandledStateTree(TNotNull<const UStateTree*>
StateTree);
```

Оба метода разбирались в главе 8; здесь отметим только контракт:

`SetExecutionRequirements()` можно звать **только до инициализации**, и функция это проверяет.

Причина понятна: требования участвуют в построении графа. Изменить их после того, как граф построен, — значит получить граф, не соответствующий реальности.

13.7. `SignalEntities()` — точка входа

cpp

```
MASSAIBEHAVIOR_API virtual void SignalEntities(FMassEntityManager&
EntityManager, FMassExecutionContext& Context, FMassSignalNameLookup&
EntitySignals) override;
```

Вместо `Execute()`. Третий параметр — `FMassSignalNameLookup`, справочник «какие сигналы пришли какой сущности».

Зачем он? Дерево может реагировать по-разному в зависимости от причины пробуждения:

cpp

```
// реконструкция фрагмента реализации
const bool bIsActivation = EntitySignals.DoesEntityContainSignal(Entity,
UE::Mass::Signals::StateTreeActivate);

if (bIsActivation)
{
    StateTreeContext.Start();
}
else
{
    StateTreeContext.Tick(DeltaTime);
}
```

Различие `Start()` и `Tick()` — минимально необходимое использование. Более тонкое: если пришёл `LookAtFinished`, можно не тикать всё дерево, а только обработать завершение конкретной задачи. В базовой реализации этого нет, но возможность есть.

Обратите внимание: **одна сущность может получить несколько сигналов за кадр**. Подсистема накапливает их, а процессор обрабатывает агента один раз. Это правильно — тикать дерево дважды за кадр не нужно.

Структура реализации

Собирая всё из глав 6, 9 и 12, получаем:

cpp

```
// реконструкция
void UMassStateTreeProcessor::SignalEntities(FMassEntityManager&
EntityManager, FMassExecutionContext& Context, FMassSignalNameLookup&
EntitySignals)
{
    UMassStateTreeSubsystem& Subsystem =
Context.GetMutableSubsystemChecked<UMassStateTreeSubsystem>();
    const double CurrentTime = GetWorld()->GetTimeSeconds();

    auto ProcessChunk = [this, &Subsystem, &EntitySignals, CurrentTime]
```

```

(FMassExecutionContext& Context)
{
    const FMassStateTreeSharedFragment& SharedFragment =
Context.GetConstSharedFragment<FMassStateTreeSharedFragment>();
    if (!HandledStateTrees.Contains(SharedFragment.StateTree))
    {
        return;    // этот ассет обслуживает другой процессор
    }

    const TArrayView<FMassStateTreeInstanceFragment> InstanceFragments =
        Context.GetMutableFragmentView<FMassStateTreeInstanceFragment>();

    FMassStateTreeExecutionContext StateTreeContext(Subsystem,
*SharedFragment.StateTree, /*временно*/ Dummy, Context);

    for (int32 i = 0; i < Context.GetNumEntities(); ++i)
    {
        FMassStateTreeInstanceFragment& InstanceFragment =
InstanceFragments[i];
        if (!InstanceFragment.InstanceHandle.IsValid())
        {
            continue;
        }

        FStateTreeInstanceData* InstanceData =
Subsystem.GetInstanceData(InstanceFragment.InstanceHandle);
        if (InstanceData == nullptr)
        {
            continue;
        }

        const FMassEntityHandle Entity = Context.GetEntity(i);
        const float DeltaTime = static_cast<float>(CurrentTime -
InstanceFragment.LastUpdateTimeInSeconds);
        InstanceFragment.LastUpdateTimeInSeconds = CurrentTime;

        StateTreeContext.SetEntity(Entity);
        // ... подставить InstanceData, выбрать Start() или
Tick(DeltaTime)
    }
};

if (bProcessEntitiesInParallel)
{
    EntityQuery.ParallelForEachEntityChunk(Context, ProcessChunk);
}

```

```

else
{
    EntityQuery.ForEachEntityChunk(Context, ProcessChunk);
}
}

```

Здесь видно всё, о чём мы говорили: двойная проверка хендла (глава 6), расчёт дельты по `LastUpdateTimeInSeconds` (глава 6), один контекст на чанк с перенацеливанием через `SetEntity()` (глава 12).

13.8. Фильтрация по ассету

cpp

```

/** The assets handled by this processor – entities utilizing any of these
assets will be processed by this processor */
UPROPERTY()
TArray<TObjectPtr<const UStateTree>> HandledStateTrees;

```

Момент, который легко упустить. Процессор создан под уникальный **набор требований**, а не под конкретный ассет. Значит, его запрос совпадёт с архетипами агентов, использующих **другие** ассеты с такими же требованиями — но обслуживаемые другим процессором с тем же хешем... нет, стоп.

Разберёмся точнее. Хеш требований — ключ в `RequirementsHashToProcessor`. Если два ассета дали одинаковый хеш, они попадают **в один** процессор, и оба лежат в `HandledStateTrees`. Тогда зачем проверка?

Затем, что запрос процессора построен из требований, а требования у разных наборов могут **пересекаться**. Процессор с требованиями {A, B} увидит в своём запросе и агентов с требованиями {A, B, C}, если те помечены как `Optional`. Проверка по `HandledStateTrees` отсекает чужих.

Проверка делается **один раз на чанк**, а не на агента: все агенты чанка гарантированно имеют один общий фрагмент, а значит один ассет.

Практическое следствие: **чем больше разных ассетов в мире, тем больше холостых проверок**. Если у вас сотня деревьев с похожими требованиями, процессоры будут перебирать чужие чанки. Обычно это дёшево, но при экстремальном разнообразии стоит замерить.

13.9. ExportRequirements()

cpp

```

MASSAIBEHAVIOR_API virtual void ExportRequirements(FMassExecutionRequirements&
OutRequirements) const override;

/**
 * Stores the additional requirements as configured by
 SetExecutionRequirements.
 * These requirements ensure the processor will be placed at the right
 location in the processing graph
 * to avoid data races.
 */
FMassExecutionRequirements ExecutionRequirements;

```

Механизм из главы 8. Подчеркну ещё раз ключевую мысль из комментария: требования нужны **не для доступа к данным**, а исключительно для размещения в графе. Доступ идёт через контекст задачи, минуя запрос процессора.

Поле не UPROPERTY — FMassExecutionRequirements содержит указатели на UScriptStruct, но это статические объекты, которые GC не собирает.

13.10. Параллельный обход

cpp

```

/** Configures whether parallel update for FMassArchetypeChunks should be used
 instead of the default single threaded update (i.e.,
 ParallelForEachEntityChunk instead of ForEachEntityChunk). */
UPROPERTY(EditDefaultsOnly, Category = Processor, config)
bool bProcessEntitiesInParallel = false;

```

По умолчанию **выключено**. Разберём, почему, и когда включать безопасно.

Что параллелится

ParallelForEachEntityChunk раздаёт **чанки** разным потокам. Внутри чанка обход последовательный.

Это важно для главы 12: один контекст создаётся на чанк, значит каждый поток получает свой контекст. Гонки по контексту нет.

Что мешает

Подсистема сигналов. Вспомните трейты из MassSignalSubsystem.h:

cpp

```

template<>
struct TMassExternalSubsystemTraits<UMassSignalSubsystem> final
{
    enum
    {
        GameThreadOnly = false,
        // @todo this subsystem not being thread-safe when writing is an
        obstacle in
        // parallelizing multiple processors
        ThreadSafeWrite = false,
    };
};

```

`ThreadSafeWrite = false`. А задачи шлют сигналы постоянно — и `BeginDelayedTransition()` тоже. Если два потока одновременно вызовут `DelaySignalEntity()`, массив `DelayedSignals` может испортиться.

Обход есть: `deferred`-варианты (`SignalEntityDeferred`) кладут команду в командный буфер, а он потокобезопасен. Но задачи движка используют прямые вызовы.

Аллокация instance data. Из главы 9: `AllocateInstanceData()` не поддерживает параллельный вызов. Если задача в `EnterState()` спавнит новую сущность с деревом, при параллельном обходе получите срабатывание MT-детектора.

Пользовательские подсистемы. Задача, обращающаяся к подсистеме без `ThreadSafeWrite`, при параллельном обходе создаёт гонку.

Когда включать

Осторожный чек-лист:

1. Все задачи вашего дерева используют только `deferred`-сигналы или не шлют сигналов вовсе.
2. Ни одна задача не создаёт и не уничтожает сущности напрямую (только через `Context.Defer()`).
3. Все подсистемы, к которым обращаются задачи, помечены `ThreadSafeWrite = true` или используются только на чтение.
4. Агентов действительно много — иначе накладные расходы на раздачу задач съедят выигрыш.

При соблюдении всех четырёх пунктов выигрыш реальный: тик деревьев — одна из самых тяжёлых частей симуляции толпы.

Включается через конфиг (спецификатор `config`):

ini

```
[/Script/MyGame.MyStateTreeProcessor]  
bProcessEntitiesInParallel=True
```

Или в C++ конструкторе своего наследника.

13.11. Как три процессора располагаются в графе

Порядок исполнения важен, и он не случаен.

Активация должна отработать до основного процессора — иначе агент проснётся на кадр позже. Формально это не ошибка (сигнал накопится и сработает в следующем кадре), но задержка накапливается.

Основной процессор должен идти до процессоров-исполнителей (движение, взгляд, анимация) — потому что он записывает намерения, а те их читают. Это обеспечивается автоматически через объявленные требования: процессор пишет `FMassMoveTargetFragment`, процессор движения его читает, солвер выстраивает порядок.

Деструктор — наблюдатель, он не в основном графе; срабатывает при флеше командного буфера.

Порядок можно подкорректировать через `ExecutionOrder` в своём наследнике:

cpp

```
// в конструкторе наследника  
ExecutionOrder.ExecuteAfter.Add(TEXT("MassPerceptionProcessor"));  
ExecutionOrder.ExecuteBefore.Add(TEXT("MassMovementProcessor"));
```

Полезно, когда солвер не может вывести порядок сам — например, когда связь между системами идёт не через фрагменты, а через сигналы.

13.12. Диагностика

«Дерево не запускается вообще». Проверьте наличие `FMassStateTreeActivatedTag` у агента. Если тега нет — процессор активации не отработал: возможно, `ExecutionFlags` не совпали (вы в PIE-клиенте), или запрос не находит агента.

«Тег есть, но дерево не тикает». Значит, сигнал ушёл, но процессор его не обработал. Варианты: ассет не в `HandledStateTrees` (процессор для него не создан — смотрите главу 9), или архетип агента не проходит запрос процессора.

«Тикнуло один раз и замерло». Классика: задача вернула `Running` и не обеспечила себе пробуждение. См. главу 12, раздел про `BeginDelayedTransition`.

«Агенты умирают, а ресурсы не освобождаются». Деструктор не отработал. Проверьте, что он зарегистрирован и что `ExecutionFlags` позволяют ему работать в вашем режиме.

Ассерт MT-детектора при параллельном обходе. См. раздел 13.10 — вы нарушили один из четырёх пунктов чек-листа.

«После добавления узла порядок процессоров изменился». Ожидаемо: состав требований дерева влияет на граф (глава 8, раздел 8.5).

13.13. Итог главы

Процессор	База	Когда работает
<code>UMassStateTreeActivationProcessor</code>	<code>UMassProcessor</code>	Каждый кадр, для агентов без тега
<code>UMassStateTreeProcessor</code>	<code>UMassSignalProcessorBase</code>	Только для сигнализированных агентов
<code>UMassStateTreeFragmentDestructor</code>	<code>UMassObserverProcessor</code>	При удалении фрагмента

Что унести в практику:

1. **Подписка на сигналы — в `InitializeInternal()`.** Свои сигналы требуют своего наследника процессора.
2. `bProcessEntitiesInParallel` **включайте по чек-листу**, а не по умолчанию.
3. `Stop()` **перед `FreeInstanceData()` обязателен** — иначе утечки логических ресурсов.
4. **Тег активации — защёлка**, ставится один раз через командный буфер.

Что дальше

Глава 14 — `UMassSignalSubsystem`, последняя в части III. Разберём все двенадцать методов отправки сигналов и то, чем они отличаются: немедленные, отложенные, `deferred` и их комбинации. Посмотрим на устройство `FDelayedSignal` и на то, как подсистема тикает свою очередь. Разберём делегаты `GetSignalDelegateByName()`, детектор доступа, и главное — трейты `TMassExternalSubsystemTraits` с их честным

@todo про потокобезопасность. Заодно соберём практические рекомендации: когда какой метод использовать и как не разбудить мёртвого агента.

Глава 14. UMassSignalSubsystem

Последняя глава части III. Подсистема сигналов — это нервная система Mass: всё, что заставляет деревья просыпаться, проходит через неё.

14.1. Заголовки и делегат

cpp

```
#if UE_ENABLE_INCLUDE_ORDER_DEPRECATED_IN_5_6
#include "MassEntityManager.h"
#endif
#include "Mass/EntityHandle.h"
#include "MassSubsystemBase.h"
#include "Misc/MTAccessDetector.h"
#include "Mass/ExternalSubsystemTraits.h"

struct FMassExecutionContext;

namespace UE::MassSignal
{
    DECLARE_MULTICAST_DELEGATE_TwoParams(FSignalDelegate, FName
/*SignalName*/, TConstArrayView<FMassEntityHandle> /*Entities*/);
}
```

Снова замена тяжёлого `MassEntityManager.h` на компактный `Mass/EntityHandle.h`.

Делегат — многоадресный, с двумя параметрами: имя сигнала и **массив** сущностей. Не одна сущность, а сразу список. Это первый признак того, что подсистема спроектирована на пакетную работу.

`TConstArrayView` вместо `TArray` — представление без владения. Ноль копирований, подписчик просто читает чужой буфер.

14.2. Объявление класса

cpp

```
/**
 * A subsystem for handling Signals in Mass
```

```
*/
UCLASS(MinimalAPI)
class UMassSignalSubsystem : public UMassTickableSubsystemBase
```

База — `UMassTickableSubsystemBase`, а не `UMassSubsystemBase`, как у `UMassStateTreeSubsystem`. Разница в наличии тика:

cpp

```
MASSIGNALS_API virtual void Tick(float DeltaTime) override;
MASSIGNALS_API virtual TStatId GetStatId() const override;
```

Тик нужен ровно для одного: отслеживать отложенные сигналы и отправлять те, у которых истёк срок. Всё остальное происходит по вызову.

`GetStatId()` — обязательный метод тикающего объекта, возвращает идентификатор для профилировщика. Именно по нему вы найдёте подсистему в `stat game`.

14.3. Делегаты по имени

cpp

```
/**
 * Retrieve the delegate dispatcher from the signal name
 * @param SignalName is the name of the signal to get the delegate dispatcher
 * from
 */
UE::MassSignal::FSignalDelegate& GetSignalDelegateByName(FName SignalName)
{
    return NamedSignals.FindOrAdd(SignalName);
}
```

Метод `inline` и делает ровно одно: находит или создаёт делегат для имени.

cpp

```
TMap<FName, UE::MassSignal::FSignalDelegate> NamedSignals;
```

`FindOrAdd` — «создать при отсутствии». Никакой предварительной регистрации сигналов не требуется: подписался на новое имя — запись появилась. Это то, о чём говорилось в главе 5: каталог сигналов расширяем без изменения кода движка.

Именно через этот метод работает `SubscribeToSignal()` в `UMassSignalProcessorBase`:

c++

```
// реконструкция
void UMassSignalProcessorBase::SubscribeToSignal(FName SignalName)
{
    SignalSubsystem->GetSignalDelegateByName(SignalName)
        .AddUObject(this, &UMassSignalProcessorBase::OnSignalReceived);
}
```

Возврат по неконстантной ссылке — подписчик должен иметь возможность добавить себя.

Практическое следствие

Подписка — это связь «имя → процессор». Если никто не подписан на сигнал, отправка проходит вхолостую: делегат пустой, вызов ничего не делает.

Это тихий отказ. Отправили сигнал `MyCustomFinished`, а забыли подписать процессор — никакой ошибки, просто агент не проснётся. Одна из самых неприятных категорий багов в Mass, потому что диагностируется только чтением кода.

14.4. Двенадцать методов отправки

Подсистема предлагает восемь публичных методов отправки, различающихся по трём независимым осям. Разложим их в таблицу:

Режим доставки	Одна сущность	Множество сущностей	Назначение и контекст применения
Немедленно	<code>SignalEntity</code>	<code>SignalEntities</code>	Прямая синхронная отправка. Исполняется в том же кадре в <code>Game Thread</code> .
С задержкой	<code>DelaySignalEntity</code>	<code>DelaySignalEntities</code>	Отложенная отправка по таймеру. Используется д. кулдаунов и таймаутов.
Отложено (Буфер)	<code>SignalEntityDeferred</code>	<code>SignalEntitiesDeferred</code>	Запись в <code>CommandBuffer</code> .

Режим доставки	Одна сущность	Множество сущностей	Назначение и контекст применения
			Потокобезопасная отправка из параллельных потоков.
С задержкой + отложено	DelaySignalEntityDeferred	DelaySignalEntitiesDeferred	Буферизируем запуск таймера. Для безопасного планирования сигналов из сторонних потоков.

Три оси: количество, задержка во времени, отложенность через командный буфер. Разберём каждую.

Ось «количество»

cpp

```
MASSSIGNALS_API void SignalEntity(FName SignalName, const FMassEntityHandle Entity);
MASSSIGNALS_API void SignalEntities(FName SignalName, TConstArrayView<FMassEntityHandle> Entities);
```

Разница не косметическая. Версия с массивом делает **один** поиск делегата в карте и **один** широковебательный вызов. Версия для одной сущности — то же самое, но для одного элемента.

Тысяча вызовов `SignalEntity()` в цикле — это тысяча поисков в `TMap` и тысяча вызовов делегата. Один вызов `SignalEntities()` с массивом из тысячи — один поиск, один вызов.

Правило: если сущностей больше одной, всегда собирайте массив. Именно так делает процессор активации (глава 13).

Ось «задержка»

cpp

```

/**
 * Inform a single entity of a signal being raised in a certain amount of
 seconds
 * @param DelayInSeconds is the amount of time before signaling the entity
 */
MASSSIGNALS_API void DelaySignalEntity(FName SignalName, const
FMassEntityHandle Entity, const float DelayInSeconds);
MASSSIGNALS_API void DelaySignalEntities(FName SignalName,
TConstArrayView<FMassEntityHandle> Entities, const float DelayInSeconds);

```

Сигнал не отправляется сразу, а кладётся в очередь с меткой времени. Это то, что использует `BeginDelayedTransition()` из главы 12.

`float` для задержки — здесь этого достаточно: задержки редко превышают минуты, а точность `float` на таких значениях избыточна. Сравните с `double` для абсолютного времени в главе 6: там значение растёт неограниченно, здесь — нет.

Ось «отложенность»

cpp

```

/**
 * Inform single entity of a signal being raised asynchronously using the Mass
 Command Buffer
 * @param Context is the Entity System execution context to push the async
 command
 */
MASSSIGNALS_API void SignalEntityDeferred(FMassExecutionContext& Context,
FName SignalName, const FMassEntityHandle Entity);
MASSSIGNALS_API void SignalEntitiesDeferred(FMassExecutionContext& Context,
FName SignalName, TConstArrayView<FMassEntityHandle> Entities);

```

Первым параметром идёт `FMassExecutionContext&` — и это подсказка о назначении. `Deferred`-версии предназначены для вызова **изнутри обхода чанков**.

Почему это важно? Обычный `SignalEntity()` немедленно вызывает делегат, а тот вызывает процессоры-подписчики. Если это происходит посреди `ForEachEntityChunk`, вы получаете реентерабельный вход в систему обработки со всеми вытекающими: изменение состава сущностей под ногами у итератора, вложенные блокировки, порча состояния.

`Deferred`-версия кладёт команду в `FMassCommandBuffer` и возвращается немедленно. Команда исполнится, когда буфер флашится, — в безопасной точке кадра.

И, что не менее важно, **командный буфер потокобезопасен**. Это единственный корректный способ послать сигнал из параллельного обхода (глава 13, раздел 13.10).

Комбинация

cpp

```
MASSIGNALS_API void DelaySignalEntityDeferred(FMassExecutionContext& Context,
FName SignalName, const FMassEntityHandle Entity, const float DelayInSeconds);
MASSIGNALS_API void DelaySignalEntitiesDeferred(FMassExecutionContext&
Context, FName SignalName, TConstArrayView<FMassEntityHandle> Entities, const
float DelayInSeconds);
```

Обе оси сразу: команда «поставить отложенный сигнал» кладётся в буфер. Само добавление в очередь произойдёт при флеше, отсчёт задержки начнётся оттуда.

Это самый безопасный вариант для использования внутри задач — и, вероятно, то, что стоило бы использовать в `BeginDelayedTransition()`.

14.5. Как выбрать метод

Практическое дерево решений:

Вы внутри `Execute()` или `SignalEntities()` процессора? → используйте deferred-версию.

Вы в параллельном обходе? → обязательно deferred.

Вы в задаче `StateTree`? → задача выполняется внутри обхода процессора, значит deferred.

Вы в обычном игровом коде (актор, подсистема, колбэк)? → можно прямой вызов.

Сущностей больше одной? → соберите массив, используйте версию для многих.

Нужна задержка? → Delay*-вариант.

Формально прямой вызов из задачи часто «работает»: `UMassSignalSubsystem` накапливает сущности, а фактическая обработка происходит позже. Но это работает случайно, а не по контракту. При параллельном обходе сломается гарантированно.

14.6. Внутреннее устройство отложенных сигналов

cpp


```
protected:
    /** Multithreading access detector to validate accesses to the list of
    delayed signals */
    UE_MT_DECLARE_RW_ACCESS_DETECTOR(DelayedSignalsAccessDetector);

    TMap<FName, UE::MassSignal::FSignalDelegate> NamedSignals;

    struct FDelayedSignal
    {
        FName SignalName;
        TArray<FMassEntityHandle> Entities;
        double TargetTimestamp;
    };

    TArray<FDelayedSignal> DelayedSignals;

    UPROPERTY(transient)
    TObjectPtr<UWorld> CachedWorld;
```

FDelayedSignal

Три поля: имя, список сущностей, момент срабатывания.

TargetTimestamp — **абсолютное время**, а не оставшаяся длительность. Разница принципиальная: с абсолютным временем не нужно каждый тик уменьшать счётчики у всех записей, достаточно сравнить с текущим временем.

Тип — `double`, по той же причине, что в главе 6: игровое время растёт неограниченно, `float` теряет точность.

Обратите внимание, что запись хранит **массив** сущностей. Один вызов `DelaySignalEntities()` создаёт одну запись, а не N. Экономия и памяти, и работы при обходе.

Тик

cpp

```
MASSSIGNALS_API virtual void Tick(float DeltaTime) override;
```

Реализация примерно такая:

cpp

```
// реконструкция
void UMassSignalSubsystem::Tick(float DeltaTime)
{
    UE_MT_SCOPED_WRITE_ACCESS(DelayedSignalsAccessDetector);

    const double CurrentTime = CachedWorld->GetTimeSeconds();

    for (int32 i = DelayedSignals.Num() - 1; i >= 0; --i)
    {
        if (DelayedSignals[i].TargetTimestamp <= CurrentTime)
        {
            SignalEntities(DelayedSignals[i].SignalName,
DelayedSignals[i].Entities);
            DelayedSignals.RemoveAtSwap(i, EAllowShrinking::No);
        }
    }
}
```

Два приёма стоит отметить.

Обход с конца. Позволяет удалять элементы прямо в цикле, не сбивая индексацию.

`RemoveAtSwap`. Удаление за $O(1)$: последний элемент переставляется на место удалённого. Порядок нарушается, но для отложенных сигналов он не важен — каждая запись срабатывает по своему времени.

Стоимость тика — линейный проход по числу **записей**, а не сущностей. Тысяча агентов, ждущих одинаковую задержку и засигналенных одним вызовом, — это одна запись.

Но если каждый агент ставит себе отложенный переход индивидуально (а именно так работает `BeginDelayedTransition()`), записей будет столько же, сколько агентов. Десять тысяч записей, обходимых каждый кадр, — заметная, хотя и не катастрофическая нагрузка. Это одна из точек, где при масштабировании стоит смотреть профилировщик.

Детектор доступа

cpp

```
UE_MT_DECLARE_RW_ACCESS_DETECTOR(DelayedSignalsAccessDetector);
```

Обратите внимание: обычный `RW`, без суффиксов `TS` (transactionally safe) и `MRSW`. Более простой вариант, чем у подсистемы `StateTree` (глава 9) и у `instance data` (глава 10).

Он ловит одновременный доступ к `DelayedSignals` из разных потоков. Как и все `UE_MT_*`, это отладочная проверка, а не синхронизация: в шипящей сборке компилируется в ничто.

CachedWorld

cpp

```
UPROPERTY(transient)
TObjectPtr<UWorld> CachedWorld;
```

Мир нужен для получения текущего времени. Кэшируется, чтобы не звать `GetWorld()` каждый тик — вызов не бесплатный, а тик частый.

`transient` (в нижнем регистре — стиль допускает оба написания) — восстанавливается при инициализации.

14.7. Трейты потокобезопасности

cpp

```
template<>
struct TMassExternalSubsystemTraits<UMassSignalSubsystem> final
{
    enum
    {
        GameThreadOnly = false,
        // @todo this subsystem not being thread-safe when writing is an
        obstacle in
        // parallelizing multiple processors
        ThreadSafeWrite = false,
    };
};
```

Этот блок мы упоминали уже трижды — в главах 8, 12 и 13. Теперь разберём его до конца.

`TMassExternalSubsystemTraits` — механизм, которым подсистема сообщает Mass о своих ограничениях. Два флага:

`GameThreadOnly = false` — подсистему можно **читать** из любого потока. Хорошая новость: обход чанков может обращаться к ней параллельно.

`ThreadSafeWrite = false` — **писать** из нескольких потоков нельзя. Отправка сигнала — это запись: она меняет `DelayedSignals` или вызывает делегаты, которые меняют состояние процессоров.

Комментарий Epic — редкий случай прямого признания архитектурного долга прямо в заголовке: небезопасность записи мешает параллелить несколько процессоров.

Что это значит на практике

Планировщик Mass, видя `ThreadSafeWrite = false`, обязан **сериализовать** все процессоры, которые объявили запись в эту подсистему. А объявляют её практически все AI-процессоры: любая задача, шлющая сигнал, добавляет

```
Builder.AddReadWrite<UMassSignalSubsystem>().
```

Результат: цепочка процессоров, которая могла бы идти параллельно, выстраивается в линию.

Обход существует и мы его знаем — `deferred`-версии через командный буфер. Но пока задачи движка используют прямые вызовы, ограничение остаётся.

Если вы упёрлись в этот потолок, есть два пути:

1. **Свои задачи писать на `deferred`-сигналах** и объявлять подсистему как `ReadOnly` в `GetDependencies()`. Тогда ваши процессоры не будут конфликтовать между собой.
2. **Ждать рефакторинга Epic** — `@todo` в коде движка обычно означает, что работа запланирована.

14.8. Жизненный цикл

c++

```
protected:
    // USubsystem implementation Begin
    MASSSIGNALS_API virtual void Initialize(FSubsystemCollectionBase&
Collection) override;
    MASSSIGNALS_API virtual void Deinitialize() override;
    // USubsystem implementation End
```

`Initialize()` кэширует мир и, вероятно, регистрирует тик. `Deinitialize()` очищает очередь и делегаты.

Важный момент: при уничтожении мира отложенные сигналы просто пропадают. Это корректно — сущностей, которым они адресованы, тоже больше нет.

14.9. Что происходит при сигнале мёртвой сущности

Ситуация штатная: агент поставил отложенный сигнал на три секунды и умер через одну.

Цепочка защиты трёхуровневая:

Уровень 1: подписчик. `UMassSignalProcessorBase` накапливает сущности и перед обработкой фильтрует их по запросу. Мёртвая сущность не пройдёт фильтр — её архетипа больше не существует.

Уровень 2: хендл сущности. Если индекс переиспользован новой сущностью, серийный номер не совпадёт, и `IsValidEntity()` вернёт `false`.

Уровень 3: хендл instance data. Даже если сигнал каким-то образом дошёл, проверка поколения в `IsValidHandle()` (глава 9) отсекает обращение к чужим данным.

Три независимых барьера. Именно поэтому в коде процессоров нет `check()` на валидность — ситуация ожидаемая и обрабатывается тихо.

Практический вывод: **отменять отложенные сигналы при смерти агента не требуется.** Это просто небольшая холостая работа при обходе очереди.

14.10. Свои сигналы: полный рецепт

Соберём всё, что нужно для добавления собственного сигнала.

Шаг 1. Объявить имя.

cpp

```
// MyGameSignals.h
#pragma once
#include "UObject/NameTypes.h"

namespace MyGame::Signals
{
    const FName CombatTargetLost = FName(TEXT("CombatTargetLost"));
    const FName ReachedDestination = FName(TEXT("ReachedDestination"));
}
```

Шаг 2. Завести свой процессор `StateTree`. Без этого подписаться не выйдет.

cpp

```
UCLASS()
class UMyStateTreeProcessor : public UMassStateTreeProcessor
```

```

{
    GENERATED_BODY()

protected:
    virtual void InitializeInternal(UObject& Owner, const
TSharedRef<FMassEntityManager>& EntityManager) override
    {
        Super::InitializeInternal(Owner, EntityManager);

        SubscribeToSignal(MyGame::Signals::CombatTargetLost);
        SubscribeToSignal(MyGame::Signals::ReachedDestination);
    }
};

```

Шаг 3. Прописать класс в настройках.

ini

```

[/Script/MassAIBehavior.MassBehaviorSettings]
DynamicStateTreeProcessorClass=/Script/MyGame.MyStateTreeProcessor

```

Шаг 4. Отправлять сигнал. Из процессора-исполнителя:

cpp

```

void UMyPerceptionProcessor::Execute(FMassEntityManager& EntityManager,
FMassExecutionContext& Context)
{
    TArray<FMassEntityHandle> LostTarget;

    EntityQuery.ForEachEntityChunk(Context, [&LostTarget]
(FMassExecutionContext& Context)
    {
        for (int32 i = 0; i < Context.GetNumEntities(); ++i)
        {
            if (/* цель потеряна */)
            {
                LostTarget.Add(Context.GetEntity(i));
            }
        }
    });

    if (!LostTarget.IsEmpty())
    {
        SignalSubsystem->SignalEntities(MyGame::Signals::CombatTargetLost,

```

```
LostTarget);  
    }  
}
```

Здесь прямой вызов допустим, потому что он **после** обхода, а не внутри.

Шаг 5. Реагировать в задаче. Задача, ожидающая события, просто вернёт `Succeeded` на ближайшем тике, обнаружив изменение во фрагменте. Либо — если нужна реакция именно на сигнал — процессор может различить его через `FMassSignalNameLookup` (глава 13).

14.11. Типичные ошибки

Забывать подписаться. Сигнал уходит, никто не слушает, агент не просыпается. Ошибки нет, диагностика только чтением кода. Заведите привычку: новый сигнал — сразу подписка.

Прямой вызов из задачи. Работает до включения параллельного обхода, потом ломается. Используйте `deferred`.

Цикл `SignalEntity()` вместо `SignalEntities()`. Тысячекратный оверхед на пустом месте.

Ожидание сигнала, который никто не шлёт. Задача вернула `Running` в расчёте на `MyCustomFinished`, а процессор-исполнитель его не отправляет (не реализован, отфильтрован, работает в другом режиме). Агент замирает.

Отложенный сигнал с нулевой задержкой. `DelaySignalEntity(..., 0.f)` создаст запись, которая сработает на следующем тике подсистемы. Если вам нужно «сейчас» — используйте обычный `SignalEntity()`.

Опечатка в имени. `FName` не проверяется компилятором. `"LookAtFinished"` и `"LookatFinished"` — разные сигналы. Всегда используйте объявленные константы, никогда не пишите строку на месте вызова.

14.12. Итог главы

Элемент	Роль
<code>GetSignalDelegateByName()</code>	Подписка без регистрации; расширяемый каталог сигналов
<code>SignalEntity</code> / <code>SignalEntities</code>	Немедленная отправка; версия с массивом на порядок дешевле

Элемент	Роль
Delay*	Отложенная отправка; основа <code>BeginDelayedTransition()</code>
*Deferred	Через командный буфер; единственный безопасный вариант внутри обхода и в параллели
FDelayedSignal	Имя + массив сущностей + абсолютная метка времени
Tick()	Линейный проход по записям, <code>RemoveAtSwap</code> для удаления
TMassExternalSubsystemTraits	Чтение параллельно можно, запись — нет; известное узкое место

Три правила на практику:

1. **Внутри обхода — только deferred.**
2. **Много сущностей — один вызов с массивом.**
3. **Каждый новый сигнал — сразу подписка, иначе он уйдёт в пустоту.**

Что дальше

Часть III закончена: мы разобрали, где живут данные, как строится контекст, кто тикает деревья и что их будит. С главы 15 начинается часть IV — практическая.

Первой пойдёт анатомия задачи: `FStateTreeTaskBase` целиком. Все восемь битовых флагов и что каждый из них реально меняет в поведении, все пять виртуальных методов с разбором того, когда именно они вызываются и в каком порядке, `TransitionHandlingPriority`, редакторные поля завершения, а также отладочный `GetDebugInfo()`. Эта глава — прямая подготовка к разбору `FMassLookAtTask` и `FMassZoneGraphPathFollowTask` в главах 16 и 17.

Глава 15. Анатомия задачи: `FStateTreeTaskBase`

Задача — основная рабочая единица `StateTree`. Файл `StateTreeTaskBase.h` короткий, 170 строк, но каждый флаг в нём меняет поведение исполнения, и понимать их нужно точно. Эта глава — фундамент для двух следующих, где мы разберём реальные Mass-задачи.

15.1. Объявление и конструктор

cpp

```
/**
 * Base struct for StateTree Tasks.
 * Tasks are logic executed in an active state.
 */
USTRUCT(meta = (Hidden))
struct FStateTreeTaskBase : public FStateTreeNodeBase
{
    GENERATED_BODY()

    FStateTreeTaskBase()
        : bShouldStateChangeOnReselect(true)
        , bShouldCallTick(true)
        , bShouldCallTickOnlyOnEvents(false)
        , bShouldCopyBoundPropertiesOnTick(true)
        , bShouldCopyBoundPropertiesOnExitState(true)
        , bShouldAffectTransitions(false)
        , bConsideredForScheduling(true)
        , bTaskEnabled(true)
        , bHasTaskCompletionDelegateDispatcher(false)
#ifdef WITH_EDITORONLY_DATA
        , bConsideredForCompletion(true)
        , bCanEditConsideredForCompletion(true)
#endif
    {
    }
}
```

Обратите внимание на форму: все флаги инициализируются в **списке инициализации конструктора**, а не значениями по умолчанию у полей. Причина в том, что это битовые поля (uint8 : 1), а для них инициализаторы по месту объявления в C++ поддерживаются, но исторически движок пишет их так.

Ключевая мысль: **разумные умолчания уже расставлены**. Обычная задача, которая тикает каждый кадр и не влияет на переходы, не требует настройки флагов вообще. Менять их нужно осознанно.

15.2. Пять виртуальных методов

Разберём каждый: когда вызывается, что должен делать, что вернуть.

EnterState

cpp

```

/**
 * Called when a new state is entered and task is part of active states.
 * @param Context Reference to current execution context.
 * @param Transition Describes the states involved in the transition
 * @return Succeed/Failed will end the state immediately and trigger to select
         new state, Running will carry on to tick the state.
 */
virtual EStateTreeRunStatus EnterState(FStateTreeExecutionContext& Context,
const FStateTreeTransitionResult& Transition) const
{
    return EStateTreeRunStatus::Running;
}

```

Вызывается один раз при входе в состояние. Порядок — **сверху вниз**: сначала задачи родительских состояний, потом дочерних.

Возврат Succeeded или Failed завершает состояние **немедленно**, не доходя до тика. Это удобный способ реализовать мгновенную задачу:

cpp

```

EStateTreeRunStatus FMyInstantTask::EnterState(FStateTreeExecutionContext&
Context, const FStateTreeTransitionResult& Transition) const
{
    FInstanceDataType& InstanceData = Context.GetInstanceData(*this);
    if (!DoTheThing(InstanceData))
    {
        return EStateTreeRunStatus::Failed;
    }
    return EStateTreeRunStatus::Succeeded;
}

```

Для Mass это **основной паттерн**. Вспомните главу 4: задача выражает намерение и уходит. Если намерение записано и ждать нечего — Succeeded прямо здесь.

Параметр Transition описывает, откуда пришли. Полезно, когда поведение зависит от предыдущего состояния.

ExitState

cpp

```

/**
 * Called when a current state is exited and task is part of active states.

```

```

*/
virtual void ExitState(FStateTreeExecutionContext& Context, const
FStateTreeTransitionResult& Transition) const
{
}

```

Зеркало `EnterState()` . Порядок — **снизу вверх**: сначала дочерние состояния, потом родительские.

Возвращаемого значения нет: выход отменить нельзя.

Это точка освобождения ресурсов. Всё, что задача захватила в `EnterState()` , здесь должно быть отпущено: занятые слоты, подписки, зарезервированные точки интереса. Именно ради корректного вызова `ExitState()` `UMassStateTreeFragmentDestructor` вызывает `Stop()` перед освобождением данных (глава 13).

Важная гарантия: `ExitState()` вызывается **всегда**, если был `EnterState()` . Даже при аварийной остановке дерева, даже при уничтожении агента.

Tick

cpp

```

/**
 * Called during state tree tick when the task is on active state.
 * Note: The method is called only if bShouldCallTick or
bShouldCallTickOnlyOnEvents is set.
 * @return Running status of the state: Running if still in progress,
Succeeded if execution is done and succeeded, Failed if execution is done and
failed.
 */
virtual EStateTreeRunStatus Tick(FStateTreeExecutionContext& Context, const
float DeltaTime) const
{
    return EStateTreeRunStatus::Running;
}

```

Вызывается на каждом тике дерева, пока состояние активно, — но только если разрешено флагами.

`DeltaTime` — время с прошлого тика **дерева**, а не кадра. В Mass это та самая величина, посчитанная по `LastUpdateTimeInSeconds` (глава 6). Может быть большой: десять секунд, если агент столько ждал.

Отсюда правило для Mass-задач: **не пишите код, предполагающий малый `DeltaTime`**. Интегрирование вида `Position += Velocity * DeltaTime` даст скачок. Такие вещи — работа процессоров, а не задач.

StateCompleted

cpp

```
/**
 * Called right after a state has been completed, but before new state has
 * been selected. StateCompleted is called in reverse order to allow to propagate
 * state to other Tasks that
 * are executed earlier in the tree. Note that StateCompleted is not called if
 * conditional transition changes the state.
 * @param CompletionStatus Describes the running status of the completed state
 * (Succeeded/Failed).
 * @param CompletedActiveStates Active states at the time of completion.
 */
virtual void StateCompleted(FStateTreeExecutionContext& Context, const
EStateTreeRunStatus CompletionStatus, const FStateTreeActiveStates&
CompletedActiveStates) const
{
}
```

Тонкий метод, и комментарий содержит две важные оговорки.

Обратный порядок — от листьев к корню. Задача-лист может записать результат в свои данные, а задача родителя это увидит и учтёт. Типичное применение: дочернее состояние «атаковать» сообщает родителю «бой», что цель уничтожена.

Не вызывается при условном переходе. Если состояние сменилось не потому, что завершилось, а потому что сработал переход по событию или тик, — `StateCompleted()` пропускается. Логично: состояние не завершилось, его прервали.

Отсюда практическое ограничение: **не полагайтесь на `StateCompleted()` для критичной логики**. Освобождение ресурсов — в `ExitState()`, который вызывается всегда.

TriggerTransitions

cpp

```
/**
 * Called when state tree triggers transitions. This method is called during
 * transition handling, before state's tick and event transitions are handled.
```

```

* Note: the method is called only if bShouldAffectTransitions is set.
*/
virtual void TriggerTransitions(FStateTreeExecutionContext& Context) const
{
};

```

Позволяет задаче самой инициировать переход, программно:

cpp

```

void FMyGuardTask::TriggerTransitions(FStateTreeExecutionContext& Context)
const
{
    const FInstanceDataType& InstanceData = Context.GetInstanceData(*this);
    if (InstanceData.ThreatLevel > 0.8f)
    {
        Context.RequestTransition(InstanceData.PanicState,
        EStateTreeTransitionPriority::High);
    }
}

```

Вызывается **до** обработки тиковых и событийных переходов — то есть у задачи есть приоритетное право высказаться.

Работает только при `bShouldAffectTransitions = true`. По умолчанию флаг выключен: большинству задач это не нужно, а лишний виртуальный вызов на каждом тике стоит денег.

Порядок среди задач определяется полем:

cpp

```

UPROPERTY()
EStateTreeTransitionPriority TransitionHandlingPriority =
EStateTreeTransitionPriority::Normal;

```

Задачи с более высоким приоритетом опрашиваются первыми.

15.3. Восемь флагов: подробно

Все флаги — битовые поля по одному биту. Одиннадцать флагов (с учётом редакторных) укладываются в два байта.

`bShouldStateChangeOnReselect`

cpp

```
/**
 * If set to true, the task will receive EnterState/ExitState even if the
 * state was previously active.
 * Generally this should be true for action type tasks, like playing
 * animation,
 * and false on state like tasks like claiming a resource that is expected to
 * be acquired on child states.
 * Default value is true.
 */
uint8 bShouldStateChangeOnReselect : 1;
```

Сценарий: агент в состоянии `Patrol` → `WalkToPoint`. Переход возвращает его в `Patrol`, и выбор снова приводит в `WalkToPoint`. Состояние «переизбрано».

При `true` (по умолчанию) задача получит `ExitState()` и снова `EnterState()`. Анимация начнётся заново — правильно для действия.

При `false` задача не заметит переизбрания и продолжит работать. Правильно для «удерживающих» задач.

Комментарий даёт точный критерий: **действие** → `true`, **удержание ресурса** → `false`.

Для Mass это особенно значимо. Задача, захватившая место в очереди или зарезервировавшая точку интереса, при `true` отпустит и тут же захватит снова — а между этими моментами ресурс может уйти другому агенту. Классический источник «мигания» поведения в толпе.

`bShouldCallTick` и `bShouldCallTickOnlyOnEvents`

cpp

```
/** If set to true, Tick() is called. Not ticking implies no property copy.
 * Default true. */
uint8 bShouldCallTick : 1;
/** If set to true, Tick() is called only when there are events. No effect if
 * bShouldCallTick is true. Not ticking implies no property copy. Default false.
 */
uint8 bShouldCallTickOnlyOnEvents : 1;
```

Три возможные комбинации:

bShouldCallTick	bShouldCallTickOnlyOnEvents	Поведение
true	любое	Тик каждый раз (второй флаг игнорируется)
false	true	Тик только при наличии событий
false	false	Тик не вызывается вообще

Фраза «Not ticking implies no property copy» важна. Привязки свойств копируются **перед** вызовом `Tick()`. Нет тика — нет копирования. Значит, задача не увидит обновлённых значений от эвалюаторов.

Для Mass это одна из главных оптимизаций. Задача, которая только записывает намерение и ждёт сигнала, тик не использует:

cpp

```
FMyIntentTask()
{
    bShouldCallTick = false;
}
```

Экономия: виртуальный вызов и копирование привязок для каждого агента на каждом тике. На десяти тысячах агентов — заметно.

Комбинация `false + true` полезна для задачи, реагирующей исключительно на события: она просыпается только когда есть что обрабатывать.

**bShouldCopyBoundPropertiesOnTick и
bShouldCopyBoundPropertiesOnExitState**

cpp

```
/** If set to true, copy the values of bound properties before calling Tick().
Default true. */
uint8 bShouldCopyBoundPropertiesOnTick : 1;
/** If set to true, copy the values of bound properties before calling
ExitState(). Default true. */
uint8 bShouldCopyBoundPropertiesOnExitState : 1;
```

Более тонкая настройка того же механизма. Задача тикает, но привязки не обновляет.

Когда это нужно: задача читает свои входные свойства только в `EnterState()`, а дальше работает с зафиксированными значениями. Копировать их каждый тик бессмысленно.

cpp

```
FMyTask()
{
    bShouldCopyBoundPropertiesOnTick = false;    // значения захвачены при
    входе
}
```

Копирование привязок — не бесплатная операция: это проход по таблице копий с разыменованием указателей. Для задачи с десятком входов на десяти тысячах агентов экономия реальная.

Аналогично `bShouldCopyBoundPropertiesOnExitState`: если `ExitState()` не читает привязанных свойств, отключайте.

bShouldAffectTransitions

cpp

```
/** If set to true, TriggerTransitions() is called during transition handling.
    Default false. */
uint8 bShouldAffectTransitions : 1;
```

Включает `TriggerTransitions()`. Разобран выше.

Выключен по умолчанию — правильный выбор: большинство задач переходами не управляют.

bConsideredForScheduling

cpp

```
/**
 * If set to true, the task is considered for scheduled tick. It will use
 * these flags: bShouldCallTick, bShouldCallTickOnlyOnEvents, and
 * bShouldAffectTransitions.
 * It doesn't affect how the task ticks.
 * Default true.
 */
uint8 bConsideredForScheduling : 1;
```


Относится к механизму запланированного тика (глава 11), который `UMassStateTreeSchema` отключает через `IsScheduledTickAllowed() = false`.

Ключевая фраза: «*It doesn't affect how the task ticks*» — флаг влияет только на расчёт, когда дереву понадобится следующий тик, но не на само тиканье.

В Mass значения не имеет. Оставляйте как есть.

`bTaskEnabled`

cpp

```
/** True if the node is Enabled (i.e. not explicitly disabled in the asset).
 */
UPROPERTY()
uint8 bTaskEnabled : 1;
```

Единственный флаг с `UPROPERTY` среди рантайм-флагов — потому что задаётся в редакторе и сохраняется в ассет.

Отключённая задача остаётся в дереве, но не исполняется. Удобно для отладки: выключить узел и посмотреть, изменится ли поведение, не перестраивая дерево.

`bHasTaskCompletionDelegateDispatcher`

cpp

```
/** True if the node is bound to a task completion delegate listener. */
UPROPERTY()
uint8 bHasTaskCompletionDelegateDispatcher : 1;
```

Тоже `UPROPERTY` — вычисляется при компиляции ассета. Помечает, что к завершению задачи привязан делегат-диспетчер.

Связано с функцией из главы 11:

cpp

```
UE_API FTaskCompletionDelegateDispatcherContainer
GetTaskCompletionDispatcher(TNotNull<const UStateTree*> StateTree, int32
NodeIndex, UE::StateTree::ETaskCompletionStatus TaskStatus);
```

Флаг — быстрая проверка «стоит ли вообще искать диспетчер». Если он `false`, поиск пропускается.

Устанавливается автоматически, вручную трогать не нужно.

15.4. Редакторные поля

cpp

```
#if WITH_EDITORONLY_DATA
/**
 * True if the task is considered for completion.
 * False if the task runs in the background without affecting the state
completion.
 */
UPROPERTY()
uint8 bConsideredForCompletion : 1;

/** True if the user can edit bConsideredForCompletion in the editor. */
UPROPERTY()
uint8 bCanEditConsideredForCompletion : 1;
#endif
```

`bConsideredForCompletion` — очень полезная настройка для `Mass`.

Сценарий: в состоянии две задачи. Первая ведёт агента к точке (завершится через несколько секунд). Вторая заставляет смотреть по сторонам (бесконечная). Если обе учитываются при завершении, состояние не завершится никогда — вторая задача всегда `Running`.

Пометив вторую как фоновую (`bConsideredForCompletion = false`), получаем правильное поведение: состояние завершается, когда завершилась первая.

Это ровно комбинация из главы 3: `ZG Path Follow` + `Mass LookAt` в одном состоянии, где взгляд — фон.

`bCanEditConsideredForCompletion` управляет доступностью настройки в редакторе. Задача может запретить её менять, если её роль однозначна.

Оба поля под `WITH_EDITORONLY_DATA`, потому что в рантайме решение уже принято и зашито в скомпилированный ассет.

Связано со схемой (глава 7):

cpp

```
/** @return True if modifying the tasks completion is allowed. If not allowed,
"any" will be used.*/
```

```
virtual bool AllowTasksCompletion() const { return true; }
```

Если схема запрещает, используется правило «любая завершившаяся задача завершает состояние».

15.5. Отладка

cpp

```
#if WITH_EDITOR
    virtual FName GetIconName() const override
    {
        return FName("StateTreeEditorStyle|Node.Task");
    }
    virtual FColor GetIconColor() const override
    {
        return UE::StateTree::Colors::Grey;
    }
#endif

#if WITH_GAMEPLAY_DEBUGGER
    UE_API virtual FString GetDebugInfo(const
FStateTreeReadOnlyExecutionContext& Context) const;
#endif
```

Иконка и цвет — оформление в редакторе. Переопределяйте, если хотите визуально выделить категории задач: боевые красным, навигационные синим. На больших деревьях это реально помогает.

GetDebugInfo() — то, что видно в Gameplay Debugger. **Для Mass стоит переопределять всегда.** Отладка тысяч агентов без осмысленной строки состояния практически невозможна:

cpp

```
#if WITH_GAMEPLAY_DEBUGGER
    virtual FString GetDebugInfo(const FStateTreeReadOnlyExecutionContext&
Context) const override
    {
        return FString::Printf(TEXT("MoveTo: %.1f m remaining"), RemainingDistance
/ 100.f);
    }
#endif
```

Обратите внимание на тип параметра: **read-only контекст** (глава 11). Отладочный вывод не должен ничего менять, и тип это гарантирует.

Сравните с эвалуатором, где старая сигнатура запечатана:

cpp

```
UE_DEPRECATED(5.8, "Use the version with the
FStateTreeReadOnlyExecutionContext.")
virtual void AppendDebugInfoString(FString& DebugString, const
FStateTreeExecutionContext& Context) const final
{
}
```

Тот же приём `virtual ... final`, что мы видели в главе 11: переопределить нельзя, только заметить, что нужно перейти на новый метод.

15.6. FStateTreeTaskCommonBase

cpp

```
/**
 * Base class (namespace) for all common Tasks that are generally applicable.
 * This allows schemas to safely include all conditions child of this struct.
 */
USTRUCT(meta = (Hidden))
struct FStateTreeTaskCommonBase : public FStateTreeTaskBase
{
    GENERATED_BODY()
};
```

Пустая структура-маркер. Разбиралась в главе 7: наследники допускаются в **любую** схему, включая Mass, потому что не трогают внешний мир.

Практическое следствие для проектирования: если ваша задача не обращается ни к фрагментам, ни к подсистемам — наследуйте от `FStateTreeTaskCommonBase`, а не от `FMassStateTreeTaskBase`. Она станет переиспользуемой между Mass-деревьями и деревьями на акторах, и не расширит требования процессора (глава 8).

15.7. Скелет Mass-задачи

Соберём всё в шаблон, от которого можно отталкиваться:

cpp

```

USTRUCT()
struct FMyTaskInstanceData
{
    GENERATED_BODY()

    /** Вход: привязывается к выходу эвалюатора или другой задачи. */
    UPROPERTY(EditAnywhere, Category = Input, meta = (Optional))
    FMassEntityHandle TargetEntity;

    /** Параметр: настраивается дизайнером. */
    UPROPERTY(EditAnywhere, Category = Parameter)
    float Duration = 0.f;

    /** Рабочее состояние: не показывается в редакторе. */
    UPROPERTY()
    float ElapsedTime = 0.f;
};

USTRUCT(meta = (DisplayName = "My Mass Task"))
struct FMyMassTask : public FMassStateTreeTaskBase
{
    GENERATED_BODY()

    using FInstanceDataType = FMyTaskInstanceData;

    FMyMassTask()
    {
        // тикаем – нужен счётчик времени
        bShouldCallTick = true;
        // привязки читаем только при входе
        bShouldCopyBoundPropertiesOnTick = false;
        // переходами не управляем
        bShouldAffectTransitions = false;
    }

protected:
    virtual const UStruct* GetInstanceDataType() const override
    {
        return FInstanceDataType::StaticStruct();
    }

    virtual bool Link(FStateTreeLinker& Linker) override
    {
        Linker.LinkExternalData(MoveTargetHandle);
        Linker.LinkExternalData(SignalSubsystemHandle);
        return true;
    }
}

```

```

    }

    // ВНИМАНИЕ: список должен совпадать с Link() выше
    virtual void
GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
override
    {
        Builder.AddReadWrite<FMassMoveTargetFragment>();
        Builder.AddReadWrite<UMassSignalSubsystem>();
    }

    virtual EStateTreeRunStatus EnterState(FStateTreeExecutionContext&
Context, const FStateTreeTransitionResult& Transition) const override;
    virtual void ExitState(FStateTreeExecutionContext& Context, const
FStateTreeTransitionResult& Transition) const override;
    virtual EStateTreeRunStatus Tick(FStateTreeExecutionContext& Context,
const float DeltaTime) const override;

#ifdef WITH_GAMEPLAY_DEBUGGER
    virtual FString GetDebugInfo(const FStateTreeReadOnlyExecutionContext&
Context) const override;
#endif

    TStateTreeExternalDataHandle<FMassMoveTargetFragment> MoveTargetHandle;
    TStateTreeExternalDataHandle<UMassSignalSubsystem> SignalSubsystemHandle;

    UPROPERTY(EditAnywhere, Category = Parameter)
    float SomeSetting = 1.f;
};

```

Пять обязательных элементов, которые легко забыть:

1. `using FInstanceDataType` — иначе `Context.GetInstanceData(*this)` не скомпилируется.
2. `GetInstanceDataType()` — иначе `StateTree` не выделит данные.
3. `Link()` — иначе хендлы внешних данных невалидны.
4. `GetDependencies()` — иначе гонка данных без ошибки компиляции.
5. `meta = (DisplayName = ...)` — иначе в редакторе будет сырое имя структуры.

15.8. Параметры узла против instance data

Тонкость, которая часто путает новичков. Посмотрите на `FMassLookAtTask`: часть настроек в instance data, часть — прямо в структуре задачи.

cpp

```
// в instance data
UPROPERTY(EditAnywhere, Category = Parameter)
float Duration = 0.f;

// в самой задаче
UPROPERTY(EditAnywhere, Category = Parameter)
FMassLookAtPriority Priority{...};
```

Разница фундаментальная:

Поле задачи — одно значение на весь ассет. Экземпляр задачи один, разделяется всеми агентами. Изменить в рантайме нельзя (методы `const`). К нему **нельзя привязать** значение от другого узла.

Поле instance data — своё у каждого агента. Меняется в рантайме. Можно привязать.

Критерий выбора: **нужна ли привязка или рантайм-изменение?** Если да — `instance data`. Если это статическая настройка, одинаковая для всех, — поле задачи, и это экономит память: одно значение вместо десяти тысяч копий.

В `FMassLookAtTask` это разделено грамотно: приоритет и режим взгляда — статические настройки поведения, длительность и цель — динамические.

15.9. Порядок вызовов: сводка

Полная последовательность для одного состояния:

Вход:

1. Проверяются условия входа.
2. Выделяется `instance data` задач.
3. Копируются привязки.
4. `EnterState()` — сверху вниз по иерархии, по порядку задач внутри состояния.
5. Если какая-то задача вернула не `Running` — переход к завершению.

Каждый тик:

1. Тикают глобальные эвалюаторы.
2. `TriggerTransitions()` у задач с `bShouldAffectTransitions`, по приоритету.
3. Обрабатываются запросы переходов и события.
4. Копируются привязки (если `bShouldCopyBoundPropertiesOnTick`).
5. `Tick()` — по порядку задач.
6. Сводится общий статус через `GetPriorityRunStatus()`.

Завершение:

1. `StateCompleted()` — снизу вверх (если завершение, а не условный переход).
2. Выбирается новое состояние.
3. Копируются привязки (если `bShouldCopyBoundPropertiesOnExitState`).
4. `ExitState()` — снизу вверх.
5. Освобождается instance data (с учётом переиспользования при `Append` с перемещением, глава 10).

15.10. Типичные ошибки

Хранить состояние в полях задачи. Методы `const`, компилятор не даст. Но можно попытаться обойти через `mutable` — не делайте этого никогда: экземпляр один на всех агентов, вы получите общую переменную для десяти тысяч.

Забыть `using FInstanceDataType`. Ошибка компиляции невнятная, ищется долго.

Рассчитывать на малый `DeltaTime`. В Mass он может быть секундами.

Освобождать ресурсы в `StateCompleted()`. Метод не вызывается при условном переходе. Только `ExitState()`.

Оставлять `bShouldCallTick = true` без надобности. Задача, которая только ставит намерение и ждёт сигнала, тикать не должна.

Забыть `bConsideredForCompletion = false` у фоновой задачи. Состояние никогда не завершится.

Не переопределять `GetDebugInfo()`. Отладка толпы без него мучительна.

15.11. Итог главы

Метод	Когда	Порядок	Возврат
<code>EnterState</code>	Вход в состояние	Сверху вниз	Статус; не-Running завершает немедленно
<code>Tick</code>	Каждый тик дерева	По порядку задач	Статус
<code>TriggerTransitions</code>	До обработки переходов	По <code>TransitionHandlingPriority</code>	—
<code>StateCompleted</code>	После завершения,	Снизу вверх	—

Метод	Когда	Порядок	Возврат
	до выбора нового		
ExitState	Выход из состояния	Снизу вверх	—

Флаг	По умолчанию	Когда менять
bShouldStateChangeOnReselect	true	false для задач, удерживающих ресурс
bShouldCallTick	true	false для задач «поставил и жду сигнала»
bShouldCallTickOnlyOnEvents	false	true для реагирующих только на события
bShouldCopyBoundPropertiesOnTick	true	false, если входы читаются только при входе
bShouldCopyBoundPropertiesOnExitState	true	false, если ExitState не читает привязок
bShouldAffectTransitions	false	true для программного управления переходами
bConsideredForCompletion	true	false для фоновых задач

Что дальше

Глава 16 — построчный разбор `FMassLookAtTask`. Пройдём по всей структуре: зачем `TargetEntity` помечен `Optional`, как работает накопление времени через `Duration` и `Time`, почему `LookAtHandle` — опциональные внешние данные и что задача делает, когда фрагмента нет, что такое системный взгляд и метод `TryActivateSystemicLookAt()`, как устроены приоритеты `FMassLookAtPriority`, режимы `EMassLookAtMode`, скорости интерполяции и случайный взгляд. Это первая настоящая задача, которую мы разберём целиком.

Глава 16. Разбор `FMassLookAtTask`

Первая настоящая задача, которую мы разберём целиком. Она удачно выбрана в качестве примера: достаточно проста, чтобы уложиться в одну главу, и при этом задействует почти все механизмы из части II — опциональные внешние данные, накопление времени, сигналы завершения, разделение параметров между задачей и instance data.

Оговорка: файла `.cpp` в нашем пакете нет, поэтому реализации методов я восстанавливаю по сигнатурам, именам полей и общей логике модуля. Контракты и структура данных — из первых рук, конкретные строки реализации — реконструкция.

16.1. Заголовки

cpp

```
#include "MassEntityTypes.h"
#include "MassStateTreeTypes.h"
#include "MassLookAtFragments.h"
#include "MassLookAtTask.generated.h"

#define UE_API MASSAIBEHAVIOR_API

class UMassSignalSubsystem;
namespace UE::MassBehavior
{
    struct FStateTreeDependencyBuilder;
};
```

Три инcludes, и каждый обязателен:

- `MassEntityTypes.h` — ради `FMassEntityHandle`, который лежит в instance data **по значению**. Форвардом не обойтись.
- `MassStateTreeTypes.h` — база `FMassStateTreeTaskBase`.
- `MassLookAtFragments.h` — здесь живут `FMassLookAtFragment`, `FMassLookAtPriority`, `EMassLookAtMode`, `EMassLookAtInterpolationSpeed`, `EMassLookAtGazeMode` и константа `UE::Mass::LookAt::DefaultCustomInterpolationSpeed`. Все они используются по значению в полях задачи.

А `UMassSignalSubsystem` объявлен форвардом — потому что нужен только как параметр шаблона `TStateTreeExternalDataHandle<>`, а шаблону достаточно неполного типа.

Лишняя точка с запятой после закрывающей скобки пространства имён — безобидная опечатка, встречается по всему модулю.

16.2. Instance data

cpp

```
/**
 * Task to assign a LookAt target for mass processing
 */
USTRUCT()
struct FMassLookAtTaskInstanceData
{
    GENERATED_BODY()

    /** Entity to set as the target for the LookAt behavior. */
    UPROPERTY(EditAnywhere, Category = Input, meta = (Optional))
    FMassEntityHandle TargetEntity;

    /** Delay before the task ends. Default (0 or any negative) will run indefinitely so it requires a transition in the state tree to stop it. */
    UPROPERTY(EditAnywhere, Category = Parameter)
    float Duration = 0.f;

    /** Accumulated time used to stop task if duration is set */
    UPROPERTY()
    float Time = 0.f;
};
```

Три поля, три разные роли — и это идеальная иллюстрация `EStateTreePropertyUsage` из главы 3.

TargetEntity — ВХОД

`Category = Input` означает, что значение приходит **извне**, через привязку свойств. Дизайнер соединяет это поле с выходом другого узла — например, эвалюатора «ближайшая угроза» или задачи «найти собеседника».

`meta = (Optional)` — привязка не обязательна. Если её нет, поле останется пустым хендлом, и задача должна это пережить.

Важно не путать два разных `Optional`, которые встречаются в этом файле:

Где	Что означает
<code>meta = (Optional)</code> у свойства	Привязка в редакторе не обязательна
<code>EStateTreeExternalDataRequirement::Optional</code> у хендла	Внешние данные могут отсутствовать в рантайме

Первое — про редактор, второе — про исполнение. В `FMassLookAtTask` используются оба, и по разным причинам.

Тип `FMassEntityHandle` в качестве входа — характерная для Mass деталь. В обычном `StateTree` целью взгляда был бы `AActor*`. Здесь — восемь байт значения, которые ничего не удерживают и ни на что не ссылаются напрямую.

Duration — параметр

`Category = Parameter` — настраивается дизайнером прямо в узле дерева.

Комментарий фиксирует важный контракт: **ноль или отрицательное значение означают «бесконечно»**. Задача будет работать, пока её не остановит переход.

Это распространённая идиома в движке: вместо отдельного флага `bInfinite` используется «неправдоподобное» значение основного параметра. Экономит поле и упрощает интерфейс, но требует внимательного чтения документации — из типа `float` это никак не следует.

Time — рабочее состояние

cpp

```
UPROPERTY()  
float Time = 0.f;
```

Ни `EditAnywhere`, ни категории. Дизайнер этого поля не видит и видеть не должен: это внутренний накопитель.

Но `UPROPERTY` всё же есть. Зачем, если поле не редактируется? Потому что оно должно быть частью `instance data` и участвовать в операциях, которые `StateTree` выполняет через рефлексии: копирование при переходах, инициализация значениями по умолчанию, отладочный вывод.

Вспомните механику из главы 10: `instance data` хранится в `FInstancedStructContainer`, а он работает через `UScriptStruct`. Поле без `UPROPERTY` в такой структуре существовало бы физически, но было бы невидимо для копирования и сериализации.

Это ключевая деталь для понимания того, зачем `instance data` вообще нужна. Поле `Time` не может лежать в самой задаче — та `const` и одна на всех агентов. Оно не может лежать во фрагменте — это внутреннее дело задачи, процессорам Mass оно не нужно. Его место — ровно здесь.

16.3. Объявление задачи

cpp

```
USTRUCT(meta = (DisplayName = "Mass LookAt Task"))
struct FMassLookAtTask : public FMassStateTreeTaskBase
{
    GENERATED_BODY()

    using FInstanceDataType = FMassLookAtTaskInstanceData;

protected:
```

Наследование от `FMassStateTreeTaskBase` — пропуск в Mass-схему (глава 7) и место для `GetDependencies()` (глава 5).

`DisplayName` — то, что дизайнер увидит в списке узлов. Без него отображалось бы `MassLookAtTask`.

`using FInstanceDataType` — обязательное объявление, без которого не заработает шаблонный `Context.GetInstanceData(*this)` (глава 11).

Что здесь отсутствует

Заметьте: **конструктора нет**. Значит, все флаги остаются в значениях по умолчанию из главы 15:

- `bShouldCallTick = true` — задача тикает каждый раз. Необходимо: `Time` надо накапливать.
- `bShouldStateChangeOnReselect = true` — при переизбрании состояния взгляд начнётся заново.
- `bShouldAffectTransitions = false` — переходами не управляет.
- `bShouldCopyBoundPropertiesOnTick = true` — привязки обновляются каждый тик.

Последнее интересно. Задача каждый тик перечитывает `TargetEntity`. Это не оптимально с точки зрения главы 15, но даёт полезное свойство: **цель может меняться на лету**. Эвалкуатор переключился на другую угрозу — взгляд последует за ней без выхода из состояния.

Осознанный компромисс: чуть больше работы за динамичность.

Всё, кроме `using`, объявлено `protected`. Задача — не библиотека функций, её методы вызывает только `StateTree`.

16.4. Обязательные методы

cpp

```
UE_API virtual bool Link(FStateTreeLinker& Linker) override;

virtual const UStruct* GetInstanceDataType() const override
{
    return FInstanceDataType::StaticStruct();
}
```

`GetInstanceDataType()` — inline и тривиален. Через него `StateTree` узнаёт, сколько памяти выделить и какой структурой её инициализировать.

`Link()` реализован в `.cpp`, потому что делает больше одной строки:

cpp

```
// реконструкция
bool FMassLookAtTask::Link(FStateTreeLinker& Linker)
{
    Linker.LinkExternalData(MassSignalSubsystemHandle);
    Linker.LinkExternalData(LookAtHandle);
    return true;
}
```

Два хендла — два вызова. Возврат `false` означал бы провал линковки всего дерева.

16.5. Внешние данные

cpp

```
TStateTreeExternalDataHandle<UMassSignalSubsystem> MassSignalSubsystemHandle;
TStateTreeExternalDataHandle<FMassLookAtFragment,
EStateTreeExternalDataRequirement::Optional> LookAtHandle;
```

Два хендла с **разными требованиями**, и это самая поучительная деталь файла.

Подсистема — обязательна

Без второго параметра шаблона требование по умолчанию — `Required`. Дерево не запустится, если подсистема сигналов недоступна.

Зачем она задаче? Для отложенного сигнала завершения. Когда `Duration` задан, задача должна проснуться ровно в момент истечения:

cpp

```
// реконструкция фрагмента EnterState
if (InstanceData.Duration > 0.f)
{
    UMassSignalSubsystem& SignalSubsystem =
Context.GetExternalData(MassSignalSubsystemHandle);
    SignalSubsystem.DelaySignalEntity(
        UE::Mass::Signals::LookAtFinished,
        MassContext.GetEntity(),
        InstanceData.Duration);
}
```

Вот где используется сигнал `LookAtFinished` из главы 5. И вот почему подсистема обязательна: без неё задача с конечной длительностью никогда бы не завершилась — дерево просто не проснулось бы (глава 12).

Обратите внимание: доступ через `GetExternalData()` — обычная форма для обязательных данных.

Фрагмент — опционален

cpp

```
TStateTreeExternalDataHandle<FMassLookAtFragment,
EStateTreeExternalDataRequirement::Optional> LookAtHandle;
```

`FMassLookAtFragment` может отсутствовать. Почему?

Потому что дерево поведения одно, а агенты разные. В толпе могут быть:

- полноценные агенты с анимированной головой — у них фрагмент есть;
- дальние агенты на низком LOD, у которых поворот головы не отображается;
- агенты-абстракции без визуального представления вообще.

Дерево общее, состояние «оглядеться» есть у всех, но физически повернуть голову может не каждый.

Отсюда — обязательное правило из главы 11:

cpp

```
// реконструкция
FMassLookAtFragment* LookAtFragment =
```

```
Context.GetExternalDataPtr(LookAtHandle);
if (LookAtFragment == nullptr)
{
    // у агента нет головы — задача бессмысленна
    return EStateTreeRunStatus::Failed;
}
```

Только `GetExternalDataPtr()`. Вызов `GetExternalData()` для опционального хендла упадёт на `check` в отладочной сборке — вспомните проверку из главы 11:

cpp

```
check(CurrentlyProcessedFrame->StateTree->ExternalDataDescs[Handle.DataHandle.GetIndex()].Requirement !=
EStateTreeExternalDataRequirement::Optional); // Optionals should query
pointer instead.
```

Что возвращать при отсутствии фрагмента — вопрос дизайна. `Failed` заставит дерево выбрать другую ветку; `Succeeded` сделает задачу мгновенно-успешной, и агент просто продолжит дальше. Судя по тому, что взгляд обычно фоновая задача, вероятнее второе или тихое `Running` без эффекта.

И `GetDependencies()`

cpp

```
UE_API virtual void
GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
override;
```

Парный метод, о котором предупреждалось в главе 5:

cpp

```
// реконструкция
void
FMassLookAtTask::GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder
& Builder) const
{
    Builder.AddReadWrite<FMassLookAtFragment>();
    Builder.AddReadWrite<UMassSignalSubsystem>();
}
```


Оба — `ReadWrite` : задача пишет во фрагмент и отправляет сигналы (а отправка — это запись, глава 14).

Сопоставьте два списка. Два хендла — две записи. Если бы кто-то добавил третий хендл и забыл про `GetDependencies()` , компилятор промолчал бы, а планировщик Mass не узнал бы о доступе. Держите методы рядом.

16.6. Параметры самой задачи

Восемь `UPROPERTY` прямо в структуре задачи. Напомню различие из главы 15: это **статические настройки**, одно значение на весь ассет, привязать к ним ничего нельзя.

Приоритет

cpp

```
/** Look At Priority */
UPROPERTY(EditAnywhere, Category = Parameter)
FMassLookAtPriority Priority{static_cast<uint8>
(EMassLookAtPriorities::LowestPriority)};
```

Приоритет — не `enum`, а **структура-обёртка** над `uint8` . Зачем такое усложнение?

Обёртка даёт три вещи, недоступных голому числу:

Своё отображение в редакторе. Можно нарисовать слайдер или выпадающий список с именованными уровнями вместо безликого числа.

Расширяемость проектом. `EMassLookAtPriorities` определяет базовые уровни, но проект может ввести промежуточные значения, не трогая перечисление движка. Числа между именованными уровнями остаются доступными.

Типобезопасность. `FMassLookAtPriority` нельзя случайно сложить с другим `uint8` или передать туда, где ждут иное число.

Значение по умолчанию — `LowestPriority` . Разумно: задача из дерева поведения не должна по умолчанию перебивать более важные источники взгляда.

Режим взгляда

cpp

```
/** Look At Mode */
UPROPERTY(EditAnywhere, Category = Parameter)
```

```
EMassLookAtMode LookAtMode = EMassLookAtMode::LookForward;
```

`LookForward` по умолчанию — «смотреть вперёд по направлению движения». Самое безопасное поведение: даже если цель не привязана, агент не будет выглядеть странно.

Другие значения перечисления (в `MassLookAtFragments.h`) наверняка включают режимы «смотреть на цель», «смотреть в заданную точку», «смотреть по траектории пути».

Скорость интерполяции

c++

```
/** Look at interpolation speed (not used by the LookAt processor but can be
forwarded to the animation system). */
UPROPERTY(EditAnywhere, Category = Parameter)
EMassLookAtInterpolationSpeed InterpolationSpeed =
EMassLookAtInterpolationSpeed::Regular;

/**
 * Look at custom interpolation speed used when 'InterpolationSpeed =
EMassLookAtInterpolationSpeed::Custom'
 * (not used by the LookAt processor but can be forwarded to the animation
system).
 */
UPROPERTY(EditAnywhere, Category = Parameter, meta =
(EditCondition="InterpolationSpeed == EMassLookAtInterpolationSpeed::Custom",
EditConditionHides))
float CustomInterpolationSpeed =
UE::Mass::LookAt::DefaultCustomInterpolationSpeed;
```

Комментарий повторяется дважды и содержит важное признание: **процессор взгляда эти значения не использует**. Он их только хранит и передаёт дальше — в систему анимации.

Это очень характерная для Mass деталь. Фрагмент выступает **шиной данных** между подсистемами: задача `StateTree` пишет, процессор взгляда вычисляет направление, анимационная система читает скорость интерполяции. Каждый участник берёт своё.

Пара метаданных заслуживает разбора:

`EditCondition` — поле доступно для редактирования только при выполнении условия. Здесь: только когда выбран режим `Custom`.

`EditConditionHides` — при невыполнении условия поле не просто отключается, а **скрывается**. Без этого флага оно висело бы серым, засоряя панель.

Хорошая практика для узлов с взаимоисключающими настройками. В своих задачах используйте её активно: панель свойств узла в StateTree и так тесная.

Значение по умолчанию берётся из именованной константы

`UE::Mass::LookAt::DefaultCustomInterpolationSpeed`, а не из литерала. Правильно: константа определена в одном месте и переиспользуется всеми, кто работает с этой системой.

Случайный взгляд

cpp

```
/** Random gaze Mode */
UPROPERTY(EditAnywhere, Category = Parameter)
EMassLookAtGazeMode RandomGazeMode = EMassLookAtGazeMode::None;

/** Random gaze yaw angle added to the look direction determined by the look
at mode. */
UPROPERTY(EditAnywhere, Category = Parameter, meta = (UIMin = 0.0, ClampMin =
0.0, UIMax = 180.0, ClampMax = 180.0))
uint8 RandomGazeYawVariation = 0;

/** Random gaze pitch angle added to the look direction determined by the look
at mode. */
UPROPERTY(EditAnywhere, Category = Parameter, meta = (UIMin = 0.0, ClampMin =
0.0, UIMax = 180.0, ClampMax = 180.0))
uint8 RandomGazePitchVariation = 0;

/** If true, allow random gaze to look at other entities too. */
UPROPERTY(EditAnywhere, Category = Parameter)
bool bRandomGazeEntities = false;
```

Механизм «живости». Толпа, где все смотрят строго вперёд, выглядит мёртвой. Случайные отклонения взгляда стоят дешево, а эффект дают заметный.

Разберём тип углов: `uint8` с диапазоном **0–180**. Почему не `float` ?

Потому что это **упаковка**. Один байт вместо четырёх, а точность в один градус для случайного отклонения избыточна с запасом. Умножьте на два поля и на десять тысяч агентов... хотя нет — стоп. Это поля **задачи**, а не instance data. Экземпляр один на весь ассет.

Значит, экономия здесь не в количестве агентов, а в другом: значения, вероятно, копируются во фрагмент, где уже размножаются по агентам. И там байт против четырёх байт — уже реальная экономия.

Метаданные `UIMin / UIMax` задают границы слайдера, `ClampMin / ClampMax` — жёсткое ограничение ввода. Разница: UI-границы можно превысить, введя число руками, `Clamp` — нельзя. Здесь заданы обе пары с одинаковыми значениями, то есть выйти за 0–180 не получится никак.

`bRandomGazeEntities` — разрешить случайному взгляду цепляться за другие сущности поблизости. Это уже не просто шум, а зачаток социального поведения: агенты «замечают» друг друга.

16.7. `TryActivateSystemicLookAt`

cpp

```
bool TryActivateSystemicLookAt(const FStateTreeExecutionContext& Context,
const FInstanceDataType& InstanceData, FMassLookAtFragment& Fragment) const;
```

Единственный не виртуальный вспомогательный метод. Разберём, что он делает, по сигнатуре и имени.

Проблема арбитража

`FMassLookAtFragment` один на агента. А источников, желающих управлять взглядом, может быть несколько:

- задача `StateTree` («смотри на собеседника»);
- система восприятия («смотри на источник шума»);
- скриптовая сцена («смотри на игрока»);
- система движения («смотри по направлению поворота»).

Одновременно активен может быть только один. Нужен арбитраж — и он строится на приоритетах.

Что делает метод

cpp

```
// реконструкция
bool FMassLookAtTask::TryActivateSystemicLookAt(const
FStateTreeExecutionContext& Context, const FInstanceDataType& InstanceData,
FMassLookAtFragment& Fragment) const
{
    // уступаем, если уже активен более приоритетный взгляд
    if (Fragment.HasActiveRequest() && Fragment.GetPriority() > Priority)
    {
```

```

        return false;
    }

    Fragment.SetLookAtRequest(Priority, LookAtMode, InstanceData.TargetEntity,
        InterpolationSpeed, CustomInterpolationSpeed,
        RandomGazeMode, RandomGazeYawVariation, RandomGazePitchVariation,
        bRandomGazeEntities);

    return true;
}

```

Возврат `bool` — «удалось ли захватить взгляд». Если нет, задача может либо провалиться, либо продолжать работать вхолостую в надежде перехватить позже.

Почему «systemic»

Термин «системный взгляд» противопоставляется «ручному»: не «повернуть голову на 30 градусов», а «объявить намерение смотреть туда-то с таким-то приоритетом, дальше система разберётся».

Это ровно та философия, о которой шла речь в главе 4: **задача выражает намерение, работу делают процессоры.**

Почему параметры такие

`const FStateTreeExecutionContext&` — константная ссылка, метод контекст не меняет.

`const FInstanceDataType&` — данные передаются явно, а не достаются внутри. Так метод можно вызвать и из `EnterState()`, и из `Tick()`, не дублируя получение данных.

`FMassLookAtFragment&` — **неконстантная** ссылка, сюда пишем. Фрагмент тоже передаётся снаружи: проверка на `nullptr` уже сделана вызывающим, метод получает гарантированно валидную ссылку.

Аккуратный дизайн: метод не знает, откуда пришли данные, и не занимается их валидацией.

16.8. Реконструкция жизненного цикла

Соберём всё в связную картину.

`EnterState`

cpp

```

// реконструкция
EStateTreeRunStatus FMassLookAtTask::EnterState(FStateTreeExecutionContext&
Context, const FStateTreeTransitionResult& Transition) const
{
    FMassStateTreeExecutionContext& MassContext =
static_cast<FMassStateTreeExecutionContext*>(Context);
    FInstanceDataType& InstanceData = Context.GetInstanceData(*this);

    InstanceData.Time = 0.f;

    FMassLookAtFragment* LookAtFragment =
Context.GetExternalDataPtr(LookAtHandle);
    if (LookAtFragment == nullptr)
    {
        return EStateTreeRunStatus::Failed;
    }

    if (!TryActivateSystemicLookAt(Context, InstanceData, *LookAtFragment))
    {
        return EStateTreeRunStatus::Failed;
    }

    if (InstanceData.Duration > 0.f)
    {
        UMassSignalSubsystem& SignalSubsystem =
Context.GetExternalData(MassSignalSubsystemHandle);
        SignalSubsystem.DelaySignalEntity(UE::Mass::Signals::LookAtFinished,
            MassContext.GetEntity(), InstanceData.Duration);
    }

    return EStateTreeRunStatus::Running;
}

```

Пять шагов: сбросить счётчик, получить фрагмент, захватить взгляд, запланировать пробуждение, вернуть `Running`.

Сброс `Time` в ноль обязателен. Instance data может **переиспользоваться** при переходах (глава 10, `Append` с перемещением), так что старое значение вполне может там оказаться.

Tick

cpp

```
// реконструкция
EStateTreeRunStatus FMassLookAtTask::Tick(FStateTreeExecutionContext& Context,
const float DeltaTime) const
{
    FInstanceDataType& InstanceData = Context.GetInstanceData(*this);

    if (InstanceData.Duration <= 0.f)
    {
        return EStateTreeRunStatus::Running;    // бесконечно
    }

    InstanceData.Time += DeltaTime;

    return InstanceData.Time < InstanceData.Duration
        ? EStateTreeRunStatus::Running
        : EStateTreeRunStatus::Succeeded;
}
```

Обратите внимание на роль `DeltaTime`. Это дельта **между тиками дерева**, посчитанная по `LastUpdateTimeInSeconds` (глава 6). Агент, разбуженный отложенным сигналом через три секунды, получит `DeltaTime ≈ 3.0`, и счётчик сойдётся правильно.

Была бы здесь кадровая дельта — таймер отстал бы в разы. Это ровно тот случай, ради которого поле `LastUpdateTimeInSeconds` существует.

Возможно, реализация ещё и переустанавливает цель, если `TargetEntity` изменился (привязки копируются каждый тик), — но это уже спекуляция.

ExitState

cpp

```
// реконструкция
void FMassLookAtTask::ExitState(FStateTreeExecutionContext& Context, const
FStateTreeTransitionResult& Transition) const
{
    if (FMassLookAtFragment* LookAtFragment =
Context.GetExternalDataPtr(LookAtHandle))
    {
        LookAtFragment->ClearRequest(Priority);
    }
}
```

Освобождение ресурса — то, ради чего `ExitState()` существует (глава 15). Задача занимала «слот взгляда»; уходя, обязана его отпустить, иначе агент так и будет пялиться в никуда до конца жизни.

Скорее всего, очистка снимает запрос только своего приоритета: если за время работы более приоритетный источник перехватил взгляд, затирать его чужую заявку неправильно.

Именно этот метод не вызвался бы, если бы `UMassStateTreeFragmentDestructor` не звал `Stop()` перед освобождением данных (глава 13). Замкнули круг.

16.9. Чему учит этот пример

Задача небольшая, но в ней собраны почти все паттерны, которые вам понадобятся.

Опциональные внешние данные — норма. Дерево одно, агенты разные. Проверка на `nullptr` — не защитное программирование, а часть контракта.

Обязательные и опциональные данные различаются способом доступа.

`GetExternalData()` и `GetExternalDataPtr()` не взаимозаменяемы.

Сигнал завершения планируется при входе. Задача сама обеспечивает своё пробуждение — иначе агент замрёт (глава 12).

Ресурс, захваченный в `EnterState()`, отпускается в `ExitState()`. Без исключений.

Параметры делятся по критерию «нужна ли привязка». Динамические — в `instance data`, статические — в задаче.

Фрагмент — шина между системами. Задача пишет намерение и параметры, разные процессоры читают то, что им нужно. Прямых вызовов между системами нет.

Приоритеты решают конфликты. Когда ресурс один, а претендентов много, нужен явный арбитраж.

16.10. Как бы вы улучшили эту задачу

Полезное упражнение: посмотреть на код движка критически.

Отключить копирование привязок на тике? Задача перечитывает `TargetEntity` каждый тик. Если ваша цель не меняется в рамках состояния, `bShouldCopyBoundPropertiesOnTick = false` сэкономит работу. Но потеряете динамическое переключение цели.

Отказаться от тика при бесконечной длительности? Если `Duration <= 0`, `Tick()` не делает ничего полезного. Но флаг `bShouldCallTick` — свойство задачи, а не `instance`

data; он одинаков для всех агентов и не может зависеть от значения параметра. Так что убрать нельзя.

Добавить `GetDebugInfo()` ? В заголовке его нет. Для отладки толпы это заметное упущение — строка вида «LookAt: 1.2/3.0s → Entity 4718» сэкономила бы много времени.

Пометить `bConsideredForCompletion = false` ? Взгляд почти всегда фоновая задача. Разумным умолчанием было бы `false`, чтобы состояние завершалось по основной задаче. Сейчас дизайнер обязан выставлять это вручную в каждом узле.

Эти наблюдения — не претензии к Epic, а материал для ваших собственных задач.

16.11. Итог главы

Элемент	Роль	Урок
<code>TargetEntity</code> (Input, Optional)	Цель взгляда, приходит по привязке	Вход может быть не привязан
<code>Duration</code> (Parameter)	Длительность; ≤ 0 = бесконечно	Идиома «неправдоподобное значение как флаг»
<code>Time</code> (без Edit)	Накопитель; UPROPERTY ради рефлексии	Рабочее состояние живёт в instance data
<code>MassSignalSubsystemHandle</code> (Required)	Планирование <code>LookAtFinished</code>	Задача обеспечивает своё пробуждение
<code>LookAtHandle</code> (Optional)	Фрагмент может отсутствовать	Только <code>GetExternalDataPtr()</code>
<code>Priority</code> (структура)	Арбитраж между источниками взгляда	Обёртка над числом даёт расширяемость и UI
<code>EditCondition</code> + <code>EditConditionHides</code>	Скрытие неактуальных настроек	Разгружает панель узла
<code>uint8</code> для углов	Упаковка данных	Точность в градус достаточна
<code>TryActivateSystemicLookAt()</code>	Захват ресурса по приоритету	Намерение вместо действия

Что дальше

Глава 17 — `FMassZoneGraphPathFollowTask`, задача существенно сложнее. У неё восемь внешних хендлов вместо двух, вспомогательная структура

`FMassZoneGraphTargetLocation` с ручным `Reset()`, вход через `FStateTreePropertyRef` вместо копии значения, работа с `TOptional` полями и метод `RequestPath()`, принимающий `Mass`-контекст напрямую. Разберём, зачем задаче столько данных, как устроен запрос пути и почему навигация в `Mass` выглядит именно так.

Глава 17. Разбор `FMassZoneGraphPathFollowTask`

Вторая задача — на порядок сложнее первой. Восемь внешних хендлов, вспомогательная структура с ручным сбросом, вход через ссылку вместо копии и приватный метод, работающий напрямую с `Mass`-контекстом. Разберём, зачем всё это нужно.

Как и в главе 16: заголовок — из первых рук, реализации методов — реконструкция по сигнатурам и логике модуля.

17.1. Заголовки

cpp

```
#include "MassNavigationTypes.h"
#include "MassMovementTypes.h"
#include "MassCommonTypes.h"
#include "MassStateTreeTypes.h"
#include "StateTreePropertyRef.h"
#include "ZoneGraphTypes.h"
#include "MassZoneGraphPathFollowTask.generated.h"

struct FMassStateTreeExecutionContext;
struct FMassZoneGraphLaneLocationFragment;
struct FMassMoveTargetFragment;
struct FMassZoneGraphPathRequestFragment;
struct FMassZoneGraphShortPathFragment;
struct FMassZoneGraphCachedLaneFragment;
struct FAgentRadiusFragment;
struct FMassMovementParameters;
class UZoneGraphSubsystem;
```

Сравните с `MassLookAtTask.h`, где было три инcludes и одна форвард-декларация. Здесь шесть инклюдов и восемь форвардов.

Пропорция показательна: **инклюды — для типов по значению, форварды — для типов по ссылке**. Все восемь фрагментов используются только как параметры шаблона `TStateTreeExternalDataHandle<>`, поэтому полные определения не нужны.

Что тянется полностью:

- `ZoneGraphTypes.h` — ради `FZoneGraphLaneHandle` и `EZoneLaneLinkType`, лежащих в структуре по значению;
- `MassNavigationTypes.h` — `EMassMovementAction`;
- `MassCommonTypes.h` — `FMassInt16Real`;
- `MassMovementTypes.h` — `FMassMovementStyleRef`;
- `StateTreePropertyRef.h` — `FStateTreePropertyRef`.

Восемь форвард-деклараций против двух хендлов у задачи взгляда — первый признак того, насколько эта задача плотнее интегрирована в систему.

Что такое ZoneGraph

Короткое отступление для тех, кто не сталкивался. `ZoneGraph` — система навигации Unreal, альтернативная `NavMesh`. Вместо сетки проходимости она описывает мир **полосами** (lanes): дороги, тротуары, коридоры. Полосы соединяются связями, у каждой есть направление, ширина, теги.

Для толпы это удобнее `NavMesh`: агент движется вдоль полосы по одному числу — расстоянию от её начала. Не нужно искать путь по сетке, не нужно сглаживать, соседи автоматически выстраиваются в потоки.

Отсюда терминология: `LaneHandle` — полоса, `DistanceAlongLane` — позиция на ней, `LinkType` — способ перехода на соседнюю.

17.2. `FMassZoneGraphTargetLocation`

cpp

```
USTRUCT()
struct FMassZoneGraphTargetLocation
{
    GENERATED_BODY()

    void Reset()
    {
        LaneHandle.Reset();
        NextLaneHandle.Reset();
        NextExitLinkType = EZoneLaneLinkType::None;
    }
}
```

```

        bMoveReverse = false;
        TargetDistance = 0.0f;
        EndOfPathPosition.Reset();
        AnticipationDistance.Set(50.0f);
        EndOfPathIntent = EMassMovementAction::Move;
    }
    // ... поля
};

```

Отдельная структура, описывающая «куда идти». Она не instance data задачи и не фрагмент — это **разделяемое описание цели**, которое одни узлы дерева заполняют, а другие читают.

Ручной Reset()

Метод сбрасывает все поля в значения по умолчанию. Зачем он, если у полей есть инициализаторы?

Инициализаторы работают **при создании**. А структура переиспользуется: одна и та же цель перезаписывается при каждом новом маршруте. Без явного сброса остались бы хвосты от предыдущего пути — например, `NextLaneHandle` от старого маршрута, ведущий агента не туда.

Обратите внимание на асимметрию: `Reset()` сбрасывает `EndOfPathPosition`, но **не трогает** `EndOfPathDirection`. Похоже на недосмотр — хотя логика может быть в том, что направление читается только при заданной позиции, так что висящее значение безвредно.

Метод `inline` в заголовке — вызывается часто, тело тривиальное.

Поля: текущая полоса

cpp

```

/** Current lane handle. (Could be debug only) */
FZoneGraphLaneHandle LaneHandle;

/** If valid, the this lane will be set as current lane after the path follow
is completed. */
FZoneGraphLaneHandle NextLaneHandle;

/** Target distance along current lane. */
float TargetDistance = 0.0f;

```

Комментарий (Could be debug only) — Epic сомневается, нужно ли поле в релизе. Похоже, текущая полоса дублируется в `FMassZoneGraphLaneLocationFragment`, и здесь она для проверок.

`NextLaneHandle` — «куда перейти после завершения». Это позволяет строить непрерывное движение: агент дошёл до конца полосы, автоматически оказался на следующей, задача завершилась, дерево выбрало следующее действие.

`TargetDistance` — расстояние вдоль полосы, а не мировая координата. Вся навигация `ZoneGraph` работает в одномерном пространстве полосы.

Опциональный конец пути

cpp

```
/** Optional end of path location. */
TOptional<FVector> EndOfPathPosition;

/** Optional end of path direction, used only if EndOfPathPosition is set. */
TOptional<FVector> EndOfPathDirection;
```

`TOptional<T>` — «значение может отсутствовать». В отличие от «магического значения» вроде `FVector::ZeroVector`, здесь отсутствие выражено явно и проверяется через `IsSet()`.

Зачем сходить с полосы? Полосы описывают магистрали движения, но цель может быть в стороне: сесть на скамейку у дорожки, подойти к прилавку, встать у стены. Агент идёт по полосе, а последние метры — свободно к точке.

Комментарий у направления фиксирует зависимость: используется только при заданной позиции. Такая связь между полями — кандидат на ошибку, если её не документировать.

Упакованное вещественное

cpp

```
/** If start or end of path is off-lane, the distance along the lane is pushed
forward/back along the lane to make smoother transition. */
FMassInt16Real AnticipationDistance = FMassInt16Real(50.0f);
```

`FMassInt16Real` — вещественное число, упакованное в `int16`. Два байта вместо четырёх.

Тип из `MassCommonTypes.h`, характерный для Mass приём. Точности хватает для расстояний в сантиметрах в разумном диапазоне, а на масштабах толпы экономия памяти реальна.

Заметьте способ обращения: `AnticipationDistance.Set(50.0f)` в `Reset()`, а не присваивание. У упакованных типов доступ через методы, потому что внутри происходит преобразование.

Смысл поля: если агент выходит на полосу не с её начала (или сходит не в конце), точка входа/выхода сдвигается на 50 см вдоль полосы, чтобы поворот получился плавным, а не под прямым углом.

Направление и намерение

cpp

```
/** True, if we're moving reverse along the lane. */
bool bMoveReverse = false;

/** Movement intent at the end of the path */
EMassMovementAction EndOfPathIntent = EMassMovementAction::Move;

/** How the next lane handle is reached relative to the current lane. */
EZoneLaneLinkType NextExitLinkType = EZoneLaneLinkType::None;
```

`bMoveReverse` — полосы направленные, но идти можно и против направления (по тротуару в обе стороны).

`EndOfPathIntent` — что делать по прибытии: продолжить движение (`Move`) или остановиться (`Stand`). Это **намерение**, которое читает процессор движения: он либо плавно затормозит, либо продолжит на скорости.

Снова паттерн из главы 4: задача не тормозит агента, она объявляет, что произойдёт в конце.

`NextExitLinkType` — тип связи с следующей полосой: продолжение, поворот, перестроение, съезд. Процессор движения по этому типу выбирает форму траектории.

17.3. Instance data

cpp

```
/**
 * Follows a path long the current lane to a specified point.
```

```

*/
USTRUCT()
struct FMassZoneGraphPathFollowTaskInstanceData
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, Category = Input, meta=(RefType =
"/Script/MassAIBehavior.MassZoneGraphTargetLocation"))
    FStateTreePropertyRef TargetLocation;

    UPROPERTY(EditAnywhere, Category = Parameter)
    FMassMovementStyleRef MovementStyle;

    UPROPERTY(EditAnywhere, Category = Parameter)
    float SpeedScale = 1.0f;
};

```

Всего три поля — при восьми внешних хендлах. Интересная пропорция: задача почти ничего не хранит сама, всё берёт извне.

FStateTreePropertyRef — ссылка вместо копии

Это главное отличие от задачи взгляда, и его стоит разобрать подробно.

В FMassLookAtTask цель была копией:

cpp

```

UPROPERTY(EditAnywhere, Category = Input, meta = (Optional))
FMassEntityHandle TargetEntity; // 8 байт, копируются при каждом обновлении
привязок

```

Здесь — ссылка:

cpp

```

UPROPERTY(EditAnywhere, Category = Input, meta=(RefType = "..."))
FStateTreePropertyRef TargetLocation;

```

Почему? Посчитаем FMassZoneGraphTargetLocation: два хендла полосы, float, два TOptional<FVector> (по 25 байт каждый с флагом), упакованное вещественное, bool и два enum. Порядка 80–90 байт с выравниванием.

Копировать это при каждом обновлении привязок — на каждом тике, для каждого агента — расточительно. Ссылка стоит столько же, сколько указатель.

Второе преимущество тоньше: **общая память вместо снимка**. Узел, вычисливший цель, и задача, её исполняющая, работают с одним объектом. Если вычислитель обновит цель, задача увидит это немедленно, без ожидания следующего копирования привязок.

Метаданные `RefType` задают тип, на который ссылка может указывать. Путь в формате `/Script/<Модуль>.<Тип>` — стандартная запись пути к типу в рефлексии Unreal. Редактор по нему проверяет совместимость при связывании: подставить туда ссылку на другую структуру не выйдет.

Обращение в коде:

c++

```
FInstanceDataType& InstanceData = Context.GetInstanceData(*this);
FMassZoneGraphTargetLocation* TargetLocation =
InstanceData.TargetLocation.GetPtr<FMassZoneGraphTargetLocation>(Context);
if (TargetLocation == nullptr)
{
    return EStateTreeRunStatus::Failed;
}
```

Ссылка может быть невалидной, если в редакторе её не связали. Проверка обязательна.

Стиль движения

c++

```
UPROPERTY(EditAnywhere, Category = Parameter)
FMassMovementStyleRef MovementStyle;
```

Ещё одна обёртка-ссылка. `FMassMovementStyleRef` указывает на именованный стиль движения, определённый в настройках проекта: «шаг», «бег трусцой», «спешка», «прогулка».

Косвенность через имя, а не значение, даёт настраиваемость: дизайнер меняет скорость «прогулки» в одном месте, и все агенты, использующие этот стиль, обновляются.

Множитель скорости

c++


```
UPROPERTY(EditAnywhere, Category = Parameter)
float SpeedScale = 1.0f;
```

Модификатор поверх стиля. Позволяет чуть варьировать: один и тот же стиль «прогулка» с масштабом 0.9 и 1.1 даст разнообразие в толпе без заведения новых стилей.

Оба поля — `Parameter`, то есть настраиваются в редакторе, но при этом лежат в **instance data**, а не в задаче. Значит, к ним **можно привязать** значение от другого узла. Скажем, эвалюатор «уровень спешки» может динамически поднимать `SpeedScale`.

Сравните с `FMassLookAtTask`, где `Priority` и `LookAtMode` лежат в самой задаче и привязать к ним ничего нельзя. Разница именно в этом: здесь параметры сделаны динамическими сознательно.

17.4. Объявление задачи

cpp

```
USTRUCT(meta = (DisplayName = "ZG Path Follow"))
struct FMassZoneGraphPathFollowTask : public FMassStateTreeTaskBase
{
    GENERATED_BODY()

    using FInstanceDataType = FMassZoneGraphPathFollowTaskInstanceData;

protected:
    UE_API virtual bool Link(FStateTreeLinker& Linker) override;
    virtual const UStruct* GetInstanceDataType() const override { return
FInstanceDataType::StaticStruct(); };
    UE_API virtual EStateTreeRunStatus EnterState(FStateTreeExecutionContext&
Context, const FStateTreeTransitionResult& Transition) const override;
    UE_API virtual EStateTreeRunStatus Tick(FStateTreeExecutionContext&
Context, const float DeltaTime) const override;
    UE_API virtual void
GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
override;
```

`DisplayName = "ZG Path Follow"` — короткое имя. На узлах дерева место ограничено, длинные имена не помещаются.

Чего здесь нет

Два отсутствия говорят больше, чем присутствия.

Нет `ExitState()`. Задача взгляда его переопределяла, чтобы отпустить фрагмент. Здесь — нет.

Почему? Потому что оставить намерение движения после выхода из состояния **безопаснее**, чем сбросить. Если задача обнулит цель движения, агент резко остановится посреди перекрёстка. А если намерение останется, он спокойно дойдёт до точки, пока дерево выбирает следующее действие.

Следующая навигационная задача всё равно перезапишет цель. Это осознанное решение, а не упущение.

Нет конструктора. Значит, флаги по умолчанию (глава 15). В частности, `bShouldCallTick = true` — задача тикает, чтобы отслеживать прогресс.

Лишняя точка с запятой после `GetInstanceDataType()` — снова опечатка Epic, безвредная.

17.5. Восемь хендлов

cpp

```
TStateTreeExternalDataHandle<FMassZoneGraphLaneLocationFragment>
LocationHandle;
TStateTreeExternalDataHandle<FMassMoveTargetFragment> MoveTargetHandle;
TStateTreeExternalDataHandle<FMassZoneGraphPathRequestFragment>
PathRequestHandle;
TStateTreeExternalDataHandle<FMassZoneGraphShortPathFragment> ShortPathHandle;
TStateTreeExternalDataHandle<FMassZoneGraphCachedLaneFragment>
CachedLaneHandle;
TStateTreeExternalDataHandle<FAgentRadiusFragment> AgentRadiusHandle;
TStateTreeExternalDataHandle<FMassMovementParameters> MovementParamsHandle;
TStateTreeExternalDataHandle<UZoneGraphSubsystem> ZoneGraphSubsystemHandle;
```

Все восемь — обязательные. Ни одного `Optional`, в отличие от задачи взгляда.

Логика простая: если у агента нет фрагментов навигации, он в принципе не может ходить по `ZoneGraph`. Дерево с такой задачей на такого агента ставить бессмысленно — пусть падает при линковке, а не тихо ничего не делает.

Разберём назначение каждого.

Хендл	Что это
<code>LocationHandle</code>	Где агент сейчас: полоса и расстояние вдоль неё

Хендл	Что это
MoveTargetHandle	Выход: куда двигаться. Читается процессором движения
PathRequestHandle	Заявка на построение пути
ShortPathHandle	Результат: короткий путь из нескольких точек
CachedLaneHandle	Кэш геометрии полосы, чтобы не дёргать подсистему
AgentRadiusHandle	Радиус агента — влияет на смещение от края полосы
MovementParamsHandle	Параметры движения: максимальная скорость, ускорение
ZoneGraphSubsystemHandle	Доступ к самому графу зон

Обратите внимание на `FMassMovementParameters`. Судя по имени и по тому, что он запрашивается как внешние данные, это, скорее всего, **общий фрагмент** — параметры движения одинаковы для всех агентов данного типа. Схема пропускает и обычные, и общие фрагменты (глава 7).

Разделение труда

Ключевое наблюдение: три фрагмента образуют конвейер.

`FMassZoneGraphPathRequestFragment` — задача записывает сюда заявку: «построй путь отсюда туда».

`FMassZoneGraphShortPathFragment` — процессор построения пути записывает сюда результат: последовательность точек.

`FMassMoveTargetFragment` — задача (или процессор) записывает сюда текущую цель движения, которую читает процессор перемещения.

То есть задача не строит путь. Она **заказывает** его и потом проверяет, готов ли. Полное воплощение принципа «дерево решает, но не делает».

Link и GetDependencies

cpp

```
// реконструкция
bool FMassZoneGraphPathFollowTask::Link(FStateTreeLinker& Linker)
{
    Linker.LinkExternalData(LocationHandle);
    Linker.LinkExternalData(MoveTargetHandle);
    Linker.LinkExternalData(PathRequestHandle);
    Linker.LinkExternalData(ShortPathHandle);
    Linker.LinkExternalData(CachedLaneHandle);
}
```

```

    Linker.LinkExternalData(AgentRadiusHandle);
    Linker.LinkExternalData(MovementParamsHandle);
    Linker.LinkExternalData(ZoneGraphSubsystemHandle);
    return true;
}

void
FMassZoneGraphPathFollowTask::GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
{
    Builder.AddReadOnly<FMassZoneGraphLaneLocationFragment>();
    Builder.AddReadWrite<FMassMoveTargetFragment>();
    Builder.AddReadWrite<FMassZoneGraphPathRequestFragment>();
    Builder.AddReadWrite<FMassZoneGraphShortPathFragment>();
    Builder.AddReadWrite<FMassZoneGraphCachedLaneFragment>();
    Builder.AddReadOnly<FAgentRadiusFragment>();
    Builder.AddReadOnly<FMassMovementParameters>();
    Builder.AddReadOnly<UZoneGraphSubsystem>();
}

```

Восемь строк в каждом методе. Здесь двойная бухгалтерия из главы 5 ощущается физически: пропустить одну строку из восьми легко, а последствия — гонка данных без единого предупреждения компилятора.

Распределение доступа: то, куда задача пишет, — `ReadWrite`; справочные данные — `ReadOnly`. Позиция агента только читается (её пишет процессор движения), радиус и параметры — константы конфигурации, подсистема `ZoneGraph` только опрашивается.

Заметьте цену: пять фрагментов на `ReadWrite`. Это значит, что процессор `StateTree`, обслуживающий дерево с этой задачей, конфликтует со всеми, кто трогает эти пять фрагментов. Ровно та ситуация из главы 8, раздел 8.9: широкое дерево — широкие требования — меньше параллелизма.

17.6. RequestPath

cpp

```

UE_API bool RequestPath(FMassStateTreeExecutionContext& Context, const
    FMassZoneGraphTargetLocation& TargetLocation) const;

```

Единственный вспомогательный метод — и его сигнатура отличается от аналога в задаче взгляда.

Почему Mass-контекст напрямую

Сравните:

cpp

```
// FMassLookAtTask
bool TryActivateSystemicLookAt(const FStateTreeExecutionContext& Context, ...)
const;

// FMassZoneGraphPathFollowTask
UE_API bool RequestPath(FMassStateTreeExecutionContext& Context, ...) const;
```

Первая принимает базовый контекст, вторая — **Mass-специфичный**, и неконстантный.

Это означает, что методу нужно что-то, чего в базовом контексте нет. Скорее всего — `GetEntity()` и `GetMassEntityExecutionContext()` (глава 12): построение пути требует знать, для какой сущности он строится, и, возможно, воспользоваться командным буфером.

Неконстантность подсказывает, что метод меняет состояние контекста или через него — состояние мира.

Приведение делает вызывающий:

cpp

```
// реконструкция
EStateTreeRunStatus
FMassZoneGraphPathFollowTask::EnterState(FStateTreeExecutionContext& Context,
const FStateTreeTransitionResult& Transition) const
{
    FMassStateTreeExecutionContext& MassContext =
static_cast<FMassStateTreeExecutionContext*>(Context);
    // ...
    if (!RequestPath(MassContext, *TargetLocation))
    {
        return EStateTreeRunStatus::Failed;
    }
    return EStateTreeRunStatus::Running;
}
```

Это тот самый `static_cast`, о котором говорилось в главе 12: безопасен только потому, что схема гарантирует исполнение Mass-контекстом.

`UE_API` у метода означает, что он экспортируется из модуля. Для приватного помощника это необычно — вероятно, наследие или задел под переиспользование в других задачах

модуля.

Что делает

cpp

```
// реконструкция
bool FMassZoneGraphPathFollowTask::RequestPath(FMassStateTreeExecutionContext&
Context, const FMassZoneGraphTargetLocation& TargetLocation) const
{
    const FMassZoneGraphLaneLocationFragment& Location =
Context.GetExternalData(LocationHandle);
    FMassZoneGraphPathRequestFragment& PathRequest =
Context.GetExternalData(PathRequestHandle);
    FMassMoveTargetFragment& MoveTarget =
Context.GetExternalData(MoveTargetHandle);
    const FAgentRadiusFragment& AgentRadius =
Context.GetExternalData(AgentRadiusHandle);
    const FMassMovementParameters& MovementParams =
Context.GetExternalData(MovementParamsHandle);

    if (Location.LaneHandle != TargetLocation.LaneHandle)
    {
        return false;    // цель на другой полосе – этой задаче не по силам
    }

    // заполнить заявку
    PathRequest.PathRequest.StartPosition = ...;
    PathRequest.PathRequest.TargetDistance = TargetLocation.TargetDistance;
    PathRequest.PathRequest.bMoveReverse = TargetLocation.bMoveReverse;
    PathRequest.PathRequest.AnticipationDistance =
TargetLocation.AnticipationDistance;
    // ...

    // выставить намерение движения
    MoveTarget.CreateNewAction(EMassMovementAction::Move, ...);

    MoveTarget.DesiredSpeed.Set(MovementParams.GenerateDesiredSpeed(MovementStyle,
...) * SpeedScale);

    return true;
}
```

Проверка «цель на той же полосе» существенна. Задача называется `PathFollow`, а не `PathFind`: она ведёт агента вдоль **текущей** полосы к точке на ней. Смена полос —

работа других узлов дерева.

Это важный принцип декомпозиции в Mass-навигации: каждая задача делает одну простую вещь, а сложное поведение собирается из них в дереве.

17.7. Реконструкция жизненного цикла

EnterState

cpp

```
// реконструкция
EStateTreeRunStatus
FMassZoneGraphPathFollowTask::EnterState(FStateTreeExecutionContext& Context,
const FStateTreeTransitionResult& Transition) const
{
    FMassStateTreeExecutionContext& MassContext =
static_cast<FMassStateTreeExecutionContext*>(Context);
    FInstanceDataType& InstanceData = Context.GetInstanceData(*this);

    FMassZoneGraphTargetLocation* TargetLocation =
        InstanceData.TargetLocation.GetPtr<FMassZoneGraphTargetLocation>
(Context);

    if (TargetLocation == nullptr || !TargetLocation->LaneHandle.IsValid())
    {
        return EStateTreeRunStatus::Failed;
    }

    if (!RequestPath(MassContext, *TargetLocation))
    {
        return EStateTreeRunStatus::Failed;
    }

    return EStateTreeRunStatus::Running;
}
```

Две проверки перед делом: ссылка связана и цель осмысленна. Возврат `Failed` при неудаче — дерево выберет другую ветку.

Tick

cpp

```

// реконструкция
EStateTreeRunStatus
FMassZoneGraphPathFollowTask::Tick(FStateTreeExecutionContext& Context, const
float DeltaTime) const
{
    const FMassZoneGraphShortPathFragment& ShortPath =
Context.GetExternalData(ShortPathHandle);
    const FMassMoveTargetFragment& MoveTarget =
Context.GetExternalData(MoveTargetHandle);

    if (ShortPath.bDone)
    {
        return EStateTreeRunStatus::Succeeded;
    }

    if (MoveTarget.GetCurrentAction() == EMassMovementAction::Stand)
    {
        return EStateTreeRunStatus::Succeeded;
    }

    return EStateTreeRunStatus::Running;
}

```

Задача **проверяет чужую работу**. Она не двигает агента и не считает расстояние — только смотрит на флаг завершения, который выставил процессор пути.

Это чистейшая иллюстрация принципа из главы 4. Сравните объём: `EnterState()` заполняет заявку, `Tick()` читает один флаг. Вся тяжёлая работа — в процессорах, которые обрабатывают тысячи агентов векторизованно.

Проблема пробуждения

Здесь есть тонкость, о которой стоит подумать. Задача возвращает `Running` и ждёт, пока процессор пути выставит флаг. Но кто разбудит дерево?

В отличие от задачи взгляда, здесь нет отложенного сигнала — время прибытия заранее неизвестно. Варианты:

1. Процессор пути шлёт сигнал при завершении (по аналогии с `LookAtFinished`).
2. Дерево тикается регулярно по другим причинам — например, из-за других задач в том же состоянии.
3. Ставится периодический отложенный сигнал для опроса.

Судя по отсутствию `UMassSignalSubsystem` среди хендлов задачи, она сама сигналов не шлёт и не планирует. Значит, пробуждение обеспечивает кто-то извне.

Это важный практический момент. Если вы вставите эту задачу в состояние в одиночку и ничто больше не будет будить дерево, агент может застрять в состоянии `Running` после фактического прибытия. Проверяйте, что в вашей конфигурации есть источник пробуждения.

17.8. Сравнение двух задач

Сведём различия в таблицу — это лучший способ увидеть спектр возможных решений.

Аспект	<code>FMassLookAtTask</code>	<code>FMassZoneGraphPathFollowTask</code>
Внешних хендлов	2	8
Опциональные хендлы	Да (фрагмент взгляда)	Нет, все обязательны
Вход	Копия (<code>FMassEntityHandle</code>)	Ссылка (<code>FStateTreePropertyRef</code>)
Параметры	В задаче (статические)	В instance data (динамические)
<code>ExitState</code>	Есть — отпускает ресурс	Нет — намерение остаётся
Завершение	Отложенный сигнал по таймеру	Проверка флага чужого процессора
Помощник принимает	Базовый контекст	Mass-контекст
Арбитраж	Да, по приоритету	Нет

Ни один вариант не «правильнее» — они отвечают разным задачам. Но выбор в каждой паре объясним:

- **Опциональность** — там, где данные могут отсутствовать по дизайну (голова есть не у всех), и обязательность там, где их отсутствие означает неприменимость задачи.
- **Ссылка вместо копии** — когда данные большие и должны быть общими.
- **Параметры в instance data** — когда нужна привязка.
- `ExitState` — когда захвачен ресурс, требующий явного освобождения.

17.9. Как строится маршрут целиком

Одна задача — маленький кирпич. Полное поведение «дойти до магазина» собирается из нескольких.

Типичная структура состояния:

Состояние "Идти к цели"

- └─ Задача: найти путь (заполняет `FMassZoneGraphTargetLocation`)
- └─ Задача: `ZG Path Follow` (читает её по ссылке)
- └─ Задача: `Mass LookAt` (фоновая, `bConsideredForCompletion = false`)

Первая задача вычисляет цель и записывает её в структуру. Вторая читает по `FStateTreePropertyRef` — без копирования. Третья работает параллельно и не влияет на завершение состояния.

Когда `Path Follow` вернёт `Succeeded`, состояние завершится, дерево выберет следующее. Если цель на другой полосе, `Path Follow` вернёт `Failed` при входе, и дерево пойдёт по ветке «искать переход на другую полосу».

Именно так и выглядит правильное Mass-поведение: **много маленьких задач, каждая из которых делает одну проверяемую вещь.**

17.10. Уроки главы

Ссылка вместо копии для больших структур. `FStateTreePropertyRef` с `RefType` — когда данные больше десятка байт или должны быть общими.

Параметры в `instance data`, если нужна привязка. Иначе — в задаче, это экономит память.

Обязательные хендлы там, где отсутствие данных означает неприменимость. Опциональные — там, где отсутствие нормально.

Не сбрасывайте намерение в `ExitState()` без нужды. Резкий сброс может дать визуальный артефакт; следующая задача всё равно перезапишет.

Задача проверяет, а не делает. Заполнить заявку, прочитать флаг — вот и весь объём работы в горячем пути.

Упакованные типы для количественных данных. `FMassInt16Real` вместо `float` там, где точность позволяет.

`TOptional` вместо магических значений. Явное отсутствие лучше договорённости «ноль означает нет».

Проверяйте источник пробуждения. Задача, ждущая чужой работы, зависит от того, что кто-то разбудит дерево.

17.11. Итог главы

Элемент	Роль
<code>FMassZoneGraphTargetLocation</code>	Разделяемое описание цели; заполняется одним узлом, читается другим
<code>Reset()</code>	Обязателен при переиспользовании — инициализаторы работают только при создании
<code>FStateTreePropertyRef + RefType</code>	Ссылка на структуру вместо копии; экономия и общая память
<code>TOptional<FVector></code>	Явное «может отсутствовать» вместо магического значения
<code>FMassInt16Real</code>	Упаковка вещественного в два байта
Восемь обязательных хендлов	Агент без навигации не может исполнять эту задачу вовсе
Конвейер <code>Request</code> → <code>ShortPath</code> → <code>MoveTarget</code>	Задача заказывает, процессоры исполняют
Отсутствие <code>ExitState()</code>	Намерение движения переживает выход из состояния намеренно
<code>RequestPath(FMassStateTreeExecutionContext&)</code>	Помощнику нужна сущность — значит, Mass-контекст напрямую

Что дальше

Глава 18 — три оставшихся вида узлов в Mass-контексте: эвалюаторы, условия и `property functions`. Разберём `FStateTreeEvaluatorBase` целиком (`TreeStart`, `TreeStop`, `Tick`), поймём, чем эвалюатор принципиально отличается от задачи и почему в Mass он используется реже, чем можно было бы ожидать. Посмотрим на особенности условий — почему у них нет обычной `instance data` и как это связано с временными данными из главы 10. И разберём `property functions` как самый дешёвый способ вычислить значение прямо в привязке.

Глава 18. Эвалюаторы, условия и `property functions`

Задачи мы разобрали. Остались три вида узлов — и все они устроены иначе. Эвалюатор работает вне состояний, условие не имеет обычной instance data, а property function вычисляется прямо внутри привязки. В Mass-контексте у каждого свои особенности и свои подводные камни.

18.1. FStateTreeEvaluatorBase целиком

Файл `StateTreeEvaluatorBase.h` — 61 строка, самый короткий в пакете.

cpp

```
/**
 * Base struct of StateTree Evaluators.
 * Evaluators calculate and expose data to be used for decision making in a
 * StateTree.
 */
USTRUCT(meta = (Hidden))
struct FStateTreeEvaluatorBase : public FStateTreeNodeBase
{
    GENERATED_BODY()

    /**
     * Called when StateTree is started.
     */
    virtual void TreeStart(FStateTreeExecutionContext& Context) const {}

    /**
     * Called when StateTree is stopped.
     */
    virtual void TreeStop(FStateTreeExecutionContext& Context) const {}

    /**
     * Called each frame to update the evaluator.
     * @param DeltaTime Time since last StateTree tick, or 0 if called during
     * preselection.
     */
    virtual void Tick(FStateTreeExecutionContext& Context, const float
    DeltaTime) const {}
}
```

Три метода против пяти у задачи. И ключевое отличие в комментариях к структуре: эвалюаторы **вычисляют и публикуют данные для принятия решений**.

Чем эвалюатор отличается от задачи

Разница фундаментальная, и её стоит проговорить явно.

Задача привязана к состоянию. Она живёт, пока состояние активно. Вошли — `EnterState()` , вышли — `ExitState()` .

Эвалюатор глобален. Он работает всё время исполнения дерева, независимо от того, какое состояние активно. Отсюда и имена методов: не `EnterState` , а `TreeStart` ; не `ExitState` , а `TreeStop` .

Задача возвращает статус. `Running` , `Succeeded` , `Failed` — она влияет на течение исполнения.

Эвалюатор ничего не возвращает. Все три метода — `void` . Он не может завершить состояние, не может запросить переход, не может провалиться. Его единственный способ повлиять на мир — записать значение в свою instance data, чтобы условия и задачи могли его прочитать через привязки.

Типичное применение: эвалюатор «расстояние до игрока» каждый тик считает дистанцию и выкладывает наружу. Условия переходов сравнивают её с порогом. Задачи используют как параметр.

Загадочный `DeltaTime = 0`

cpp

```
* @param DeltaTime Time since last StateTree tick, or 0 if called during preselection.
```

Эвалюатор тикается не только в обычном тике, но и **во время предварительного выбора состояния**. Дерево «примеряет» состояние, ему нужны свежие данные для проверки условий входа — и оно прогоняет эвалюаторы с нулевой дельтой.

Практическое следствие: **эвалюатор должен корректно работать при `DeltaTime == 0`** . Любое деление на дельту или накопление вида `Value += Rate * DeltaTime` должно это учитывать.

cpp

```
void FMyEvaluator::Tick(FStateTreeExecutionContext& Context, const float DeltaTime) const
{
    FInstanceDataType& InstanceData = Context.GetInstanceData(*this);

    // вычисление, не зависящее от дельты — безопасно
    InstanceData.DistanceToTarget = ComputeDistance(Context);
```

```

// накопление – требует проверки
if (DeltaTime > 0.f)
{
    InstanceData.SmoothedValue =
FMath::FInterpTo(InstanceData.SmoothedValue,
    InstanceData.DistanceToTarget, DeltaTime, 2.f);
}
}

```

Это одна из тех деталей, которые не всплывают при написании и стреляют потом: значение «дёргается», потому что при предвыборе сглаживание не сработало, а при обычном тике сработало.

Отладочный блок

cpp

```

#ifdef WITH_GAMEPLAY_DEBUGGER
    UE_API virtual FString GetDebugInfo(const
FStateTreeReadOnlyExecutionContext& Context) const;

    UE_DEPRECATED(5.8, "Use the version with the
FStateTreeReadOnlyExecutionContext.")
    virtual void AppendDebugInfoString(FString& DebugString, const
FStateTreeExecutionContext& Context) const final
    {
    }
}
#endif // WITH_GAMEPLAY_DEBUGGER

```

Тот же приём `virtual ... final`, что мы разбирали в главах 11 и 15: старый метод запечатан, переопределить нельзя — компилятор заставит перейти на новый.

Обратите внимание на разницу сигнатур. Старая: `void` плюс выходной параметр `FString&`. Новая: возврат `FString` по значению. Второе чище и, благодаря оптимизации возвращаемого значения, не дороже.

`GetDebugInfo()` для эвалюатора **особенно ценен**. В отличие от задачи, эвалюатор ничего видимого не делает — он только считает числа. Понять, что он насчитал, без отладочного вывода невозможно.

FStateTreeEvaluatorCommonBase

cpp

```

/**
 * Base class (namespace) for all common Evaluators that are generally
 * applicable.
 * This allows schemas to safely include all Evaluators child of this struct.
 */
USTRUCT(Meta=(Hidden))
struct FStateTreeEvaluatorCommonBase : public FStateTreeEvaluatorBase
{
    GENERATED_BODY()
};

```

Маркер «безопасен в любой схеме», разобранный в главе 7. Заметьте написание `Meta` с большой буквы — в `StateTreeTaskBase.h` было `meta`. Обе формы работают; непоследовательность в коде движка обычное дело.

18.2. Эвалюаторы в Mass: почему их мало

Здесь стоит остановиться. Эвалюатор кажется естественным инструментом: «посчитать расстояние и выложить наружу». Но в реальных Mass-деревьях их используют реже, чем можно ожидать. Разберём почему.

Причина первая: эвалюатор тикает всегда

Задача с `bShouldCallTick = false` не тикает вообще. Эвалюатор такого флага не имеет — он тикается на каждом тике дерева, пока дерево работает.

Для агента, который спит между сигналами, это не проблема: дерево не тикает, значит и эвалюатор не тикает. Но как только дерево просыпается по любой причине, тикают **все** эвалюаторы, даже если их результат никому сейчас не нужен.

Причина вторая: работа дублирует процессоры

Вот главное. Эвалюатор считает что-то для **одного** агента внутри тика дерева. Процессор считает то же самое для **всей толпы** одним проходом по плотным массивам.

Сравните два способа получить расстояние до игрока:

Через эвалюатор. На каждом тике дерева каждого агента: получить позицию агента, получить позицию игрока, посчитать. Разрозненные обращения к памяти, виртуальный вызов, всё внутри исполнения дерева.

Через процессор. Один проход по чанкам: читаем массив трансформов подряд, пишем массив расстояний подряд. Векторизуется, параллелится, кэш работает идеально.

Второй способ на порядок эффективнее. Поэтому в Mass типичная схема такая: **процессор считает и кладёт во фрагмент, задача или условие читает фрагмент как внешние данные.**

cpp

```
// вместо эвалюатора
USTRUCT()
struct FMyDistanceToPlayerFragment : public FMassFragment
{
    GENERATED_BODY()
    float Distance = 0.f;
};

// условие читает его напрямую
struct FMyDistanceCondition : public FMassStateTreeConditionBase
{
    // ...
    TStateTreeExternalDataHandle<FMyDistanceToPlayerFragment> DistanceHandle;
};
```

Когда эвалюатор всё же уместен

Три случая:

Данные нужны только этому дереву. Заводить фрагмент и процессор ради значения, которое использует одно поведение из двадцати, — оверинжиниринг. Эвалюатор проще.

Вычисление зависит от параметров дерева. Процессор одинаков для всех агентов чанка; эвалюатор может учитывать глобальные параметры конкретного экземпляра дерева.

Нужны TreeStart / TreeStop. Эвалюатор — единственный узел, который получает уведомления о старте и остановке всего дерева. Если нужно что-то захватить на всё время жизни поведения — только он.

Практическое правило: **если значение может понадобиться больше чем одному дереву — делайте фрагмент и процессор. Если это внутреннее дело одного поведения — эвалюатор.**

FMassStateTreeEvaluatorBase

Mass-версия из главы 5:

cpp


```

USTRUCT(meta = (Hidden, DisplayName = "Mass Evaluator Base"))
struct FMassStateTreeEvaluatorBase : public FStateTreeEvaluatorBase
{
    GENERATED_BODY()

    virtual void
    GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
    {
    }
};

```

Всё как у задачи: место для объявления зависимостей плюс пропуск в схему. Та же двойная бухгалтерия `Link()` / `GetDependencies()`.

18.3. Условия

`StateTreeConditionBase.h` в наш пакет не попал, но структуру условия мы можем восстановить по использованию и по тому, что уже знаем.

cpp

```

// реконструкция
USTRUCT(meta = (Hidden))
struct FStateTreeConditionBase : public FStateTreeNodeBase
{
    GENERATED_BODY()

    virtual bool TestCondition(FStateTreeExecutionContext& Context) const {
return false; }

    // операнд для комбинирования с предыдущим условием
    UPROPERTY()
    EStateTreeExpressionOperand Operand = EStateTreeExpressionOperand::And;

    // глубина вложенности в выражении (скобки)
    UPROPERTY()
    int8 DeltaIndent = 0;
};

```

Один метод, возвращающий `bool`. Плюс поля, описывающие, как условие комбинируется с соседними.

Выражения из условий

Помните `EStateTreeExpressionOperand` из главы 3?

cpp

```
enum class EStateTreeExpressionOperand : uint8
{
    Copy UMETA(Hidden),
    And,           // для условий — AND, для полезности — Min(a, b)
    Or,            // для условий — OR, для полезности — Max(a, b)
    Multiply,      // (a * b), только для полезности
};
```

Условия в списке образуют логическое выражение. Первое имеет операнд `Copy` (просто взять значение), остальные — `And` или `Or`. Поле отступа даёт скобки: `A AND (B OR C)`.

Это существенно отличает `StateTree` от `Behavior Tree`, где для комбинирования нужны узлы-декораторы. Здесь всё выражается плоским списком с операндами — компактнее и дешевле.

Почему у условий нет обычной instance data

Вот действительно неочевидный момент, и он связан с главой 10.

Условие вычисляется **до** входа в состояние. В этот момент instance data состояния ещё не выделена — состояние только «примеряется».

Отсюда особые типы источников данных из главы 3:

cpp

```
SharedInstanceData, SharedInstanceDataObject    // общие данные условий
EvaluationScopeInstanceData, ...Object          // временные, живут одно
вычисление
```

`SharedInstanceData` — данные, разделяемые всеми экземплярами условия. Условие одно на ассет, его настройки одинаковы для всех агентов, менять их в рантайме нельзя.

`EvaluationScopeInstanceData` — временные данные, создаваемые на время одного вычисления и немедленно уничтожаемые. Именно для них в контексте существует стек областей (глава 11):

cpp

```

UE_API void
PushEvaluationScopeInstanceContainer(UE::StateTree::InstanceData::FEvaluationScopeInstanceContainer& Container, const FStateTreeExecutionFrame& Frame);
UE_API void
PopEvaluationScopeInstanceContainer(UE::StateTree::InstanceData::FEvaluationScopeInstanceContainer& Container);

```

Практическое следствие: условие не может ничего запомнить между вызовами.

Никаких счётчиков, никакого гистерезиса, никакого «в прошлый раз было так». Каждое вычисление — с чистого листа.

Если нужна память — она должна жить снаружи: во фрагменте Mass или в instance data эвалюатора.

Условия в Mass

cpp

```

USTRUCT(meta = (Hidden, DisplayName = "Mass Condition Base"))
struct FMassStateTreeConditionBase : public FStateTreeConditionBase
{
    GENERATED_BODY()

    virtual void
    GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
    {
    }
};

```

Условие может запрашивать внешние данные — фрагменты и подсистемы — точно так же, как задача. Это основной способ читать состояние агента:

cpp

```

USTRUCT(meta = (DisplayName = "Has Valid Target"))
struct FMyHasTargetCondition : public FMassStateTreeConditionBase
{
    GENERATED_BODY()

    using FInstanceDataType = FMyHasTargetConditionInstanceData;

protected:
    virtual bool Link(FStateTreeLinker& Linker) override
    {
    }
};

```

```

        Linker.LinkExternalData(TargetHandle);
        return true;
    }

    virtual void
    GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
    override
    {
        Builder.AddReadOnly<FMyTargetFragment>();
    }

    virtual bool TestCondition(FStateTreeExecutionContext& Context) const
    override
    {
        const FMyTargetFragment* Target =
        Context.GetExternalDataPtr(TargetHandle);
        return Target != nullptr && Target->TargetEntity.IsSet();
    }

    TStateTreeExternalDataHandle<FMyTargetFragment,
    EStateTreeExternalDataRequirement::Optional> TargetHandle;
};

```

Обратите внимание: доступ **только на чтение**. Условие, которое что-то меняет, — почти всегда ошибка проектирования: оно вызывается непредсказуемое число раз, в том числе при проверке состояний, которые в итоге не выберут.

Стоимость условий

Условия — самая часто вызываемая часть дерева. При выборе состояния проверяются условия входа на каждом уровне иерархии, для каждого кандидата. Одно переключение состояния может означать десятки вычислений условий.

Отсюда правило: **условия должны быть дешёвыми**. Сравнение чисел, проверка флага, чтение поля фрагмента. Никаких трассировок, поисков по миру, обходов массивов.

Если проверка дорогая — вынесите вычисление в процессор, положите результат во фрагмент, а условие пусть читает готовое значение.

18.4. Property functions

Третий вид узла — самый малоизвестный и при этом очень полезный.

cpp

```

USTRUCT(meta = (Hidden, DisplayName = "Mass Property Function Base"))
struct FMassStateTreePropertyFunctionBase : public
FStateTreePropertyFunctionBase
{
    GENERATED_BODY()

    virtual void
    GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
    {
    }
};

```

Базовый FStateTreePropertyFunctionBase живёт в StateTreePropertyFunctionBase.h (подключён из MassStateTreeTypes.h, но в пакет не попал). По назначению восстанавливается однозначно.

Что это такое

Property function — маленький вычислитель, встраиваемый **прямо в привязку свойства**.

Обычная привязка соединяет выход одного узла с входом другого напрямую:

```
Эвалюатор.Distance → Задача.Duration
```

С property function между ними встает преобразование:

```
Эвалюатор.Distance → [Divide by 100] → Задача.Duration
```

Или значение генерируется без источника:

```
[Random Float 1.0..3.0] → Задача.Duration
```

Типичные функции: арифметика, случайные числа, преобразования типов, выборка из массива, интерполяция.

Зачем это нужно

Без property functions каждое такое преобразование требовало бы отдельного узла-эвалюатора. Дерево из десяти состояний обрастало бы двадцатью служебными эвалюаторами, которые ничего не делают, кроме «умножить на два».

Property function не является отдельным узлом дерева — она часть привязки. Не занимает места в списке эвалюаторов, не тикает отдельно, вычисляется ровно тогда,

когда копируется привязка.

Особенность для Mass: случайность

Самое частое применение в толпе — **разнообразие**.

Тысяча агентов с одинаковым деревом ведут себя синхронно: все встают одновременно, все ждут ровно две секунды, все идут с одной скоростью. Выглядит как марширующие клоны.

Property function `Random` решает это в одну привязку:

```
[Random Float 1.5 .. 4.0] → LookAt.Duration
```

Каждый агент получит своё значение при копировании привязки. Никакого кода, никаких дополнительных фрагментов.

Здесь важно вспомнить про `RandomSeed` из главы 11:

cpp

```
struct FStartParameters
{
    /** Optional override of initial seed for RandomStream. By default
    FPlatformTime::Cycles() will be used. */
    TOptional<int32> RandomSeed;
};
```

Случайность в дереве берётся из потока, инициализированного при `Start()`. По умолчанию сид — от таймера, то есть недетерминирован. Для сетевой игры это источник рассинхрона: сервер и клиент получают разные значения.

Если используете случайные property functions в сетевом Mass-проекте — задавайте сид явно, например производный от индекса сущности.

Зависимости у property function

Метод `GetDependencies()` есть и здесь. Значит, property function может обращаться к фрагментам:

cpp

```
// гипотетическая функция «скорость агента»
USTRUCT(meta = (DisplayName = "Agent Speed"))
```

```

struct FMyAgentSpeedFunction : public FMassStateTreePropertyFunctionBase
{
    GENERATED_BODY()

    // ...
    virtual void
    GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
    override
    {
        Builder.AddReadOnly<FMassVelocityFragment>();
    }

    TStateTreeExternalDataHandle<FMassVelocityFragment> VelocityHandle;
};

```

Полезно, но помните о цене из главы 8: каждый фрагмент в требованиях сужает параллелизм. Для функции, которая вычисляется в каждой привязке, стоит трижды подумать, нельзя ли обойтись без внешних данных.

18.5. Сводная таблица четырёх видов узлов

Критерий	Задача (Task)	Эвалюатор (Evaluator)	Условие (Condition)	Функция свойства (Property Function)
Привязан к	Состоянию	Дереву	Проверке (Переход / Вход)	Привязке (Property Binding)
Возвращает	EStateTree RunStatus	void	bool	Значение
Instance Data	Полноценная, персональная	Полноценная, персональная	Общая и временная	Обычно нет
Может помнить между вызовами	Да	Да	Нет	Нет
Влияет на исполнение	Да, статусом (RunStatus)	Нет, только данными	Да, выбором состояния	Нет
Частота вызова	Каждый тик состояния	Каждый тик дерева + предвыбор	Множественно при выборе	При копировании привязки
Может писать во фрагменты	Да	Да	Не следует	Не следует

Критерий	Задача (Task)	Эвалюатор (Evaluator)	Условие (Condition)	Функция свойства (Property Function)
Mass-база	FMassStateTreeTaskBase	FMassStateTreeEvaluatorBase	FMassStateTreeConditionBase	FMassStateTreePropertyFunction

18.6. Как выбрать вид узла

Практическое дерево решений для Mass.

Нужно совершить действие, занимающее время? → Задача.

Нужно принять решение о переходе? → Условие.

Нужно вычислить значение, которое используют несколько узлов? → Сначала подумайте, не сделать ли фрагмент плюс процессор. Если данные нужны только этому дереву — эвалюатор.

Нужно преобразовать одно значение в другое прямо в привязке? → Property function.

Нужно захватить что-то на всё время жизни поведения? → Эвалюатор с `TreeStart` / `TreeStop`.

Нужно, чтобы значение было разным у разных агентов? → Property function `Random` в привязке, либо фрагмент, заполняемый при спавне.

18.7. Типичные ошибки

Эвалюатор вместо процессора. Самая частая архитектурная ошибка в Mass. Если вычисление применимо ко всем агентам — это процессор, а не эвалюатор.

Состояние в условии. Условие не может помнить. Попытка завести счётчик «сколько раз проверяли» не работает — данные временные.

Дорогое условие. Трассировка луча в условии входа — гарантированная просадка. Условия вызываются каскадом при каждом выборе состояния.

Забыть про `DeltaTime == 0` в эвалюаторе. При предвыборе дельта нулевая. Любое сглаживание или интегрирование должно это учитывать.

Property function с внешними данными в горячей привязке. Требования расширяются, параллелизм падает, а выигрыш от «удобной привязки» ничтожен.

Случайность без сида в сетевом проекте. Сервер и клиент разойдутся.

Условие, которое пишет. Побочные эффекты в условии срабатывают непредсказуемое число раз, включая примерки состояний, которые не будут выбраны.

18.8. Итог главы

Узел	Ключевая особенность в Mass
Эвалюатор	Тикает всегда, пока тикает дерево; часто проигрывает процессору; единственный получает <code>TreeStart / TreeStop</code> ; должен пережить <code>DeltaTime == 0</code>
Условие	Без памяти между вызовами; вызывается каскадом; обязано быть дешёвым; только чтение
Property function	Вычисление внутри привязки; главное применение — разнообразие через случайность; сид задавайте явно

И три правила:

1. **Вычисление для всех агентов — процессор, не эвалюатор.**
2. **Условие читает готовое, а не считает.**
3. **Разнообразие толпы — через случайные привязки, а не через дублирование деревьев.**

Что дальше

Глава 19 закрывает часть IV — это практический рецепт: как написать свою Mass-задачу от пустого файла до работающего узла. Пройдём весь путь по шагам, с полным кодом: объявление `instance data`, выбор флагов, `Link()` и `GetDependencies()` в папе, реализация трёх методов, регистрация модуля, отладочный вывод. Отдельно разберём чек-лист проверки перед коммитом и полный каталог типичных ошибок с их симптомами — чтобы вы узнавали проблему по признакам, а не искали вслепую.

Глава 19. Своя Mass-задача: полный рецепт

Часть IV закрывается практикой. Напишем задачу с нуля — от пустого файла до работающего узла в дереве, — а потом соберём чек-лист и каталог ошибок с симптомами.

Пример возьмём такой, чтобы он задействовал максимум механизмов: задача занимает место у точки интереса, ставит агента там на заданное время и корректно освобождает

место при выходе. Здесь есть захват ресурса, опциональные данные, отложенный сигнал, накопление времени и осмысленный отладочный вывод.

19.1. Подготовка модуля

Прежде чем писать код, убедитесь, что модуль подключён к нужным зависимостям. В

`MyGame.Build.cs` :

csharp

```
PublicDependencyModuleNames.AddRange(new string[]
{
    "Core",
    "CoreUObject",
    "Engine",
    "StateTreeModule",
    "MassEntity",
    "MassCommon",
    "MassSignals",
    "MassAIBehavior",
    "MassMovement",
    "MassNavigation",
    "GameplayTags",
});
```

Минимально обязательны первые шесть: без `StateTreeModule` не будет базовых узлов, без `MassAIBehavior` — Mass-баз и билдера зависимостей, без `MassSignals` — подсистемы сигналов.

Забыть зависимость — самая быстрая ошибка: линковщик скажет о неразрешённых символах, и всё станет ясно за минуту. Гораздо хуже ошибки, которые компилируются молча, — к ним мы вернёмся в разделе 19.9.

19.2. Фрагмент точки интереса

Задаче нужны данные, с которыми работать. Заведём фрагмент — он будет шиной между задачей и процессорами (глава 6):

cpp

```
// MyPointOfInterestFragments.h
#pragma once

#include "MassEntityTypes.h"
#include "MyPointOfInterestFragments.generated.h"
```

```

/** Занятое агентом место у точки интереса. */
USTRUCT()
struct FMyPointOfInterestFragment : public FMassFragment
{
    GENERATED_BODY()

    /** Сущность точки интереса, у которой стоим. INDEX_NONE-подобное
    состояние = не занято. */
    FMassEntityHandle PointOfInterest;

    /** Индекс слота внутри точки. */
    int32 SlotIndex = INDEX_NONE;

    bool IsOccupied() const { return SlotIndex != INDEX_NONE; }

    void Release()
    {
        PointOfInterest = FMassEntityHandle();
        SlotIndex = INDEX_NONE;
    }
};

```

Фрагмент маленький: шестнадцать байт. Никаких контейнеров, никаких указателей — ровно то, что требуется от данных в чанке.

Метод `Release()` вынесен сюда, а не в задачу: освобождение может понадобиться и другим системам (например, процессору, который убирает мёртвых агентов из очередей).

19.3. Instance data задачи

cpp

```

// MyOccupySlotTask.h
#pragma once

#include "MassEntityTypes.h"
#include "MassStateTreeTypes.h"
#include "MyPointOfInterestFragments.h"
#include "MyOccupySlotTask.generated.h"

class UMassSignalSubsystem;
struct FMassMoveTargetFragment;

namespace UE::MassBehavior

```

```

{
    struct FStateTreeDependencyBuilder;
}

USTRUCT()
struct FMyOccupySlotTaskInstanceData
{
    GENERATED_BODY()

    /** Точка интереса, у которой нужно встать. Привязывается к выходу
поискового узла. */
    UPROPERTY(EditAnywhere, Category = Input)
    FMassEntityHandle TargetPoint;

    /** Сколько стоять. Ноль или меньше — бесконечно, до внешнего перехода. */
    UPROPERTY(EditAnywhere, Category = Parameter)
    float Duration = 5.f;

    /** Накопленное время. Служебное поле, в редакторе не показывается. */
    UPROPERTY()
    float Time = 0.f;
};

```

Три поля, три роли — по образцу `FMassLookAtTaskInstanceData` из главы 16.

`TargetPoint` в категории `Input` без `meta = (Optional)`: привязка обязательна. Без цели задача бессмысленна, и пусть редактор об этом скажет.

`Duration` в категории `Parameter` — настраивается дизайнером и при этом может быть привязан к случайной property function (глава 18), чтобы агенты не расходились синхронно.

`Time` без `EditAnywhere`, но с `UPROPERTY` — рабочее состояние, живущее в instance data по причинам из главы 16.

19.4. Объявление задачи

cpp

```

USTRUCT(meta = (DisplayName = "Occupy POI Slot"))
struct FMyOccupySlotTask : public FMassStateTreeTaskBase
{
    GENERATED_BODY()

    using FInstanceDataType = FMyOccupySlotTaskInstanceData;

```

```

FMyOccupySlotTask()
{
    // Тикаем: нужно накапливать Time.
    bShouldCallTick = true;

    // Входные привязки читаем только при входе – цель в рамках состояния
    не меняется.
    bShouldCopyBoundPropertiesOnTick = false;

    // ExitState читает только внешние данные, привязки ему не нужны.
    bShouldCopyBoundPropertiesOnExitState = false;

    // Переходами не управляем.
    bShouldAffectTransitions = false;

    // Задача удерживает ресурс: при переизбрании состояния не надо
    // отпускать и заново занимать слот – это вызвало бы мигание.
    bShouldStateChangeOnReselect = false;
}

protected:
    virtual const UStruct* GetInstanceDataType() const override
    {
        return FInstanceDataType::StaticStruct();
    }

    virtual bool Link(FStateTreeLinker& Linker) override;
    virtual void
GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
override;

    virtual EStateTreeRunStatus EnterState(FStateTreeExecutionContext&
Context, const FStateTreeTransitionResult& Transition) const override;
    virtual void ExitState(FStateTreeExecutionContext& Context, const
FStateTreeTransitionResult& Transition) const override;
    virtual EStateTreeRunStatus Tick(FStateTreeExecutionContext& Context,
const float DeltaTime) const override;

#ifdef WITH_GAMEPLAY_DEBUGGER
    virtual FString GetDebugInfo(const FStateTreeReadOnlyExecutionContext&
Context) const override;
#endif

    // ВНИМАНИЕ: список хендлов ниже обязан совпадать с GetDependencies().
    TStateTreeExternalDataHandle<FMyPointOfInterestFragment> SlotHandle;

```

```

TStateTreeExternalDataHandle<FMassMoveTargetFragment,
EStateTreeExternalDataRequirement::Optional> MoveTargetHandle;
TStateTreeExternalDataHandle<UMassSignalSubsystem> SignalSubsystemHandle;

/** Статическая настройка: одинакова для всех агентов, привязать нельзя.
*/
UPROPERTY(EditAnywhere, Category = Parameter)
bool bStopMovementWhileOccupying = true;
};

```

Разберём принятые решения.

Конструктор с флагами. В отличие от обеих задач движка, здесь флаги настроены явно — и каждый выбор обоснован комментарием. `bShouldStateChangeOnReselect = false` особенно важен: это ровно тот случай «удержание ресурса», о котором предупреждает комментарий в `StateTreeTaskBase.h` (глава 15).

Три хендла с разными требованиями. Фрагмент слота обязателен — без него задача не может работать. Фрагмент движения опционален: агент без движения (сидящий, статичный) всё равно может занимать место. Подсистема сигналов обязательна — она обеспечивает пробуждение.

Комментарий-якорь перед хендлами. Простой приём, который спасает от рассогласования с `GetDependencies()` (глава 5).

`bStopMovementWhileOccupying` **в самой задаче.** Это статическая настройка поведения: либо задача останавливает агента, либо нет. Привязывать её не нужно, значит держим в задаче, а не в instance data — экономим память (глава 15, раздел 15.8).

19.5. Линковка и зависимости

cpp

```

// MyOccupySlotTask.cpp

bool FMyOccupySlotTask::Link(FStateTreeLinker& Linker)
{
    Linker.LinkExternalData(SlotHandle);
    Linker.LinkExternalData(MoveTargetHandle);
    Linker.LinkExternalData(SignalSubsystemHandle);
    return true;
}

void
FMyOccupySlotTask::GetDependencies(UE::MassBehavior::FStateTreeDependencyBuild

```

```
er& Builder) const
{
    Builder.AddReadWrite<FMyPointOfInterestFragment>();
    Builder.AddReadWrite<FMassMoveTargetFragment>();
    Builder.AddReadWrite<UMassSignalSubsystem>();
}
```

Держите эти два метода рядом в файле. Не разносите их по разным местам, не прячьте один в заголовок. Правило простое: правите один — сразу смотрите на второй.

Уровни доступа: слот пишем (занимаем и освобождаем), цель движения пишем (останавливаем агента), подсистему пишем (отправка сигнала — это запись, глава 14).

Если бы задача только читала слот — был бы `AddReadOnly`, и параллелизм был бы лучше (глава 8, раздел 8.6). Здесь пишем, так что платим.

19.6. Реализация методов

EnterState

cpp

```
EStateTreeRunStatus FMyOccupySlotTask::EnterState(FStateTreeExecutionContext&
Context, const FStateTreeTransitionResult& Transition) const
{
    FMassStateTreeExecutionContext& MassContext =
static_cast<FMassStateTreeExecutionContext>(Context);
    FInstanceData& InstanceData = Context.GetInstanceData(*this);

    // Сброс обязателен: instance data может быть переиспользована при
переходах.
    InstanceData.Time = 0.f;

    if (!InstanceData.TargetPoint.IsSet())
    {
        return EStateTreeRunStatus::Failed;
    }

    FMyPointOfInterestFragment& Slot = Context.GetExternalData(SlotHandle);

    // Захват ресурса. Если место не досталось — задача проваливается,
// дерево выберет другую ветку.
    const int32 ClaimedIndex = ClaimSlot(MassContext,
InstanceData.TargetPoint);
    if (ClaimedIndex == INDEX_NONE)
    {
```

```

        return EStateTreeRunStatus::Failed;
    }

    Slot.PointOfInterest = InstanceData.TargetPoint;
    Slot.SlotIndex = ClaimedIndex;

    // Опциональные данные — только через GetExternalDataPtr.
    if (bStopMovementWhileOccupying)
    {
        if (FMassMoveTargetFragment* MoveTarget =
Context.GetExternalDataPtr(MoveTargetHandle))
        {
            MoveTarget->CreateNewAction(EMassMovementAction::Stand,
*MassContext.GetEntityManager().GetWorld());
        }
    }

    // Обеспечиваем себе пробуждение. Без этого агент замрёт навсегда.
    if (InstanceData.Duration > 0.f)
    {
        UMassSignalSubsystem& SignalSubsystem =
Context.GetExternalData(SignalSubsystemHandle);
        SignalSubsystem.DelaySignalEntityDeferred(
            MassContext.GetMassEntityExecutionContext(),
            MyGame::Signals::SlotOccupationFinished,
            MassContext.GetEntity(),
            InstanceData.Duration);
    }

    return EStateTreeRunStatus::Running;
}

```

Шесть содержательных моментов:

Сброс Time. Из главы 10: instance data переиспользуется, старое значение может остаться.

Проверка входа. Даже при обязательной привязке значение может оказаться пустым, если источник его не заполнил.

GetExternalData для обязательных, GetExternalDataPtr для опциональных. Перепутать — значит получить ассерт в отладочной сборке (глава 11).

Захват через отдельную функцию. ClaimSlot() — деталь реализации; важно, что она может вернуть неудачу, и задача это корректно обрабатывает.

Deferred-сигнал. Задача выполняется внутри обхода чанков процессора, значит прямой вызов небезопасен (глава 14). Используем `DelaySignalEntityDeferred` с передачей Mass-контекста.

Возврат `Running`. Задача продолжает жить, ожидая либо истечения времени, либо внешнего перехода.

Tick

cpp

```
EStateTreeRunStatus FMyOccupySlotTask::Tick(FStateTreeExecutionContext&
Context, const float DeltaTime) const
{
    FInstanceDataType& InstanceData = Context.GetInstanceData(*this);

    if (InstanceData.Duration <= 0.f)
    {
        return EStateTreeRunStatus::Running;    // бесконечно, ждём внешнего
перехода
    }

    // DeltaTime – время между тиками ДЕРЕВА, а не кадрами.
    // При пробуждении по отложенному сигналу здесь будет вся длительность
разом.
    InstanceData.Time += DeltaTime;

    return InstanceData.Time >= InstanceData.Duration
        ? EStateTreeRunStatus::Succeeded
        : EStateTreeRunStatus::Running;
}
```

Комментарий про `DeltaTime` стоит писать всегда — это то, обо что спотыкаются все, кто приходит из обычного Unreal.

ExitState

cpp

```
void FMyOccupySlotTask::ExitState(FStateTreeExecutionContext& Context, const
FStateTreeTransitionResult& Transition) const
{
    FMassStateTreeExecutionContext& MassContext =
static_cast<FMassStateTreeExecutionContext*>(Context);
    FMypointOfInterestFragment& Slot = Context.GetExternalData(SlotHandle);
```

```

    if (Slot.IsOccupied())
    {
        ReleaseSlot(MassContext, Slot.PointOfInterest, Slot.SlotIndex);
        Slot.Release();
    }

    // Цель движения намеренно НЕ сбрасываем: следующая задача её перезапишет,
    // а резкий сброс дал бы визуальный артефакт.
}

```

Освобождение ресурса — единственная обязанность этого метода. Всё, что захвачено в `EnterState()`, отпускается здесь и только здесь. Не в `StateCompleted()` — тот не вызывается при условном переходе (глава 15).

Комментарий про цель движения фиксирует осознанное решение — по образцу `FMassZoneGraphPathFollowTask`, у которой `ExitState()` нет вовсе (глава 17).

Именно ради этого метода `UMassStateTreeFragmentDestructor` вызывает `Stop()` перед освобождением instance data (глава 13). Без него слот остался бы занятым навсегда.

Отладочный вывод

cpp

```

#ifdef WITH_GAMEPLAY_DEBUGGER
FString FMyOccupySlotTask::GetDebugInfo(const
FStateTreeReadOnlyExecutionContext& Context) const
{
    // Read-only контекст: менять ничего нельзя, и это правильно.
    // Instance data здесь недоступна напрямую — выводим то, что знаем.
    return FString::Printf(TEXT("OccupySlot: duration %.1fs, stop=%s"),
        /* Duration недоступен без instance data — см. примечание ниже */ 0.f,
        bStopMovementWhileOccupying ? TEXT("yes") : TEXT("no"));
}
#endif

```

Небольшая ловушка: read-only контекст не даёт полноценного доступа к instance data узла — метод `GetInstanceData()` объявлен в полном контексте (глава 11). На практике движок предоставляет отладочному выводу нужные данные через другие каналы; если вам критично показать `Time` и `Duration`, продублируйте их во фрагмент или используйте трассировку `StateTree`.

Даже урезанный отладочный вывод лучше отсутствующего: в Gameplay Debugger вы хотя бы увидите, какая задача активна и с какими настройками.

19.7. Регистрация: чего делать не нужно

Приятная новость: **никакой регистрации не требуется**.

Как только модуль собран, USTRUCT-задача попадает в рефлекссию. Схема UMassStateTreeSchema пропустит её через IsStructAllowed(), потому что она наследник FMassStateTreeTaskBase (глава 7). Редактор покажет её в списке узлов под именем из DisplayName.

Единственное, что требует настройки, — **собственные сигналы**. Как разбиралось в главе 14, нужен свой процессор:

cpp

```
// MyStateTreeProcessor.h
UCLASS()
class UMyStateTreeProcessor : public UMassStateTreeProcessor
{
    GENERATED_BODY()

protected:
    virtual void InitializeInternal(UObject& Owner, const
TSharedRef<FMassEntityManager>& EntityManager) override
    {
        Super::InitializeInternal(Owner, EntityManager);
        SubscribeToSignal(MyGame::Signals::SlotOccupationFinished);
    }
};
```

и прописать его в настройках проекта:

ini

```
[/Script/MassAIBehavior.MassBehaviorSettings]
DynamicStateTreeProcessorClass=/Script/MyGame.MyStateTreeProcessor
```

Забыть этот шаг — самая коварная ошибка из всех: код компилируется, задача появляется в редакторе, дерево запускается, а агент просто застревает. Никаких сообщений.

19.8. Чек-лист перед коммитом

Пройдитесь по списку — он покрывает почти все ошибки, которые компилятор не ловит.

Структура узла

- ☐ Наследование от `FMassStateTreeTaskBase` (не от `FStateTreeTaskBase`)
- ☐ `USTRUCT()` и `GENERATED_BODY()` на месте
- ☐ `using FInstanceDataType = ...;` объявлен
- ☐ `GetInstanceDataType()` переопределён
- ☐ `meta = (DisplayName = "...")` задан
- ☐ Не скопирован случайно `meta = (Hidden)` из базы

Данные

- ☐ Каждое поле instance data имеет осознанную категорию: `Input`, `Parameter` или без категории для рабочего состояния
- ☐ Рабочие поля сбрасываются в `EnterState()`
- ☐ Статические настройки лежат в задаче, динамические — в instance data
- ☐ Ни одно поле задачи не `mutable`

Внешние данные

- ☐ Каждый хендл зарегистрирован в `Link()`
- ☐ Каждый хендл отражён в `GetDependencies()` — списки совпадают один в один
- ☐ Уровни доступа минимально достаточны: `ReadOnly` там, где не пишем
- ☐ Опциональные хендлы читаются только через `GetExternalDataPtr()`
- ☐ Обязательные — через `GetExternalData()`
- ☐ Результат `GetExternalDataPtr()` проверяется на `nullptr`

Флаги

- ☐ `bShouldCallTick` выключен, если задача не тикает по существу
- ☐ `bShouldStateChangeOnReselect = false`, если задача удерживает ресурс
- ☐ `bShouldCopyBoundPropertiesOnTick` выключен, если входы читаются только при входе
- ☐ `bConsideredForCompletion` продуман, если задача фоновая

Жизненный цикл

- ☐ Всё захваченное в `EnterState()` освобождается в `ExitState()`
- ☐ Освобождение не вынесено в `StateCompleted()`
- ☐ `Tick()` не предполагает малого `DeltaTime`
- ☐ Задача, возвращающая `Running`, обеспечила себе пробуждение

Сигналы

- ☐ Внутри задачи используются только deferred-варианты отправки
- ☐ Для нескольких сущностей — версия с массивом
- ☐ Собственные сигналы объявлены константами, не строками на месте
- ☐ Процессор подписан на собственные сигналы

Прочее

- ☐ `GetDebugInfo()` переопределён
- ☐ Модуль подключён в `Build.cs`

19.9. Каталог ошибок по симптомам

Раздел для чтения задом наперёд: находите свой симптом, читаете причину.

«Узел не появляется в списке в редакторе»

Причина	Проверка
Наследование не от Mass-базы	Смотрите объявление структуры
Скопирован <code>meta = (Hidden)</code> из базы	Смотрите метаданные <code>USTRUCT</code>
Ассет создан с другой схемой	Свойства ассета → Schema должна быть «Mass Behavior»
Модуль не пересобран	Перезапустите редактор с полной сборкой

«Дерево не запускается вообще»

Причина	Проверка
Вы в PIE-клиенте	<code>UE::MassStateTree::ExecutionFlags</code> — только Standalone и Server (глава 5)
Не выделена instance data	Смотрите <code>InstanceHandle.IsValid()</code> во фрагменте
Не создан процессор для ассета	<code>StateTreeToProcessor</code> в подсистеме (глава 9)
Не поставлен тег активации	Наличие <code>FMassStateTreeActivatedTag</code> у агента (глава 13)

«Дерево тикнуло один раз и замерло»

Почти всегда одно: **задача вернула `Running` и не обеспечила себе пробуждение.**

Проверьте: поставлен ли отложенный сигнал, шлёт ли процессор-исполнитель сигнал завершения, подписан ли ваш процессор на этот сигнал.

Это настолько частая ошибка, что стоит завести привычку: написали `return EStateTreeRunStatus::Running` — сразу ответьте себе на вопрос «кто меня разбудит».

«Падает на `check` при обращении к внешним данным»

Симптом	Причина
Ассерт про <code>Optional</code>	Вызвали <code>GetExternalData()</code> для опционального хендла — нужен <code>GetExternalDataPtr()</code>
Ассерт про <code>CurrentlyProcessedFrame</code>	Обращаетесь к внешним данным вне метода узла
Ассерт про <code>InstanceDataHandle</code>	Просите instance data чужого узла — так нельзя (глава 11)

«Значения дёргаются, поведение случайно ломается под нагрузкой»

Гонка данных. Почти наверняка **рассогласование `Link()` и `GetDependencies()`** (глава 8, раздел 8.7).

Сядьте и сверьте два списка построчно. Компилятор здесь не помощник.

Второй кандидат: включён `bProcessEntitiesInParallel`, а задачи шлют сигналы напрямую (глава 13, раздел 13.10).

«Ресурсы утекают: слоты заняты, места не освобождаются»

Причина	Проверка
Освобождение в <code>StateCompleted()</code> вместо <code>ExitState()</code>	Метод не вызывается при условном переходе
<code>bShouldStateChangeOnReselect = true</code> у удерживающей задачи	Задача отпускает и захватывает при каждом переизбрании
Деструктор не отработал	Проверьте <code>ExecutionFlags</code> у <code>UMassStateTreeFragmentDestructor</code>

«Таймеры считают неправильно»

Используете кадровую дельту вместо той, что пришла в `Tick()` . Либо копите время в задаче, которая не тикает (`bShouldCallTick = false`).

«Состояние никогда не завершается»

В состоянии есть фоновая задача, вечно возвращающая `Running` , у которой не выставлен `bConsideredForCompletion = false` (глава 15).

«Агенты ведут себя синхронно, как клоны»

Нет разнообразия. Добавьте случайную `property function` в привязку `Duration` (глава 18) или разнесите параметры через фрагменты, заполняемые при спавне.

«После правки дерева в PIE поведение странное»

Ассет перекомпилирован, а процессор остался со старыми требованиями. Комментарий Epic в `MassStateTreeSubsystem.h` признаёт эту проблему (глава 9). Перезапустите PIE.

«Процессоров стало неожиданно много»

Требования деревьев различаются сильнее, чем вы думали. Смотрите, какие узлы вносят уникальные зависимости (глава 8, раздел 8.8).

«Симуляция стала однопоточной»

Слишком широкие требования. Проверьте: нет ли лишних `AddReadWrite` там, где хватило бы `AddReadOnly` ; не тянет ли одно дерево требования всех веток сразу (глава 8, раздел 8.9).

19.10. Три привычки, которые окупаются

Комментарий-якорь у хендлов. Одна строка `// ВНИМАНИЕ: список обязан совпадать с GetDependencies()` экономит часы отладки гонок.

Вопрос «кто меня разбудит» при каждом `return Running` . Задайте его вслух — и половина ошибок с застрявшими агентами исчезнет.

Отладочный вывод сразу, а не потом. Написать `GetDebugInfo()` в момент создания задачи стоит две минуты. Дописывать его, когда тысяча агентов ведёт себя непонятно, — совсем другое дело.

19.11. Итог главы

Порядок работы над своей задачей:

1. Подключить модули в `Build.cs` .

2. Спроектировать данные: что во фрагмент, что в instance data, что в задачу.
 3. Объявить задачу с `using FInstanceDataType` и осознанными флагами.
 4. Написать `Link()` и `GetDependencies()` **рядом и одновременно**.
 5. Реализовать `EnterState()` — захват, намерение, планирование пробуждения.
 6. Реализовать `Tick()` — проверка, а не работа.
 7. Реализовать `ExitState()` — освобождение всего захваченного.
 8. Добавить `GetDebugInfo()` .
 9. Для своих сигналов — наследник процессора плюс строка в настройках.
 10. Пройти чек-лист из раздела 19.8.
-

Что дальше

Часть IV закончена. С главы 20 начинается часть V — сборка и эксплуатация.

Глава 20 разберёт последнее недостающее звено: как ассет дерева вообще попадает на агента. Трейты `UMassEntityTraitBase` и метод `BuildTemplate()` , конфигурационный ассет `UMassEntityConfigAsset` , шаблоны сущностей и их реестр, фильтрация ассетов по схеме через `RequiredAssetDataTags` , создание константного общего фрагмента через `GetOrCreateConstSharedFragment` . После неё у нас будет полная картина от редактора до исполнения, и можно будет собирать сквозной пример.

Глава 20. Трейты и конфигурация: как дерево попадает на агента

Последнее недостающее звено. Мы разобрали, что происходит в рантайме, но не разобрали, откуда там берутся `FMassStateTreeInstanceFragment` и `FMassStateTreeSharedFragment` . Эта глава закрывает пробел между редактором и исполнением.

Заголовков трейтов в нашем пакете нет, поэтому конкретные сигнатуры я привожу по общей структуре системы Mass; принципы и связи с уже разобранным кодом — надёжны.

20.1. Проблема, которую решают трейты

Представьте, что трейтов нет. Чтобы заспавнить агента-горожанина, нужно вручную перечислить всё:

cpp


```
// как это выглядело бы без трейтов
TArray<const UScriptStruct*> Fragments = {
    FTransformFragment::StaticStruct(),
    FMassVelocityFragment::StaticStruct(),
    FMassForceFragment::StaticStruct(),
    FAgentRadiusFragment::StaticStruct(),
    FMassMoveTargetFragment::StaticStruct(),
    FMassZoneGraphLaneLocationFragment::StaticStruct(),
    FMassZoneGraphPathRequestFragment::StaticStruct(),
    FMassZoneGraphShortPathFragment::StaticStruct(),
    FMassZoneGraphCachedLaneFragment::StaticStruct(),
    FMassStateTreeInstanceFragment::StaticStruct(),
    FMassLookAtFragment::StaticStruct(),
    FMassRepresentationFragment::StaticStruct(),
    // ... и ещё два десятка
};
```

Три проблемы очевидны:

Хрупкость. Забыли один фрагмент — задача при линковке дерева упадёт, потому что не найдёт обязательные внешние данные (глава 11).

Неизвестность зависимостей. Чтобы навигация работала, нужны все четыре фрагмента ZoneGraph плюс FAgentRadiusFragment плюс FMassMovementParameters. Знать это наизусть невозможно.

Недоступность для дизайнера. Список в C++ — значит, любое изменение конфигурации агента требует программиста.

Трейт решает всё три: он инкапсулирует **связный набор данных плюс их конфигурацию** и выставляет наружу только осмысленные настройки.

20.2. UMassEntityTraitBase

cpp

```
// упрощённое объявление
UCLASS(Abstract, EditInlineNew, CollapseCategories, meta = (DisplayName =
"Mass Trait Base"))
class MASSSPAWNER_API UMassEntityTraitBase : public UObject
{
    GENERATED_BODY()

public:
    /** Добавляет в шаблон фрагменты, теги и общие фрагменты, нужные этому
```

```

трейту. */
    virtual void BuildTemplate(FMassEntityTemplateBuildContext& BuildContext,
    const UWorld& World) const {}

    /** Проверка корректности настроек; вызывается при валидации конфигурации.
    */
    virtual void ValidateTemplate(FMassEntityTemplateBuildContext&
    BuildContext, const UWorld& World) const {}

    /** Уничтожение шаблона. */
    virtual void DestroyTemplate() const {}
};

```

Три спецификатора класса стоит разобрать — мы уже видели такие у схемы (глава 7):

Abstract — базовый трейт бесполезен, только наследники.

EditInlineNew — экземпляр создаётся внутри владеющего ассета конфигурации, а не как отдельный объект. Трейт — часть конфигурации, не самостоятельная сущность.

CollapseCategories — свойства показываются плоским списком. У трейтов обычно два-три свойства, группировать нечего.

Главный метод — `BuildTemplate()`. Он вызывается **один раз** при построении шаблона, не при каждом спавне. Это важно: тяжёлые операции здесь допустимы, они не в горячем пути.

20.3. Трейт StateTree

Восстановим трейт, который добавляет агенту поведение:

cpp

```

UCLASS(meta = (DisplayName = "StateTree"))
class MASSAIBEHAVIOR_API UMassStateTreeTrait : public UMassEntityTraitBase
{
    GENERATED_BODY()

protected:
    virtual void BuildTemplate(FMassEntityTemplateBuildContext& BuildContext,
    const UWorld& World) const override;

    /** Ассет поведения. Фильтруется по схеме — см. раздел 20.4. */
    UPROPERTY(EditAnywhere, Category = "", meta = (RequiredAssetDataTags =
    "Schema=/Script/MassAIBehavior.MassStateTreeSchema"))

```

```
TObjectPtr<UStateTree> StateTree;  
};
```

И реализация:

cpp

```
void UMassStateTreeTrait::BuildTemplate(FMassEntityTemplateBuildContext&  
BuildContext, const UWorld& World) const  
{  
    // Персональные данные: у каждого агента свой хендл instance data.  
    BuildContext.AddFragment<FMassStateTreeInstanceFragment>();  
  
    // Общие данные: один указатель на ассет для всей группы.  
    const FMassEntityManager& EntityManager =  
    UE::Mass::Utils::GetEntityManagerChecked(World);  
    const FConstSharedStruct StateTreeFragment =  
  
    EntityManager.GetOrCreateConstSharedFragment(FMassStateTreeSharedFragment{  
        StateTree });  
    BuildContext.AddConstSharedFragment(StateTreeFragment);  
}
```

Две строки — и вся связь между редактором и рантаймом.

Что здесь важно

Обычный фрагмент добавляется по типу. `AddFragment<T>()` не требует значения: у каждого агента будет свой экземпляр, инициализированный по умолчанию. Хендл `instance data` пока пустой — его заполнит наблюдатель.

Общий фрагмент добавляется по значению. `GetOrCreateConstSharedFragment()` считает хеш содержимого, ищет существующий экземпляр и при совпадении возвращает его (глава 6). Сто конфигураций с одним ассетом дадут один общий фрагмент в памяти.

Тег активации не добавляется. Это состояние рантайма, его ставит процессор (глава 13). Трейт описывает конфигурацию, а не текущее состояние.

20.4. Фильтрация ассетов по схеме

Вернёмся к метаданным свойства:

cpp

```
meta = (RequiredAssetDataTags =  
"Schema=/Script/MassAIBehavior.MassStateTreeSchema")
```

Это ровно тот механизм, который описан в комментарии `StateTreeSchema.h` (глава 7):

cpp

```
* Each StateTree asset saves the schema class name in asset data tags, which  
can be  
* used to limit which StatTree assets can be selected per use case, i.e.:  
*  
* UPROPERTY(EditDefaultsOnly, Category = AI, meta=  
(RequiredAssetDataTags="Schema=StateTreeSchema_SupaDupa"))  
* UStateTree* StateTree;
```

Как это работает технически: при сохранении ассета `StateTree` записывает имя класса своей схемы в теги данных ассета — метаданные, доступные без загрузки самого ассета. Редактор при построении выпадающего списка фильтрует по этим тегам.

Практический эффект: **дизайнер физически не может выбрать дерево не той схемы.** В списке будут только ассеты с «Mass Behavior», деревья для компонентов туда не попадут.

Это защита от ошибки, которая иначе проявилась бы только в рантайме — и проявилась бы непонятно: узлы дерева попытались бы получить `AActor*`, которого нет.

Помните про имена тегов из главы 7:

cpp

```
namespace UE::StateTree  
{  
    inline const FName SchemaTag(TEXT("Schema"));  
    inline const FName SchemaCanBeOverridenTag(TEXT("SchemaCanBeOverriden"));  
}
```

Второй тег помечает деревья с «общей» схемой (метка `CommonSchema` у `UMassStateTreeSchema`), которые можно подключать как `LinkedAsset` в деревья с другими схемами.

20.5. Шаблон сущности

BuildTemplate() наполняет FMassEntityTemplateBuildContext, а результат — FMassEntityTemplate:

cpp

```
// упрощённо
struct FMassEntityTemplate
{
    /** Композиция: какие фрагменты, теги, общие фрагменты. */
    FMassArchetypeCompositionDescriptor Composition;

    /** Значения общих фрагментов. */
    FMassArchetypeSharedFragmentValues SharedFragmentValues;

    /** Начальные значения обычных фрагментов. */
    TArray<FInstancedStruct> InitialFragmentValues;

    /** Готовый архетип. */
    FMassArchetypeHandle Archetype;
};
```

Шаблон — это **рецепт архетипа плюс начальные значения**. Он строится один раз и переиспользуется для всех агентов данной конфигурации.

Обратите внимание на разделение из главы 2: `Composition` отвечает на вопрос «какие данные есть», `SharedFragmentValues` — «какие у общих данных значения». Битсет не может хранить значения, поэтому нужны обе структуры.

Шаблоны кэшируются в `UMassEntityTemplateRegistry` — подсистеме, которая хранит их по ключу конфигурации. Повторный запрос той же конфигурации вернёт готовый шаблон без перестроения.

20.6. Конфигурационный ассет

cpp

```
UCLASS(BlueprintType, meta = (DisplayName = "Mass Entity Config"))
class MASSSPAWNER_API UMassEntityConfigAsset : public UDataAsset
{
    GENERATED_BODY()

public:
    const FMassEntityTemplate& GetOrCreateEntityTemplate(const UWorld& World)
    const;
```

```
protected:
    /** Родительская конфигурация: трейты наследуются. */
    UPROPERTY(EditAnywhere, Category = "Mass")
    FMassEntityConfig Config;
};
```

А внутри FMassEntityConfig:

cpp

```
USTRUCT()
struct FMassEntityConfig
{
    GENERATED_BODY()

    /** Наследование от другой конфигурации. */
    UPROPERTY(EditAnywhere, Category = "Mass")
    TObjectPtr<UMassEntityConfigAsset> Parent;

    /** Список трейтов. */
    UPROPERTY(EditAnywhere, Instanced, Category = "Mass")
    TArray<TObjectPtr<UMassEntityTraitBase>> Traits;
};
```

Два механизма композиции.

Список трейтов — горизонтальная композиция. Агент собирается из независимых кусков: движение, представление, поведение, восприятие.

Родительская конфигурация — вертикальное наследование. Базовая конфигурация «пешеход» задаёт общее, дочерние «турист», «курьер», «полицейский» добавляют своё.

Спецификатор `Instanced` у массива трейтов существен: он означает, что каждый элемент — отдельный объект, принадлежащий этой конфигурации, а не ссылка на общий. Настройки трейта в одной конфигурации не влияют на другую.

Типичная конфигурация горожанина выглядит примерно так:

Трейт	Что добавляет
Mass Movement	Скорость, силы, параметры движения
Mass ZoneGraph Navigation	Фрагменты полос, путей, кэша
Mass Avoidance	Данные обхода препятствий
Mass Representation	Визуальное представление и LOD

Трейт	Что добавляет
Mass LookAt	Фрагмент взгляда
StateTree	Instance data и ассет поведения
Mass Simulation LOD	Уровни детализации симуляции

Порядок в списке значения не имеет: композиция — множество, а не последовательность.

20.7. Проверка совместимости

Здесь возникает вопрос: что если дизайнер добавил трейт StateTree с деревом, которое использует навигацию, но забыл трейт навигации?

Три уровня защиты, и все мы уже разбирали.

Уровень 1: линковка дерева. Обязательные внешние данные (EStateTreeExternalDataRequirement::Required) должны разрешиться. Если FMassZoneGraphLaneLocationFragment не найден — линковка провалится, дерево не запустится (глава 11).

Уровень 2: ValidateTemplate(). Трейт может проверить наличие нужных фрагментов и выдать понятное сообщение:

cpp

```
void UMassStateTreeTrait::ValidateTemplate(FMassEntityTemplateBuildContext&
BuildContext, const UWorld& World) const
{
    if (StateTree == nullptr)
    {
        UE_LOG(LogMass, Error, TEXT("%s: не задан ассет StateTree"),
*GetNameSafe(this));
        return;
    }

    // проверить, что все обязательные внешние данные дерева
    // присутствуют в композиции шаблона
    // ...
}
```

Уровень 3: рантайм. Опциональные данные проверяются на nullptr в самой задаче (глава 16).

Первый уровень работает всегда, второй — если разработчик трейта его реализовал, третий — забота автора задачи.

Практическое следствие для вас: **если пишете свой трейт, реализуйте**

`ValidateTemplate()`. Внятное сообщение «дереву нужен фрагмент X, добавьте трейт Y» экономит дизайнеру часы.

20.8. Полный путь от редактора к рантайму

Соберём цепочку целиком — она проходит через всё, что мы разобрали.

Шаг 1. Дизайнер создаёт дерево. Новый ассет `StateTree`, схема «Mass Behavior». В теги ассета записывается имя класса схемы.

Шаг 2. Дизайнер собирает поведение. Схема через `IsStructAllowed()` показывает только Mass-узлы и общеприменимые (глава 7). Через `IsExternalItemAllowed()` разрешается доступ к фрагментам и подсистемам.

Шаг 3. Компиляция ассета. Дерево превращается в плоские массивы. Узлы линкуются, схема в `Link()` собирает `Dependencies` через `GetDependencies()` каждого узла (глава 8).

Шаг 4. Дизайнер создаёт конфигурацию. Добавляет трейты, в трейте `StateTree` выбирает ассет — список отфильтрован по схеме.

Шаг 5. Построение шаблона. При первом обращении вызываются `BuildTemplate()` всех трейтов, формируется композиция, создаётся архетип, шаблон кэшируется в реестре.

Шаг 6. Спавн. Сущность создаётся по шаблону. У неё есть `FMassStateTreeInstanceFragment` (пустой хендл) и `FMassStateTreeSharedFragment` (указатель на ассет).

Шаг 7. Выделение instance data. Наблюдатель на добавление фрагмента вызывает `AllocateInstanceData()`. Попутно подсистема при первом обращении к ассету создаёт для него динамический процессор (глава 9).

Шаг 8. Активация. `UMassStateTreeActivationProcessor` ставит тег и шлёт `StateTreeActivate` (глава 13).

Шаг 9. Первый тик. Процессор строит `FMassStateTreeExecutionContext`, вызывает `SetEntity()` и `Start()`. Подставляется `FMassExecutionExtension` с хендлом сущности (глава 12).

Шаг 10. Поведение работает. Задачи пишут намерения во фрагменты, процессоры Mass их исполняют, сигналы будят дерево.

Десять шагов, из которых первые четыре — редактор, остальные — рантайм. Полная картина.

20.9. Спавнеры

Осталось упомянуть, кто именно создаёт сущности.

`AMassSpawner` — актор, размещаемый на уровне. У него список конфигураций с весами и правило размещения:

cpp

```
// упрощённо
UCLASS()
class AMassSpawner : public AActor
{
    /** Что спавнить и в какой пропорции. */
    UPROPERTY(EditAnywhere, Category = "Mass|Spawn")
    TArray<FMassSpawnedEntityType> EntityTypes;

    /** Сколько всего. */
    UPROPERTY(EditAnywhere, Category = "Mass|Spawn")
    int32 Count = 0;

    /** Где размещать: точки, объёмы, ZoneGraph. */
    UPROPERTY(EditAnywhere, Instanced, Category = "Mass|Spawn")
    TArray<TObjectPtr<UMassEntitySpawnDataGenerator>> SpawnDataGenerators;
};
```

Веса позволяют смешивать типы: 70% обычных горожан, 20% спешащих, 10% с колясками. Разные конфигурации — разные архетипы, но спавнер разбирается сам.

Генераторы данных размещения определяют, где появятся агенты: в объёме, вдоль полос ZoneGraph, в точках навигации.

Программно спавн выглядит так:

cpp

```
const FMassEntityTemplate& Template = ConfigAsset-
>GetOrCreateEntityTemplate(*World);
```

```
TArray<FMassEntityHandle> SpawnedEntities;
EntityManager.BatchCreateEntities(Template.GetArchetype(), Count,
SpawnedEntities);
EntityManager.BatchSetEntityFragmentValues(SpawnedEntities,
Template.GetInitialFragmentValues());
```

Пакетное создание, а не по одному. Это принципиально: `BatchCreateEntities` выделяет место сразу под все сущности, а поштучный спавн вызывал бы перевыделение чанков многократно.

Вспомните ограничение из главы 9: аллокация instance data не параллелится. При спавне десяти тысяч агентов все выделения пойдут последовательно. Если это заметно — спавните пачками по кадрам.

20.10. Свой трейт: рецепт

Допустим, вашему поведению нужен персональный «уровень тревоги» — фрагмент, который пишет процессор восприятия и читают условия дерева.

cpp

```
// MyAlertFragments.h
USTRUCT()
struct FMyAlertFragment : public FMassFragment
{
    GENERATED_BODY()

    float Level = 0.f;
};

/** Общие параметры тревоги: одинаковы для всех агентов данного типа. */
USTRUCT()
struct FMyAlertParamsFragment : public FMassConstSharedFragment
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, meta = (ClampMin = 0.0))
    float RiseRate = 1.f;

    UPROPERTY(EditAnywhere, meta = (ClampMin = 0.0))
    float DecayRate = 0.5f;
};
```

cpp

```

// MyAlertTrait.h
UCLASS(meta = (DisplayName = "Alert Level"))
class UMyAlertTrait : public UMassEntityTraitBase
{
    GENERATED_BODY()

protected:
    virtual void BuildTemplate(FMassEntityTemplateBuildContext& BuildContext,
const UWorld& World) const override
    {
        BuildContext.AddFragment<FMyAlertFragment>();

        const FMassEntityManager& EntityManager =
UE::Mass::Utils::GetEntityManagerChecked(World);
        const FConstSharedStruct ParamsFragment =
            EntityManager.GetOrCreateConstSharedFragment(Params);
        BuildContext.AddConstSharedFragment(ParamsFragment);
    }

    /** Настройки, выставляемые дизайнером. */
    UPROPERTY(EditAnywhere, Category = "")
    FMyAlertParamsFragment Params;
};

```

Три решения, которые стоит отметить:

Персональное значение — обычный фрагмент. У каждого агента свой уровень тревоги.

Параметры — константный общий фрагмент. Скорость нарастания одинакова для всех агентов данной конфигурации и не меняется. Правило из главы 6.

Параметры редактируются в трейте. Дизайнер видит два числа, а не структуру фрагмента напрямую.

Дальше остаётся написать процессор, который обновляет `FMyAlertFragment`, и условие для дерева, читающее его как внешние данные (глава 18).

20.11. Типичные ошибки конфигурации

Забыть трейт, от которого зависит дерево. Симптом: дерево не запускается, в логе ошибка линковки о неразрешённых внешних данных. Лечение: добавить нужный трейт.

Слишком много разных конфигураций. Каждая уникальная композиция — свой архетип. Пятьдесят слегка различающихся конфигураций дадут пятьдесят архетипов,

чанки станут разреженными, обходы — менее эффективными. Лучше меньше конфигураций с большей вариативностью внутри (через случайные значения при спавне).

Много разных ассетов деревьев. Помимо архетипов это влияет на количество процессоров (глава 8). Часто лучше одно дерево с ветвлением по условиям, чем десять почти одинаковых.

Ассет не той схемы. При правильных метаданных невозможно выбрать в редакторе. Но если свойство объявлено без `RequiredAssetDataTags`, ошибка станет рантаймовой.

Изменяемый общий фрагмент вместо константного. Ограничивает параллелизм, отключает дедупликацию (глава 6).

Тяжёлая работа в `BuildTemplate()` без нужды. Метод вызывается редко, но при десятках конфигураций и горячей перезагрузке это заметно. Не грузите там ассеты синхронно.

20.12. Итог главы

Элемент	Роль
<code>UMassEntityTraitBase::BuildTemplate()</code>	Добавляет связный набор фрагментов, тегов и общих фрагментов
<code>AddFragment<T>()</code>	Персональные данные, по одному экземпляру на агента
<code>GetOrCreateConstSharedFragment()</code>	Общие данные с дедупликацией по содержимому
<code>RequiredAssetDataTags</code>	Фильтрация ассетов по схеме прямо в редакторе
<code>FMassEntityTemplate</code>	Рецепт архетипа плюс начальные значения; кэшируется
<code>UMassEntityConfigAsset</code>	Список трейтов плюс наследование от родительской конфигурации
<code>ValidateTemplate()</code>	Ранняя диагностика несовместимых конфигураций
<code>AMassSpawner</code>	Размещение агентов с весами по типам

Три правила:

1. **Персональное — обычный фрагмент, общее и неизменное — константный общий.**

2. **Реализуйте `ValidateTemplate()` в своих трейтах** — это экономит время дизайнерам.
 3. **Меньше конфигураций и деревьев** — плотнее чанки и меньше процессоров.
-

Что дальше

Глава 21 — сквозной пример. Соберём агента-патрульного целиком: спроектируем поведение, распишем дерево по состояниям, напишем недостающие узлы, соберём конфигурацию из трейтов, разберём, что происходит на каждом кадре, и проследим полный цикл от спавна до смерти. Всё, что мы разбирали двадцать глав, встретится там в работе.

Глава 21. Сквозной пример: патрульный агент

Соберём всё вместе. Спроектируем поведение, напишем недостающие узлы, соберём дерево и конфигурацию, а потом проследим, что происходит покадрово — от спавна до смерти. Почти каждый механизм из предыдущих двадцати глав встретится здесь в работе.

21.1. Постановка задачи

Агент патрулирует маршрут по полосам `ZoneGraph`. На точках маршрута останавливается, осматривается, идёт дальше. Если уровень тревоги превышает порог — бросает патруль, идёт к источнику тревоги, осматривается там и возвращается к маршруту.

Требования, которые определяют архитектуру:

- Агентов много (тысячи), значит всё должно спать между действиями.
- Маршруты разные у разных агентов.
- Пауза на точке должна быть случайной, чтобы толпа не двигалась синхронно.
- Уровень тревоги считается отдельной системой, дерево его только читает.

21.2. Проектирование данных

Первое решение — что куда положить. Применяем правила из глав 6 и 20.

Маршрут агента. У каждого свой → обычный фрагмент.

cpp

```

// MyPatrolFragments.h
#pragma once

#include "MassEntityTypes.h"
#include "ZoneGraphTypes.h"
#include "MyPatrolFragments.generated.h"

/** Точка маршрута: полоса и расстояние вдоль неё. */
USTRUCT()
struct FMyPatrolPoint
{
    GENERATED_BODY()

    FZoneGraphLaneHandle Lane;
    float DistanceAlongLane = 0.f;
};

/** Персональный маршрут агента. */
USTRUCT()
struct FMyPatrolRouteFragment : public FMassFragment
{
    GENERATED_BODY()

    /** Точки маршрута. Обычно 2–6, инлайн-аллокатор избавляет от кучи. */
    TArray<FMyPatrolPoint, TInlineAllocator<4>> Points;

    /** Индекс следующей точки. */
    int32 NextIndex = 0;
};

```

`TInlineAllocator<4>` — тот же приём, что мы видели в движке (главы 10 и 11). Маршрут из четырёх точек не потребует ни одного обращения к куче, а массив останется внутри фрагмента.

Параметры патрулирования. Одинаковы для всех агентов данного типа, не меняются → константный общий фрагмент.

cpp

```

/** Настройки поведения, общие для конфигурации. */
USTRUCT()
struct FMyPatrolParamsFragment : public FMassConstSharedFragment
{
    GENERATED_BODY()

```

```

    /** Минимальная пауза на точке. */
    UPROPERTY(EditAnywhere, meta = (ClampMin = 0.0))
    float MinPauseTime = 2.f;

    /** Максимальная пауза на точке. */
    UPROPERTY(EditAnywhere, meta = (ClampMin = 0.0))
    float MaxPauseTime = 6.f;

    /** Порог тревоги для перехода к расследованию. */
    UPROPERTY(EditAnywhere, meta = (ClampMin = 0.0, ClampMax = 1.0))
    float AlertThreshold = 0.7f;
};

```

Уровень тревоги. Персональный, но вычисляется процессором восприятия → обычный фрагмент, который дерево только читает. Возьмём его из главы 20:

cpp

```

USTRUCT()
struct FMyAlertFragment : public FMassFragment
{
    GENERATED_BODY()

    float Level = 0.f;

    /** Куда идти разбираться. */
    FVector SourceLocation = FVector::ZeroVector;
};

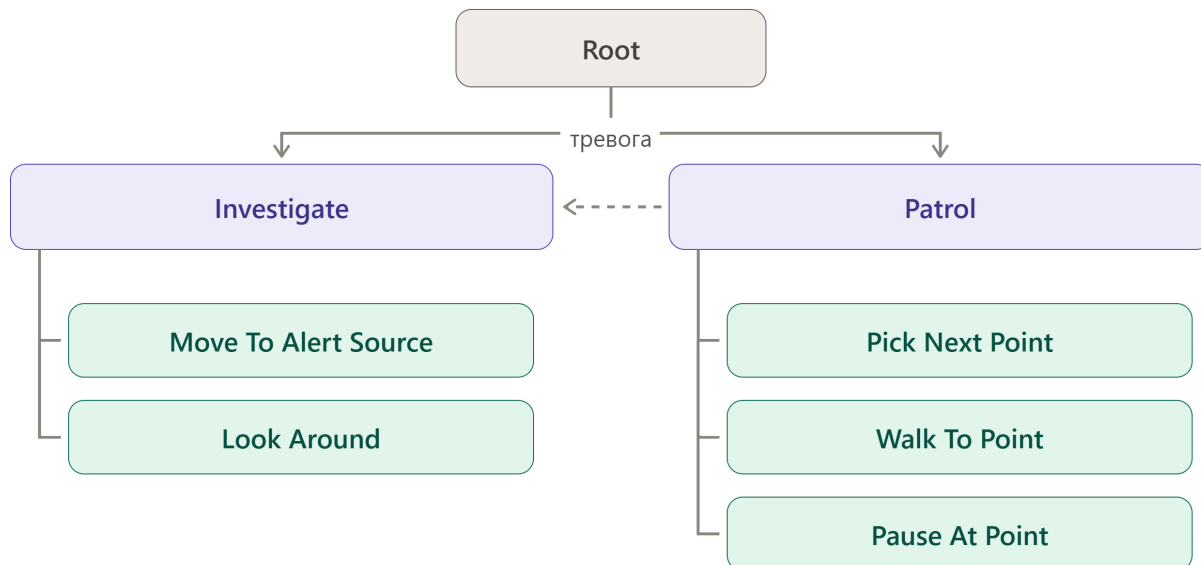
```

Цель движения. Здесь важное решение: `FMassZoneGraphTargetLocation` (глава 17) — **не фрагмент**. Она живёт в дереве как параметр состояния, а узлы обмениваются ею через `FStateTreePropertyRef`.

Почему не фрагмент? Потому что это внутренний обмен между узлами одного дерева. Процессорам Mass она не нужна — им нужен `FMassZoneGraphPathRequestFragment`, который заполнит `ZG Path Follow`. Класть во фрагмент то, что нужно только дереву, — лишние 90 байт на каждом агенте.

21.3. Структура дерева

Спроектируем поведение до написания кода.



Расшифруем структуру.

Root — `TrySelectChildrenInOrder`. Порядок важен: `Investigate` объявлен первым, значит проверяется первым.

Investigate — условие входа: уровень тревоги выше порога. Если условие не проходит, выбор откатывается и переходит к `Patrol` (глава 3).

Patrol — без условий входа, выбирается всегда, если `Investigate` не подошёл.

Дочерние состояния в каждой ветке идут последовательно. Переход у каждого: триггер `OnStateCompleted`, тип `NextSelectableState`. Последнее состояние в ветке возвращается к родителю через `Parent`.

Пунктирная стрелка — переход по тревоге: триггер `OnTick`, условие «тревога выше порога», цель `Investigate`, приоритет `High`. Он висит на состоянии `Patrol`, значит работает из любого его дочернего состояния.

Задачи в состояниях

Состояние	Задачи
Pick Next Point	<code>FMyPickPatrolPointTask</code> — вычисляет цель и завершается немедленно
Walk To Point	<code>ZG Path Follow</code> + <code>Mass LookAt</code> (фоновая)
Pause At Point	<code>Delay</code> со случайной длительностью + <code>Mass LookAt</code> со случайным взглядом

Состояние	Задачи
Move To Alert Source	ZG Path Follow
Look Around	Delay + Mass LookAt с широким разбросом

Обратите внимание на Mass LookAt в Walk To Point: у неё должен стоять `bConsideredForCompletion = false`, иначе состояние никогда не завершится (глава 15).

21.4. Задача выбора следующей точки

Единственный узел, который придётся написать. Всё остальное — готовые задачи модуля.

cpp

```
// MyPickPatrolPointTask.h
#pragma once

#include "MassStateTreeTypes.h"
#include "StateTreePropertyRef.h"
#include "MyPatrolFragments.h"
#include "MyPickPatrolPointTask.generated.h"

struct FMassZoneGraphLaneLocationFragment;

namespace UE::MassBehavior
{
    struct FStateTreeDependencyBuilder;
}

USTRUCT()
struct FMyPickPatrolPointTaskInstanceData
{
    GENERATED_BODY()

    /** Выход: сюда пишем цель, отсюда её читает ZG Path Follow. */
    UPROPERTY(EditAnywhere, Category = Output, meta = (RefType =
"/Script/MassAIBehavior.MassZoneGraphTargetLocation"))
    FStateTreePropertyRef TargetLocation;
};

USTRUCT(meta = (DisplayName = "Pick Patrol Point"))
struct FMyPickPatrolPointTask : public FMassStateTreeTaskBase
{

```

```

GENERATED_BODY()

using FInstanceDataType = FMyPickPatrolPointTaskInstanceData;

FMyPickPatrolPointTask()
{
    // Вся работа делается при входе, тикать незачем.
    bShouldCallTick = false;
    bShouldCopyBoundPropertiesOnExitState = false;
}

protected:
    virtual const UStruct* GetInstanceDataType() const override
    {
        return FInstanceDataType::StaticStruct();
    }

    virtual bool Link(FStateTreeLinker& Linker) override;
    virtual void
GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
override;
    virtual EStateTreeRunStatus EnterState(FStateTreeExecutionContext&
Context, const FStateTreeTransitionResult& Transition) const override;

#if WITH_GAMEPLAY_DEBUGGER
    virtual FString GetDebugInfo(const FStateTreeReadOnlyExecutionContext&
Context) const override;
#endif

    // ВНИМАНИЕ: список обязан совпадать с GetDependencies().
    TStateTreeExternalDataHandle<FMyPatrolRouteFragment> RouteHandle;
    TStateTreeExternalDataHandle<FMassZoneGraphLaneLocationFragment>
LocationHandle;
};

```

Реализация:

cpp

```

// MyPickPatrolPointTask.cpp

bool FMyPickPatrolPointTask::Link(FStateTreeLinker& Linker)
{
    Linker.LinkExternalData(RouteHandle);
    Linker.LinkExternalData(LocationHandle);
    return true;
}

```

```

}

void
FMyPickPatrolPointTask::GetDependencies(UE::MassBehavior::FStateTreeDependency
Builder& Builder) const
{
    Builder.AddReadWrite<FMyPatrolRouteFragment>();    // двигаем NextIndex
    Builder.AddReadOnly<FMassZoneGraphLaneLocationFragment>();
}

EStateTreeRunStatus
FMyPickPatrolPointTask::EnterState(FStateTreeExecutionContext& Context, const
FStateTreeTransitionResult& Transition) const
{
    FInstanceDataType& InstanceData = Context.GetInstanceData(*this);

    FMassZoneGraphTargetLocation* Target =
        InstanceData.TargetLocation.GetPtr<FMassZoneGraphTargetLocation>
(Context);
    if (Target == nullptr)
    {
        return EStateTreeRunStatus::Failed;
    }

    FMyPatrolRouteFragment& Route = Context.GetExternalData(RouteHandle);
    if (Route.Points.IsEmpty())
    {
        return EStateTreeRunStatus::Failed;
    }

    const FMassZoneGraphLaneLocationFragment& Location =
Context.GetExternalData(LocationHandle);

    const FMyPatrolPoint& Point = Route.Points[Route.NextIndex];

    // Обязательный сброс – структура переиспользуется (глава 17).
    Target->Reset();
    Target->LaneHandle = Point.Lane;
    Target->TargetDistance = Point.DistanceAlongLane;
    Target->bMoveReverse = Point.DistanceAlongLane <
Location.DistanceAlongLane;
    Target->EndOfPathIntent = EMassMovementAction::Stand;

    // Циклический маршрут.
    Route.NextIndex = (Route.NextIndex + 1) % Route.Points.Num();
}

```

```
    return EStateTreeRunStatus::Succeeded;
}
```

Три момента, которые стоит отметить.

`bShouldCallTick = false`. Задача делает всё в `EnterState()` и завершается там же. Тик ей не нужен — и это экономит виртуальный вызов и копирование привязок на каждом тике для каждого агента (глава 15).

Возврат `Succeeded` из `EnterState()`. Состояние завершается немедленно, не дойдя до тика. Дерево тут же переходит к следующему.

`Target->Reset()` перед заполнением. Ровно по причине из главы 17: инициализаторы полей работают при создании, а структура переиспользуется.

`EndOfPathIntent = Stand`. Намерение остановиться в конце. Процессор движения плавно затормозит агента — задача сама этого не делает (глава 4).

21.5. Условие тревоги

cpp

```
USTRUCT()
struct FMyAlertConditionInstanceData
{
    GENERATED_BODY()

    /** Порог. Привязывается к параметру дерева или задаётся вручную. */
    UPROPERTY(EditAnywhere, Category = Parameter)
    float Threshold = 0.7f;
};

USTRUCT(meta = (DisplayName = "Alert Above Threshold"))
struct FMyAlertCondition : public FMassStateTreeConditionBase
{
    GENERATED_BODY()

    using FInstanceDataType = FMyAlertConditionInstanceData;

protected:
    virtual const UStruct* GetInstanceDataType() const override
    {
        return FInstanceDataType::StaticStruct();
    }
}
```

```

virtual bool Link(FStateTreeLinker& Linker) override
{
    Linker.LinkExternalData(AlertHandle);
    return true;
}

virtual void
GetDependencies(UE::MassBehavior::FStateTreeDependencyBuilder& Builder) const
override
{
    Builder.AddReadOnly<FMyAlertFragment>();
}

virtual bool TestCondition(FStateTreeExecutionContext& Context) const
override
{
    const FMyAlertFragment* Alert =
Context.GetExternalDataPtr(AlertHandle);
    if (Alert == nullptr)
    {
        return false;    // нет системы тревоги — нечего расследовать
    }

    const FInstanceDataType& InstanceData =
Context.GetInstanceData(*this);
    return Alert->Level > InstanceData.Threshold;
}

TStateTreeExternalDataHandle<FMyAlertFragment,
EStateTreeExternalDataRequirement::Optional> AlertHandle;
};

```

Условие максимально дешёвое: получить указатель, сравнить два числа. Именно таким оно и должно быть (глава 18) — условия вызываются каскадом при каждом выборе состояния.

Доступ `ReadOnly` — условие ничего не меняет. Это не только правильно по смыслу, но и лучше для параллелизма (глава 8).

Фрагмент опционален: агент без системы восприятия просто никогда не будет расследовать.

21.6. Разнообразие через привязки

Пауза на точке должна быть случайной. Пишем ли мы для этого задачу? Нет — используем `property function` (глава 18).

В состоянии `Pause At Point` стоит стандартная задача `Delay`. Её параметр `Duration` привязывается не к константе, а к функции:

```
[Random Float] → Delay.Duration
    Min ← Params.MinPauseTime
    Max ← Params.MaxPauseTime
```

А `MinPauseTime` и `MaxPauseTime` приходят из глобальных параметров дерева, которые, в свою очередь, можно связать с общим фрагментом.

Ноль строк кода — и тысяча агентов расходится по фазе.

Здесь стоит вспомнить предупреждение из главы 11: случайность берётся из потока, инициализированного при `Start()`. По умолчанию сид — от таймера. Для сетевого проекта задавайте его явно, например производным от индекса сущности.

21.7. Конфигурация агента

Собираем `UMassEntityConfigAsset` из трейтов (глава 20):

Трейт	Зачем
Mass Movement	Скорость, силы, <code>FMassMovementParameters</code>
Mass ZoneGraph Navigation	Все четыре фрагмента навигации
Mass Avoidance	Обход препятствий и друг друга
Mass LookAt	<code>FMassLookAtFragment</code>
Mass Representation	Визуальное представление, LOD
StateTree	<code>FMassStateTreeInstanceFragment</code> + ассет дерева
My Patrol	<code>FMyPatrolRouteFragment</code> + параметры
My Alert	<code>FMyAlertFragment</code>
Mass Simulation LOD	Уровни детализации симуляции

Три трейта наши, остальные из движка.

Трейт патрулирования:

cpp

```
UCLASS(meta = (DisplayName = "Patrol"))
class UMyPatrolTrait : public UMassEntityTraitBase
{
```

```

GENERATED_BODY()

protected:
    virtual void BuildTemplate(FMassEntityTemplateBuildContext& BuildContext,
    const UWorld& World) const override
    {
        BuildContext.AddFragment<FMyPatrolRouteFragment>();

        const FMassEntityManager& EntityManager =
        UE::Mass::Utils::GetEntityManagerChecked(World);
        BuildContext.AddConstSharedFragment(
            EntityManager.GetOrCreateConstSharedFragment(Params));
    }

    virtual void ValidateTemplate(FMassEntityTemplateBuildContext&
    BuildContext, const UWorld& World) const override
    {
        if (Params.MinPauseTime > Params.MaxPauseTime)
        {
            UE_LOG(LogMass, Warning, TEXT("%s: MinPauseTime больше
            MaxPauseTime"), *GetNameSafe(this));
        }
    }

    UPROPERTY(EditAnywhere, Category = "")
    FMyPatrolParamsFragment Params;
};

```

Маршрут заполняется при спавне — генератором данных размещения или отдельным процессором инициализации, который назначает агенту ближайшие точки интереса.

21.8. Что происходит покадрово

Теперь самое интересное — проследим исполнение.

Кадр 0: спавн

`AMassSpawner` создаёт сущности пакетно. Архетип уже построен, шаблон закэширован. У каждого агента появляются все фрагменты, включая `FMassStateTreeInstanceFragment` с пустым хендлом.

Кадр 0, позже: выделение данных

Наблюдатель на добавление `FMassStateTreeInstanceFragment` вызывает `AllocateInstanceData()` (глава 9). Попутно, при первом обращении к ассету, подсистема:

1. Берёт схему, вызывает `GetDependencies()` ;
2. Конвертирует зависимости в требования `Mass`;
3. Считает хеш, ищет процессор, не находит;
4. Создаёт динамический `UMassStateTreeProcessor` , настраивает требования, регистрирует.

Требования нашего дерева — объединение всех узлов:

<code>FMassZoneGraphLaneLocationFragment</code>	RO
<code>FMassMoveTargetFragment</code>	RW
<code>FMassZoneGraphPathRequestFragment</code>	RW
<code>FMassZoneGraphShortPathFragment</code>	RW
<code>FMassZoneGraphCachedLaneFragment</code>	RW
<code>FAgentRadiusFragment</code>	RO
<code>FMassMovementParameters</code>	RO
<code>FMassLookAtFragment</code>	RW
<code>FMyPatrolRouteFragment</code>	RW
<code>FMyAlertFragment</code>	RO
<code>UZoneGraphSubsystem</code>	RO
<code>UMassSignalSubsystem</code>	RW

Двенадцать записей. Солвер по ним размещает процессор в графе так, чтобы он шёл до процессоров движения (которые читают `FMassMoveTargetFragment`) и после процессора восприятия (который пишет `FMyAlertFragment`).

Кадр 1: активация

`UMassStateTreeActivationProcessor` находит агентов без тега, ставит `FMassStateTreeActivatedTag` через командный буфер и шлёт `StateTreeActivate` пачкой на все сущности сразу.

Кадр 1, позже: первый тик

Процессор просыпается. Для каждого чанка:

1. Достаёт `FMassStateTreeSharedFragment` , проверяет ассет в `HandledStateTrees` ;
2. Создаёт **один** `FMassStateTreeExecutionContext` на чанк;
3. В цикле по агентам: проверяет хендл, получает `instance data`, считает дельту, вызывает `SetEntity()` и `Start()` .

При `Start()` подставляется `FMassExecutionExtension` с хендлом сущности — теперь логи будут привязаны к агенту (глава 12).

Дерево выбирает состояние. Root пробует `Investigate` — условие «тревога > 0.7» не проходит (тревога равна нулю). Откат, пробуем `Patrol`. Внутри — `Pick Next Point`.

`FMyPickPatrolPointTask::EnterState()` заполняет цель, двигает индекс, возвращает `Succeeded`. Состояние завершается немедленно.

Срабатывает переход `OnStateCompleted` → `NextSelectableState`. Выбирается `Walk To Point`.

`ZG Path Follow::EnterState()` заполняет заявку на путь и выставляет намерение движения. `Mass LookAt::EnterState()` захватывает фрагмент взгляда и планирует `LookAtFinished`. Обе возвращают `Running`.

Дерево засыпает.

Кадры 2–200: агент идёт

Дерево **не тикает вообще**. Работают процессоры Mass:

- процессор построения пути видит заявку, строит короткий путь;
- процессор движения читает `FMassMoveTargetFragment` и двигает агента;
- процессор обхода корректирует траекторию;
- процессор взгляда крутит голову;
- процессор представления обновляет визуал по LOD.

Двести кадров работы — и ноль тиков дерева. Это и есть выигрыш событийной модели (глава 12).

Кадр ~200: прибытие

Процессор пути выставляет флаг завершения. Дерево должно проснуться — и здесь важный практический момент, который мы отмечали в главе 17: `ZG Path Follow` сама сигналов не шлёт.

Пробуждение приходит от `Mass LookAt`: её `Duration` истекла, отложенный `LookAtFinished` сработал. Процессор тикает дерево, `ZG Path Follow::Tick()` видит флаг и возвращает `Succeeded`.

Если бы взгляда в состоянии не было, понадобился бы другой источник пробуждения — например, сигнал от процессора пути. **Проверяйте это в своих деревьях.**

Кадр ~200, продолжение

Состояние завершается, переход к `Pause At Point`. Задача `Delay` получает случайную длительность через `property function`, планирует отложенный переход.

BeginDelayedTransition() ставит DelayedTransitionWakeup (глава 12).

Дерево снова засыпает — на три-четыре секунды.

Кадр ~400: тревога

Процессор восприятия обнаружил шум, записал в FMyAlertFragment уровень 0.85 и позицию источника.

Но дерево спит. Оно не узнает о тревоге, пока не проснётся.

Это ключевой момент проектирования. Процессор восприятия обязан **послать сигнал**:

cpp

```
void UMyPerceptionProcessor::Execute(FMassEntityManager& EntityManager,
FMassExecutionContext& Context)
{
    TArray<FMassEntityHandle> NewlyAlerted;

    EntityQuery.ForEachEntityChunk(Context, [&NewlyAlerted]
(FMassExecutionContext& Context)
    {
        const TArrayView<FMyAlertFragment> Alerts =
Context.GetMutableFragmentView<FMyAlertFragment>();
        for (int32 i = 0; i < Context.GetNumEntities(); ++i)
        {
            const float Previous = Alerts[i].Level;
            Alerts[i].Level = ComputeAlert(Context, i);

            // сигналим только на пересечении порога, а не каждый кадр
            if (Previous <= 0.7f && Alerts[i].Level > 0.7f)
            {
                NewlyAlerted.Add(Context.GetEntity(i));
            }
        }
    });

    if (!NewlyAlerted.IsEmpty())
    {
        SignalSubsystem->SignalEntities(MyGame::Signals::AlertRaised,
NewlyAlerted);
    }
}
```

Обратите внимание на **сигнал по пересечению порога**, а не по каждому изменению. Иначе тревожный агент будил бы дерево каждый кадр, и вся экономия исчезла бы.

И, разумеется, наш процессор должен быть подписан на `AlertRaised` :

cpp

```
UCLASS()
class UMyStateTreeProcessor : public UMassStateTreeProcessor
{
    GENERATED_BODY()
protected:
    virtual void InitializeInternal(UObject& Owner, const
TSharedRef<FMassEntityManager>& EntityManager) override
    {
        Super::InitializeInternal(Owner, EntityManager);
        SubscribeToSignal(MyGame::Signals::AlertRaised);
    }
};
```

Плюс строка в настройках:

ini

```
[/Script/MassAIBehavior.MassBehaviorSettings]
DynamicStateTreeProcessorClass=/Script/MyGame.MyStateTreeProcessor
```

Кадр ~400, продолжение: переход

Дерево просыпается. Обработываются переходы. Переход на `Patrol` с триггером `OnTick` и условием тревоги срабатывает, приоритет `High` .

Задачам текущих состояний приходит `ExitState()` — снизу вверх. `Mass LookAt` отпускает фрагмент взгляда, `Delay` отменяет отсчёт. `ZG Path Follow` метода `ExitState()` не имеет, намерение движения остаётся — и это правильно (глава 17).

Выбирается `Investigate` → `Move To Alert Source` . Агент разворачивается и идёт разбираться.

Смерть агента

Сущность уничтожается. `UMassStateTreeFragmentDestructor` перехватывает удаление фрагмента:

1. Строит контекст, вызывает `Stop()` ;

2. Всем активным задачам приходит `ExitState()` — фрагмент взгляда отпускается корректно;
3. Вызывается `FreeInstanceData()`, слот возвращается в пул, поколение растёт.

Отложенные сигналы, адресованные этому агенту, останутся в очереди, но не сработают: три уровня проверки их отсекут (глава 14).

21.9. Что показал пример

Соберём наблюдения.

Кода написано мало. Одна задача, одно условие, три трейта, один процессор восприятия, один наследник процессора `StateTree`. Всё остальное — готовые узлы и конфигурация в редакторе.

Дерево спит большую часть времени. Из четырёхсот кадров оно тикнуло раз пять. Вся работа — в процессорах, обрабатывающих толпу пачками.

Пробуждение — главная забота проектировщика. Дважды в примере возникал вопрос «кто разбудит»: при прибытии и при тревоге. В обоих случаях ответ пришлось искать явно.

Разнообразие достигается привязками, а не кодом. Случайная пауза — одна `property function`.

Требования дерева широки. Двенадцать записей, из них шесть на запись. Процессор конфликтует со многими — это цена универсального поведения (глава 8).

Данные разложены по трём уровням. Персональное — фрагменты, общее и неизменное — константный общий фрагмент, внутреннее для дерева — `instance data` и `property ref`.

21.10. Как это масштабировать

Если агентов станет десятки тысяч, посмотрите на следующее.

Разделить дерево. Патрулирование и расследование — разные наборы требований. Два ассета вместо одного дадут два процессора с более узкими требованиями и лучшим параллелизмом. Цена — переключение поведения придётся делать сменой ассета, что нетривиально.

Сократить одновременно активные задачи. Каждая активная задача — её `instance data` в буфере (глава 10). Замерьте через `GetEstimatedMemoryUsage()`.

Проверить частоту сигналов. Если процессор восприятия шлёт сигнал слишком часто, деревья перестают спать. Пороговая логика обязательна.

Рассмотреть параллельный обход. По чек-листу из главы 13: все сигналы deferred, никаких спавнов из задач, подсистемы безопасны.

Уменьшить число конфигураций. Каждая уникальная композиция — свой архетип. Вариативность лучше делать значениями фрагментов, а не разными наборами трейтов.

21.11. Итог главы

Что делали	Где это разбиралось
Разложили данные по фрагментам и общим фрагментам	Главы 6, 20
Спроектировали дерево с условиями входа и приоритетным переходом	Глава 3
Написали задачу без тика, завершающуюся в <code>EnterState()</code>	Главы 15, 16
Использовали <code>FStateTreePropertyRef</code> для обмена целью	Глава 17
Написали дешёвое условие только на чтение	Глава 18
Добавили разнообразие через <code>property function</code>	Глава 18
Собрали конфигурацию из трейтов	Глава 20
Обеспечили пробуждение сигналами по порогу	Главы 12, 14
Подписали свой процессор на свой сигнал	Главы 13, 14
Проследили корректное завершение при смерти	Главы 9, 13

Что дальше

Глава 22 — отладка. Разберём инструменты: `Gameplay Debugger` и его категории для `Mass`, `StateTree Debugger` с трассировкой, консольные команды `mass.debug`, визуальный лог. Посмотрим, что даёт `GetInstanceDescription()` и как читать логи толпы. И, главное, соберём таблицу «симптом → где смотреть» — чтобы диагностика шла по признакам, а не перебором.

Глава 22. Отладка

Отладка Mass-поведения отличается от отладки обычного AI принципиально. Нет актора, на котором можно поставить точку останова. Нет `Tick`, куда можно вставить лог. Агентов тысячи, и ломается обычно не «этот», а «примерно каждый двадцатый в определённых обстоятельствах».

Эта глава — про инструменты и про методику.

22.1. Почему обычные приёмы не работают

Точка останова в задаче. Поставили — и она срабатывает для первого попавшегося агента из десяти тысяч. Условная точка останова по хендлу сущности помогает, но хендл ещё надо узнать.

`UE_LOG` в `Tick()`. Получите поток из тысяч строк в секунду, в котором ничего не разобрать. И, что хуже, само логирование замедлит симуляцию настолько, что баг с гонкой перестанет воспроизводиться.

Пошаговая отладка. Отладчик останавливает все потоки. Гонки данных под ним не воспроизводятся никогда — а это как раз самый частый класс ошибок в Mass (глава 8).

Отсюда методика: **сначала сузить область до одного агента, потом смотреть на него подробно**. Инструменты ниже выстроены именно в этом порядке.

22.2. Первое, что надо проверить

Прежде чем лезть в инструменты — три вопроса, которые закрывают большую часть обращений.

Вы не в PIE-клиенте?

cpp

```
namespace UE::MassStateTree
{
    static constexpr EProcessorExecutionFlags ExecutionFlags(
        EProcessorExecutionFlags::Standalone | EProcessorExecutionFlags::Server);
}
```

Деревья исполняются **только** в Standalone и на сервере (глава 5). Если вы запустили PIE с клиентом и смотрите на клиентское окно — там не тикает ничего и не должно.

Это причина номер один для «у меня совсем ничего не работает».

Ассет скомпилирован?

Если дерево правили и не сохранили, схема могла не пересобирать `Dependencies`. Плюс помните комментарий Eric из главы 9:

cpp

```
// @todo ask Patrick how would this behave when ST asset gets
changed/recompiled or whatnot
```

После существенных правок дерева перезапускайте PIE. Горячая перезагрузка не всегда пересоздаёт процессоры с новыми требованиями.

Кто должен разбудить дерево?

Если агент «замер» — почти наверняка ни один сигнал не запланирован (глава 12). Прежде чем искать инструменты, просто перечитайте свою задачу и ответьте: кто пришлёт следующий сигнал.

22.3. Gameplay Debugger

Основной инструмент. Включается клавишей апостроф (`'`) по умолчанию, категории переключаются цифрами.

Для Mass есть выделенная категория, показывающая под агентом:

- активные состояния дерева (весь стек, от корня до листа);
- отладочные строки задач — то, что возвращает `GetDebugInfo()`;
- значения ключевых фрагментов;
- хендл сущности.

Вот здесь и окупается переопределение отладочного вывода, о котором я говорил в главах 15, 16 и 19:

cpp

```
#if WITH_GAMEPLAY_DEBUGGER
FString FMyPickPatrolPointTask::GetDebugInfo(const
FStateTreeReadOnlyExecutionContext& Context) const
{
    return TEXT("PickPatrolPoint");
}
#endif
```

Без него в отладчике будет только имя узла. С ним — осмысленное состояние.

Выбор агента

Отладчик показывает данные **выделенного** агента. Выбор идёт по направлению взгляда камеры или командой. В толпе прицелиться сложно, поэтому полезны консольные команды фильтрации по типу и радиусу — ищите их в разделе `mass.debug` (точные имена различаются между версиями движка).

Что смотреть в первую очередь

Стек активных состояний. Показывает, где агент застрял. Если состояние то же, что минуту назад, — либо задача честно работает, либо она вернула `Running` без пробуждения.

Отладочные строки задач. Показывают внутренние счётчики: сколько прошло, сколько осталось.

Значения фрагментов. Показывают, дошло ли намерение до исполнителей. Задача записала цель движения, а `FMassMoveTargetFragment` пустой — значит, где-то разрыв.

22.4. StateTree Debugger

Отдельный инструмент, работающий на трассировке. Включается под макросом:

cpp

```
#if WITH_STATETREE_TRACE
    UE_API FStateTreeInstanceDebugId GetInstanceDebugId() const;
    FString GetInstanceDebugDescription() const { return
GetInstanceDescriptionInternal(); }
    void SetOuterTraceId(const uint64 Id) const;
    // ...
#endif
```

Он записывает историю исполнения конкретного экземпляра дерева: какие состояния выбирались, какие условия проходили, какие переходы срабатывали, какие события приходили. Потом эту историю можно проматывать назад и вперёд.

Для Mass это **самый мощный инструмент из доступных**, потому что отвечает на вопрос «почему выбралось это состояние, а не то» — а именно он чаще всего и стоит.

Идентификация экземпляра

Здесь работает `FMassExecutionExtension` из главы 12:

cpp


```
virtual FString GetInstanceDescription(const FContextParameters& Context)
const override;

FMassEntityHandle Entity;
```

Без расширения все экземпляры назывались бы именем владельца — то есть `MassStateTreeSubsystem`, одинаково для всех. Расширение подставляет хендл сущности, и в списке экземпляров отладчика вы видите различные записи.

Ограничения

Трассировка недёшева. На тысячах агентов она заметно замедлит симуляцию и завалит буфер. Практический подход: воспроизвести проблему на **малом числе агентов** (десятки), включив трассировку, и разбираться там.

Если баг воспроизводится только на тысячах — трассировка не поможет, и это признак, что вы имеете дело с гонкой или с проблемой планирования, а не с логикой дерева.

22.5. Консольные команды

Раздел `mass.debug` содержит набор команд для инспекции. Точные имена стоит смотреть через автодополнение консоли (наберите `mass.debug` и нажмите Tab) — они меняются между версиями. Категории, которые вам понадобятся:

Граф процессоров. Вывод порядка исполнения и групп. Здесь видно, где встал ваш динамический процессор `StateTree` и с кем он конфликтует. Незаменимо при разборе «почему всё стало последовательным» (глава 8).

Требования процессоров. Метод `UMassProcessor::DebugOutputDescription()` печатает объявленные требования. Для нашего процессора это объединение собственного запроса и `ExecutionRequirements` (глава 13).

Список архетипов. Показывает, сколько архетипов в мире и сколько сущностей в каждом. Если архетипов подозрительно много при небольшом числе агентов — у вас слишком дробная конфигурация (глава 20).

Переключатель динамических процессоров.

cpp

```
namespace UE::Mass::StateTree
{
    extern bool bDynamicSTProcessorsEnabled;
}
```

Аварийный выключатель из главы 9. Если после обновления движка порядок процессоров развалился — выключите и посмотрите, изменится ли картина. Быстро локализует проблему.

22.6. Собственная диагностика

Штатных инструментов часто не хватает. Три приёма, которые стоит завести в проекте.

Дамп зависимостей при загрузке

Временно вставьте в `Link()` своей схемы (или в наследнике `UMassStateTreeSchema`) вывод собранных зависимостей:

cpp

```
bool UMySchema::Link(FStateTreeLinker& Linker)
{
    if (!Super::Link(Linker))
    {
        return false;
    }

    UE_LOG(LogMyGame, Log, TEXT("Зависимости дерева %s:"),
    *GetNameSafe(GetOuter()));
    for (const FMassStateTreeDependency& Dep : GetDependencies())
    {
        UE_LOG(LogMyGame, Log, TEXT("  %s : %s"),
            *Dep.Type->GetName(),
            Dep.Access == EMassFragmentAccess::ReadWrite ? TEXT("RW") :
            TEXT("RO"));
    }
    return true;
}
```

Один раз при загрузке, ноль стоимости в рантайме. Показывает, что именно требует ваше дерево, — и часто выясняется, что там есть неожиданные записи от узлов, о которых вы забыли.

Автотест соответствия `Link` и `GetDependencies`

Ошибка рассогласования (глава 8, раздел 8.7) не ловится компилятором и почти не ловится тестированием. Но её можно поймать автоматически.

Идея: для каждого Mass-узла через рефлекссию собрать все поля типа `TStateTreeExternalDataHandle<>`, извлечь их типы данных, сравнить с результатом

```
GetDependencies() .
```

Технически это непросто — тип хендла шаблонный, — но выполнимо через анализ `UScriptStruct` узла и разбор имён свойств. В движке такого нет; если ваш проект активно пишет свои узлы, вложение окупится.

Более простой вариант: соглашение в код-ревью. Правило «`Link()` и `GetDependencies()` смотрим всегда вместе» дёшево и работает.

Замер памяти instance data

cpp

```
UE_API int32 GetEstimatedMemoryUsage() const;
```

Метод из главы 10. Периодически замеряйте:

cpp

```
// в отладочной команде
int32 Total = 0;
int32 Count = 0;
// пройти по InstanceDataArray подсистемы...
UE_LOG(LogMyGame, Log, TEXT("Instance data: %d агентов, %.1f КБ, в среднем %d Б"),
    Count, Total / 1024.f, Count > 0 ? Total / Count : 0);
```

Резкий рост среднего размера обычно означает, что кто-то добавил в дерево состояние с большим количеством одновременно активных задач.

22.7. Как читать логи толпы

Логирование в Mass требует дисциплины. Три правила.

Логируйте события, а не состояния. Не «агент идёт», а «агент начал идти», «агент прибыл». События редки, состояния постоянны.

Ограничивайте по сущности. Заведите отладочную переменную «интересующий хендл» и логируйте только его:

cpp

```
#if !UE_BUILD_SHIPPING
    if (UE::MyGame::Debug::WatchedEntity == Entity)
```

```

    {
        UE_LOG(LogMyGame, Log, TEXT("[%s] вошёл в состояние Patrol"),
        *Entity.DebugGetDescription());
    }
#endif

```

Ограничивайте по частоте. Если событие может произойти тысячу раз за кадр, логируйте счётчик, а не каждое:

c++

```

static int32 FrameCounter = 0;
++FrameCounter;
// и раз в секунду выводите итог

```

Префикс с хендлом сущности приходит бесплатно, если вы пользуетесь макросами `STATETREE_LOG` внутри узлов — они подставляют описание из `FMassExecutionExtension`.

22.8. Таблица «симптом → где смотреть»

Основная практическая часть главы. Читайте по симптому.

Ничего не работает вообще

Проверить	Как
Режим запуска	Standalone или сервер, не PIE-клиент
Наличие фрагментов	Gameplay Debugger: есть ли <code>FMassStateTreeInstanceFragment</code>
Выделение instance data	Хендл валиден?
Создание процессора	Дамп графа процессоров: есть ли динамический процессор <code>StateTree</code>
Тег активации	Есть ли <code>FMassStateTreeActivatedTag</code>

Агент застрял в состоянии

Проверить	Как
Стек состояний	Gameplay Debugger: меняется ли он
Источник пробуждения	Чтение кода задачи: кто пришлёт сигнал
Подписка процессора	Ваш сигнал есть в <code>InitializeInternal()</code> ?

Проверить	Как
Класс процессора в настройках	<code>DynamicStateTreeProcessorClass</code> прописан?
Фоновая задача	Есть ли вечно- Running задача без <code>bConsideredForCompletion = false</code>

Выбирается не то состояние

Проверить	Как
Порядок детей	Первый подходящий выигрывает при <code>TrySelectChildrenInOrder</code>
Условия входа	StateTree Debugger: какое условие не прошло
Приоритеты переходов	Более приоритетный перебивает
Данные условия	Значение фрагмента, которое читает условие

Поведение «дёргается»

Проверить	Как
Переизбрание состояния	<code>bShouldStateChangeOnReselect</code> у удерживающих задач
Гонка данных	Сверить <code>Link()</code> и <code>GetDependencies()</code> построчно
Параллельный обход	<code>bProcessEntitiesInParallel</code> включён при небезопасных задачах?
Конкуренция за ресурс	Приоритеты в системе арбитража (как у <code>FMassLookAtPriority</code>)

Утечка логических ресурсов

Проверить	Как
Освобождение не в том методе	Должно быть в <code>ExitState()</code> , не в <code>StateCompleted()</code>
Деструктор не работает	Зарегистрирован ли <code>UMassStateTreeFragmentDestructor</code> , совпадают ли <code>ExecutionFlags</code>
<code>Stop()</code> не вызывается	Проверить порядок в деструкторе: <code>Stop()</code> до <code>FreeInstanceData()</code>

Таймеры считают неверно

Проверить	Как
Источник дельты	Используется параметр <code>Tick()</code> , а не кадровое время
Задача не тикает	<code>bShouldCallTick</code> выключен, а счётчик копится
Поле не сброшено	<code>Time = 0</code> в начале <code>EnterState()</code>

Производительность просела

Проверить	Как
Граф стал линейным	Дамп графа: где последовательные связи
Слишком широкие требования	Дамп зависимостей дерева
Деревья не спят	Частота сигналов; шлёт ли процессор сигнал каждый кадр
Много архетипов	Список архетипов: разреженные чанки
Много процессоров	По одному на ассет вместо одного на группу

Агенты движутся синхронно

Отсутствие разнообразия. Добавьте случайную `property function` в привязку длительностей (глава 18) или разнесите параметры при спавне.

Ассерт многопоточного детектора

Детектор	Причина
<code>InstanceDataMTDetector</code> в подсистеме	Аллокация <code>instance data</code> не из игрового потока или параллельно с чтением
<code>AccessDetector</code> в хранилище	Запись в <code>instance data</code> из нескольких потоков
<code>DelayedSignalsAccessDetector</code>	Отправка сигналов из нескольких потоков — используйте <code>deferred</code>

22.9. Методика: как локализовать сложный баг

Когда симптом не попадает в таблицу, работает такая последовательность.

Шаг 1. Свести к минимуму. Один агент вместо тысячи. Если баг исчез — это гонка или проблема масштаба, переходите к шагу 5. Если остался — логическая ошибка, продолжайте.

Шаг 2. Упростить дерево. Отключите узлы через `bTaskEnabled` (глава 15) или уберите состояния. Найдите минимальную конфигурацию, в которой баг воспроизводится.

Шаг 3. Включить трассировку. На одном агенте StateTree Debugger покажет всю историю решений.

Шаг 4. Проверить данные на границе. Задача записала во фрагмент — процессор его прочитал? Смотрите значения в Gameplay Debugger до и после.

Шаг 5. Для гонок: отключить параллелизм. `bProcessEntitiesInParallel = false`. Исчезло — вы нашли класс проблемы, теперь ищите незадекларированный доступ.

Шаг 6. Для гонок: сверить объявления. Построчно, для каждого узла дерева. Это скучно, но это единственный надёжный способ.

Шаг 7. Проверить порядок процессоров. Дамп графа. Возможно, ваш процессор оказался не там, где вы думали, и читает данные прошлого кадра.

22.10. Что стоит встроить заранее

Три вещи, которые дешевле сделать до появления проблем.

`GetDebugInfo()` в каждом узле. Две минуты при написании, часы экономии потом.

Отладочная команда «показать агента». Печать всех фрагментов и состояния дерева для выделенной сущности. Понадобится обязательно.

Счётчики в профилировщике. Обернуть тик деревьев в `SCOPE_CYCLE_COUNTER` с отдельной статистикой. Без этого вы не отличите «дерево тормозит» от «тормозит движение».

22.11. Итог главы

Инструмент	Для чего
Проверка режима запуска	Первое, что смотреть при «ничего не работает»
Gameplay Debugger	Состояние конкретного агента здесь и сейчас
StateTree Debugger	История решений: почему выбралось это состояние
Дамп графа процессоров	Порядок исполнения и конфликты
Дамп зависимостей дерева	Что реально требует ваше дерево

Инструмент	Для чего
bDynamicSTProcessorsEnabled	Аварийный выключатель для локализации
GetEstimatedMemoryUsage()	Вес поведения в памяти
Логи по одной сущности	Единственный читаемый способ логировать толпу

Три правила:

1. **Сначала сузить до одного агента, потом смотреть подробно.**
2. **Гонки не воспроизводятся под отладчиком** — ищите их сверкой объявлений, а не пошаговым исполнением.
3. `GetDebugInfo()` пишется **вместе с задачей**, а не когда всё сломалось.

Что дальше

Глава 23 — производительность. Разберём, из чего складывается стоимость Mass-поведения: тик деревьев, сигналы, память instance data, построение графа. Посмотрим на параллельный обход и условия его безопасности, на влияние состава требований на пропускную способность, на стратегии LOD для поведения, на шардирование процессоров и на то, когда стоит разделять деревья, а когда объединять. С цифрами и порядками величин там, где их можно оценить.

Глава 23. Производительность

Mass существует ради масштаба. Эта глава — про то, из чего складывается стоимость поведения и где искать резервы.

Сразу оговорка о цифрах: конкретные значения зависят от железа, версии движка и содержимого ваших деревьев. Я привожу порядки величин и соотношения, а не абсолютные числа — измерять всё равно придётся на своём проекте.

23.1. Из чего складывается стоимость

Пять статей расхода, и они очень разные по природе.

Тик деревьев. Виртуальные вызовы задач, копирование привязок, вычисление условий при выборе состояний. Пропорционален числу **пробуждений**, а не числу агентов.

Сигналы. Отправка, накопление, обход очереди отложенных. Пропорционален числу событий.

Память instance data. Пул в подсистеме плюс буферы активных задач. Пропорционален числу агентов и глубине стека состояний.

Планирование. Построение графа процессоров и его исполнение. Не пропорционально ничему — это фиксированная стоимость, зависящая от структуры требований.

Потерянный параллелизм. Самая коварная статья: она не видна в профилировщике как отдельная строка, но растягивает кадр.

Порядок в списке примерно соответствует тому, в каком порядке эти статьи обычно становятся проблемой.

23.2. Тик деревьев: считаем пробуждения

Ключевая метрика Mass-поведения — не «сколько агентов», а **сколько тиков дерева в секунду**.

Прикинем на примере из главы 21. Патрульный агент за цикл маршрута (скажем, 30 секунд) тикает дерево:

- один раз на выбор точки;
- один раз на прибытие;
- один раз на завершение паузы;
- плюс изредка на события.

Порядка четырёх-пяти тиков за 30 секунд. Это примерно **0.15 тика в секунду на агента**.

Десять тысяч агентов дают полторы тысячи тиков дерева в секунду — около 25 тиков за кадр при 60 FPS. Совершенно незначительная нагрузка.

Сравните с покадровым тиком: $10\,000 \times 60 = 600\,000$ тиков в секунду. Разница в четыреста раз.

Вывод: если тик деревьев виден в профилировщике, у вас деревья не спят. Ищите, кто их будит.

Кто чаще всего будит зря

Процессор, шлющий сигнал каждый кадр. Классика из главы 21: восприятие пересчитывает уровень тревоги и сигнализирует при любом изменении вместо пересечения порога.

cpp

```
// ПЛОХО: сигнал каждый кадр, пока значение меняется
if (Alerts[i].Level != Previous)
{
    NewlyAlerted.Add(Entity);
}

// ХОРОШО: сигнал только при пересечении порога
if (Previous <= Threshold && Alerts[i].Level > Threshold)
{
    NewlyAlerted.Add(Entity);
}
```

Задача с коротким отложенным переходом. Переход с задержкой 0.1 секунды — это десять пробуждений в секунду на агента. На десяти тысячах агентов — сто тысяч тиков в секунду, и вся экономия исчезла.

Периодический опрос. Задача, которая ставит себе отложенный сигнал «проверить через полсекунды», превращает событийную модель в опрос с меньшей частотой. Иногда это необходимо, но каждый такой случай стоит пересмотреть: нельзя ли получить сигнал от того, кто действительно знает о наступлении события.

Стоимость одного тика

Внутри одного тика дерева работа складывается из:

- копирования привязок для тикающих задач;
- виртуальных вызовов `Tick()`;
- при смене состояния — вычисления условий, `ExitState()` / `EnterState()`, перестройки буфера `instance data`.

Смена состояния существенно дороже обычного тика: перестраивается

`FInstancedStructContainer` (глава 10), вызываются условия по всей цепочке выбора.

Практическое следствие: **дерево с частыми переключениями состояний дороже дерева с длинными состояниями**, даже при одинаковом числе пробуждений.

23.3. Флаги задач как инструмент оптимизации

Из главы 15 — четыре флага, которые прямо влияют на стоимость.

cpp

```

FMyTask()
{
    bShouldCallTick = false;           // нет виртуального вызова
и копирования
    bShouldCopyBoundPropertiesOnTick = false; // нет копирования, но тик
есть
    bShouldCopyBoundPropertiesOnExitState = false; // нет копирования при
выходе
    bShouldAffectTransitions = false; // нет вызова
TriggerTransitions
}

```

Экономия на одной задаче ничтожна. Но задач в активном стеке может быть пять-десять, а тиков — тысячи в секунду.

`bShouldCallTick = false` — **самый ценный**. Задача типа «записал намерение и жду сигнала» не должна тикать вообще. В примере из главы 21 такова `FMyPickPatrolPointTask`.

Проверьте свои задачи: если `Tick()` состоит из `return EStateTreeRunStatus::Running;` — флаг надо выключить.

`bShouldCopyBoundPropertiesOnTick = false` — когда входы читаются только при входе. Копирование привязок это проход по таблице копий с разыменованием указателей; для задачи с восемью входами это заметно.

Цена — теряется динамическое обновление входа. В `FMassLookAtTask` (глава 16) флаг оставлен включённым именно ради возможности сменить цель на лету.

23.4. Сигналы

Стоимость сигналов складывается из трёх частей.

Отправка. Поиск делегата в `TMap` плюс широковебательный вызов. Дёшево, но пропорционально числу **вызовов**, а не сущностей.

Отсюда правило из главы 14: собирайте массив.

cpp

```

// ПЛОХО: 1000 поисков в TMap, 1000 вызовов делегата
for (const FMassEntityHandle Entity : Entities)
{
    SignalSubsystem->SignalEntity(SignalName, Entity);
}

```

```
// ХОРОШО: один поиск, один вызов
SignalSubsystem->SignalEntities(SignalName, Entities);
```

Разница на порядки.

Накопление. `UMassSignalProcessorBase` собирает сущности и фильтрует по запросу. Пропорционально числу сигнализированных агентов.

Очередь отложенных. Здесь есть неочевидная проблема.

cpp

```
struct FDelayedSignal
{
    FName SignalName;
    TArray<FMassEntityHandle> Entities;
    double TargetTimestamp;
};

TArray<FDelayedSignal> DelayedSignals;
```

Тик подсистемы — линейный проход по **записям**. Если тысяча агентов получила один отложенный сигнал через `DelaySignalEntities()`, это одна запись.

Но `BeginDelayedTransition()` (глава 12) ставит сигнал **индивидуально** для каждого агента, потому что моменты истечения у всех разные. Десять тысяч агентов с отложенными переходами — десять тысяч записей, обходимых каждый кадр.

Обход дешёвый (сравнение `double`), но десять тысяч итераций в кадре — не ноль. Если профилировщик показывает `UMassSignalSubsystem::Tick` заметной строкой, вы упёрлись именно в это.

Смягчение: **избегайте очень коротких отложенных переходов**. Запись живёт в очереди от постановки до срабатывания; чем короче задержка, тем чаще очередь перестраивается.

23.5. Память

Три источника.

Фрагменты на сущности. `FMassStateTreeInstanceFragment` — 16 байт (глава 6). На десяти тысячах агентов — 160 КБ. Ничтожно.

Пул instance data. Здесь основной вес. Замеряйте:

cpp

```
UE_API int32 GetEstimatedMemoryUsage() const;
```

Порядок величины зависит от числа одновременно активных задач и размера их instance data. Задача с десятком полей даёт сотню байт; пять активных задач — полкилобайта; десять тысяч агентов — пять мегабайт.

Приемлемо, но не бесплатно. И заметьте: **массив в подсистеме только растёт** (глава 9). Пик численности определяет постоянное потребление.

Общие фрагменты. Один экземпляр на уникальное значение благодаря дедупликации (глава 6). Пренебрежимо.

Как сократить

Меньше одновременно активных задач. Каждая задача в активном стеке держит свою instance data. Состояние с тремя задачами вместо шести — вдвое меньше данных.

Меньше полей в instance data. Всё, что не меняется в рантайме и не привязывается, переносите в саму задачу (глава 15, раздел 15.8) — там одно значение на весь ассет.

Прогрев пула. Если знаете пиковую численность — заспавните её сразу при старте уровня, а не наращивайте постепенно. Избежите многократного перевыделения TArray с копированием тяжёлых структур.

23.6. Параллелизм: главный резерв

Самая большая потенциальная экономия — и самый сложный участок.

Как теряется параллелизм

Планировщик Mass строит граф по объявленным требованиям (глава 8). Два процессора, пишущие в один фрагмент, идут последовательно.

Процессор StateTree объявляет **объединение требований всех узлов всех обслуживаемых деревьев**. Для дерева из главы 21 это двенадцать записей, шесть на запись.

Каждая запись ReadWrite — потенциальный конфликт со всеми, кто трогает этот фрагмент. А конфликты **транзитивны**: ваш процессор блокирует X, X блокирует Y, цепочка вытягивается в линию.

Что с этим делать

Минимизировать ReadWrite. Пересмотрите `GetDependencies()` каждого узла. Задача, которая только читает фрагмент, должна объявлять `AddReadOnly`. Это самая дешёвая оптимизация из всех.

cpp

```
// Проверьте каждую строку: точно ли пишем?
Builder.AddReadOnly<FMassZoneGraphLaneLocationFragment>(); // только читаем
позицию
Builder.AddReadWrite<FMassMoveTargetFragment>();           // пишем
намерение — да, RW
Builder.AddReadOnly<FAgentRadiusFragment>();               // константа
конфигурации
```

Разделить широкие деревья. Дерево, объединяющее бой, навигацию и социальное поведение, объявляет требования всех трёх. Три отдельных ассета дадут три процессора с более узкими требованиями.

Цена: переключение между деревьями нетривиально (нужна смена общего фрагмента, то есть смена архетипа). Плюс больше процессоров — больше узлов в графе. Это компромисс, а не универсальное решение.

Помнить про подсистему сигналов.

cpp

```
template<>
struct TMassExternalSubsystemTraits<UMassSignalSubsystem> final
{
    enum
    {
        GameThreadOnly = false,
        // @todo this subsystem not being thread-safe when writing is an
        obstacle in
        // parallelizing multiple processors
        ThreadSafeWrite = false,
    };
};
```

Признанное узкое место (глава 14). Все процессоры, объявившие запись в подсистему сигналов, сериализуются между собой.

Обход: используйте **только deferred-варианты** отправки в своих узлах и объявляйте подсистему как `ReadOnly`. Тогда ваши процессоры не будут конфликтовать друг с другом

по этому основанию.

Параллельный обход внутри процессора

cpp

```
UPROPERTY(EditDefaultsOnly, Category = Processor, config)
bool bProcessEntitiesInParallel = false;
```

Раздаёт **чанки** разным потокам (глава 13). Выигрыш реальный: тик деревьев — одна из самых тяжёлых частей симуляции толпы.

Чек-лист безопасности (повторю из главы 13, он критичен):

1. Все задачи используют deferred-сигналы или не шлют сигналов.
2. Ни одна задача не создаёт и не уничтожает сущности напрямую.
3. Все подсистемы, к которым обращаются задачи, безопасны для параллельного использования.
4. Агентов достаточно много, чтобы окупить раздачу задач.

Нарушение любого пункта даёт либо ассерт MT-детектора, либо тихую гонку.

23.7. LOD для поведения

Приём, который в Mass используется повсеместно: **чем дальше агент, тем реже он думает**.

Базовые механизмы LOD в Mass относятся к представлению (визуал, анимация). Для поведения их придётся строить самому, и есть два подхода.

Подход первый: разные деревья по дистанции

Агент вблизи получает полное дерево, вдали — упрощённое. Переключение — смена конфигурации, то есть смена архетипа.

Дорого при частых переключениях: смена архетипа копирует все фрагменты агента (глава 2). Годится, если пороги дистанции с гистерезисом и переключения редки.

Подход второй: фильтрация сигналов

Более щадящий вариант. Процессоры, шлющие сигналы, учитывают LOD агента:

cpp

```

// в процессоре восприятия
const TArrayView<FMassRepresentationLODFragment> LODs =
    Context.GetFragmentView<FMassRepresentationLODFragment>();

for (int32 i = 0; i < Context.GetNumEntities(); ++i)
{
    if (LODs[i].LOD >= EMassLOD::Far)
    {
        continue;    // дальним агентам сигналы не шлём
    }
    // ...
}

```

Дерево дальнего агента просто не будит никто, и оно спит. При приближении сигналы возобновляются, дерево оживает.

Изящно: ноль изменений в дереве, ноль смен архетипа, полный контроль в процессорах.

Подход третий: увеличение задержек

Если поведение должно продолжаться, но может быть менее отзывчивым — увеличивайте задержки для дальних агентов. Пауза на точке 3 секунды вблизи и 15 секунд вдали. Разница незаметна визуально, а число пробуждений падает впятеро.

23.8. Архетипы и чанки

Косвенный, но существенный фактор.

Каждая уникальная композиция фрагментов и тегов — свой архетип (глава 2). Агенты одного архетипа лежат вместе, обход по ним эффективен.

Пятьдесят слегка различающихся конфигураций дадут пятьдесят архетипов. При тысяче агентов это по двадцать агентов на архетип — чанки полупустые, обход неэффективен, накладные расходы на итерацию по архетипам растут.

Правило: меньше конфигураций, больше вариативности внутри. Различия задавайте значениями фрагментов при спавне, а не разными наборами трейтов.

Помните также, что **общий фрагмент участвует в композиции**. Пятьдесят разных ассетов дерева — минимум пятьдесят архетипов, даже при одинаковых наборах фрагментов.

23.9. Количество процессоров

Из главы 8: процессор создаётся на каждый уникальный **набор требований**, а не на каждый ассет.

Если у вас двадцать деревьев с одинаковыми требованиями — один процессор. Если у всех двадцати требования разные — двадцать процессоров.

Двадцать процессоров означают:

- двадцать узлов в графе, которые солвер должен разместить;
- двадцать обходов архетипов (каждый процессор ищет свои чанки);
- двадцать проверок `HandledStateTrees` на чанк.

Как проверить: дамп графа процессоров (глава 22). Если динамических процессоров `StateTree` подозрительно много — смотрите, какие узлы вносят уникальные зависимости.

Как сократить: унифицируйте набор используемых узлов между похожими деревьями. Дерево, где вместо специфичной задачи используется общая, попадёт в тот же хеш требований.

23.10. Что мерить и в каком порядке

Практическая последовательность оптимизации.

1. Профилировщик: сколько занимает тик деревьев.

Если строка `UMassStateTreeProcessor` заметна — деревья не спят. Идите к пункту 2. Если не заметна — оптимизировать поведение бессмысленно, узкое место в другом.

2. Число пробуждений.

Заведите счётчик тиков дерева за секунду. Поделите на число агентов. Если больше единицы на агента в секунду — ищите, кто будит.

3. Профилировщик: тик подсистемы сигналов.

Если замечен — слишком много записей в очереди отложенных. Смотрите на короткие отложенные переходы.

4. Дамп графа процессоров.

Если граф линейный там, где мог бы ветвиться, — смотрите требования. Ищите лишние `ReadWrite`.

5. Память `instance data`.

`GetEstimatedMemoryUsage()` . Если много — сокращайте число одновременно активных задач.

6. Список архетипов.

Если их много при малом числе агентов — сокращайте конфигурации.

7. Параллельный обход.

Только после того, как всё остальное сделано. И только по чек-listy безопасности.

23.11. Чего делать не стоит

Оптимизировать до измерения. Тик деревьев в правильно спроектированном Mass-поведении почти незаметен. Если вы тратите время на микрооптимизацию задач, а узкое место в рендеринге толпы — работа впустую.

Отключать флаги наугад. `bShouldCallTick = false` у задачи, которая должна тикать, даст неработающий таймер, а не ускорение.

Включать параллелизм без проверки. Гонки данных обойдутся дороже любого выигрыша.

Дробить деревья ради параллелизма без нужды. Больше ассетов — больше архетипов и процессоров. Выигрыш от параллелизма может не окупить эти накладные расходы.

Строить свой LOD, не разобравшись с пробуждениями. Если деревья и так спят, LOD для поведения ничего не даст.

23.12. Итог главы

Статья	Пропорциональна	Основной резерв
Тик деревьев	Числу пробуждений	Сигналы по порогу, а не по изменению; длинные задержки
Сигналы	Числу вызовов и записей в очереди	Пакетная отправка; избегать очень коротких задержек
Память instance data	Агентам × активные задачи	Меньше задач в стеке; параметры в задаче, не в данных
Планирование	Структуре требований	Минимальные <code>ReadWrite</code> ; узкие деревья
Параллелизм	Конфликтам в графе	<code>ReadOnly</code> где можно; <code>deferred</code> -сигналы; параллельный обход

Статья	Пропорциональна	Основной резерв
Обход архетипов	Числу архетипов	Меньше конфигураций и ассетов

Три правила:

1. **Считайте пробуждения, а не агентов.** Это главная метрика Mass-поведения.
2. `ReadOnly` **вместо** `ReadWrite` — **самая дешёвая оптимизация.** Пересмотрите каждый узел.
3. **Измеряйте до оптимизации.** Хорошо спроектированное поведение почти не видно в профилировщике.

Что дальше

Глава 24 — заключительная. Соберём справочный материал: полную карту заголовков с указанием, что где искать; шпаргалку по всем типам и перечислениям, встреченным в книге; сводные чек-листы по написанию узлов, проектированию деревьев и конфигурации агентов; список известных ограничений и открытых `@todo` движка. Это глава, к которой вы будете возвращаться в работе.

Глава 24. Приложения

Заключительная глава — справочная. Здесь собрано то, к чему возвращаются в работе: карта заголовков, шпаргалка по типам, сводные чек-листы и список известных ограничений.

24.1. Карта заголовков: где что искать

Ядро `StateTree` (модуль `StateTreeModule`)

Файл	Что там
<code>StateTreeTypes.h</code>	Все перечисления: типы состояний, поведение выбора, триггеры и типы переходов, приоритеты, источники данных, требования внешних данных, назначение свойств. Хендлы состояний и данных
<code>StateTreeExecutionTypes.h</code>	<code>EStateTreeRunStatus</code> , <code>FStateTreeTransitionResult</code> , <code>FStateTreeExecutionState</code> ,

Файл	Что там
	FStateTreeExecutionFrame , FStateTreeExecutionExtension
StateTreeInstanceData.h	FStateTreeInstanceStorage , FStateTreeInstanceData , временные экземпляры, TStateTreeInstanceDataStructRef
StateTreeExecutionContext.h	Три уровня контекста, FOnCollectStateTreeExternalData , FStartParameters , весь API исполнения
StateTreeNodeBase.h	Общая база узлов, InstanceDataHandle , BindingsBatch
StateTreeTaskBase.h	FStateTreeTaskBase с флагами и методами, FStateTreeTaskCommonBase
StateTreeEvaluatorBase.h	FStateTreeEvaluatorBase , FStateTreeEvaluatorCommonBase
StateTreeConditionBase.h	FStateTreeConditionBase , операнды выражений
StateTreePropertyFunctionBase.h	База property functions
StateTreeSchema.h	UStateTreeSchema — все виртуальные методы фильтрации
StateTreeLinker.h	FStateTreeLinker::LinkExternalData()
StateTreePropertyRef.h	FStateTreePropertyRef — ссылка на чужое свойство
StateTreeEvents.h	FStateTreeEvent , FStateTreeEventQueue
StateTree.h	UStateTree — сам ассет, скомпилированные массивы

Ядро Mass (модули MassEntity , MassSignals)

Файл	Что там
Mass/EntityHandle.h	FMassEntityHandle
MassElement.h	FMassFragment , FMassTag , FMassChunkFragment , FMassSharedFragment , FMassConstSharedFragment
MassEntityTypes.h	FMassArchetypeCompositionDescriptor , FMassArchetypeSharedFragmentValues , EMassObservedOperation , параметры создания архетипов
MassEntityManager.h	Создание и уничтожение сущностей, доступ к фрагментам, наблюдатели

Файл	Что там
MassEntityQuery.h	FMassEntityQuery , AddRequirement , ForEachEntityChunk
MassExecutionContext.h	FMassExecutionContext — доступ к чанку, Defer()
MassProcessor.h	UMassProcessor , UMassCompositeProcessor , FMassProcessorExecutionOrder
MassObserverProcessor.h	UMassObserverProcessor
MassProcessorDependencySolver.h	FMassProcessorDependencySolver , FMassExecutionRequirements
MassCommandBuffer.h	Отложенные команды
MassEntityTraitBase.h	UMassEntityTraitBase::BuildTemplate()
MassEntityTemplate.h	FMassEntityTemplate , FMassEntityTemplateBuildContext
MassSignalSubsystem.h	Все двенадцать методов отправки, FDelayedSignal , трейты подсистемы
MassSignalProcessorBase.h	UMassSignalProcessorBase , SubscribeToSignal() , SignalEntities()

Связка (модуль MassAIBehavior)

Файл	Что там
MassStateTreeTypes.h	Четыре базы узлов, каталог сигналов, ExecutionFlags , FMassStateTreeInstanceHandle
MassStateTreeFragments.h	FMassStateTreeInstanceFragment , FMassStateTreeSharedFragment
MassStateTreeSchema.h	UMassStateTreeSchema , GetDependencies()
MassStateTreeDependency.h	FMassStateTreeDependency , FStateTreeDependencyBuilder
MassStateTreeSubsystem.h	Пул instance data, создание динамических процессоров
MassStateTreeExecutionContext.h	FMassStateTreeExecutionContext , FMassExecutionExtension
MassStateTreeProcessors.h	Три процессора, FMassStateTreeActivatedTag
MassLookAtTask.h	Пример задачи с опциональными данными и таймером
MassZoneGraphPathFollowTask.h	Пример задачи с восемью хендлами и FStateTreePropertyRef

24.2. Шпаргалка по перечислениям

EStateTreeRunStatus

Running · Failed · Succeeded · Stopped · Unset

Возвращают EnterState() и Tick(). Не-Running из EnterState() завершает состояние немедленно.

EStateTreeStateType

State · Group · Linked · LinkedAsset · Subtree

EStateTreeStateSelectionBehavior

None · TryEnterState · TrySelectChildrenInOrder · TrySelectChildrenAtRandom · TrySelectChildrenWithHighestUtility · TrySelectChildrenAtRandomWeightedByUtility · TryFollowTransitions

EStateTreeTransitionTrigger (битовая маска)

None · OnStateCompleted (= Succeeded | Failed) · OnStateSucceeded · OnStateFailed · OnTick · OnEvent · OnDelegate

EStateTreeTransitionType

None · Succeeded · Failed · GotoState · Parent · NextState · NextSelectableState · NextParent · NextSelectableParent

EStateTreeTransitionPriority

Low · Normal · Medium · High · Critical

EStateTreeExpressionOperand

Copy · And (условия: AND; полезность: Min) · Or (OR; Max) · Multiply (только полезность)

EStateTreeExternalDataRequirement

Required → GetExternalData() · Optional → GetExternalDataPtr()

EStateTreePropertyUsage

Invalid · Context · Input · Parameter · Output

EStateTreeDataSourceType (ОСНОВНЫЕ)

Глобальные: GlobalInstanceData , GlobalInstanceDataObject

Активного состояния: ActiveInstanceData , ActiveInstanceDataObject

Общие: SharedInstanceData , SharedInstanceDataObject

Временные: EvaluationScopeInstanceData , ...Object

Рантайм: ExecutionRuntimeData , ...Object , ...Any

Прочие: ContextData , ExternalData , GlobalParameterData ,
ExternalGlobalParameterData , SubtreeParameterData , StateParameterData ,
TransitionEvent , StateEvent

EMassFragmentAccess

ReadOnly · ReadWrite

EMassFragmentPresence

All · Any · None · Optional

EMassObservedOperation

AddElement · RemoveElement · DestroyEntity · CreateEntity (плюс устаревшие Add ,
Remove)

EProcessorExecutionFlags

Standalone · Server · Client · Editor

Для StateTree в Mass: Standalone | Server

EMassProcessingPhase

PrePhysics (по умолчанию) · StartPhysics · DuringPhysics · EndPhysics ·
PostPhysics · FrameEnd

Сигналы UE::Mass::Signals

StateTreeActivate · LookAtFinished · NewStateTreeTaskRequired · StandTaskFinished ·
AnimateTaskFinished · DelayedTransitionWakeup · ContextualAnimTaskFinished

24.3. Флаги задачи: справочник

Флаг	По умолчанию	Ставить false , когда
bShouldStateChangeOnReselect	true	Задача удерживает ресурс
bShouldCallTick	true	Задача не тикает по существу

Флаг	По умолчанию	Ставить false , когда
<code>bShouldCallTickOnlyOnEvents</code>	<code>false</code>	(ставить true для реакции только на события)
<code>bShouldCopyBoundPropertiesOnTick</code>	<code>true</code>	Входы читаются только при входе
<code>bShouldCopyBoundPropertiesOnExitState</code>	<code>true</code>	<code>ExitState()</code> не читает привязок
<code>bShouldAffectTransitions</code>	<code>false</code>	(ставить true для <code>TriggerTransitions()</code>)
<code>bConsideredForScheduling</code>	<code>true</code>	В Mass значения не имеет
<code>bConsideredForCompletion</code>	<code>true</code>	Задача фоновая

24.4. Чек-лист: написание узла

Структура

- Наследование от Mass-базы, не от базы StateTree
- `USTRUCT()` + `GENERATED_BODY()`
- `using FInstanceDataType = ...;`
- `GetInstanceDataType()` переопределён
- `meta = (DisplayName = "...")` задан
- Не скопирован `meta = (Hidden)` из базы

Данные

- Категории полей осознаны: `Input` / `Parameter` / без категории
- Рабочие поля сбрасываются в `EnterState()`
- Статическое — в задаче, динамическое — в `instance data`
- Большие структуры передаются через `FStateTreePropertyRef` , не копией

Внешние данные

- Каждый хендл в `Link()`
- Каждый хендл в `GetDependencies()` — списки совпадают
- `ReadOnly` там, где не пишем
- `Optional` читается через `GetExternalDataPtr()` с проверкой на `nullptr`
- `Required` читается через `GetExternalData()`

Флаги

- `bShouldCallTick` выключен, если тик пустой
- `bShouldStateChangeOnReselect = false` для удерживающих задач
- `bConsideredForCompletion = false` для фоновых

Жизненный цикл

- Захваченное в `EnterState()` освобождается в `ExitState()`
- Освобождение не в `StateCompleted()`
- `Tick()` не предполагает малого `DeltaTime`
- На каждый `return Running` есть ответ на вопрос «кто разбудит»

Сигналы

- Внутри узла — только `deferred`
- Для многих сущностей — версия с массивом
- Имена — константы, не строки на месте
- Процессор подписан на свои сигналы

Прочее

- `GetDebugInfo()` переопределён
- Модули в `Build.cs`

24.5. Чек-лист: проектирование дерева

- Состояния соответствуют осмысленным фазам поведения, а не отдельным действиям
- Фоновые задачи помечены `bConsideredForCompletion = false`
- Условия входа дешёвые: сравнения, а не вычисления
- Каждое состояние с задачами, возвращающими `Running`, имеет источник пробуждения
- Отложенные переходы не короче секунды, если агентов много
- Длительности привязаны к случайным `property functions` для разнообразия
- `RandomSeed` задан явно, если нужен детерминизм
- Дерево не объединяет несвязанные области поведения (иначе широкие требования)
- Переопределения связанных деревьев не расширяют набор требований

24.6. Чек-лист: конфигурация агента

- Все трейты, от которых зависят обязательные внешние данные дерева, добавлены
- Персональные данные — обычные фрагменты
- Общее и неизменное — константные общие фрагменты

- `ValidateTemplate()` реализован в своих трейтах
- Число конфигураций минимально; вариативность через значения, а не композицию
- Ассет дерева выбирается из отфильтрованного по схеме списка
- Спавн пакетный, не поштучный
- `DynamicStateTreeProcessorClass` прописан, если есть свои сигналы

24.7. Известные ограничения движка

Собрано из комментариев и `@todo` в разобранных исходниках.

Аллокация `instance data` не параллелится.

cpp

```
// @todo Instance data creation needs to be refactored in order to allow
parallelization of StateTree activations
```

Массовый спавн агентов с деревьями идёт последовательно (глава 9).

Подсистема сигналов небезопасна для параллельной записи.

cpp

```
// @todo this subsystem not being thread-safe when writing is an obstacle in
parallelizing multiple processors
```

Ограничивает параллелизм всех процессоров, шлющих сигналы (глава 14).

Поведение при перекомпиляции ассета не определено.

cpp

```
// @todo ask Patrick how would this behave when ST asset gets
changed/recompiled or whatnot
```

После правок дерева в PIE процессор может остаться со старыми требованиями (глава 9).

Сигнал контекстных анимаций в неправильном модуле.

cpp

```
// @todo MassStateTree: move this to its game plugin when possible
```

`ContextualAnimTaskFinished` логически не часть базового Mass AI (глава 5).

Состояние деревьев не сохраняется. `FStateTreeInstanceData` поддерживает сериализацию только для замены ссылок на объекты, не для записи на диск (глава 10).

Массив instance data только растёт. Пиковая численность определяет постоянное потребление памяти (глава 9).

Клиент поведение не исполняет. `ExecutionFlags` — только `Standalone` | `Server` (глава 5).

Опечатки в исходниках, о которых стоит знать: поле `EntityManager` с юникодным `Ё` в подсистеме; двойные точки с запятой в нескольких местах; `InScriptStruct` как имя параметра типа `UClass*` в `IsClassAllowed()`.

24.8. Двадцать утверждений, которые стоит помнить

1. Узлы `StateTree` не имеют изменяемого состояния — только `const`-методы и `instance data`.
2. `Instance data` живёт в подсистеме; на сущности — только 8-байтовый хендл.
3. Ассет дерева разделяется всеми агентами через константный общий фрагмент.
4. Внешние данные объявляются дважды: `Link()` для `StateTree`, `GetDependencies()` для `Mass`.
5. Рассогласование этих двух методов даёт гонку без единой ошибки компиляции.
6. Тик происходит только по сигналу; отсутствие сигнала — это «навсегда», а не «пауза».
7. Задача выражает намерение; работу делают процессоры.
8. `Optional` внешние данные читаются только через `GetExternalDataPtr()`.
9. `DeltaTime` в `Mass` — время между тиками дерева, может быть секундами.
10. Освобождение ресурсов — в `ExitState()`, никогда в `StateCompleted()`.
11. Указатель на `instance data` нельзя сохранять — только `TStateTreeInstanceDataStructRef`.
12. Контекст исполнения временный; один на чанк, перенацеливается через `SetEntity()`.
13. Схема запрещает всё по умолчанию; `Mass`-база узла — пропуск в дерево.
14. Тег меняет архетип, поэтому ставится один раз и через командный буфер.
15. Внутри обхода чанков сигналы отправляются только `deferred`-вариантами.
16. Для нескольких сущностей — один вызов с массивом, не цикл.
17. Состав требований дерева влияет на порядок исполнения всей симуляции.
18. `ReadOnly` вместо `ReadWrite` — самая дешёвая оптимизация параллелизма.
19. Главная метрика производительности — число пробуждений, а не число агентов.

20. Разнообразие толпы достигается случайными привязками, а не дублированием деревьев.

24.9. Что осталось за рамками

Честный список тем, которые в книге не разобраны и куда стоит смотреть дальше.

Blueprint-узлы StateTree. В Mass не применяются (глава 7), но существуют для деревьев на компонентах.

Асинхронный контекст. `StateTreeAsyncExecutionContext.h` и `FStateTreeWeakExecutionContext` — для завершения задач из колбэков и других потоков.

Utility AI в StateTree. Considerations, нормировка полезности, взвешенный случайный выбор — механизм упомянут (глава 3), но не разобран детально.

Остальные задачи MassAIBehavior. Стояние, анимация, поиск точек интереса, работа со слотами — устроены по разобранным принципам.

ZoneGraph как система. Построение графа зон, теги полос, аннотации, поиск пути между полосами.

Mass Representation и LOD. Визуальное представление толпы, ISM/HISM, переключение уровней детализации.

Сетевая репликация Mass. Как авторитетное состояние сервера доходит до клиентов.

Редактор StateTree. Работа с деревьями в редакторе: привязки, отладочные точки, параметры.

24.10. Как учиться дальше

Три практических совета напоследок.

Читайте .cpp там, где мы читали только заголовки. Реализации `MassStateTreeSubsystem.cpp`, `MassStateTreeProcessors.cpp` и `MassStateTreeSchema.cpp` подтвердят или уточнят реконструкции из глав 7, 9 и 13. Теперь вы знаете, что там искать.

Разберите оставшиеся задачи модуля. Каждая добавит один-два новых приёма к тем, что мы видели в главах 16 и 17.

Соберите собственный минимальный проект. Один агент, одно дерево из двух состояний, одна своя задача. Проведите его через все стадии — от трейта до отладчика. Это закрепит связи между главами лучше любого чтения.

Заключение

Мы прошли путь от «зачем вообще эта связка» до собственных узлов, конфигурации агентов и оптимизации.

Ключевая идея, вокруг которой всё построено, стоит того, чтобы повторить её последний раз: **StateTree в Mass не исполняет поведение — он его планирует**. Дерево просыпается, принимает решение, записывает намерение и засыпает. Всю работу делают процессоры, обходящие плотные массивы данных пачками.

Из этой идеи следует всё остальное: и вынос instance data в подсистему, и событийная модель тика, и система зависимостей, и требование, чтобы задачи были тонкими. Когда очередной механизм модуля покажется странным, попробуйте объяснить его через эту идею — почти всегда получится.

Удачи с толпами.